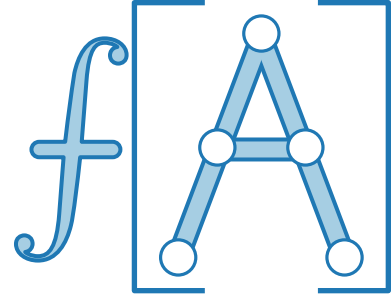


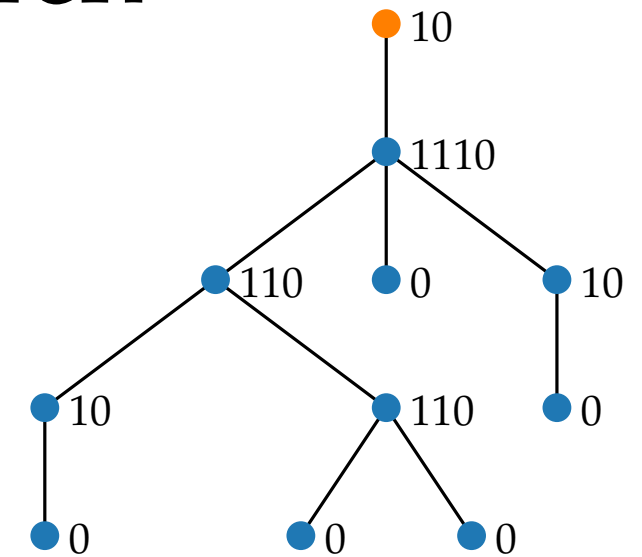


Fortgeschrittene Algorithmen

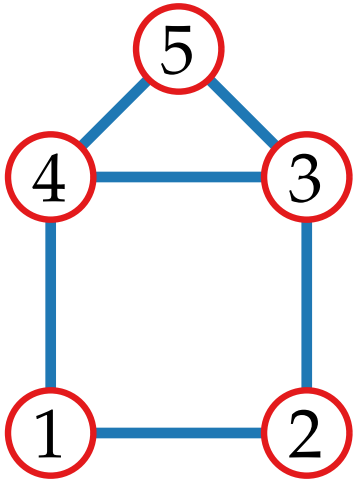
16. Vorlesung Datenstrukturen II:

Platzeffiziente Datenstrukturen (Succinct Data Structures)

Teil I: Problemstellung



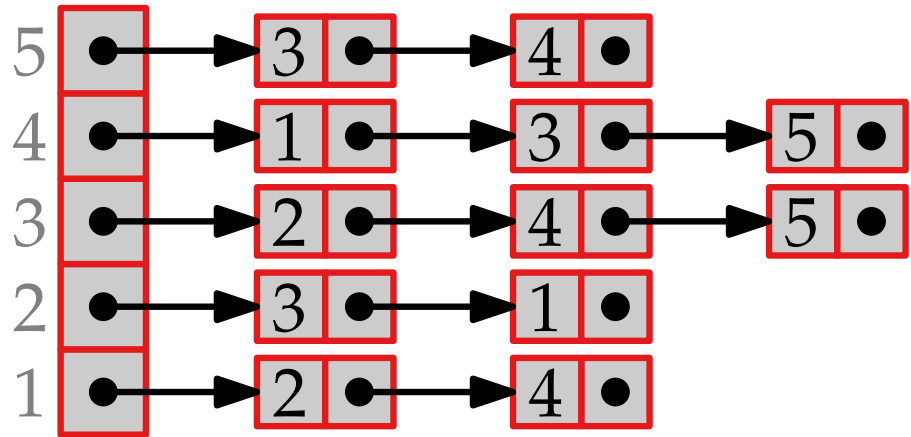
Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

$$E = \{ \{1,2\}, \{2,3\}, \{3,4\}, \{1,4\}, \{3,5\}, \{4,5\} \}$$

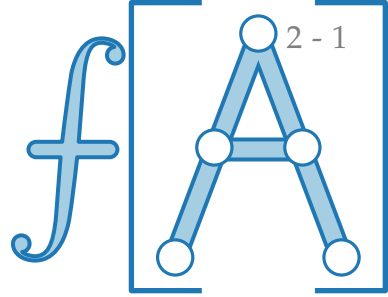
Mengen



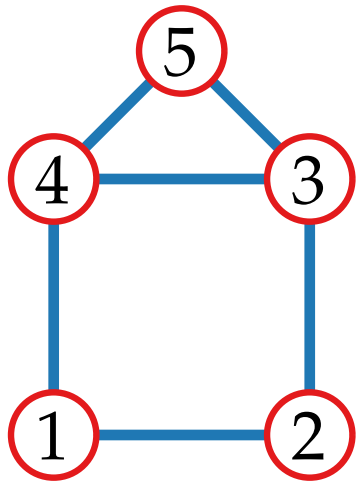
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix



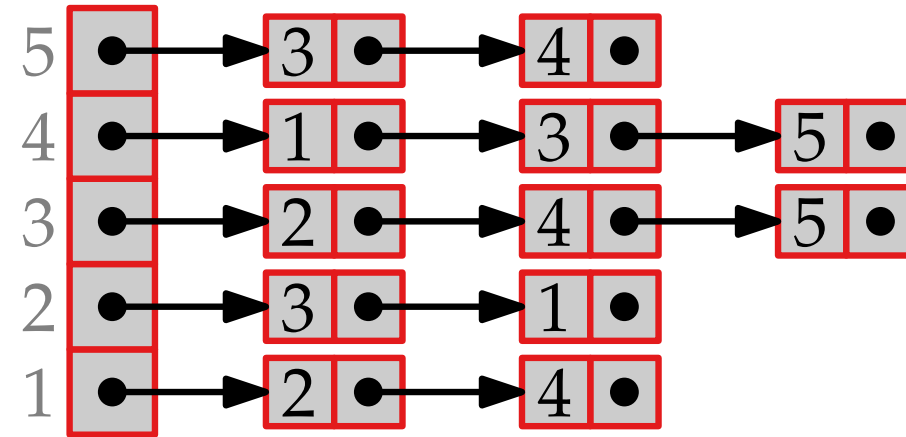
Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

$$E = \{ \{1,2\}, \{2,3\}, \{3,4\}, \{1,4\}, \{3,5\}, \{4,5\} \}$$

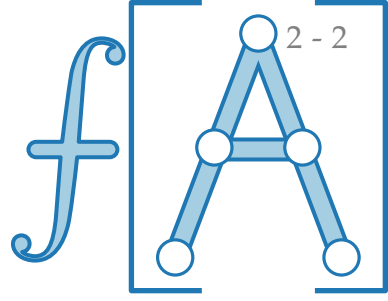
Mengen



Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

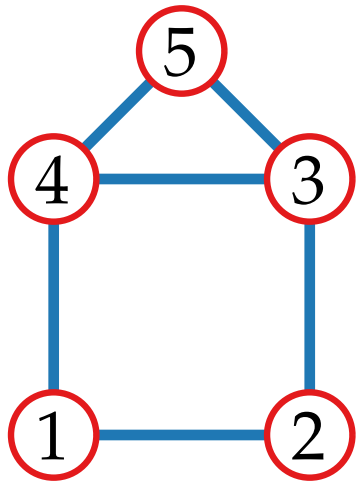
Adjazenzmatrix



Speicher

Zeit für Frage:
sind Knoten i
und j adjazent?

Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

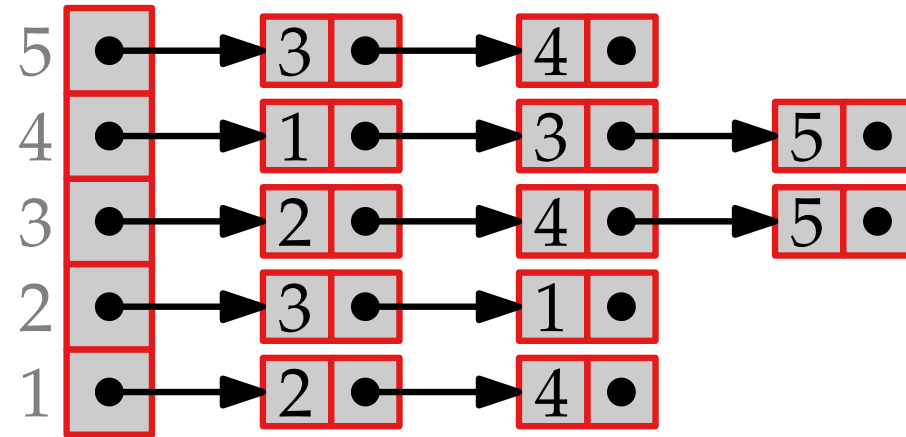
$$E = \{ \{1,2\}, \{2,3\}, \{3,4\}, \{1,4\}, \{3,5\}, \{4,5\} \}$$

Mengen

Speicher

$$\mathcal{O}(m)$$

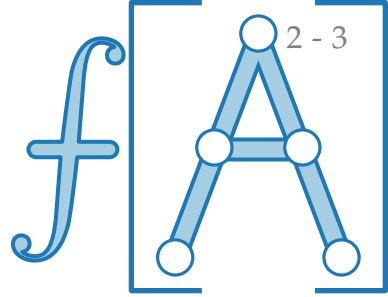
Zeit für Frage:
sind Knoten i
und j adjazent?



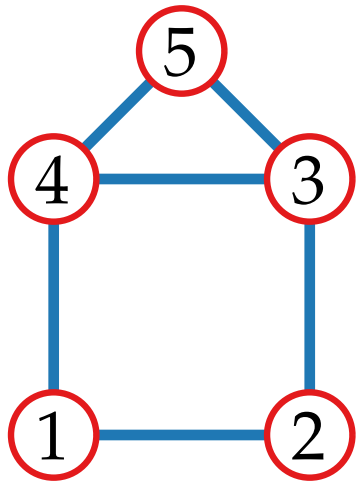
Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix



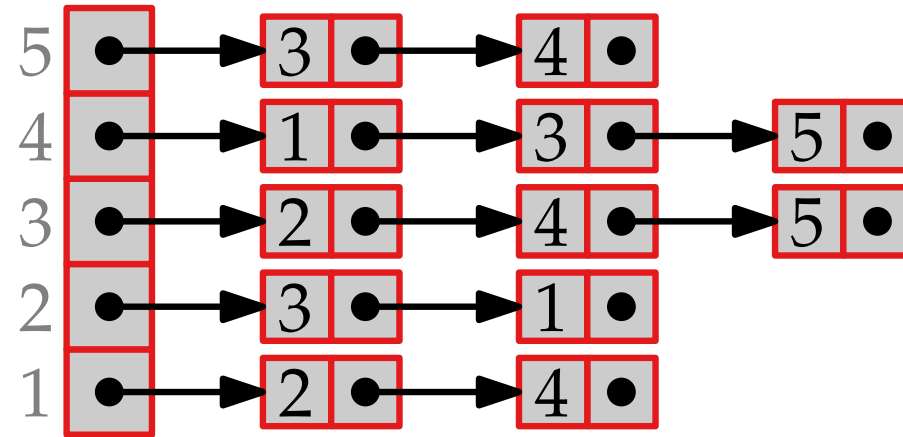
Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

$$E = \{ \{1,2\}, \{2,3\}, \{3,4\}, \{1,4\}, \{3,5\}, \{4,5\} \}$$

Mengen



Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

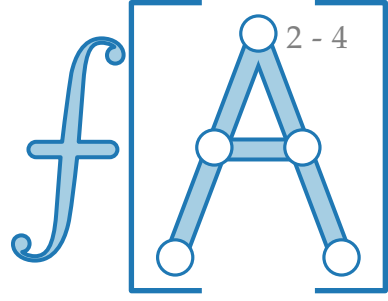
Adjazenzmatrix

Speicher

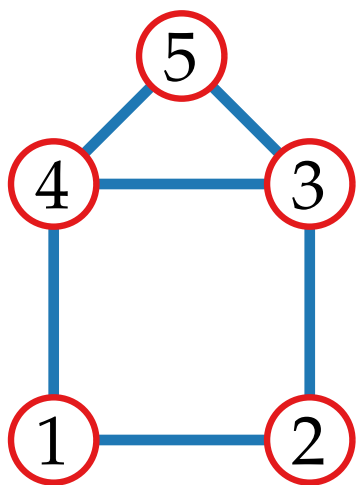
$$\mathcal{O}(m)$$

Zeit für Frage:
sind Knoten i
und j adjazent?

$$\mathcal{O}(m)$$



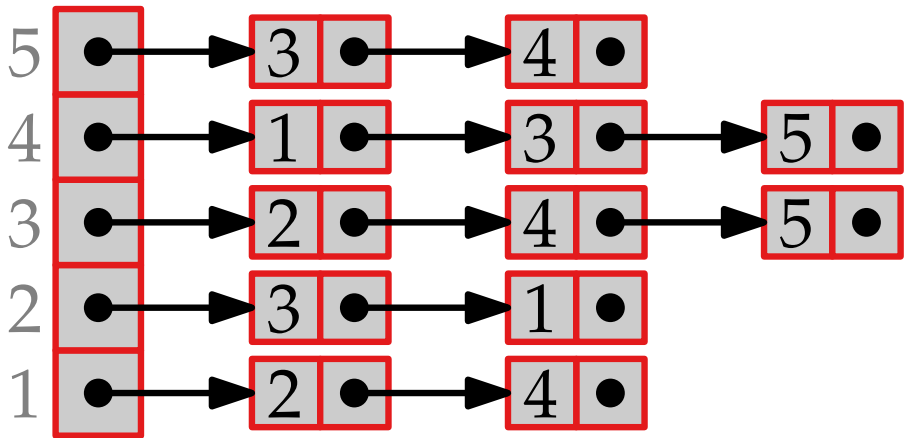
Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

$$E = \{ \{1,2\}, \{2,3\}, \{3,4\}, \{1,4\}, \{3,5\}, \{4,5\} \}$$

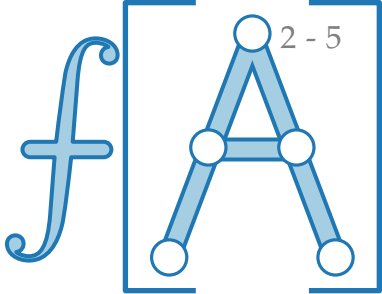
Mengen



Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix



Speicher

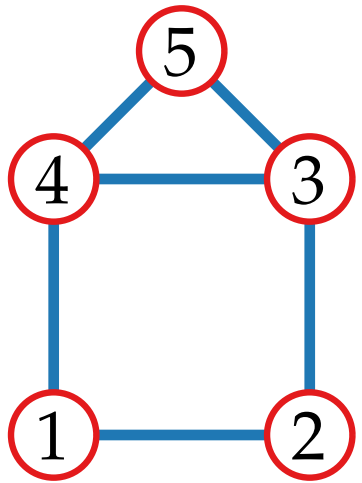
$$\mathcal{O}(m)$$

$$\mathcal{O}(n + m)$$

Zeit für Frage:
sind Knoten i
und j adjazent?

$$\mathcal{O}(m)$$

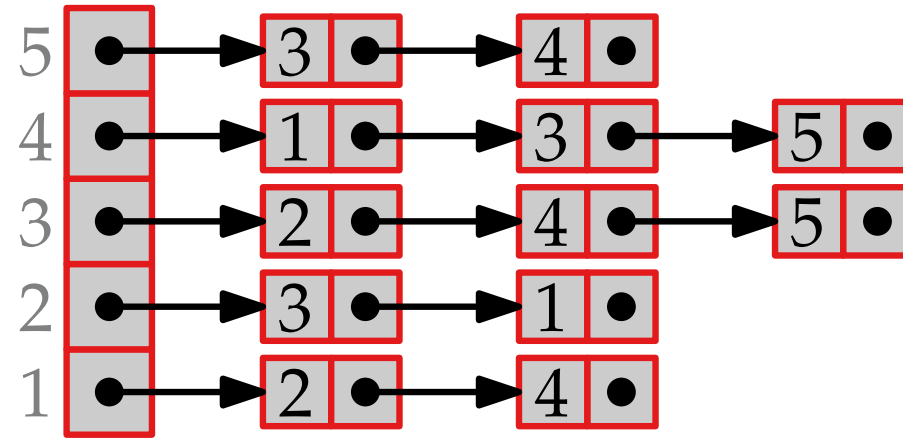
Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

$$E = \{ \{1, 2\}, \{2, 3\}, \{3, 4\}, \{1, 4\}, \{3, 5\}, \{4, 5\} \}$$

Mengen



Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

Speicher

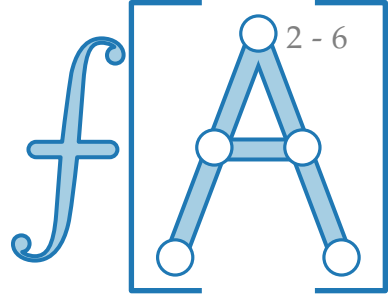
$$\mathcal{O}(m)$$

$$\mathcal{O}(n + m)$$

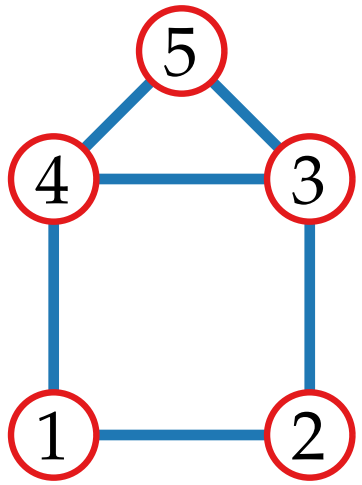
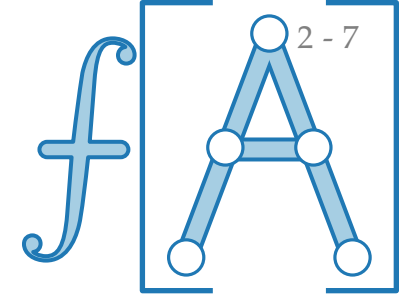
Zeit für Frage:
sind Knoten i
und j adjazent?

$$\mathcal{O}(m)$$

$$\mathcal{O}(n)$$



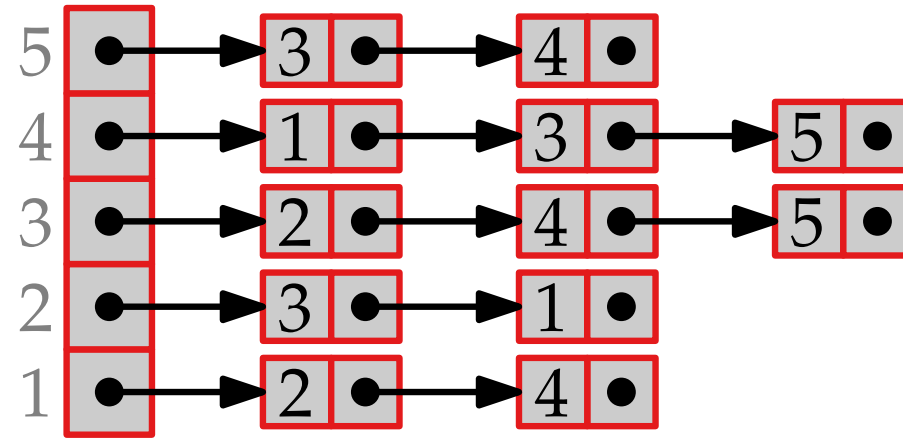
Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

$$E = \{ \{1, 2\}, \{2, 3\}, \{3, 4\}, \{1, 4\}, \{3, 5\}, \{4, 5\} \}$$

Mengen



Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

Speicher

$$\mathcal{O}(m)$$

$$\mathcal{O}(n + m)$$

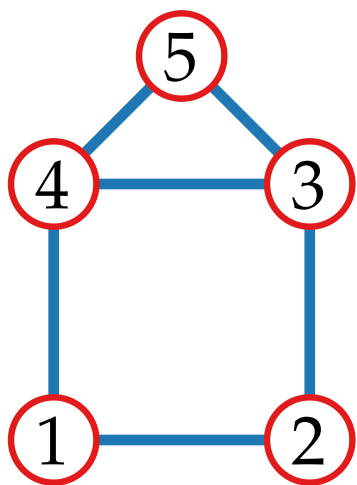
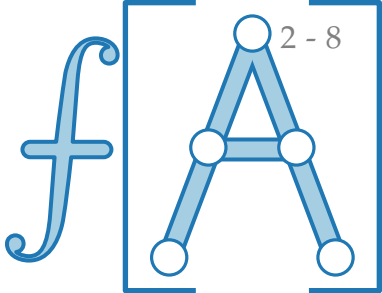
$$\mathcal{O}(n^2)$$

Zeit für Frage:
sind Knoten i
und j adjazent?

$$\mathcal{O}(m)$$

$$\mathcal{O}(n)$$

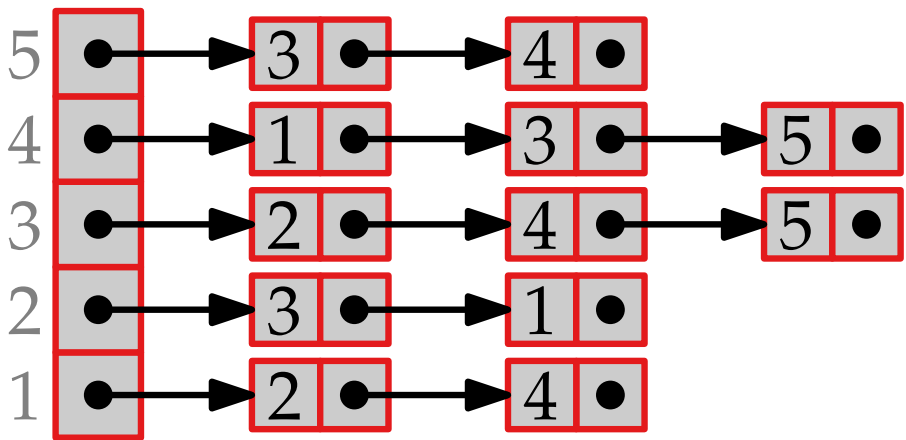
Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

$$E = \{ \{1, 2\}, \{2, 3\}, \{3, 4\}, \{1, 4\}, \{3, 5\}, \{4, 5\} \}$$

Mengen



Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

Speicher

$$\mathcal{O}(m)$$

$$\mathcal{O}(n + m)$$

$$\mathcal{O}(n^2)$$

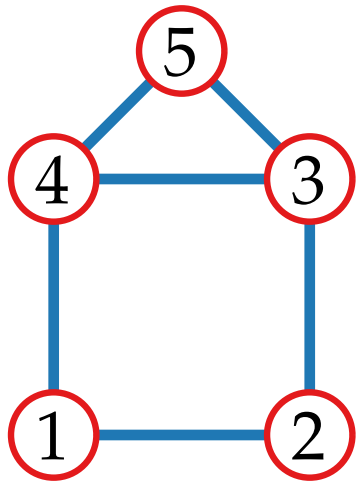
Zeit für Frage:
sind Knoten i
und j adjazent?

$$\mathcal{O}(m)$$

$$\mathcal{O}(n)$$

$$\mathcal{O}(1)$$

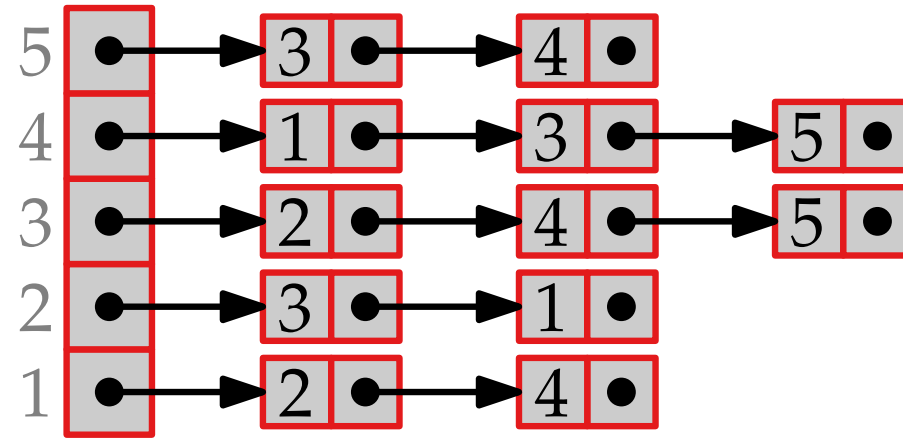
Trade-Off zwischen Speicher und Anfragezeit



ungerichteter
zshg. Graph

$$E = \{ \{1, 2\}, \{2, 3\}, \{3, 4\}, \{1, 4\}, \{3, 5\}, \{4, 5\} \}$$

Mengen



Adjazenzlisten

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	1
5	0	0	1	1	0

Adjazenzmatrix

Speicher

$$\mathcal{O}(m)$$

$$\mathcal{O}(n + m)$$

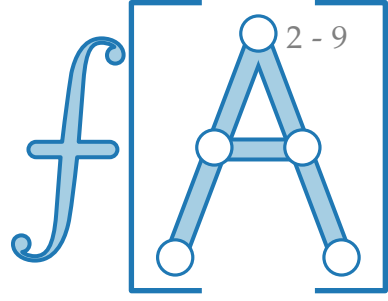
$$\mathcal{O}(n^2)$$

Zeit für Frage:
sind Knoten i
und j adjazent?

$$\mathcal{O}(m)$$

$$\mathcal{O}(n)$$

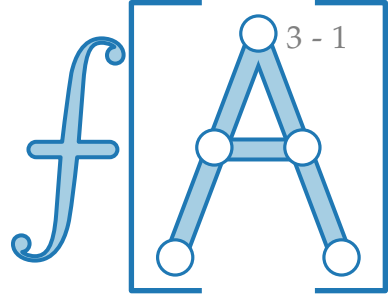
$$\mathcal{O}(1)$$



Platzeffiziente Datenstrukturen

Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

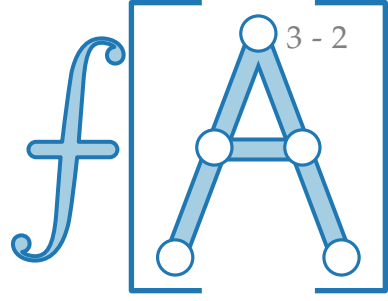


Platzeffiziente Datenstrukturen

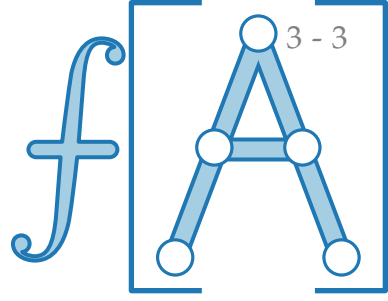
Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.



Platzeffiziente Datenstrukturen



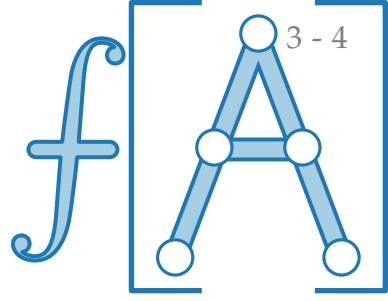
Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

Platzeffiziente Datenstrukturen



Ziel.

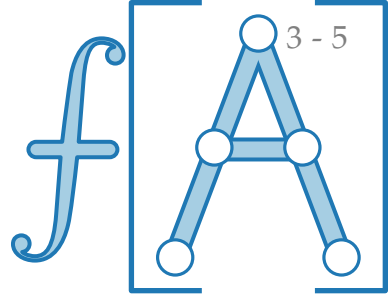
- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

- **implizit**, wenn sie $L + \mathcal{O}(1)$ Bits Speicher braucht;

Platzeffiziente Datenstrukturen



Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

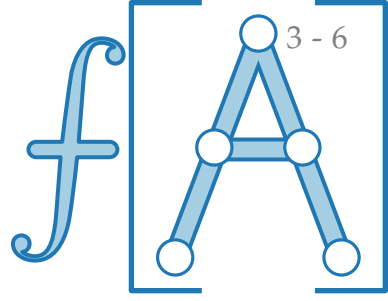
Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

- **implizit**, wenn sie $L + \mathcal{O}(1)$ Bits Speicher braucht;

z.B.

$$L + 5$$

Platzeffiziente Datenstrukturen



Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

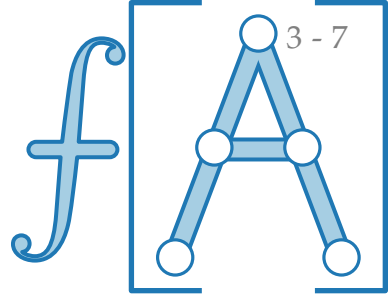
Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

z.B.

- **implizit**, wenn sie $L + \mathcal{O}(1)$ Bits Speicher braucht;
- **platzeffizient** (en. succinct), wenn sie $L + o(L)$ Bits Speicher braucht;

$L + 5$

Platzeffiziente Datenstrukturen



Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

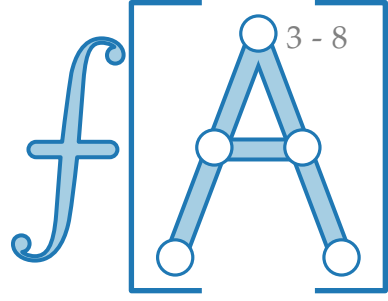
- **implizit**, wenn sie $L + \mathcal{O}(1)$ Bits Speicher braucht;
- **platzeffizient** (en. succinct), wenn sie $L + o(L)$ Bits Speicher braucht;

z.B.

$$L + 5$$

$$L + \log L$$

Platzeffiziente Datenstrukturen



Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

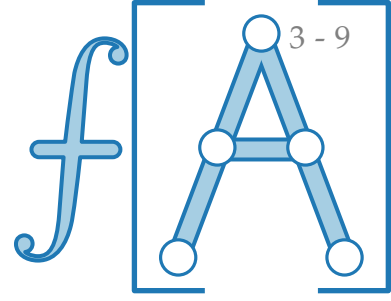
- **implizit**, wenn sie $L + \mathcal{O}(1)$ Bits Speicher braucht;
- **platzeffizient** (en. succinct), wenn sie $L + o(L)$ Bits Speicher braucht;

z.B.

$$L + 5$$

$$L + \log L$$
$$L + \sqrt{L}$$

Platzeffiziente Datenstrukturen



Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

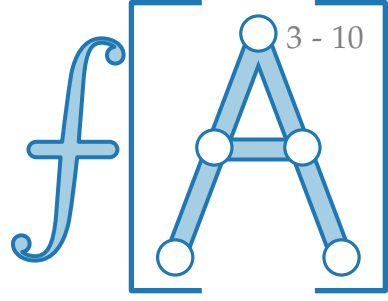
z.B.

- **implizit**, wenn sie $L + \mathcal{O}(1)$ Bits Speicher braucht;
- **platzeffizient** (en. succinct), wenn sie $L + o(L)$ Bits Speicher braucht;
- **kompakt**, wenn sie $\mathcal{O}(L)$ Bits Speicher braucht.

$$L + 5$$

$$L + \log L$$
$$L + \sqrt{L}$$

Platzeffiziente Datenstrukturen



Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

- **implizit**, wenn sie $L + \mathcal{O}(1)$ Bits Speicher braucht;
- **platzeffizient** (en. succinct), wenn sie $L + o(L)$ Bits Speicher braucht;
- **kompakt**, wenn sie $\mathcal{O}(L)$ Bits Speicher braucht.

z.B.

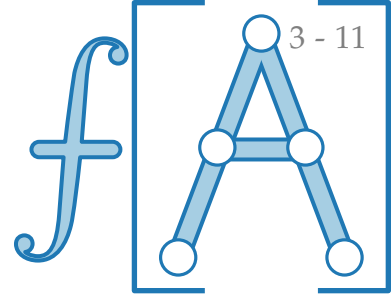
$$L + 5$$

$$L + \log L$$

$$L + \sqrt{L}$$

$$2L$$

Platzeffiziente Datenstrukturen



Ziel.

- Speicher „nah“ am informationstheoretischen Minimum
- aber trotzdem noch zeiteffiziente Operationen.

Sei L die informationstheoretische untere Schranke, um eine Klasse von Objekten zu repräsentieren.

Eine Datenstruktur, die **zeiteffiziente** Operationen zulässt, heißt

- **implizit**, wenn sie $L + \mathcal{O}(1)$ Bits Speicher braucht;
- **platzeffizient** (en. succinct), wenn sie $L + o(L)$ Bits Speicher braucht;
- **kompakt**, wenn sie $\mathcal{O}(L)$ Bits Speicher braucht.

z.B.

$$L + 5$$

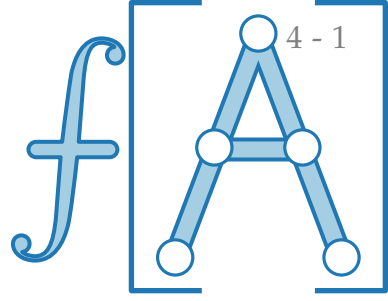
$$L + \log L$$

$$L + \sqrt{L}$$

$$2L$$

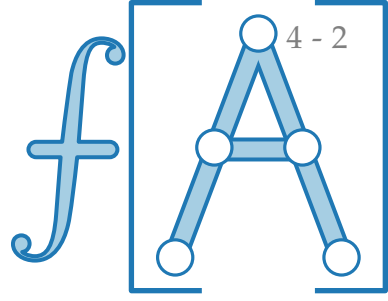
Beispiele?

Beispiele für implizite Datenstrukturen



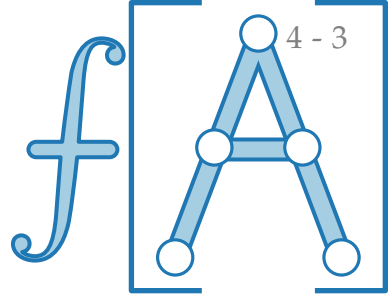
Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren



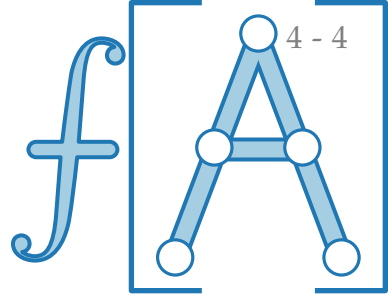
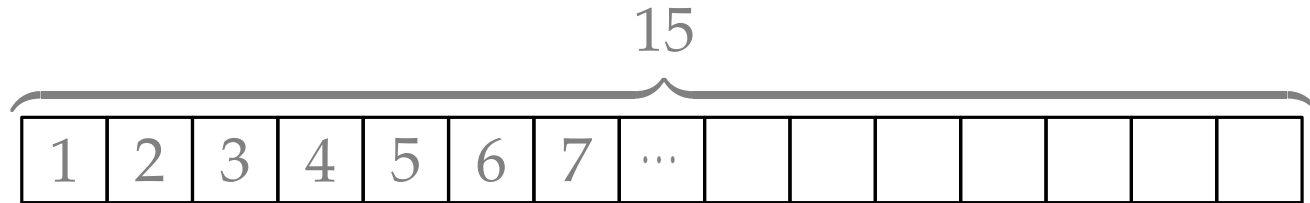
Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren



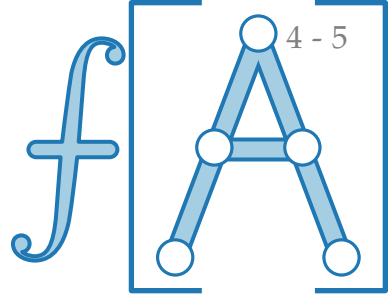
Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren

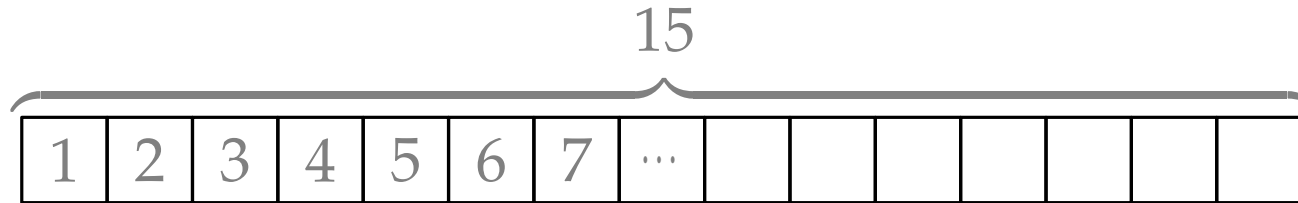


Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren

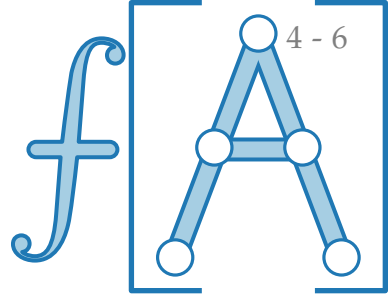


Speicher

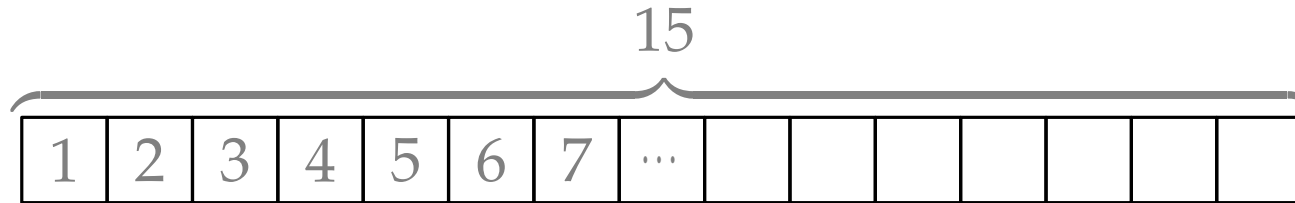


Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren



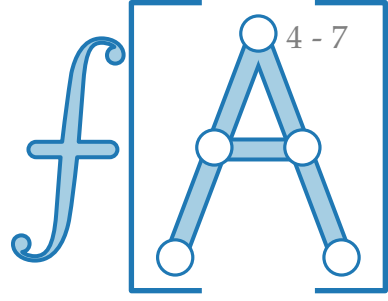
Speicher



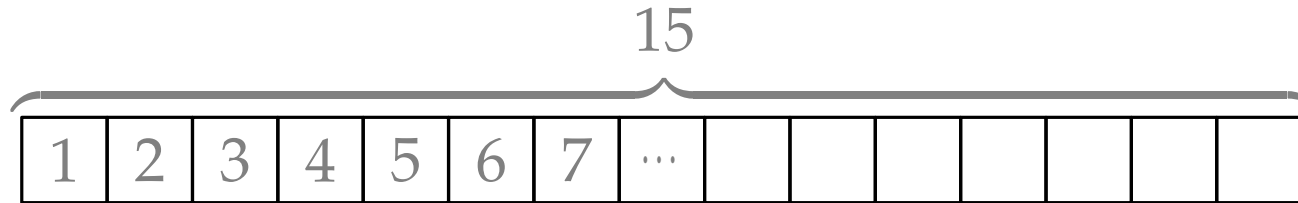
L

Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren



Speicher

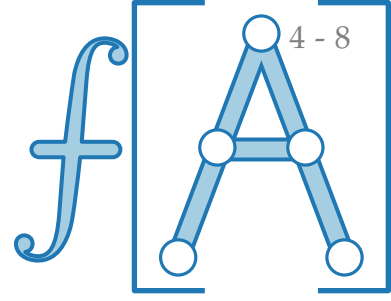


L

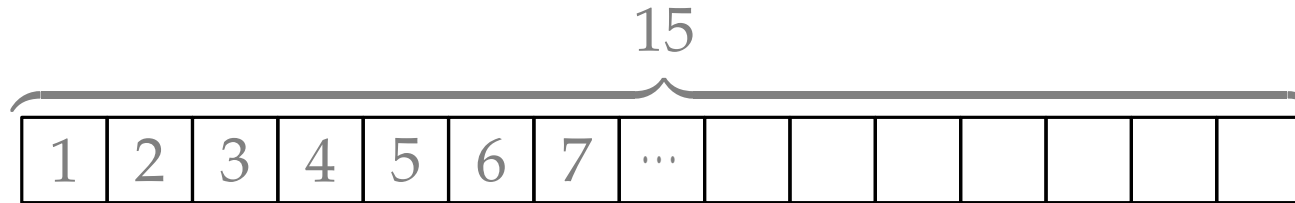
implizit

Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?



Speicher

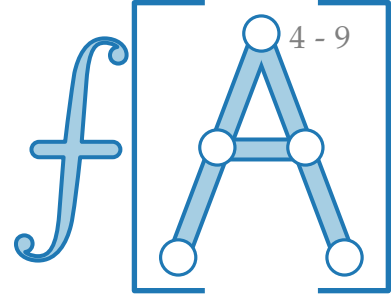


L

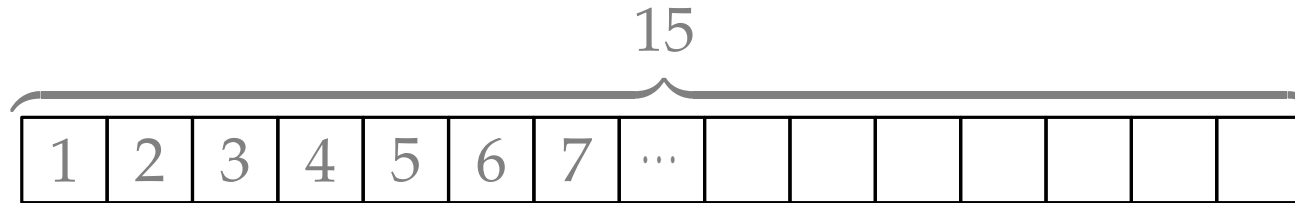
implizit

Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?

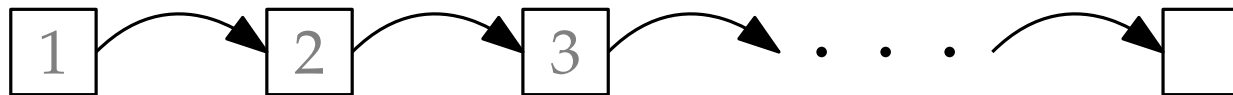


Speicher



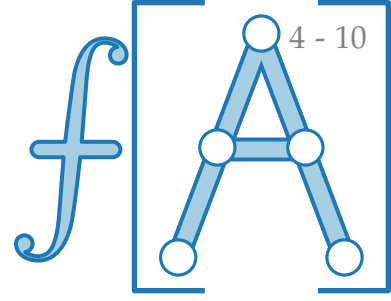
L

implizit

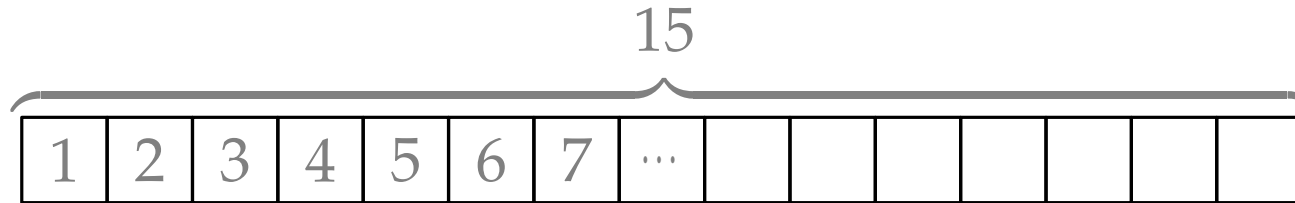


Beispiele für implizite Datenstrukturen

- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?

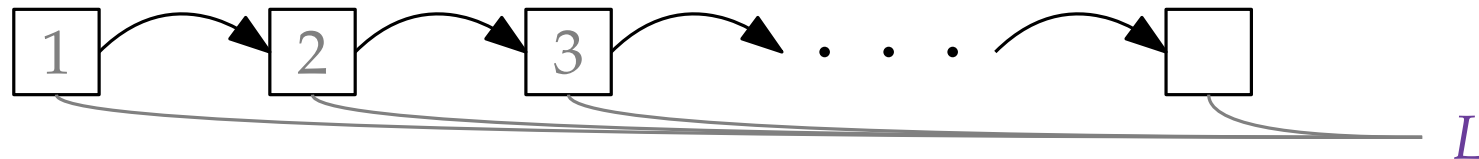


Speicher



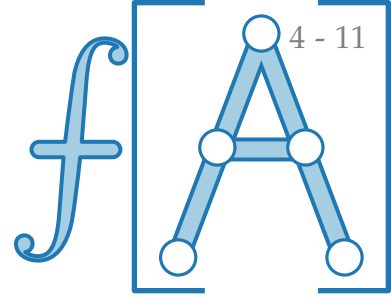
L

implizit



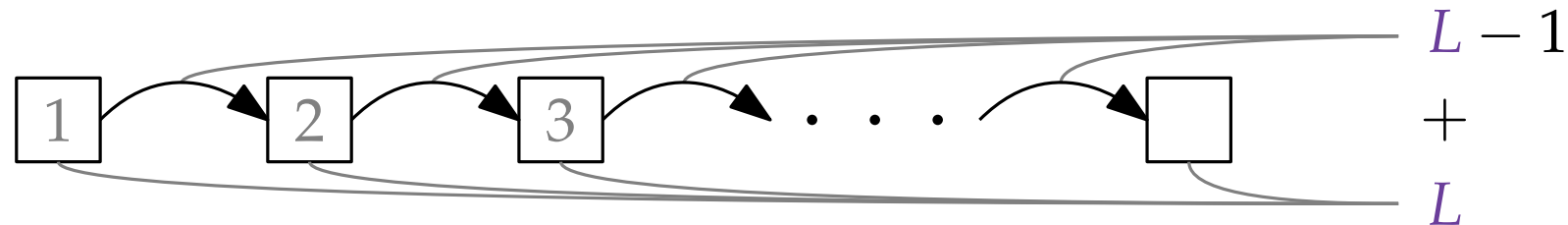
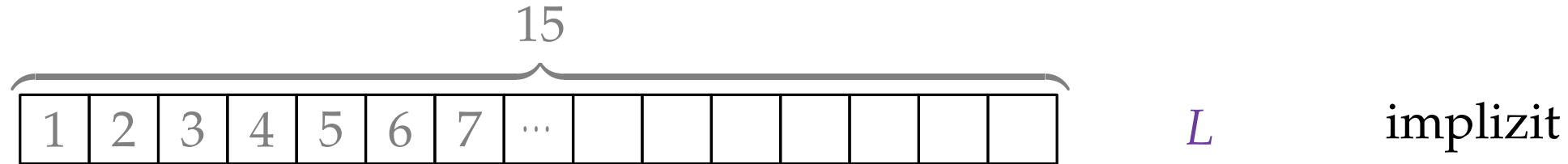
L

Beispiele für implizite Datenstrukturen

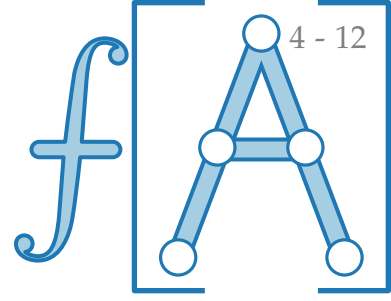


- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?

Speicher

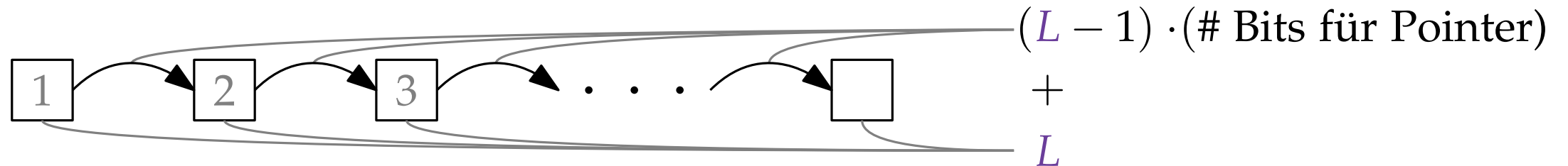
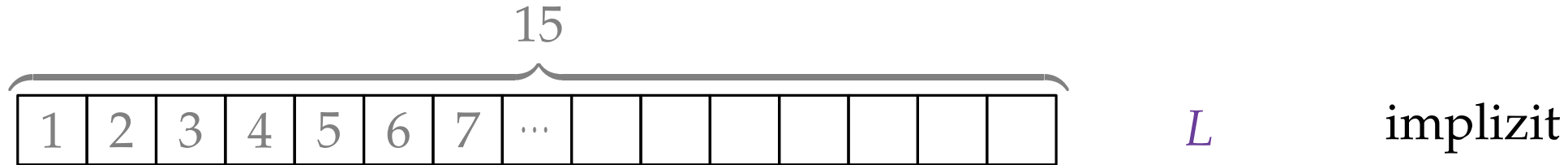


Beispiele für implizite Datenstrukturen

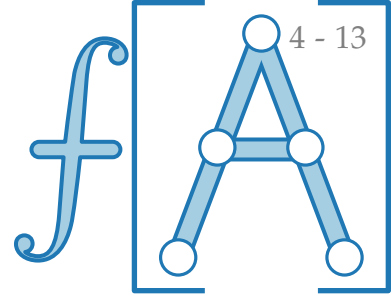


- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?

Speicher

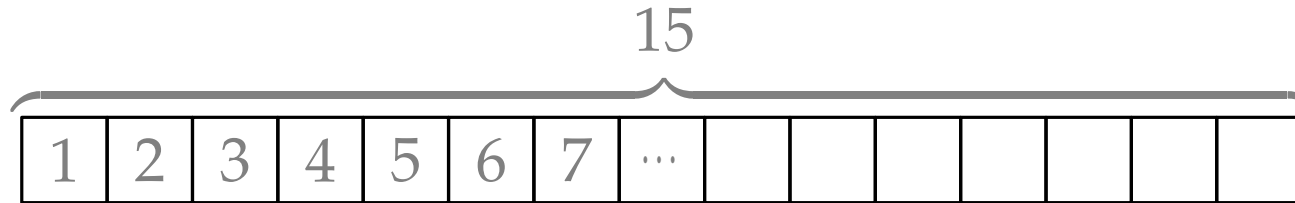


Beispiele für implizite Datenstrukturen



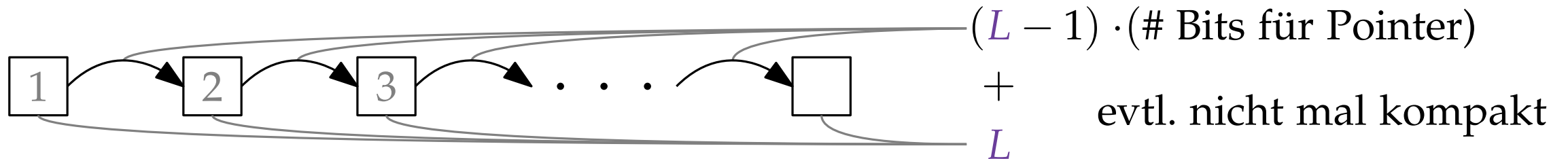
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?

Speicher

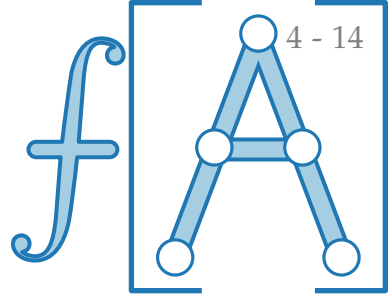


L

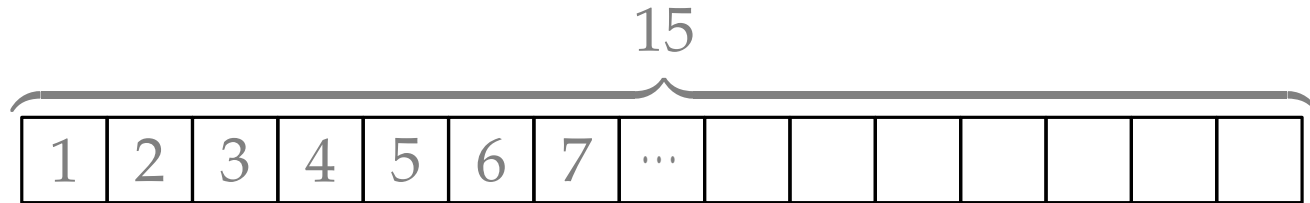
implizit



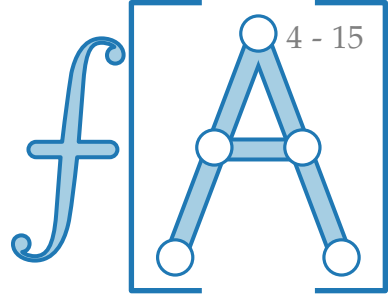
Beispiele für implizite Datenstrukturen



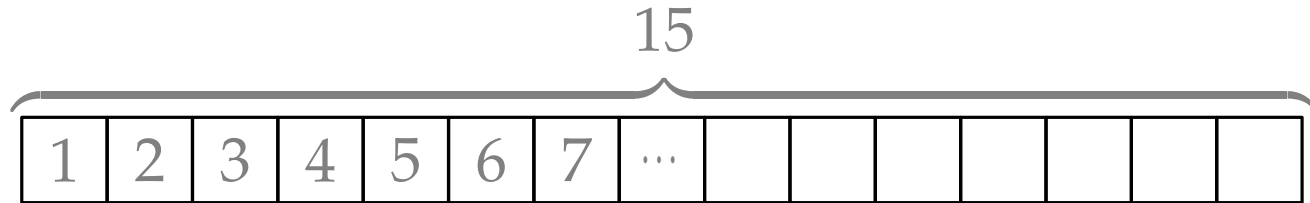
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren



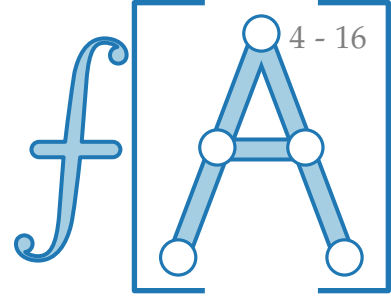
Beispiele für implizite Datenstrukturen



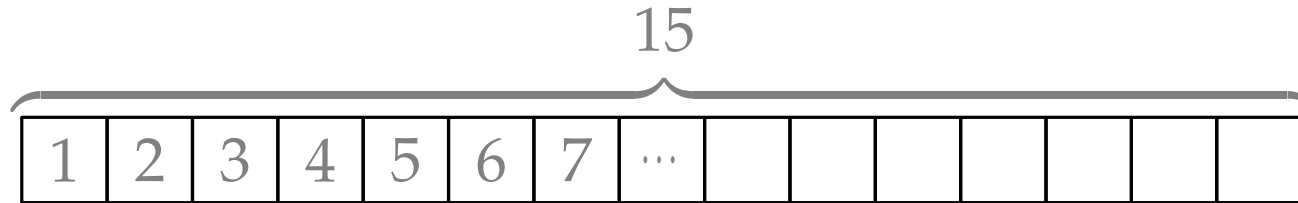
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren



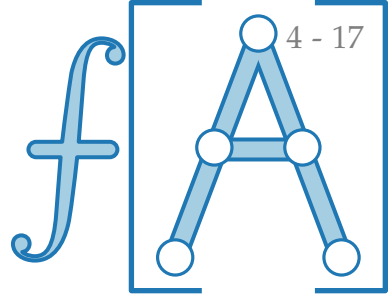
Beispiele für implizite Datenstrukturen



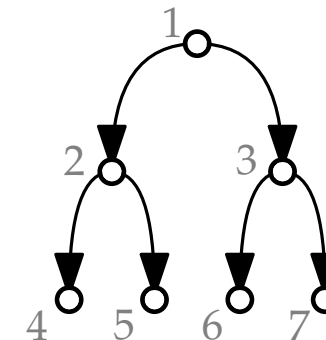
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren
 - Keine binären Suchbäume aus gleichem Grund wie richtige Listen



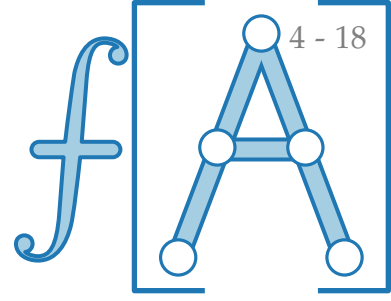
Beispiele für implizite Datenstrukturen



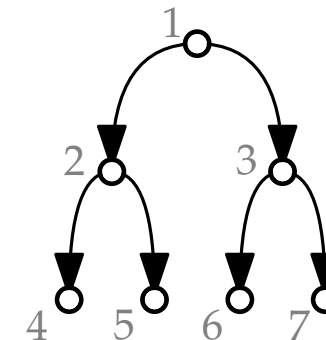
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren
 - Keine binären Suchbäume aus gleichem Grund wie richtige Listen
- **Arrays**, um vollständige Binärbäume und Heaps zu repräsentieren



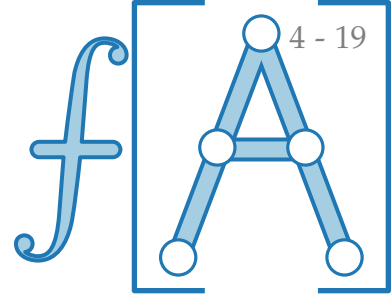
Beispiele für implizite Datenstrukturen



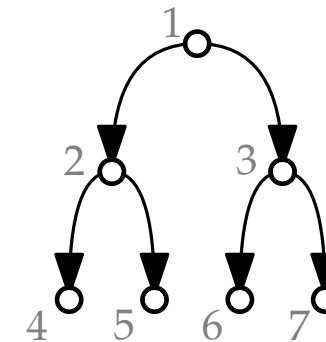
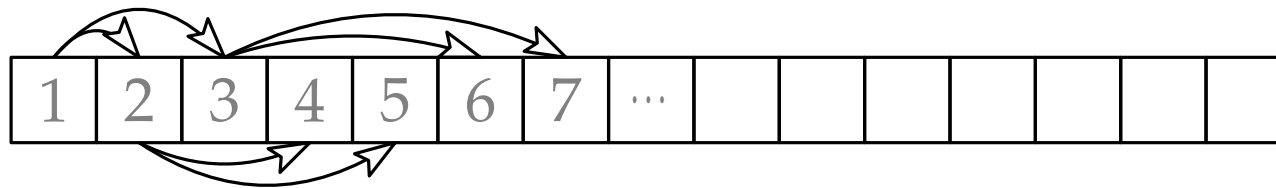
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren
 - Keine binären Suchbäume aus gleichem Grund wie richtige Listen
- **Arrays**, um vollständige Binärbäume und Heaps zu repräsentieren



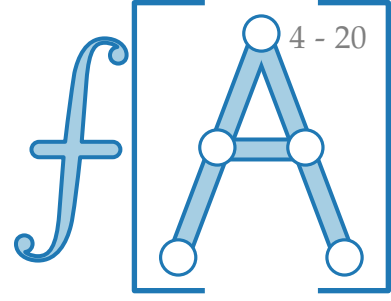
Beispiele für implizite Datenstrukturen



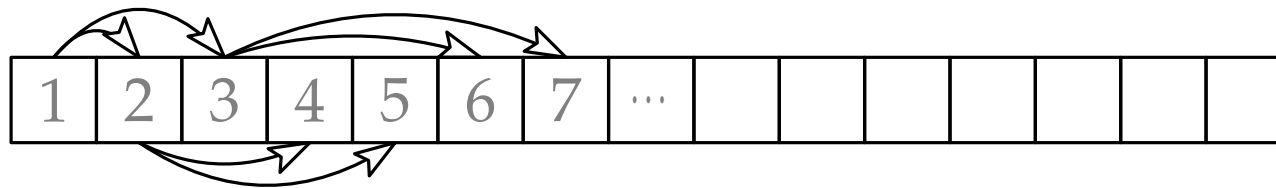
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren
 - Keine binären Suchbäume aus gleichem Grund wie richtige Listen
- **Arrays**, um vollständige Binärbäume und Heaps zu repräsentieren



Beispiele für implizite Datenstrukturen



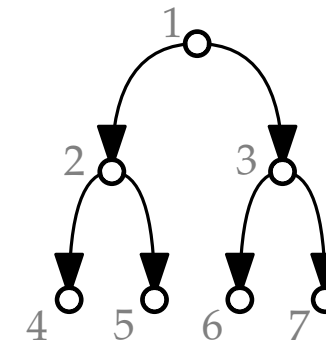
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren
 - Keine binären Suchbäume aus gleichem Grund wie richtige Listen
- **Arrays**, um vollständige Binärbäume und Heaps zu repräsentieren



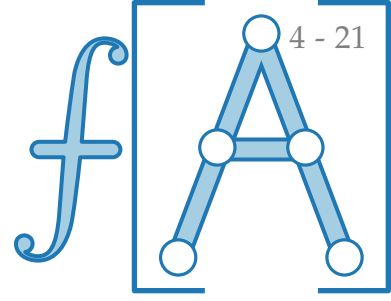
LEFTCHILD(i) =

RIGHTCHILD(i) =

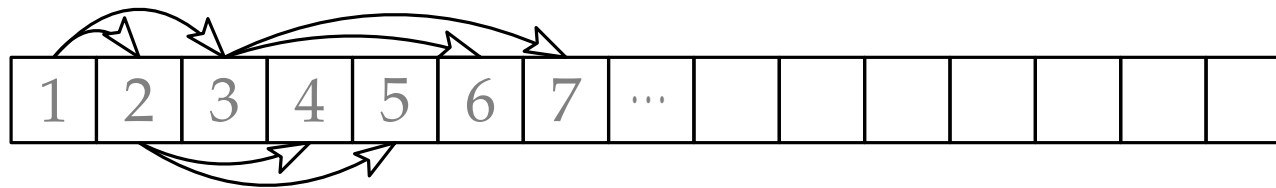
PARENT(i) =



Beispiele für implizite Datenstrukturen



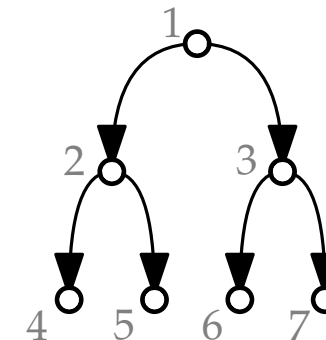
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren
 - Keine binären Suchbäume aus gleichem Grund wie richtige Listen
- **Arrays**, um vollständige Binärbäume und Heaps zu repräsentieren



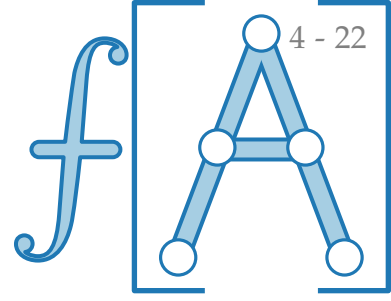
$$\text{LEFTCHILD}(i) = 2i$$

$$\text{RIGHTCHILD}(i) = 2i + 1$$

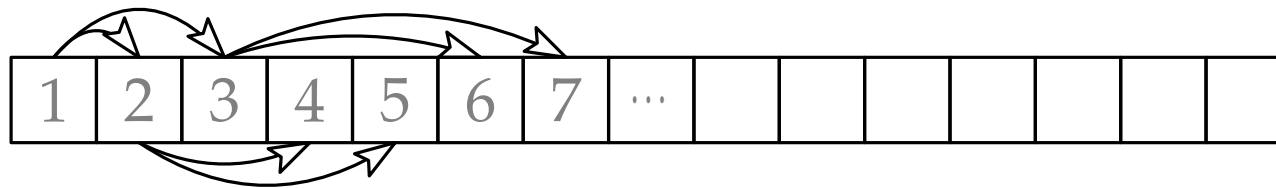
$$\text{PARENT}(i) =$$



Beispiele für implizite Datenstrukturen



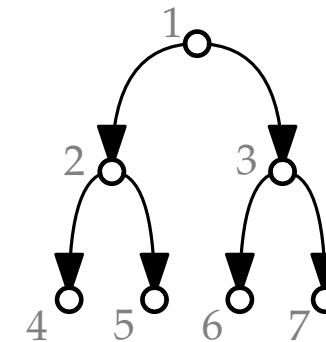
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren
 - Keine binären Suchbäume aus gleichem Grund wie richtige Listen
- **Arrays**, um vollständige Binärbäume und Heaps zu repräsentieren



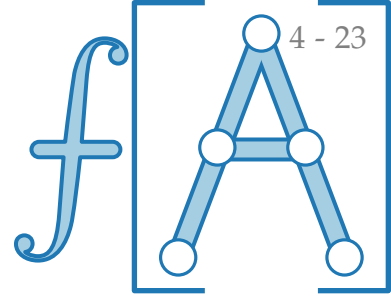
$$\text{LEFTCHILD}(i) = 2i$$

$$\text{RIGHTCHILD}(i) = 2i + 1$$

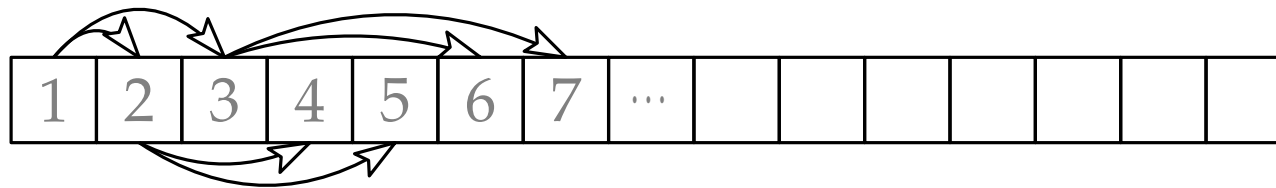
$$\text{PARENT}(i) = \lfloor \frac{i}{2} \rfloor$$



Beispiele für implizite Datenstrukturen



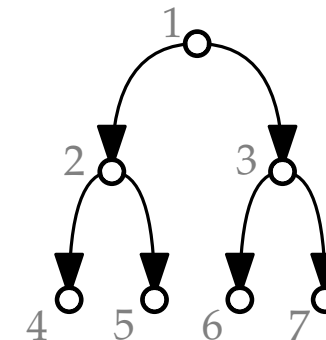
- **Arrays**, um Listen zu repräsentieren
 - aber warum keine richtigen Listen?
- **1-dimensionale Arrays**, um multi-dimensionale Arrays zu repräsentieren
- **Sortierte Arrays**, um sortierte Listen zu repräsentieren
 - Keine binären Suchbäume aus gleichem Grund wie richtige Listen
- **Arrays**, um vollständige Binärbäume und Heaps zu repräsentieren



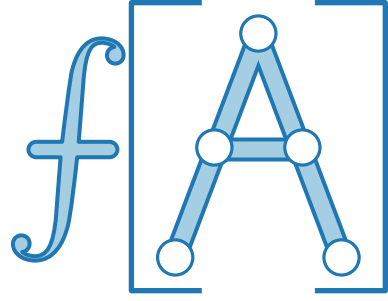
$$\text{LEFTCHILD}(i) = 2i$$

$$\text{RIGHTCHILD}(i) = 2i + 1$$

$$\text{PARENT}(i) = \lfloor \frac{i}{2} \rfloor$$



Und unbalancierte
Bäume?

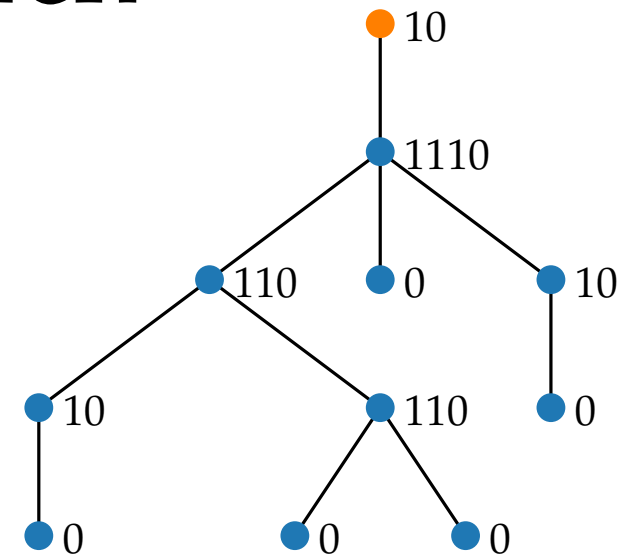


Fortgeschrittene Algorithmen

16. Vorlesung Datenstrukturen II:

Platzeffiziente Datenstrukturen (Succinct Data Structures)

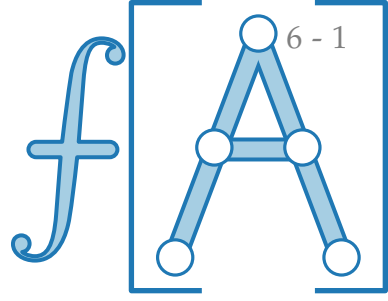
Teil II: Wörterbücher



Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

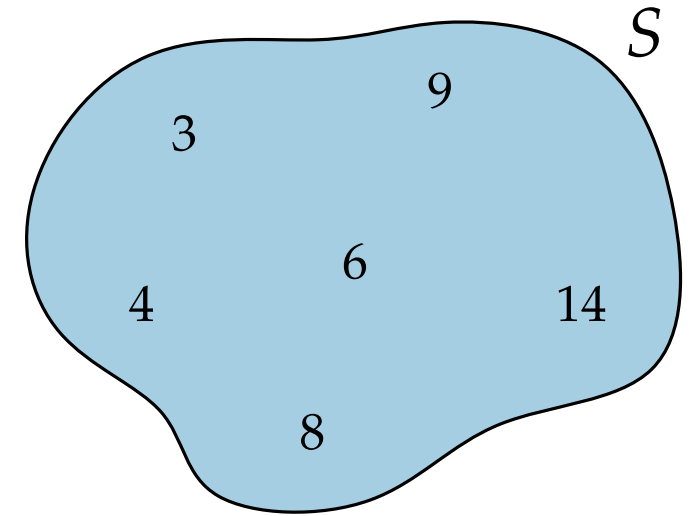
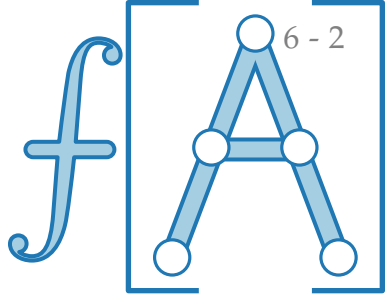
Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen



Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

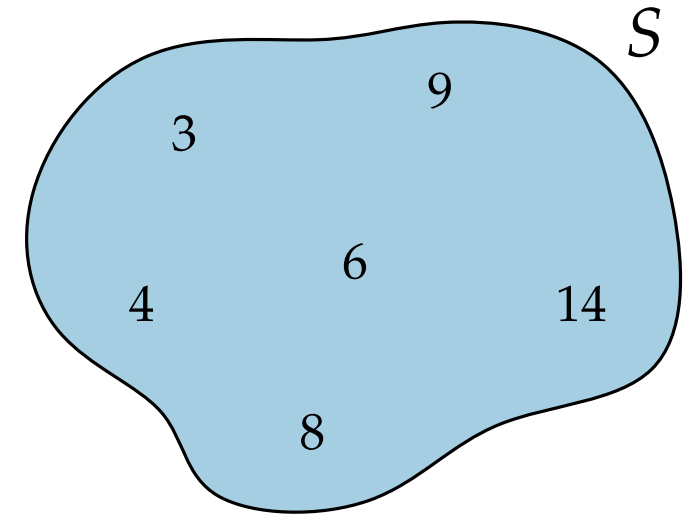
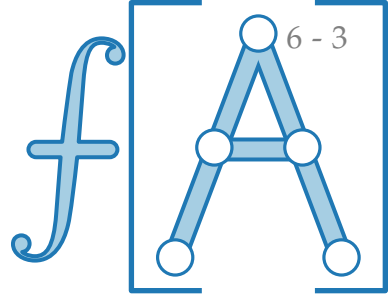


Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- MEMBER(i) überprüft, ob $i \in S$

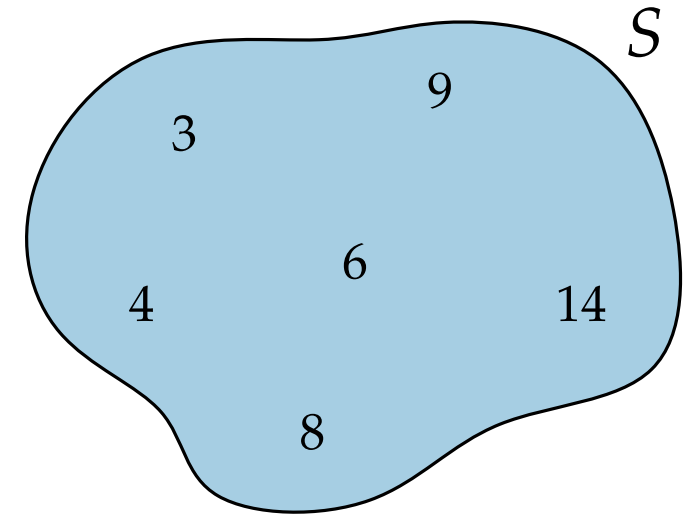
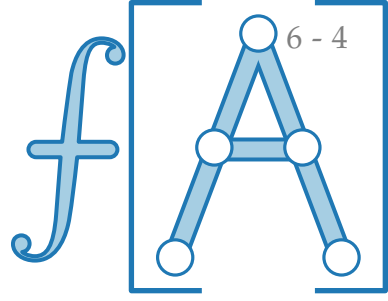


Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- **MEMBER**(i) überprüft, ob $i \in S$



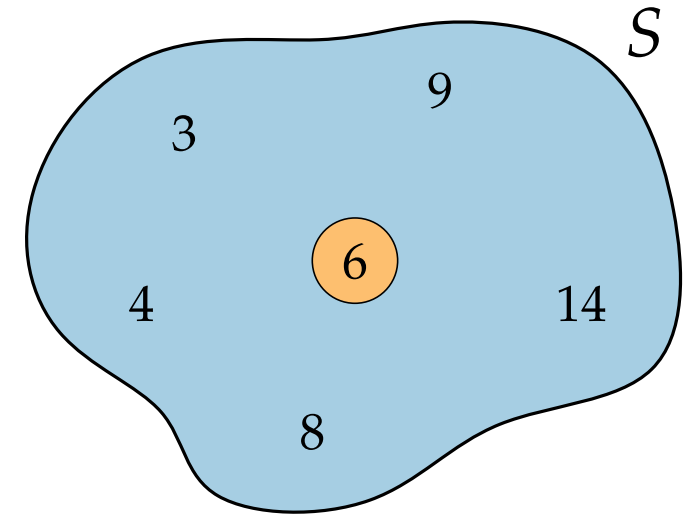
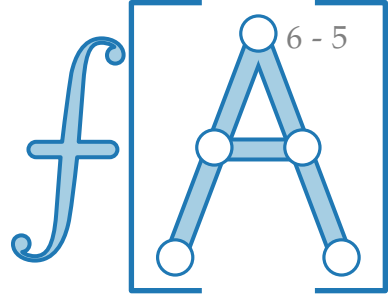
MEMBER(6)

Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- **MEMBER**(i) überprüft, ob $i \in S$



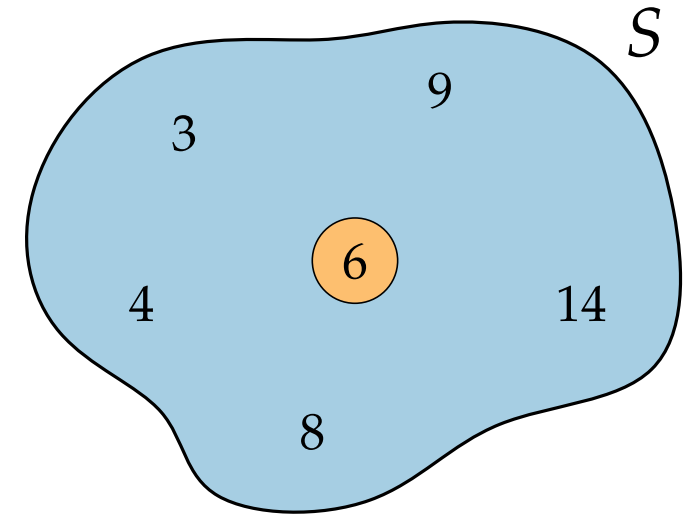
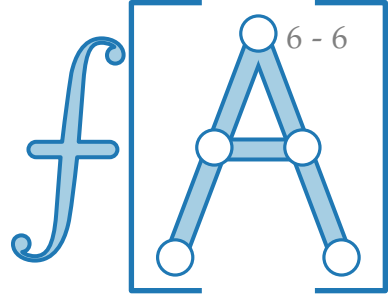
MEMBER(6)

Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- **MEMBER**(i) überprüft, ob $i \in S$



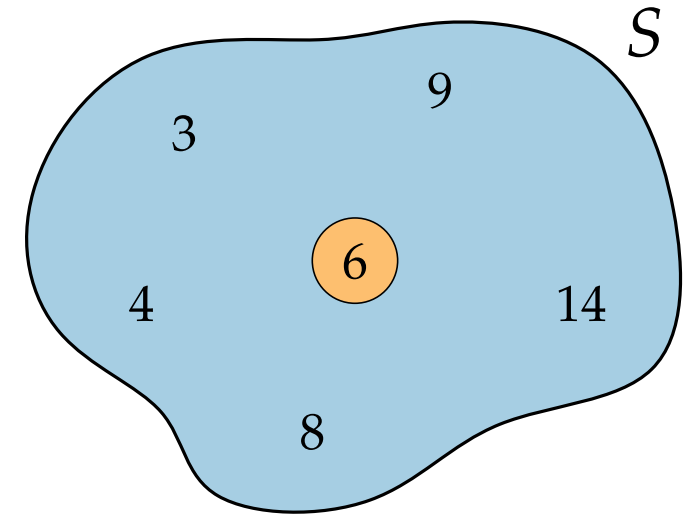
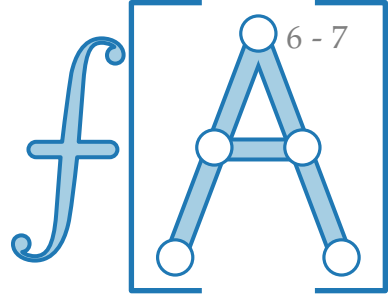
MEMBER(6) ✓

Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- **MEMBER**(i) überprüft, ob $i \in S$



MEMBER(6) ✓

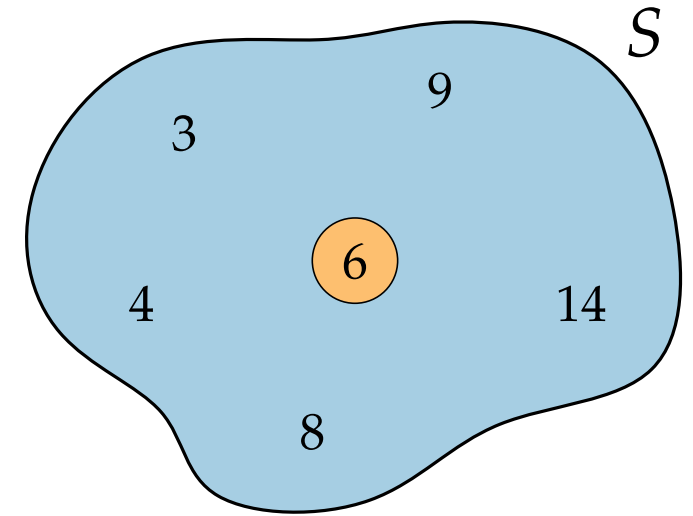
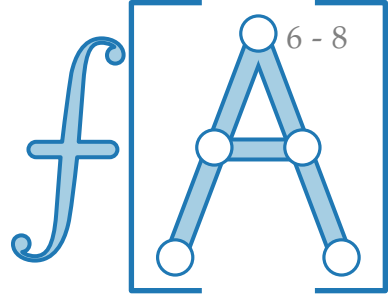
MEMBER(2)

Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$



$\text{MEMBER}(6)$ ✓

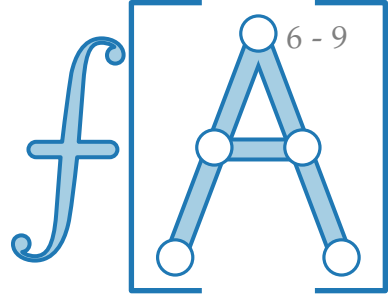
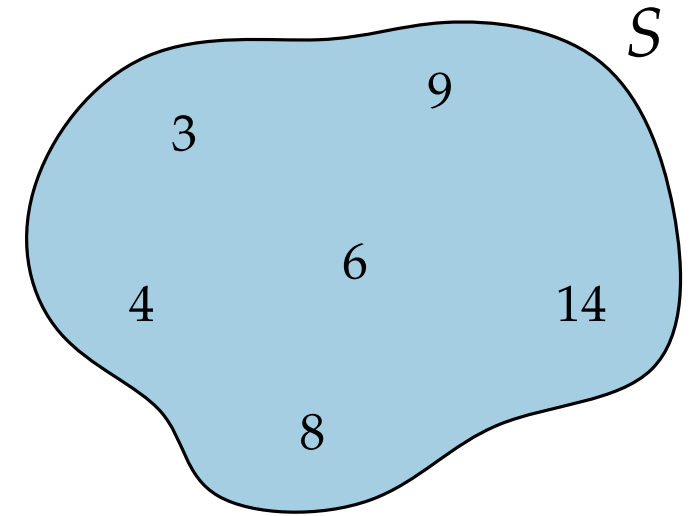
$\text{MEMBER}(2)$ ✗

Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$

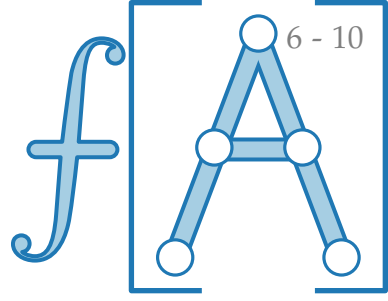
Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$



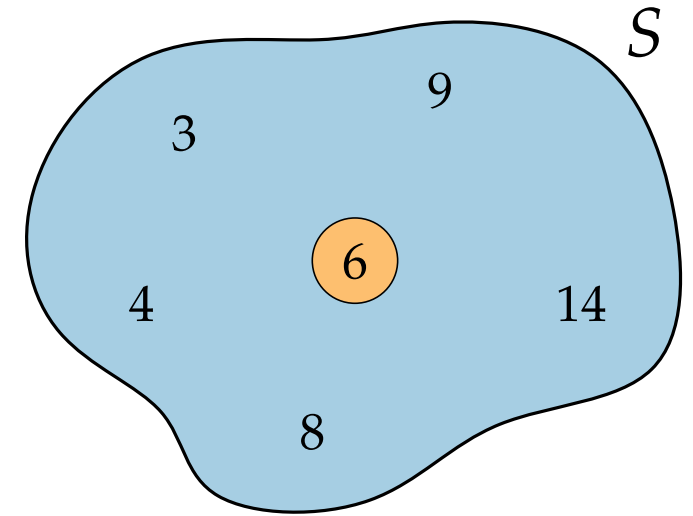
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

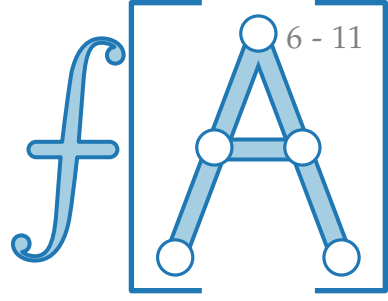
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$



$\text{PREDECESSOR}(6)$

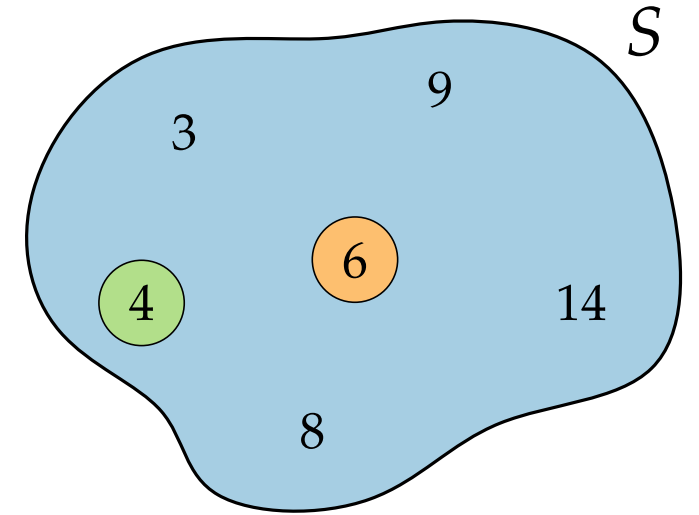
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

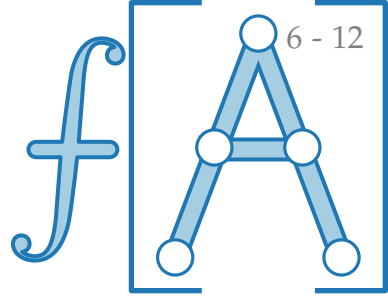
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$



$$\text{PREDECESSOR}(6) = 4$$

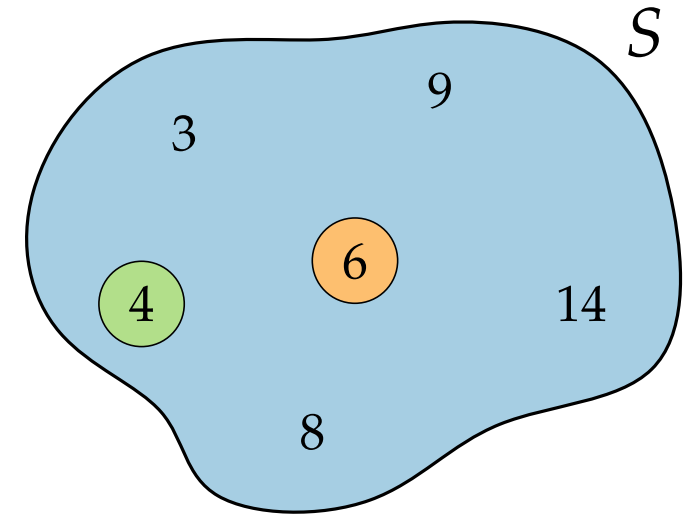
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

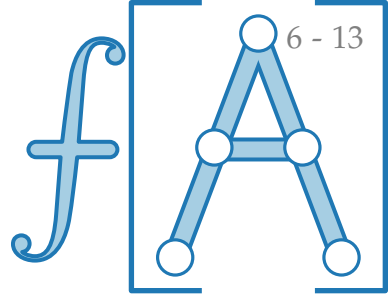
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$



$$\text{PREDECESSOR}(6) = 4$$

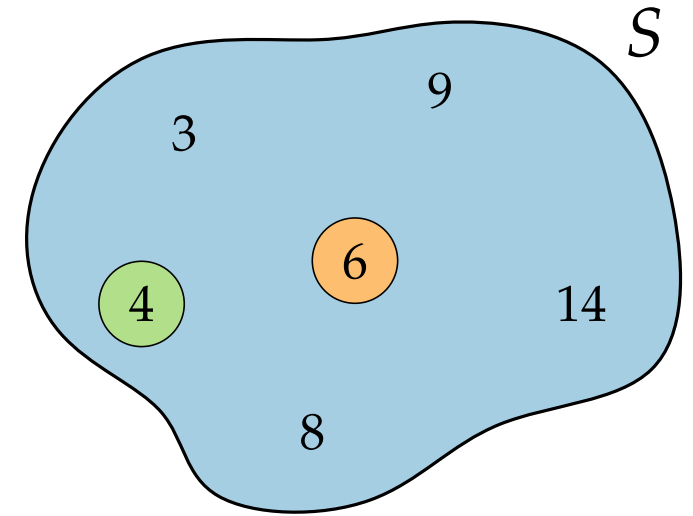
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

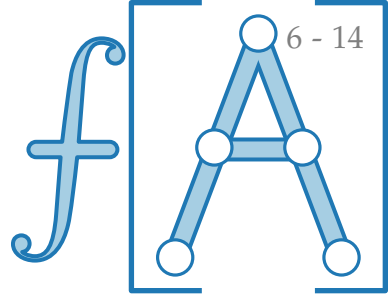
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$



$\text{PREDECESSOR}(6) = 4$
 $\text{SUCCESSOR}(6)$

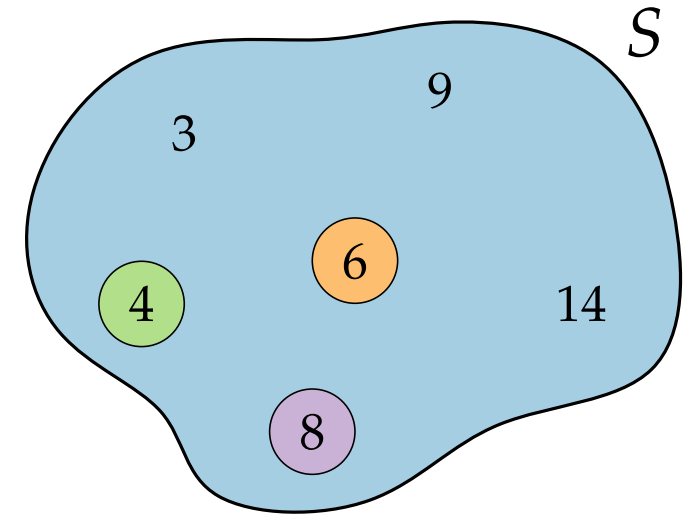
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

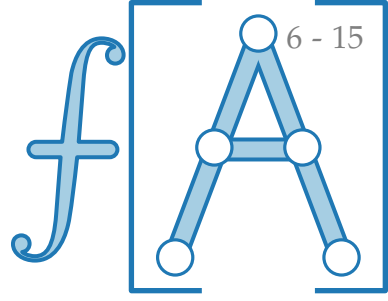
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$



$\text{PREDECESSOR}(6) = 4$
 $\text{SUCCESSOR}(6) = 8$

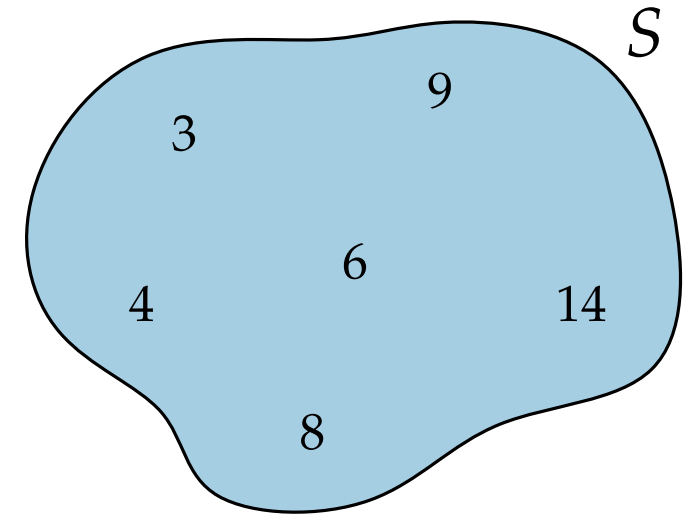
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



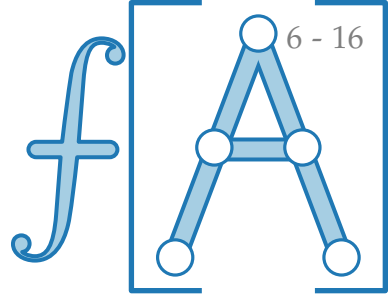
Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$



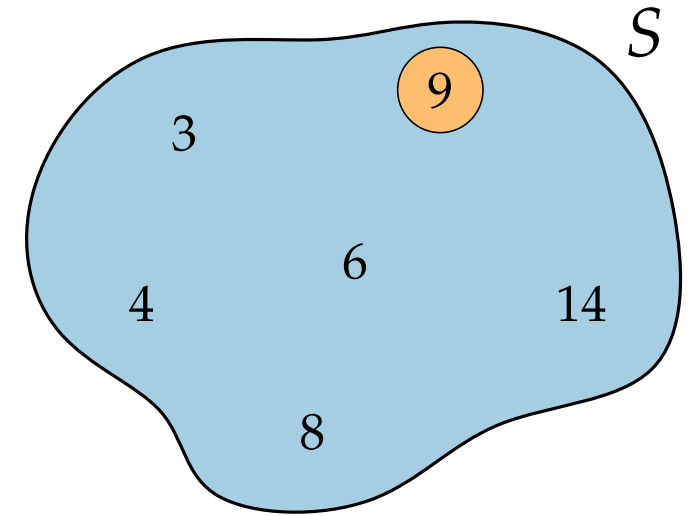
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

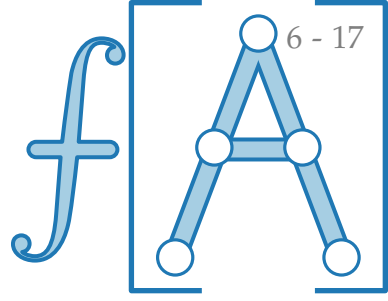
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$



$\text{RANK}_1(9)$

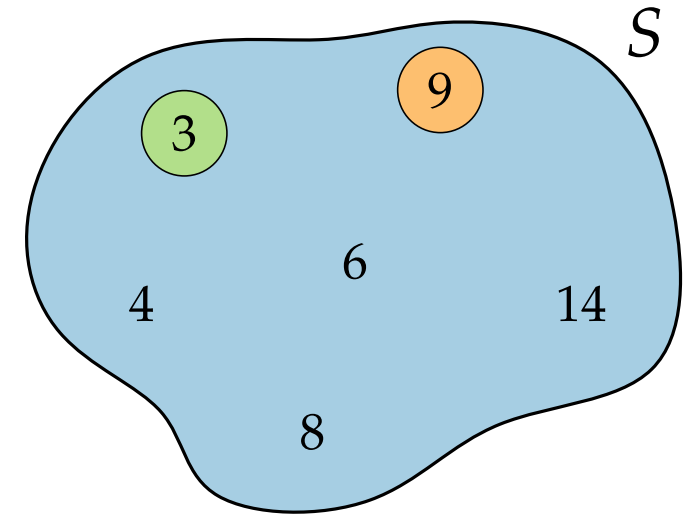
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

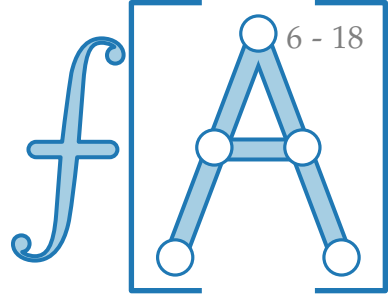
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$



$\text{RANK}_1(9)$

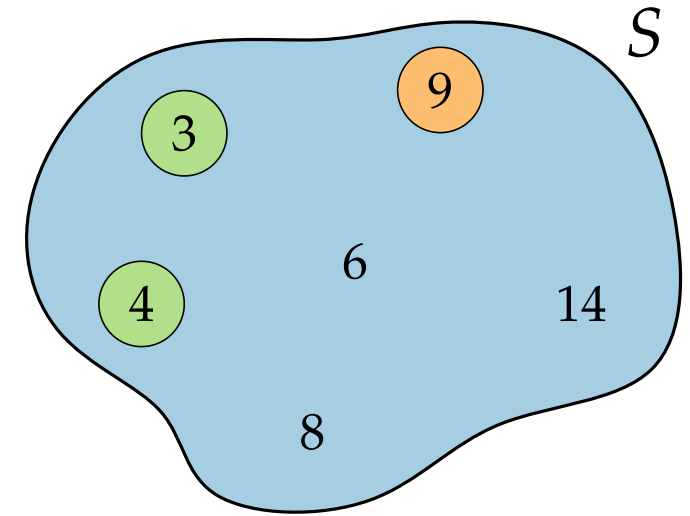
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

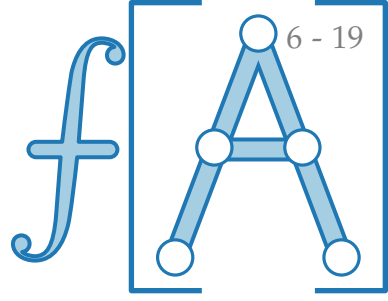
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$



$\text{RANK}_1(9)$

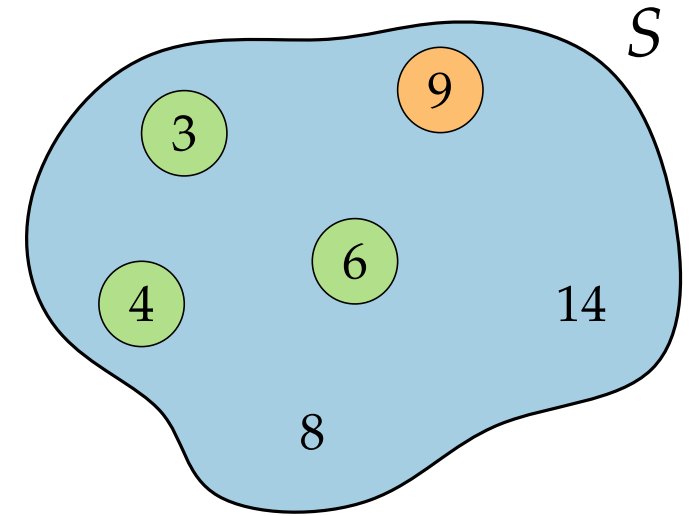
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

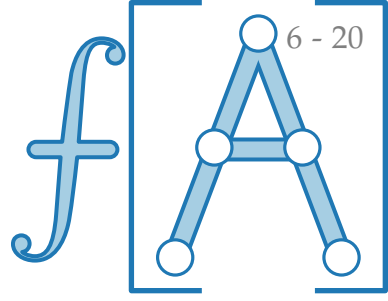
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$



$\text{RANK}_1(9)$

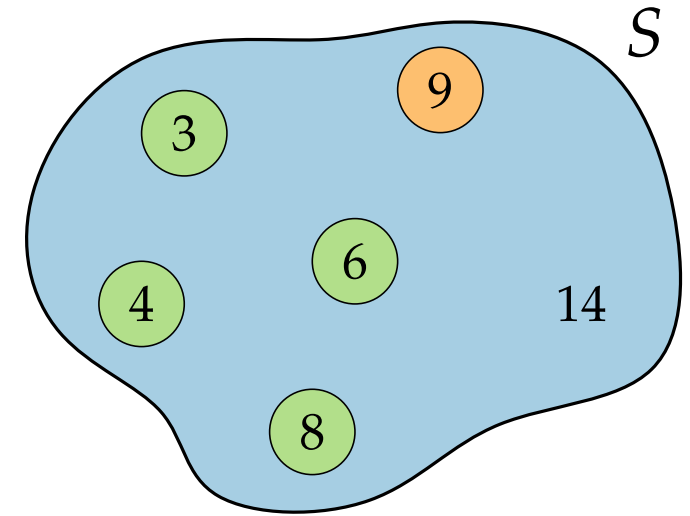
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

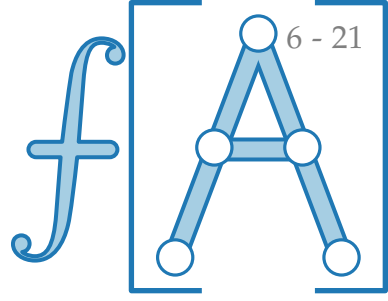
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$



$\text{RANK}_1(9)$

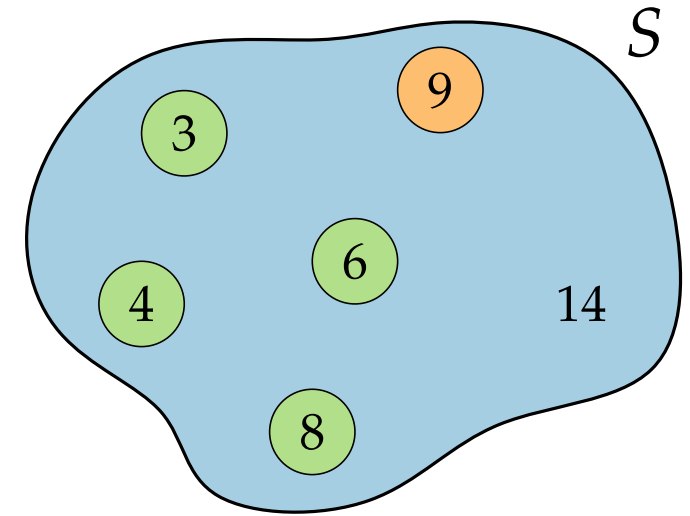
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

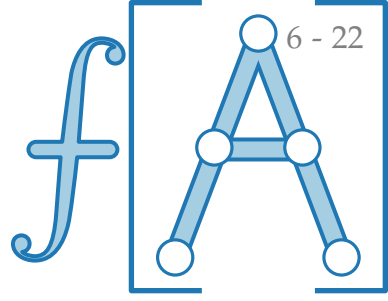
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$



$$\text{RANK}_1(9) = 5$$

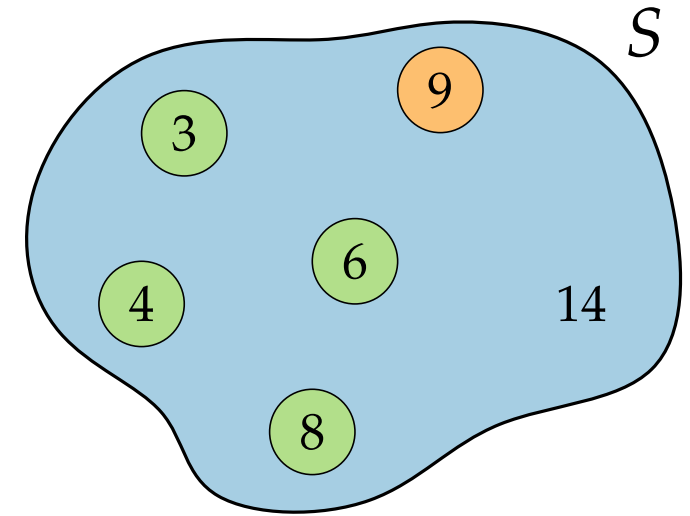
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

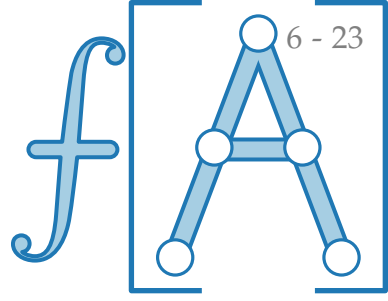
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$



$$\text{RANK}_1(9) = 5$$

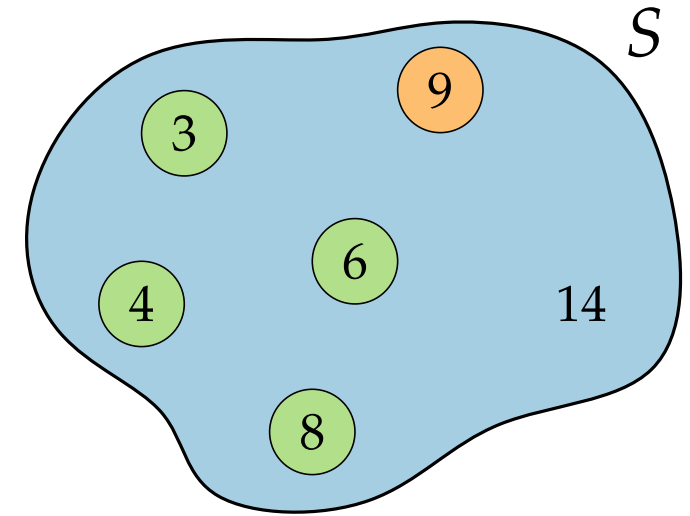
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

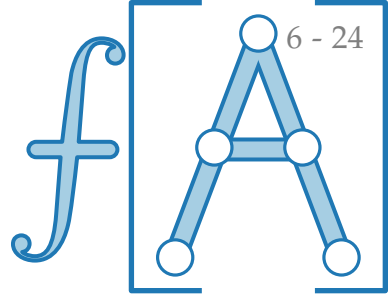
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i - 1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i + 1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$



$$\text{RANK}_1(9) = 5$$
$$\text{RANK}_0(9)$$

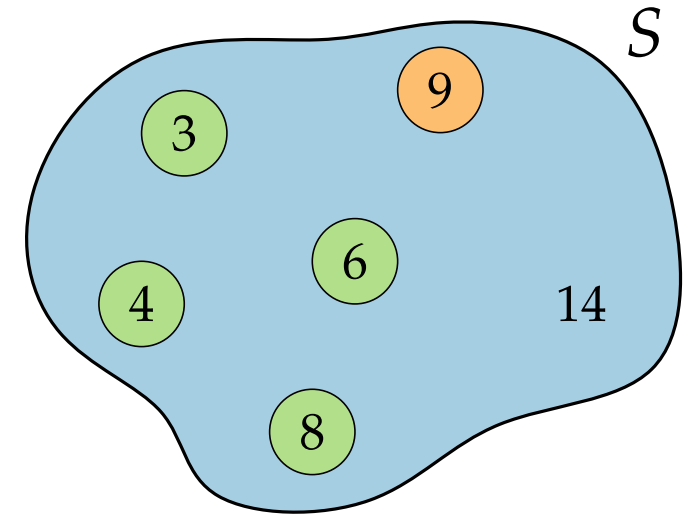
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$

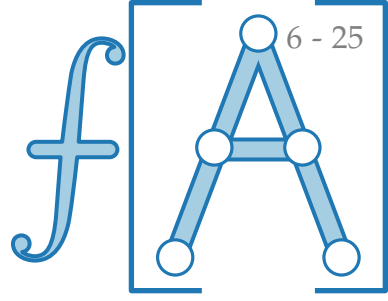


$$\text{RANK}_1(9) = 5$$

$$\text{RANK}_0(9) = 4$$

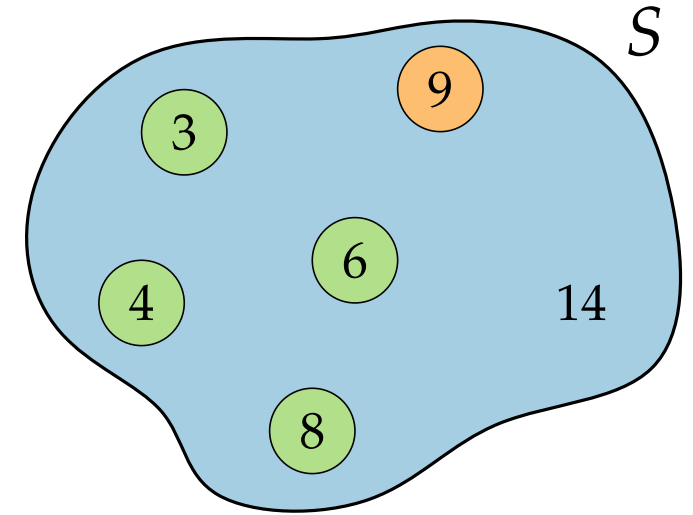
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$



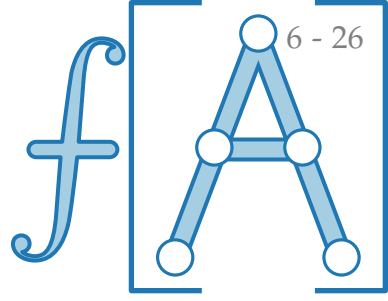
$$\text{RANK}_1(9) = 5$$

$$\text{RANK}_0(9) = 4$$

$$\text{RANK}_1(7)$$

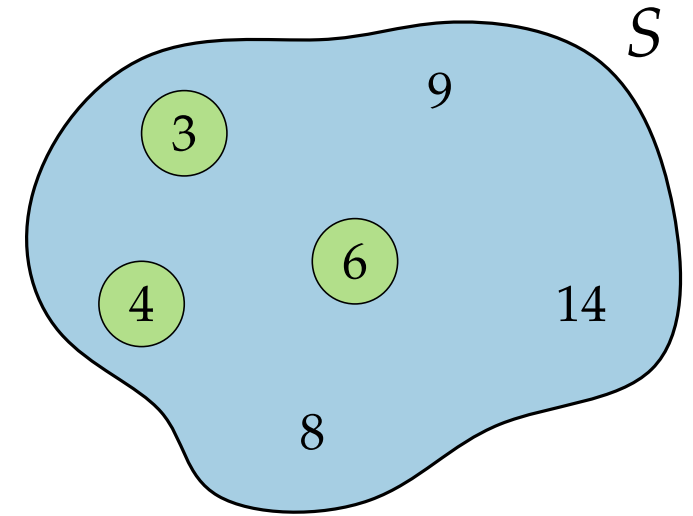
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$



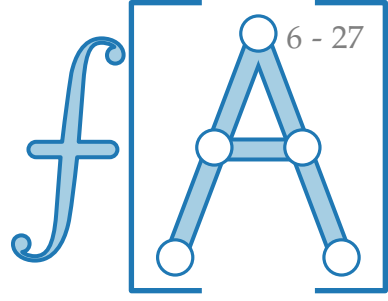
$$\text{RANK}_1(9) = 5$$

$$\text{RANK}_0(9) = 4$$

$$\text{RANK}_1(7)$$

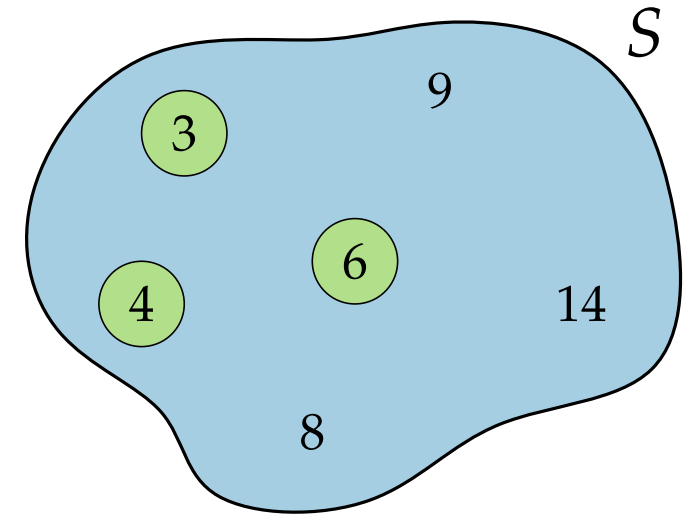
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$



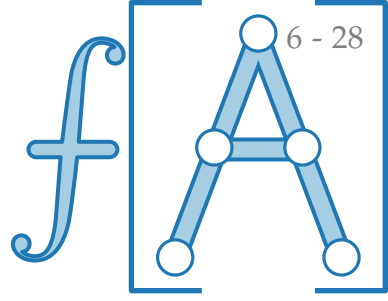
$$\text{RANK}_1(9) = 5$$

$$\text{RANK}_0(9) = 4$$

$$\text{RANK}_1(7) = 3$$

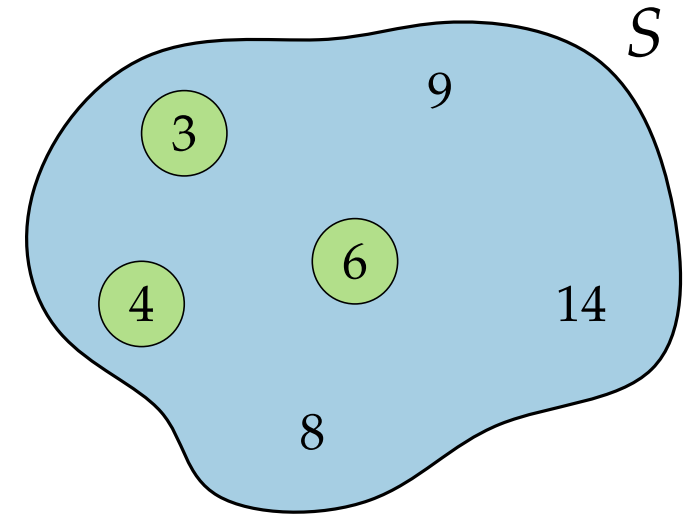
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$



$$\text{RANK}_1(9) = 5$$

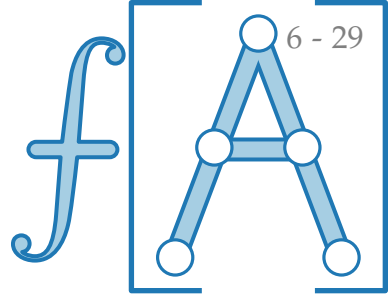
$$\text{RANK}_0(9) = 4$$

$$\text{RANK}_1(7) = 3$$

$$\text{RANK}_0(7)$$

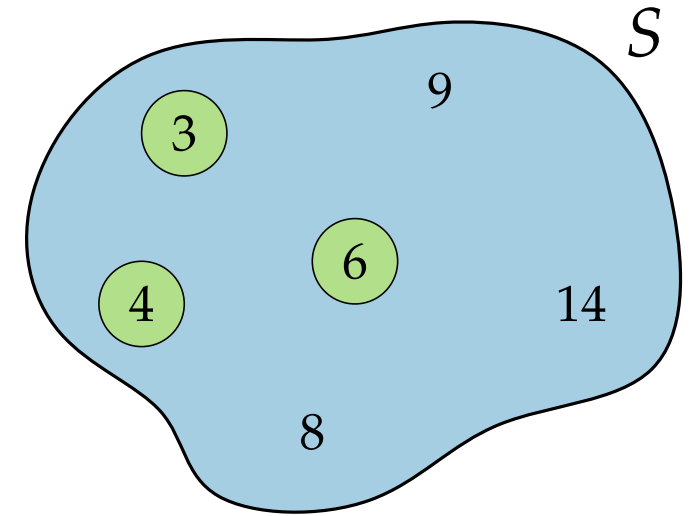
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$



$$\text{RANK}_1(9) = 5$$

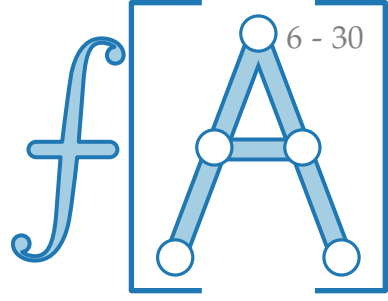
$$\text{RANK}_0(9) = 4$$

$$\text{RANK}_1(7) = 3$$

$$\text{RANK}_0(7) = 4$$

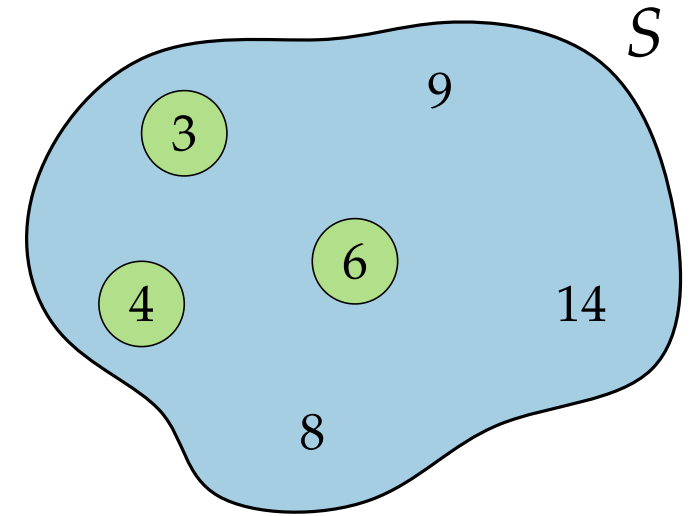
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$



$$\text{RANK}_1(9) = 5$$

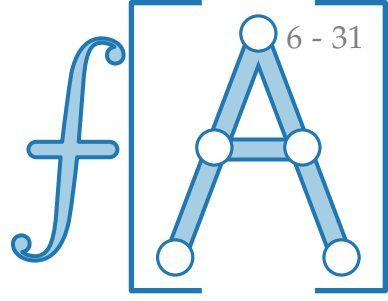
$$\text{RANK}_0(9) = 4$$

$$\text{RANK}_1(7) = 3$$

$$\text{RANK}_0(7) = 4$$

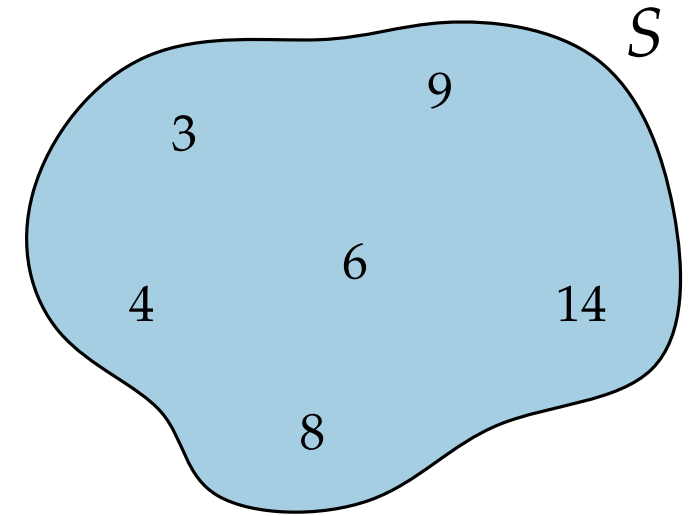
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



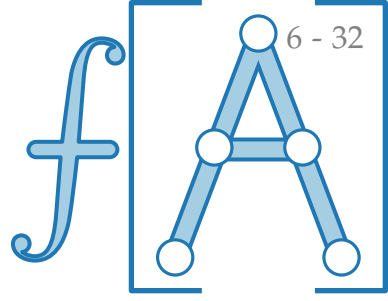
Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



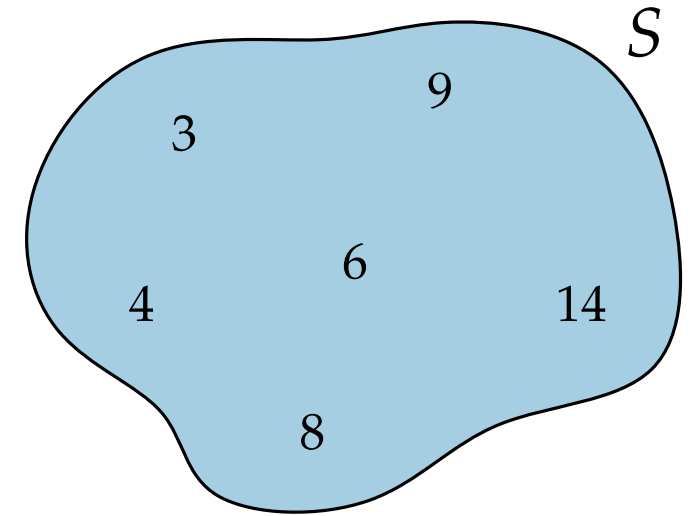
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

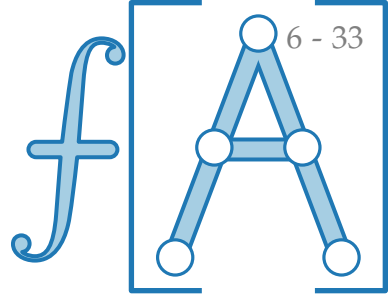
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$\text{SELECT}_1(1)$

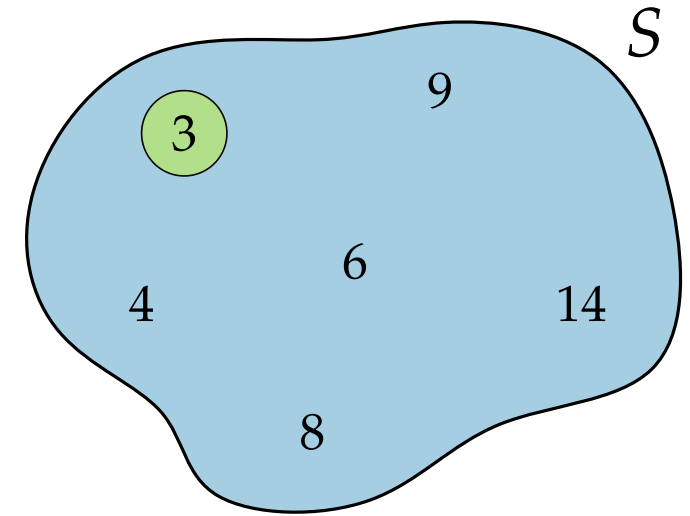
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

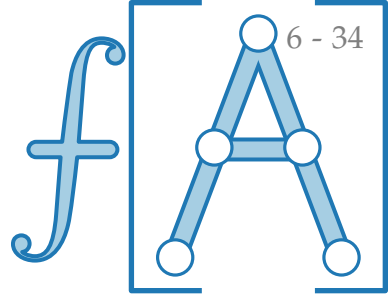
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$$\text{SELECT}_1(1) = 3$$

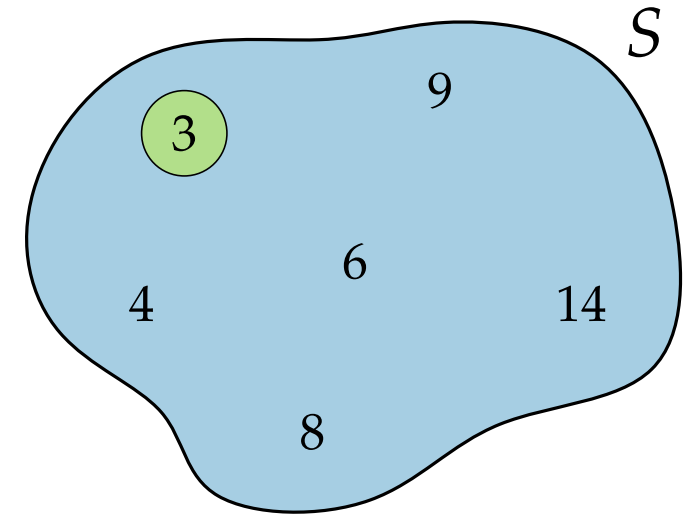
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

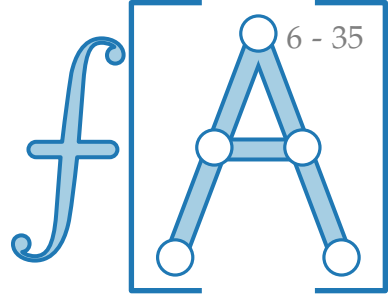
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$\text{SELECT}_1(1) = 3$
 $\text{SELECT}_1(2)$

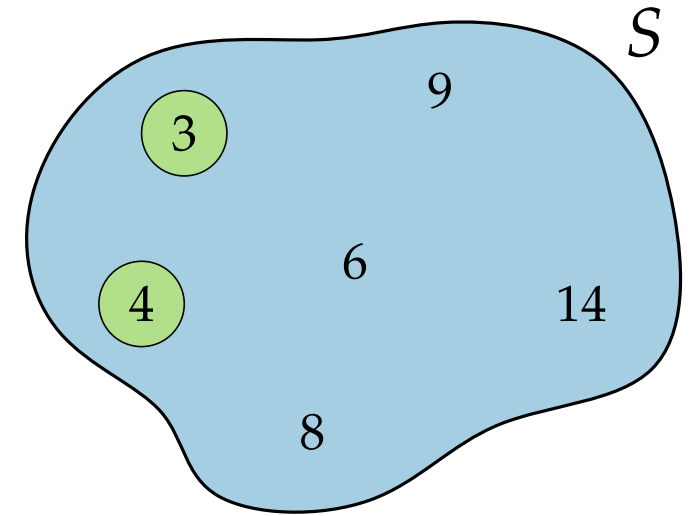
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$

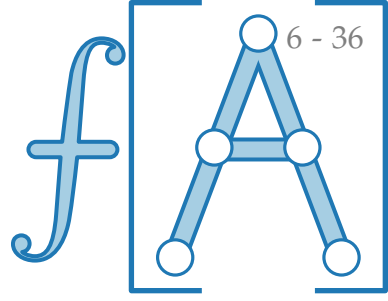


$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

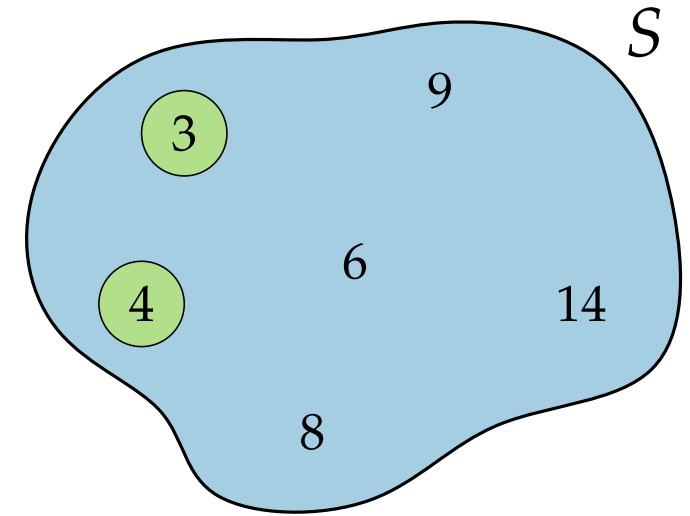
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



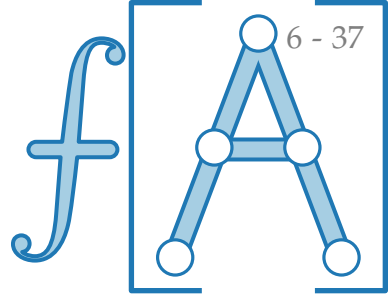
$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

$$\text{SELECT}_1(3)$$

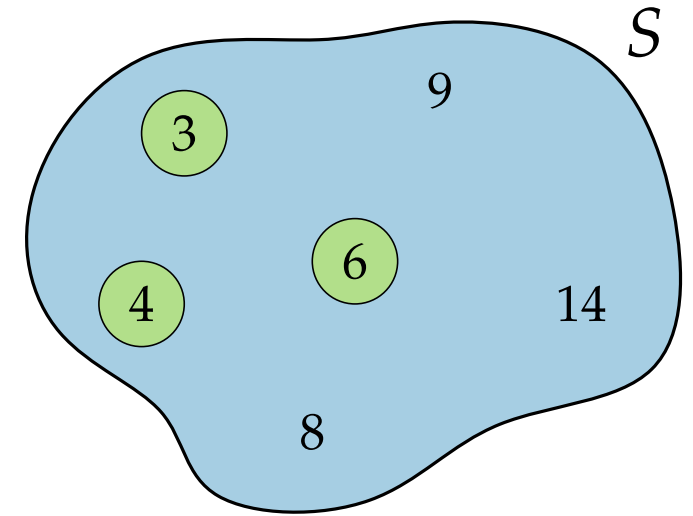
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



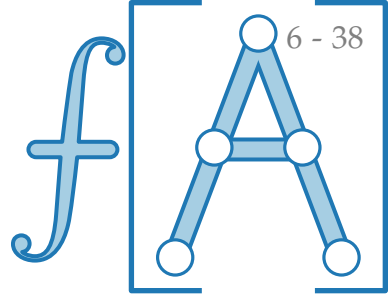
$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

$$\text{SELECT}_1(3) = 6$$

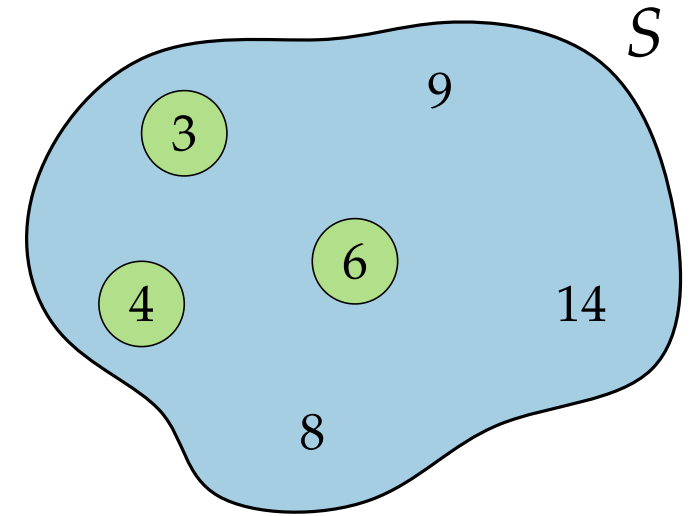
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

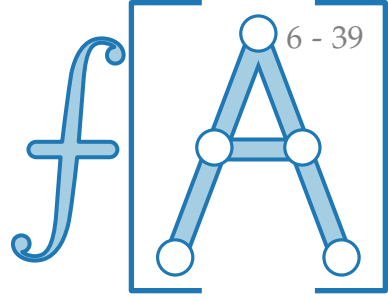
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$$\begin{aligned}\text{SELECT}_1(1) &= 3 \\ \text{SELECT}_1(2) &= 4 \\ \text{SELECT}_1(3) &= 6 \\ \text{SELECT}_1(4) &= 8\end{aligned}$$

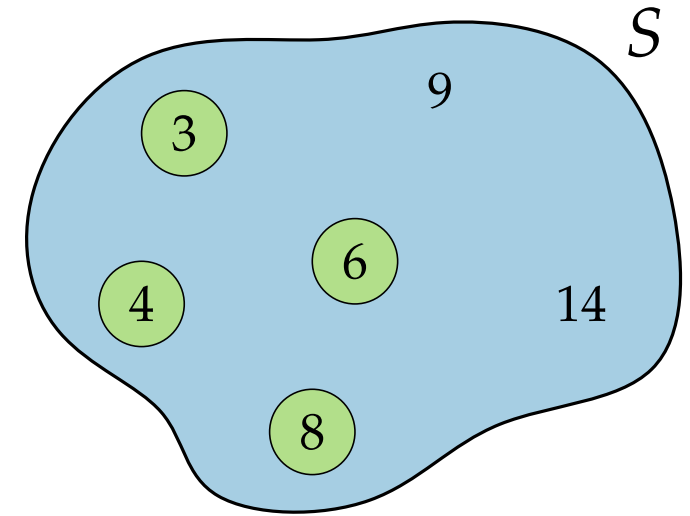
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

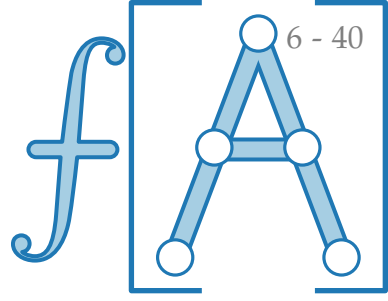
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$$\begin{aligned}\text{SELECT}_1(1) &= 3 \\ \text{SELECT}_1(2) &= 4 \\ \text{SELECT}_1(3) &= 6 \\ \text{SELECT}_1(4) &= 8\end{aligned}$$

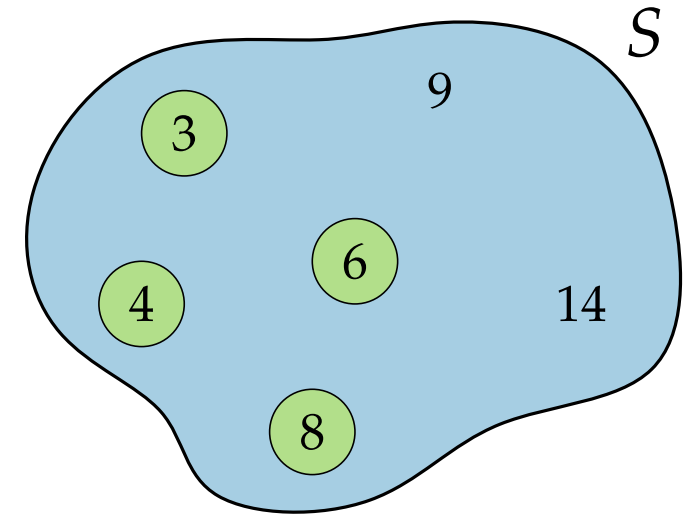
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

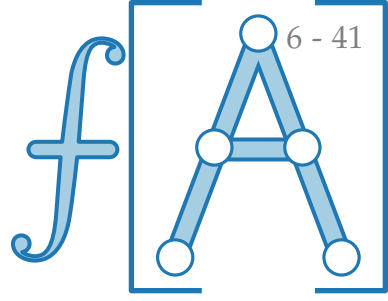
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



$$\begin{aligned}\text{SELECT}_1(1) &= 3 \\ \text{SELECT}_1(2) &= 4 \\ \text{SELECT}_1(3) &= 6 \\ \text{SELECT}_1(4) &= 8\end{aligned}$$

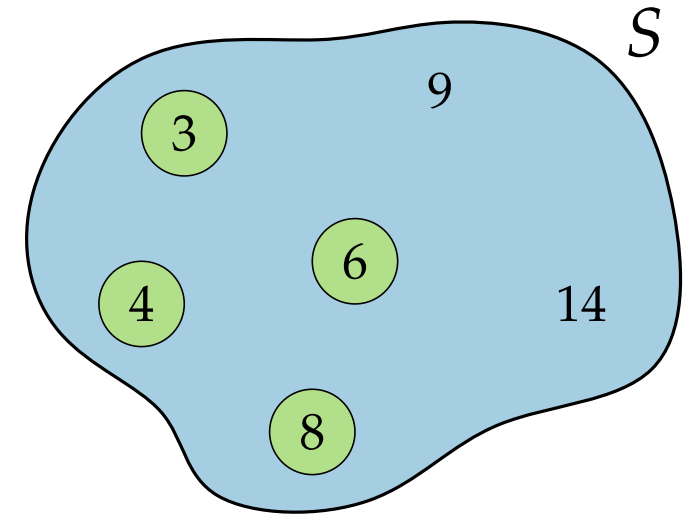
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

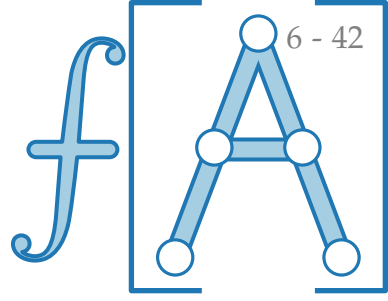
$$\text{SELECT}_1(3) = 6$$

$$\text{SELECT}_1(4) = 8$$

$$\text{SELECT}_0(1)$$

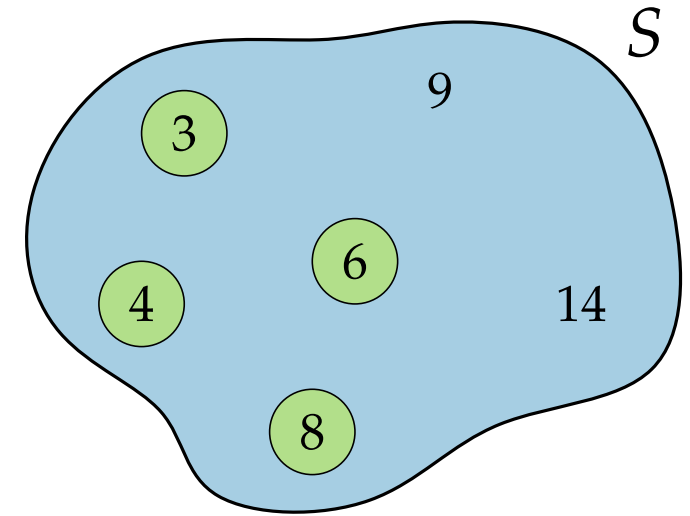
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

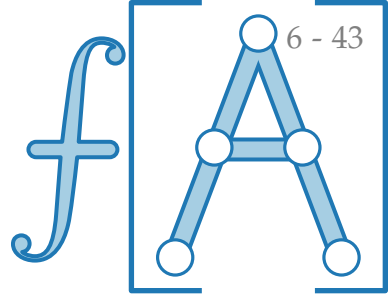
$$\text{SELECT}_1(3) = 6$$

$$\text{SELECT}_1(4) = 8$$

$$\text{SELECT}_0(1) = 1$$

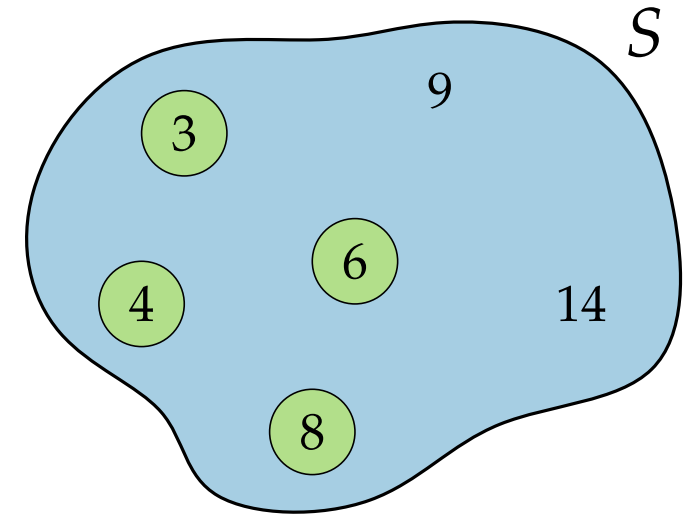
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

$$\text{SELECT}_1(3) = 6$$

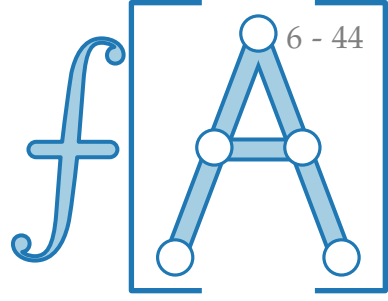
$$\text{SELECT}_1(4) = 8$$

$$\text{SELECT}_0(1) = 1$$

$$\text{SELECT}_0(2)$$

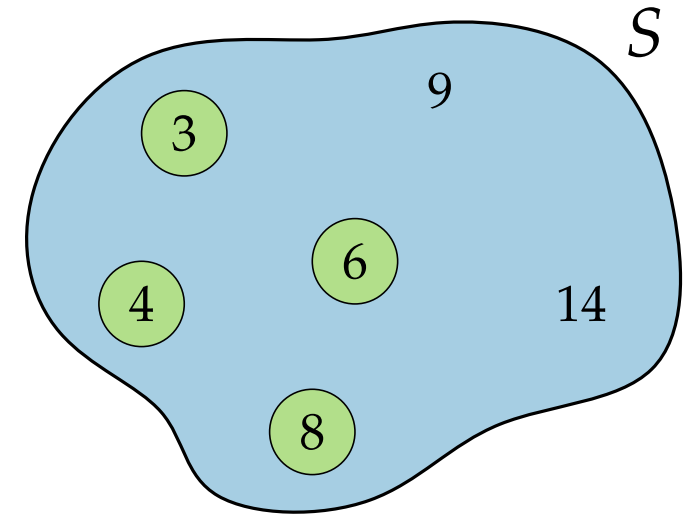
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

$$\text{SELECT}_1(3) = 6$$

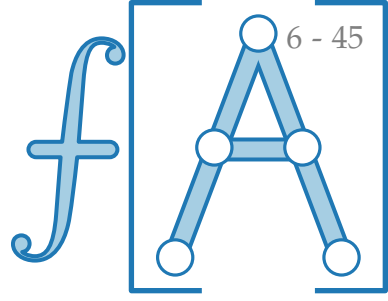
$$\text{SELECT}_1(4) = 8$$

$$\text{SELECT}_0(1) = 1$$

$$\text{SELECT}_0(2) = 2$$

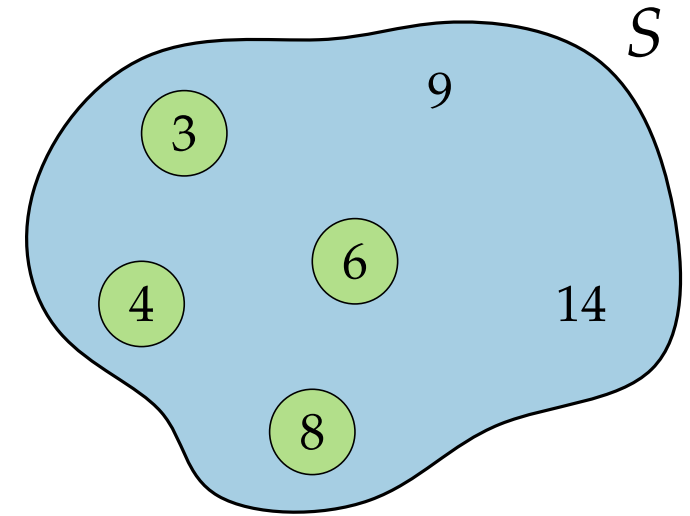
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

$$\text{SELECT}_1(3) = 6$$

$$\text{SELECT}_1(4) = 8$$

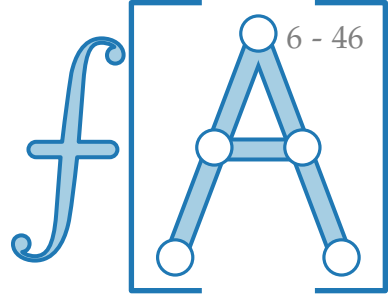
$$\text{SELECT}_0(1) = 1$$

$$\text{SELECT}_0(2) = 2$$

$$\text{SELECT}_0(3)$$

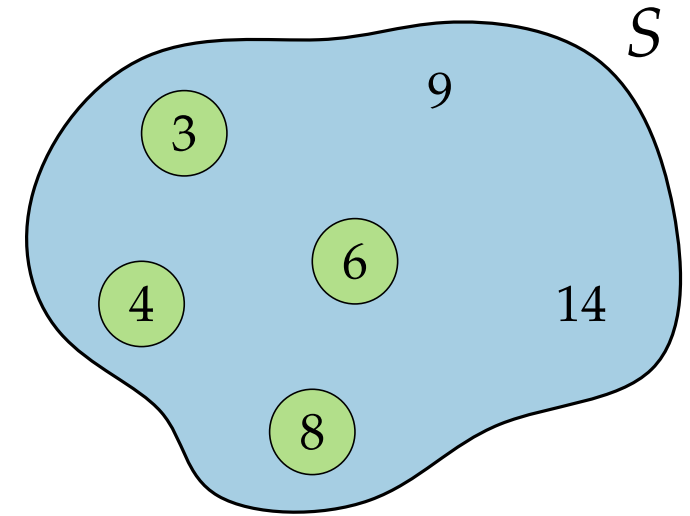
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



$$\text{SELECT}_1(1) = 3$$

$$\text{SELECT}_1(2) = 4$$

$$\text{SELECT}_1(3) = 6$$

$$\text{SELECT}_1(4) = 8$$

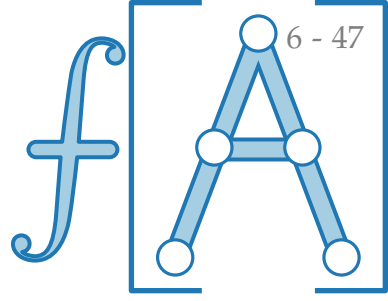
$$\text{SELECT}_0(1) = 1$$

$$\text{SELECT}_0(2) = 2$$

$$\text{SELECT}_0(3) = 5$$

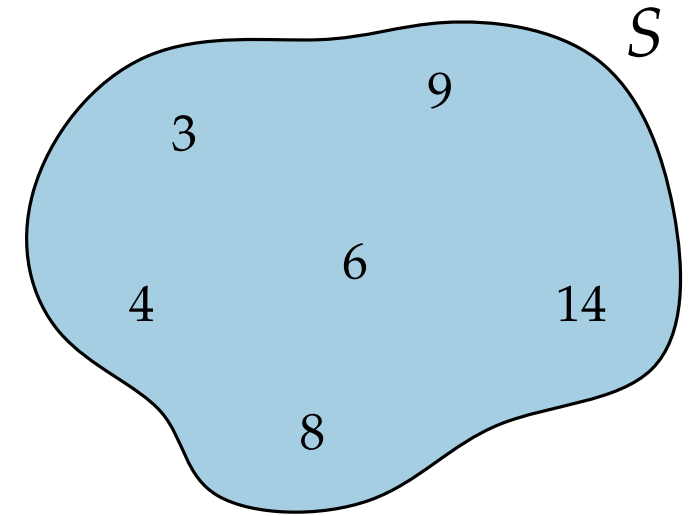
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

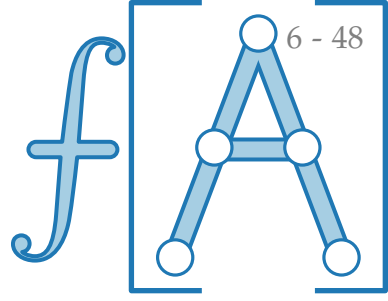
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



Wie viele unterschiedliche Teilmengen von $[n]$ gibt es?

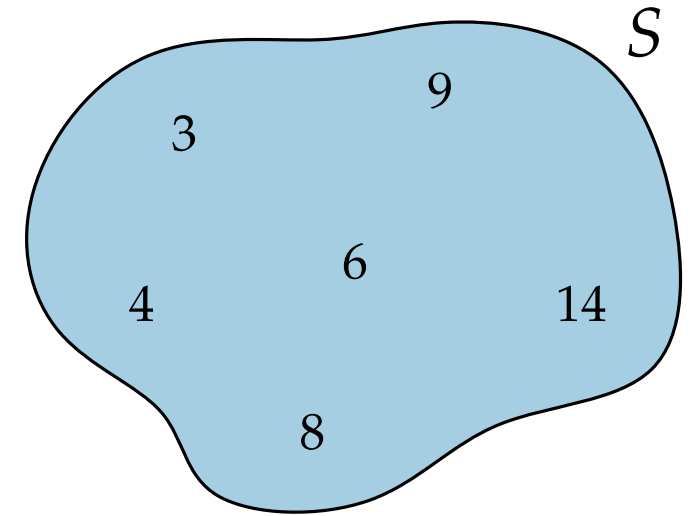
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

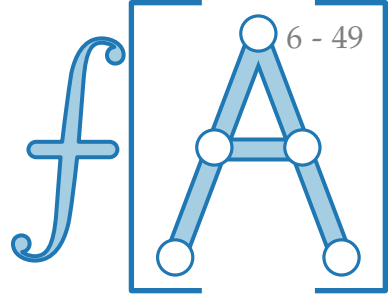
- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



Wie viele unterschiedliche Teilmengen von $[n]$ gibt es? 2^n

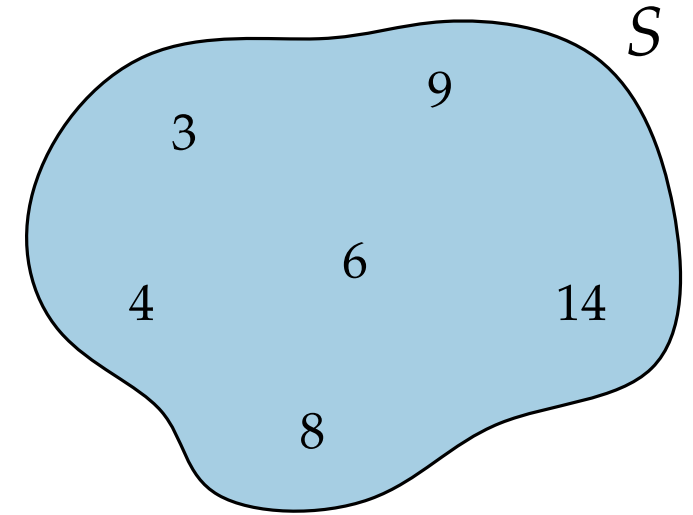
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$

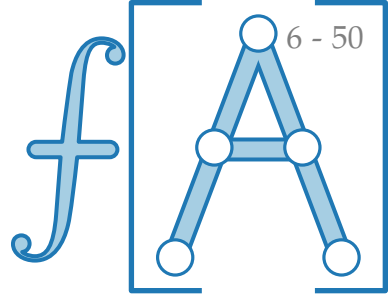


Wie viele unterschiedliche Teilmengen von $[n]$ gibt es? 2^n

Wie viele Bits brauchen wir, um sie zu unterscheiden?

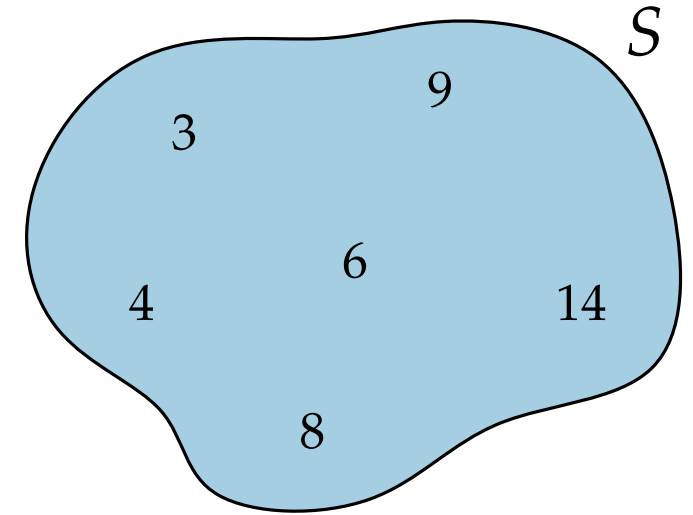
Platzeffiziente Wörterbücher (indizierbare Zuordnungstabellen; en. dictionary)

$$[n] = 1, \dots, n$$



Repräsentiere Menge $S \subset [n]$ mit folgenden $\mathcal{O}(1)$ -Zeit Operationen

- $\text{MEMBER}(i)$ überprüft, ob $i \in S$
- $\text{PREDECESSOR}(i) = \max\{k \in [i-1] \mid k \in S\}$
- $\text{SUCCESSOR}(i) = \min\{k \in [i+1, \dots, n] \mid k \in S\}$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}| = j - \text{RANK}_1(j)$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$



Wie viele unterschiedliche Teilmengen von $[n]$ gibt es?

$$2^n$$

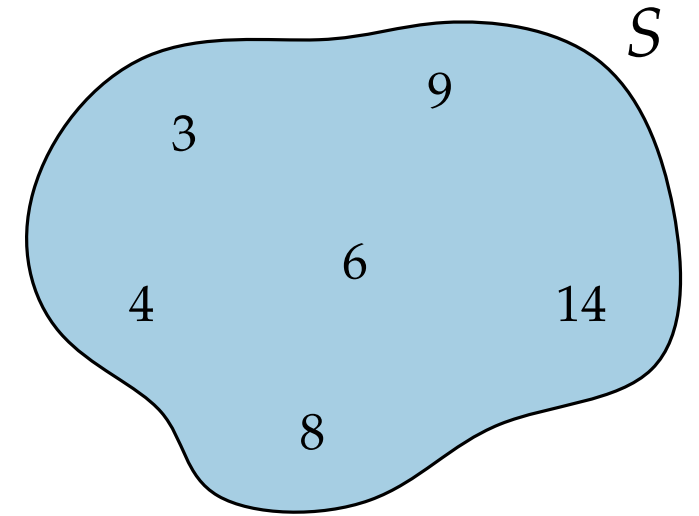
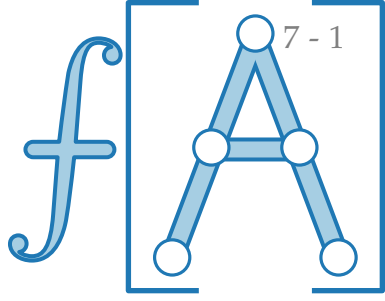
Wie viele Bits brauchen wir, um sie zu unterscheiden?

$$\log 2^n = n \text{ Bits}$$

Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

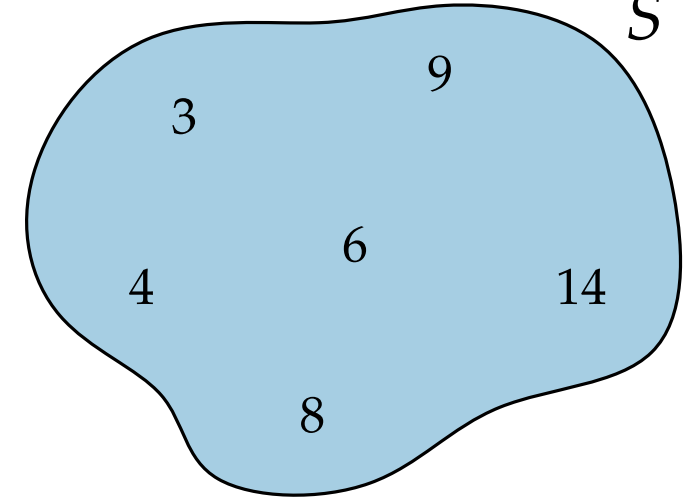
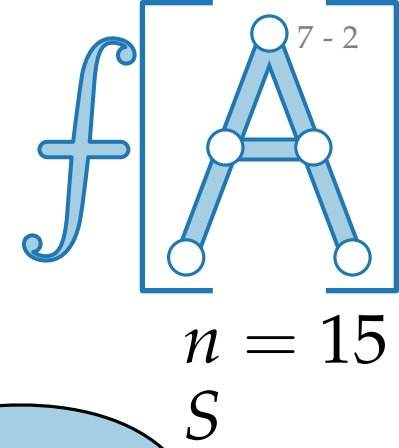
$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$



Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

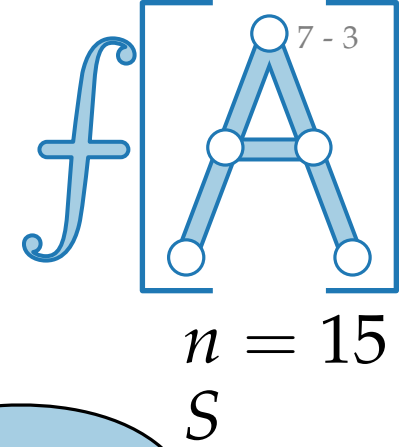
$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$



Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0

Bitvektor Repräsentation

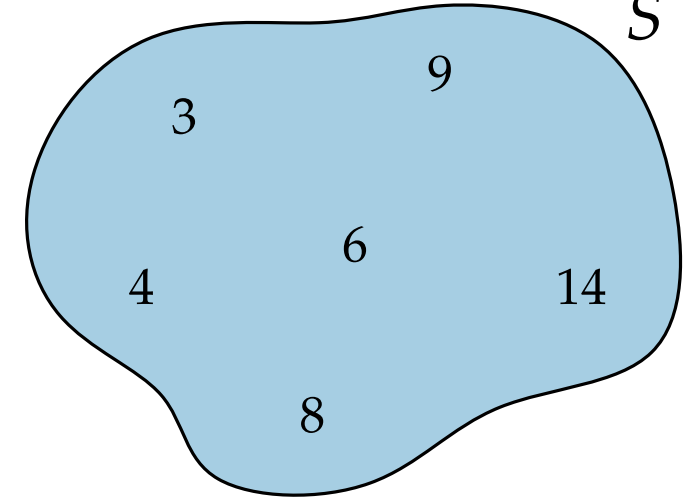
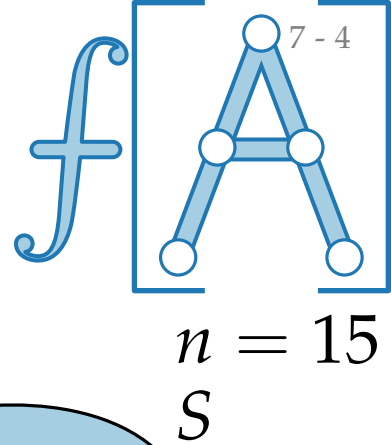
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



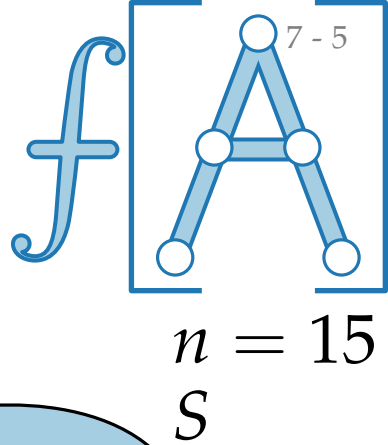
Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$\text{SELECT}_1(5) =$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0

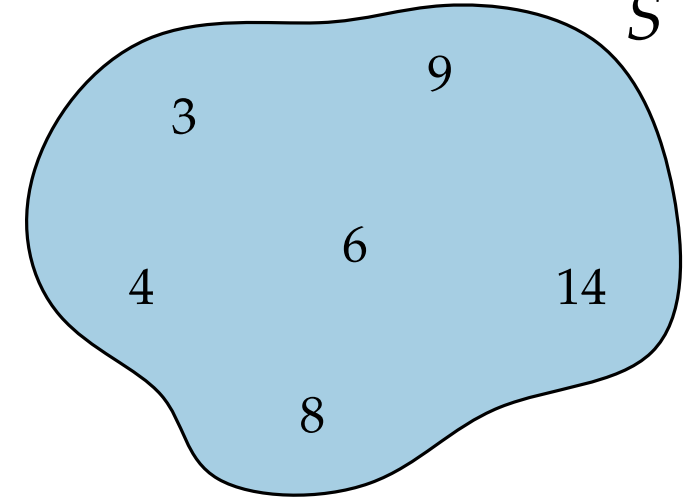
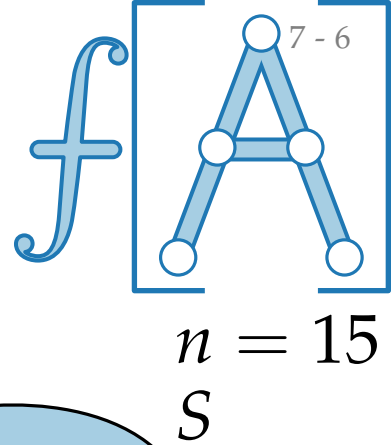
Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$\text{SELECT}_1(5) =$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0

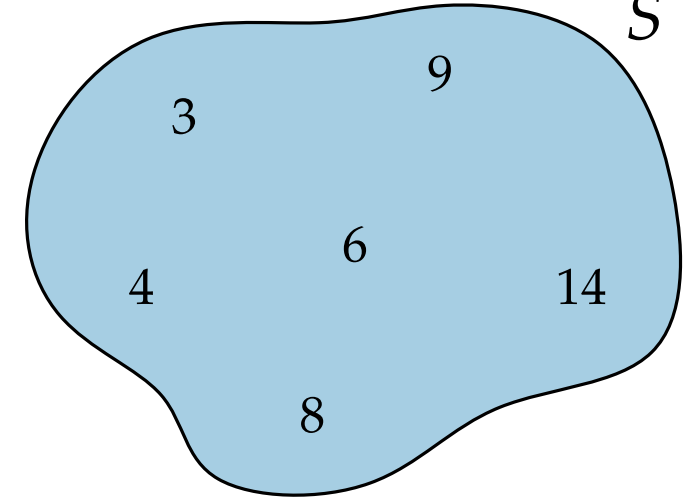
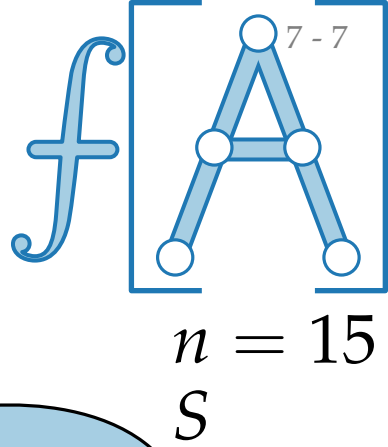
Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$$\text{SELECT}_1(5) = 9$$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0

Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

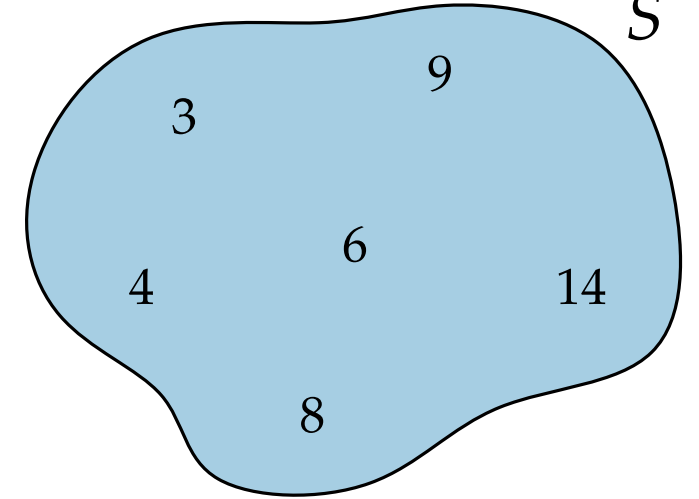
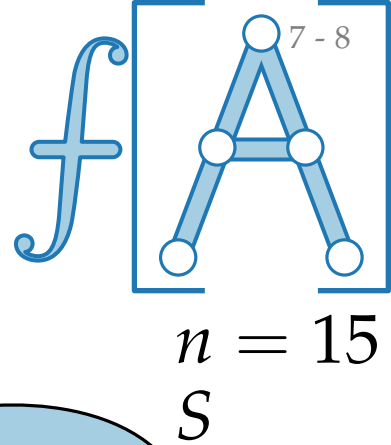
plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$

$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) =$$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



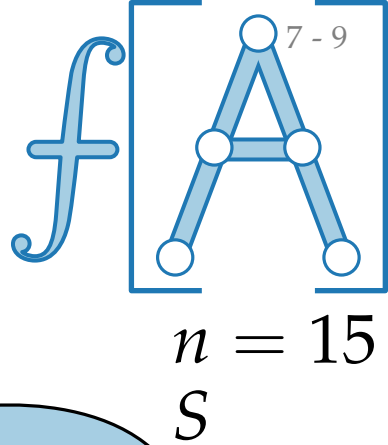
Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5$$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0

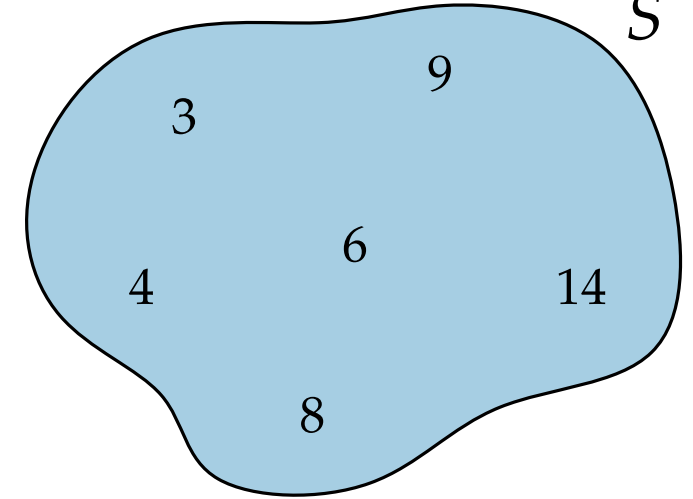
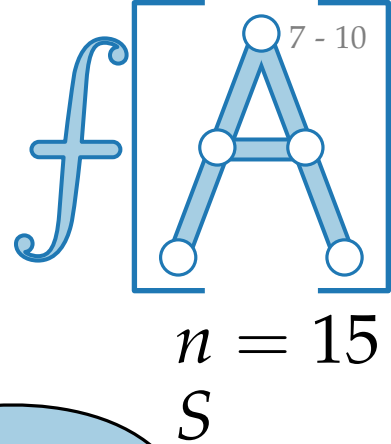
Bitvektor Repräsentation

Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0

Bitvektor Repräsentation

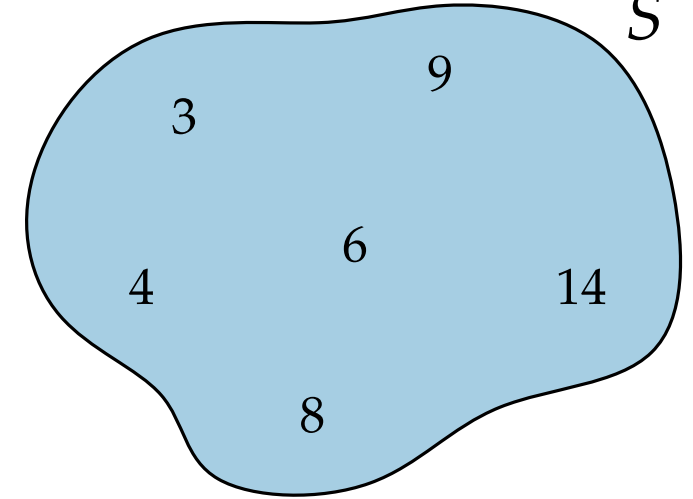
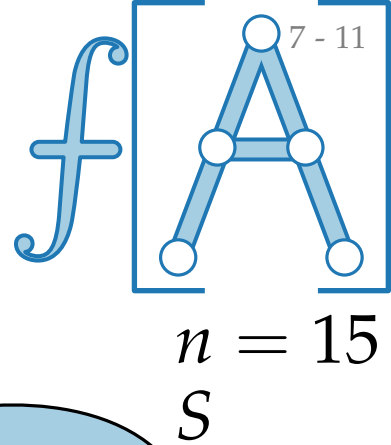
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) =$$

Bitvektor Repräsentation

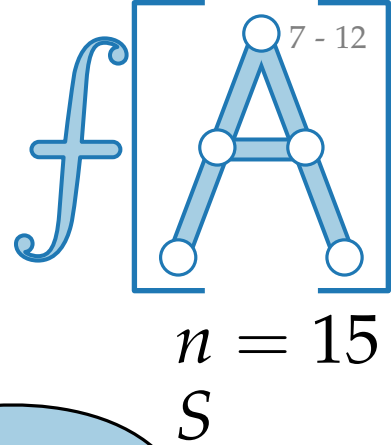
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) =$$

Bitvektor Repräsentation

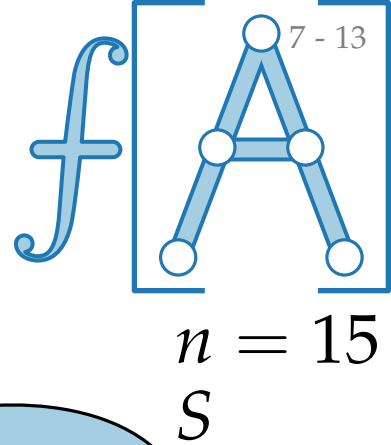
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$\text{SELECT}_1(5) = 9$
 $\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$
 $\text{SELECT}_0(7) = 12$

Bitvektor Repräsentation

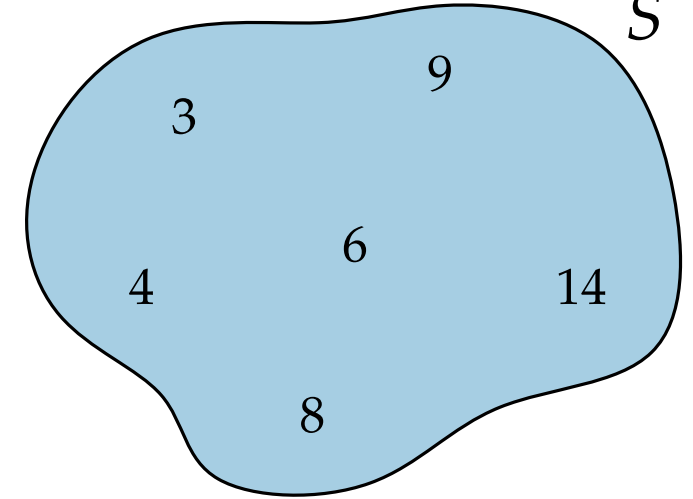
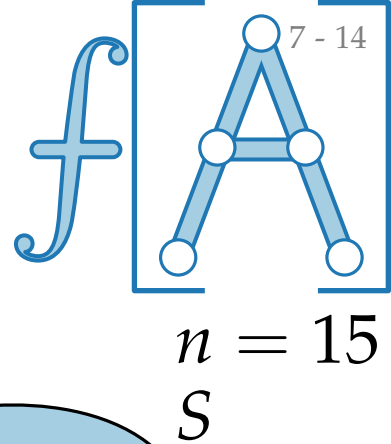
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{PREDECESSOR}(i) = ?$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) = 12$$

Bitvektor Repräsentation

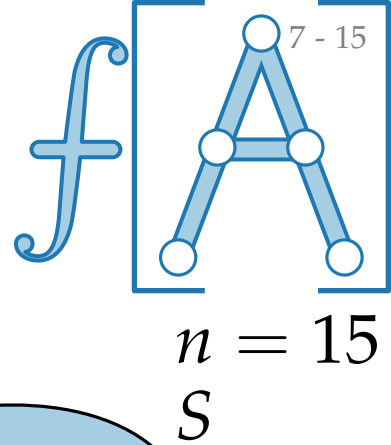
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{PREDECESSOR}(i) = ?$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) = 12$$

$$\text{SELECT}_1(\text{RANK}_1(i)) = ?$$

Bitvektor Repräsentation

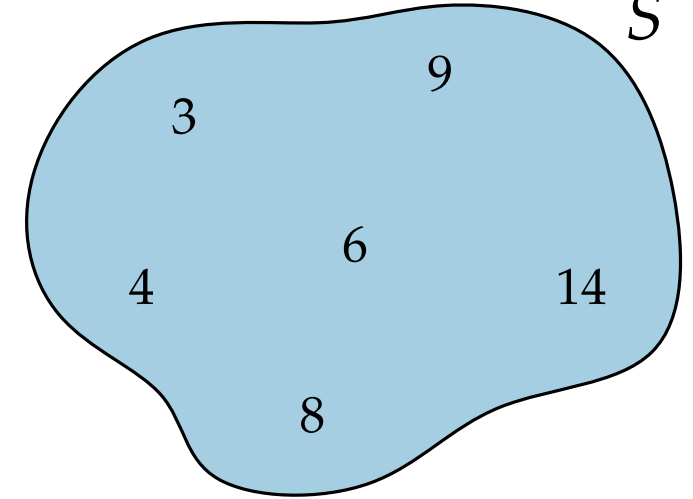
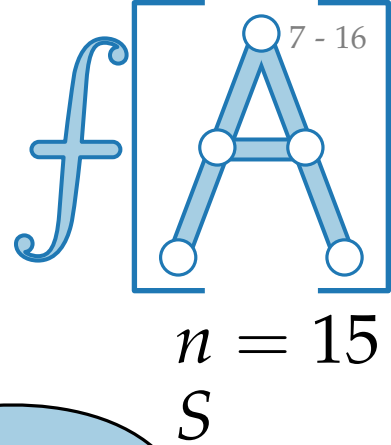
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{PREDECESSOR}(i) = ?$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) = 12$$

$$\text{SELECT}_1(\text{RANK}_1(i)) = i \quad \text{falls } b[i] = 1$$

Bitvektor Repräsentation

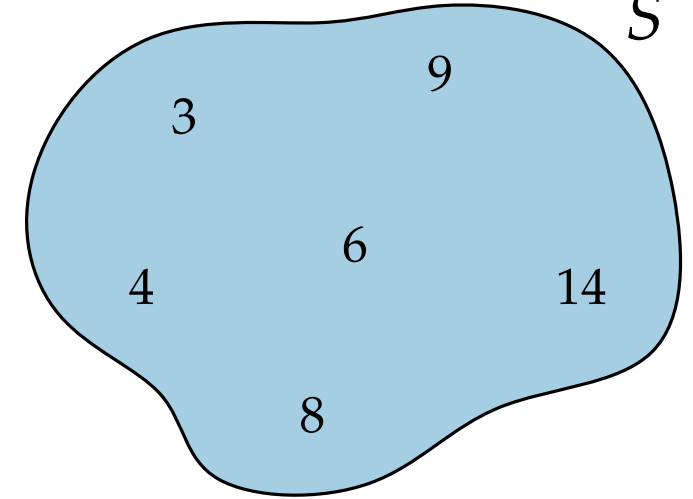
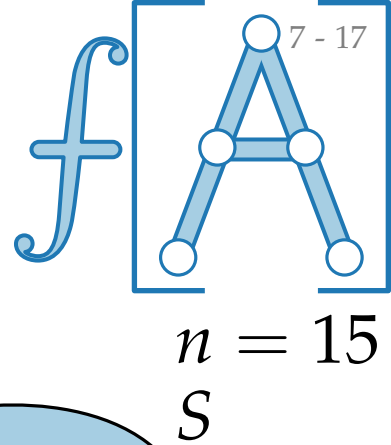
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{PREDECESSOR}(i) = ?$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) = 12$$

$$\begin{aligned} \text{SELECT}_1(\text{RANK}_1(i)) &= i && \text{falls } b[i] = 1 \\ &= \text{PREDECESSOR}(i) && \text{sonst} \end{aligned}$$

Bitvektor Repräsentation

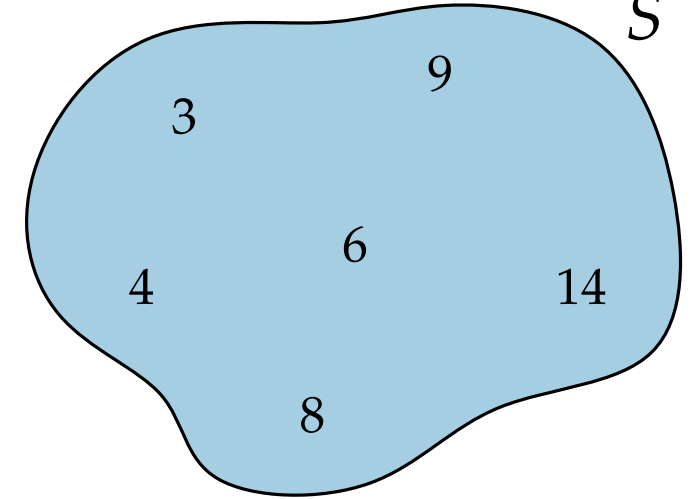
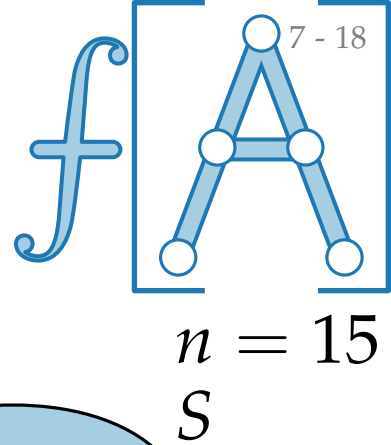
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{PREDECESSOR}(i) = \text{SELECT}_1(\quad)$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) = 12$$

$$\begin{aligned} \text{SELECT}_1(\text{RANK}_1(i)) &= i && \text{falls } b[i] = 1 \\ &= \text{PREDECESSOR}(i) && \text{sonst} \end{aligned}$$

Bitvektor Repräsentation

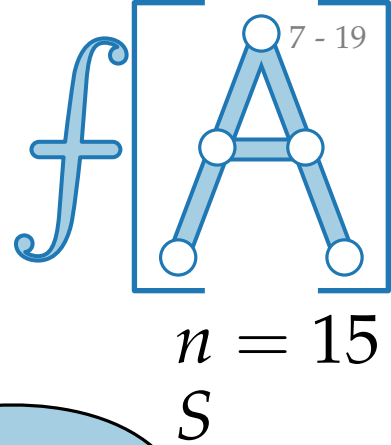
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{PREDECESSOR}(i) = \text{SELECT}_1(\text{RANK}_1(i - 1))$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) = 12$$

$$\begin{aligned} \text{SELECT}_1(\text{RANK}_1(i)) &= i && \text{falls } b[i] = 1 \\ &= \text{PREDECESSOR}(i) && \text{sonst} \end{aligned}$$

Bitvektor Repräsentation

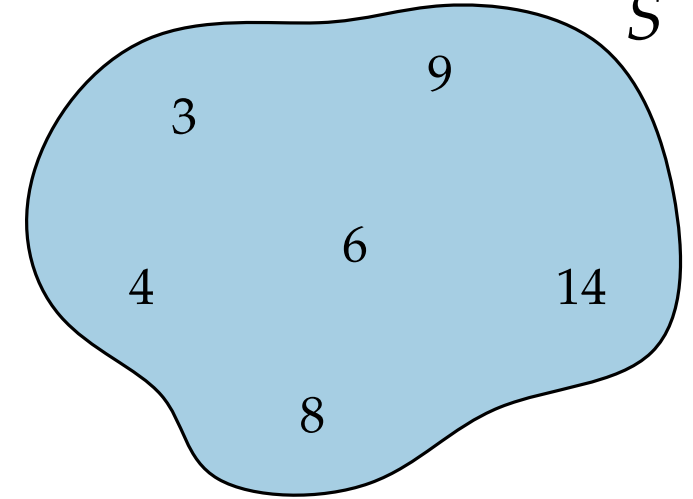
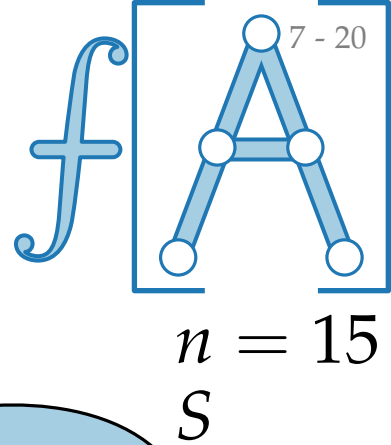
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{PREDECESSOR}(i) = \text{SELECT}_1(\text{RANK}_1(i - 1))$
- $\text{SUCCESSOR}(i) = ?$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0



$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) = 12$$

$$\begin{aligned} \text{SELECT}_1(\text{RANK}_1(i)) &= i && \text{falls } b[i] = 1 \\ &= \text{PREDECESSOR}(i) && \text{sonst} \end{aligned}$$

Bitvektor Repräsentation

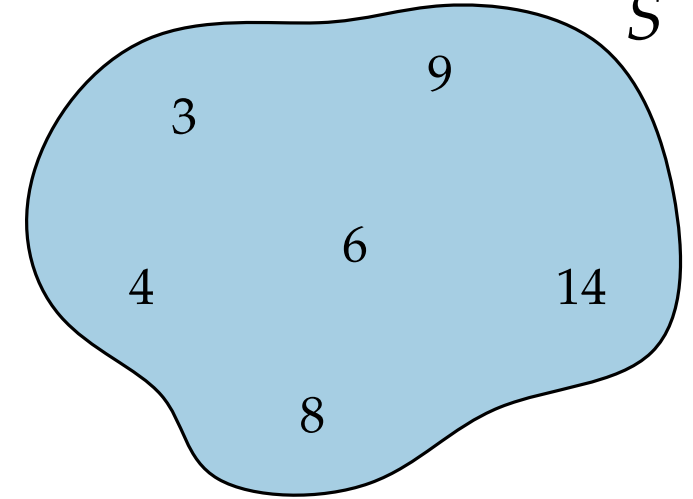
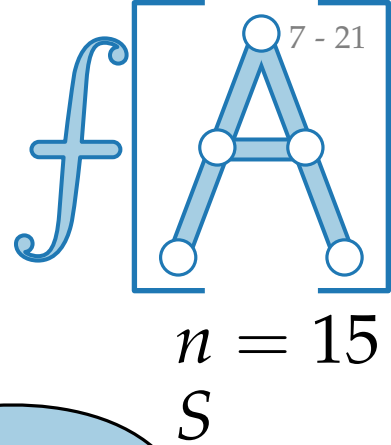
Repräsentiere S mit einem Bitvektor b der Länge n , wobei

$$b[i] = \begin{cases} 1 & \text{falls } i \in S \\ 0 & \text{falls } i \notin S \end{cases}$$

plus $o(n)$ -Platz Strukturen, um in $\mathcal{O}(1)$ Zeit zu beantworten:

- $\text{RANK}_0(j) = |\{k \in [j] \mid k \notin S\}|$
- $\text{RANK}_1(j) = |\{k \in [j] \mid k \in S\}|$
- $\text{SELECT}_0(i) = \min\{k \in [n] \mid \text{RANK}_0(k) = i\}$
- $\text{SELECT}_1(i) = \min\{k \in [n] \mid \text{RANK}_1(k) = i\}$
- $\text{PREDECESSOR}(i) = \text{SELECT}_1(\text{RANK}_1(i - 1))$
- $\text{SUCCESSOR}(i) = \text{SELECT}_1(\text{RANK}_1(i) + 1)$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
b	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0

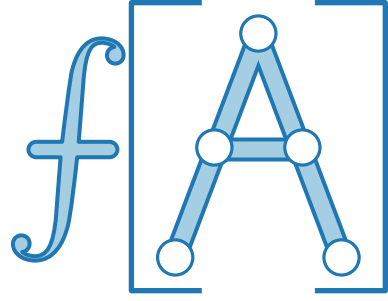


$$\text{SELECT}_1(5) = 9$$

$$\text{RANK}_1(9) = 5 = \text{RANK}_1(13)$$

$$\text{SELECT}_0(7) = 12$$

$$\begin{aligned} \text{SELECT}_1(\text{RANK}_1(i)) &= i && \text{falls } b[i] = 1 \\ &= \text{PREDECESSOR}(i) && \text{sonst} \end{aligned}$$

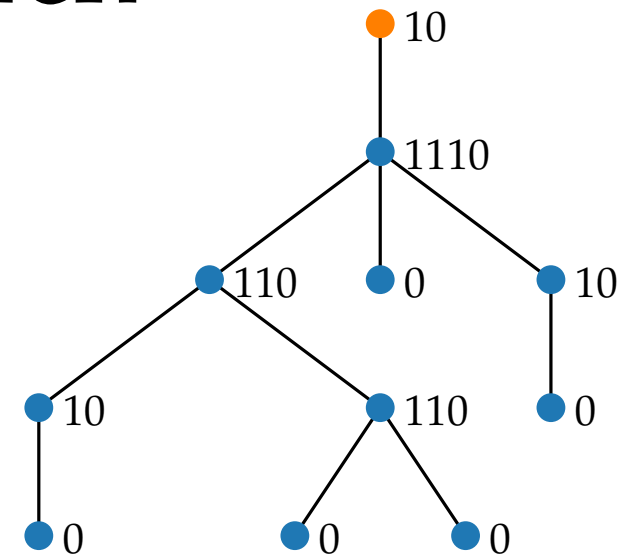


Fortgeschrittene Algorithmen

16. Vorlesung Datenstrukturen II:

Platzeffiziente Datenstrukturen (Succinct Data Structures)

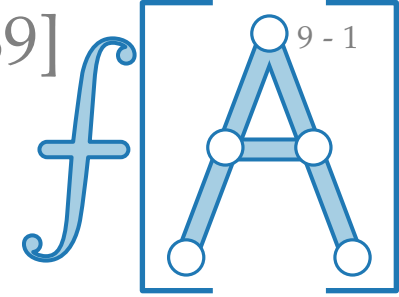
Teil III: RANK und SELECT



RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]

b



nicht prüfungsrelevant

RANK in $o(n)$ Bits

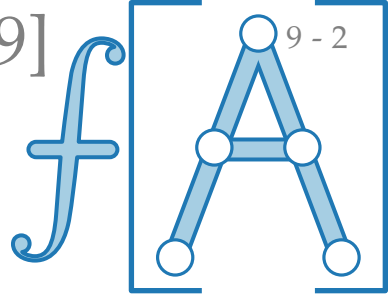
[Guy Jacobson, FOCS 1989]

$\log^2 n$

b



1. Zerlege in $(\log^2 n)$ -Bit **Stücke**



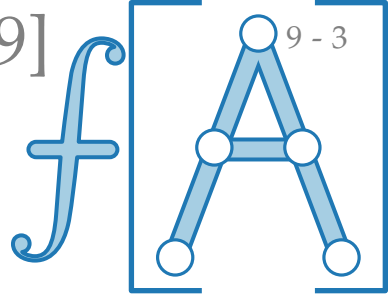
nicht prüfungsrelevant

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



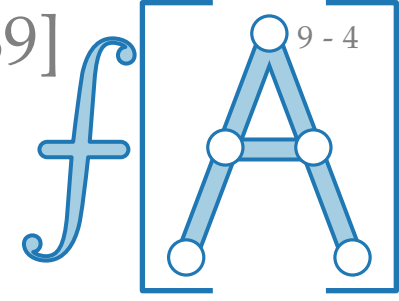
1. Zerlege in $(\log^2 n)$ -Bit **Stücke** und speichere kumulativen RANK:



nicht prüfungsrelevant

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]

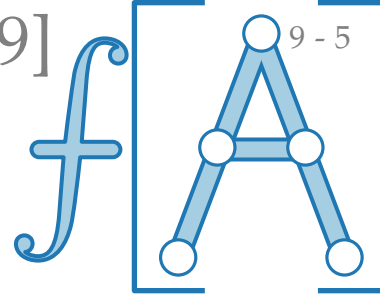


nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke** und speichere kumulativen RANK:

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke** und speichere kumulativen RANK:

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]

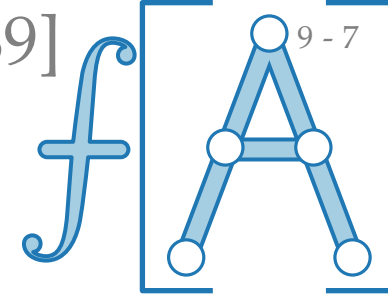


1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

nicht prüfungsrelevant

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke** und speichere kumulativen RANK:

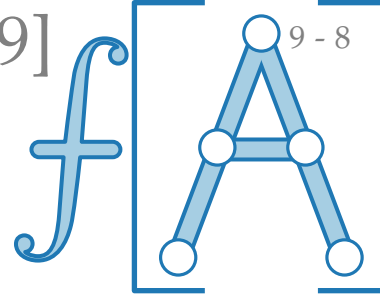
je $\log n$ Bits

$$\rightarrow \mathcal{O}\left(\underbrace{\log n}_{\text{\# Stücke}} \cdot \underbrace{\log n}_{\text{RANK}}\right)$$

Bits

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK:

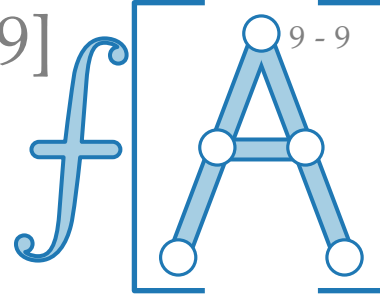
je $\log n$ Bits

$$\rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log^2 n}}_{\text{\# Stücke}} \cdot \underbrace{\quad}_{\text{RANK}}\right)$$

Bits

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK:

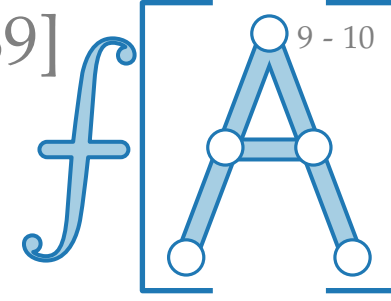
je $\log n$ Bits

$$\rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log^2 n}}_{\text{\# Stücke}} \underbrace{\log n}_{\text{RANK}}\right)$$

Bits

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK:

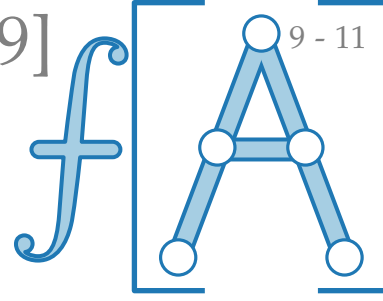
je $\log n$ Bits

$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\text{\# Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right)$$

Bits

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

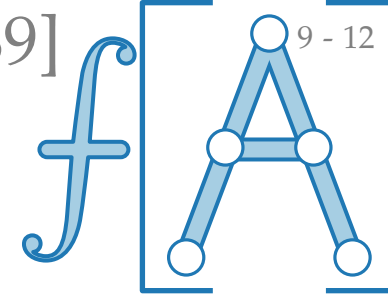
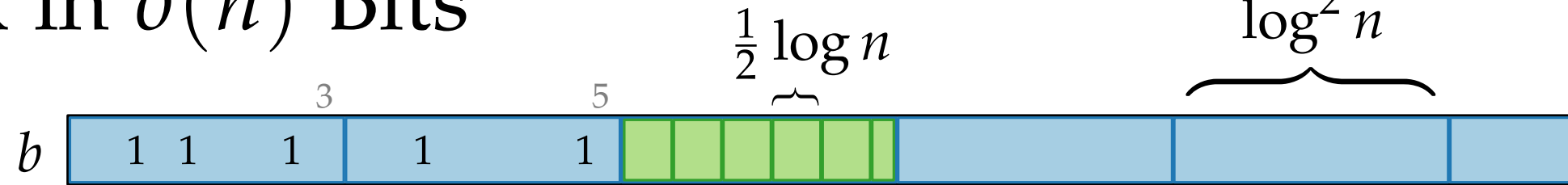
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK:

je $\log n$ Bits

$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\text{\# Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]

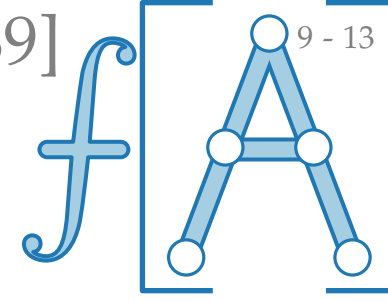
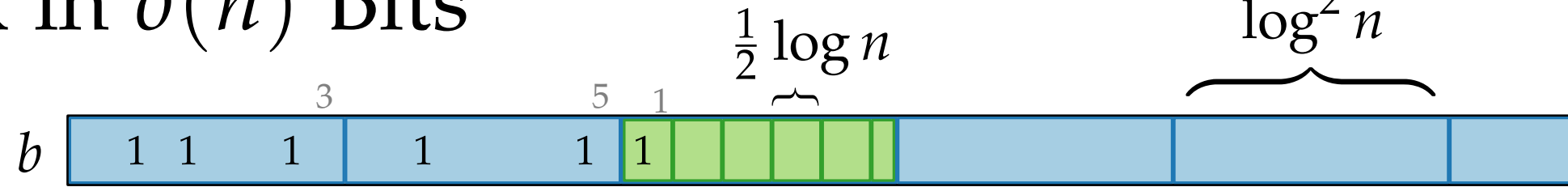


nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits
 $\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**:

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**

und speichere kumulativen RANK: je $\log n$ Bits

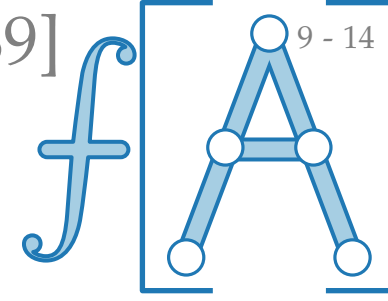
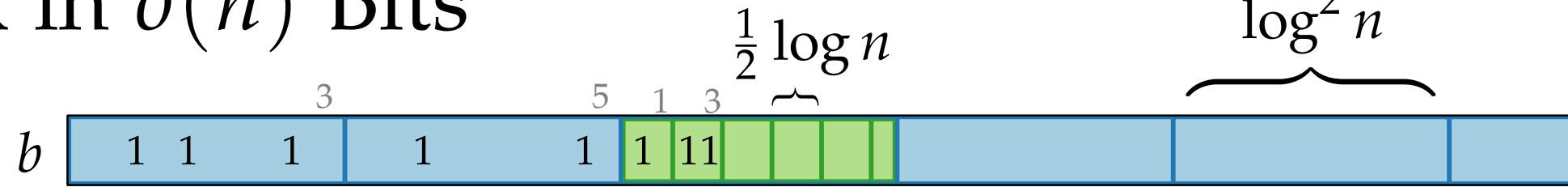
$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$

2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**

und speichere kumulativen RANK innerhalb **Stück**:

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**

und speichere kumulativen RANK: je $\log n$ Bits

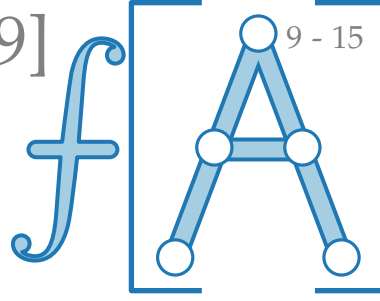
$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$

2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**

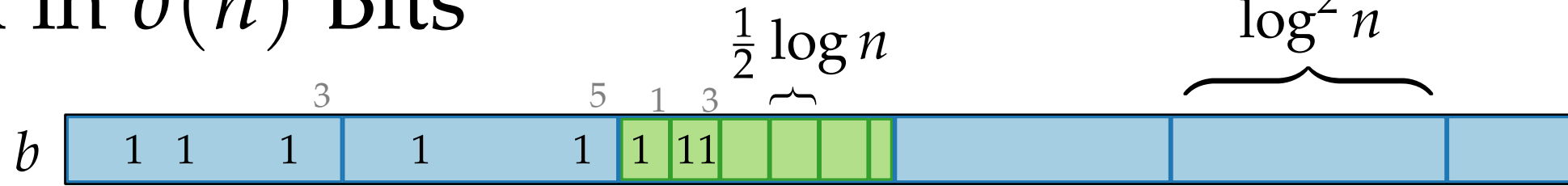
und speichere kumulativen RANK innerhalb **Stück**:

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant



1. Zerlege in $(\log^2 n)$ -Bit **Stücke**

und speichere kumulativen RANK: je $\log n$ Bits

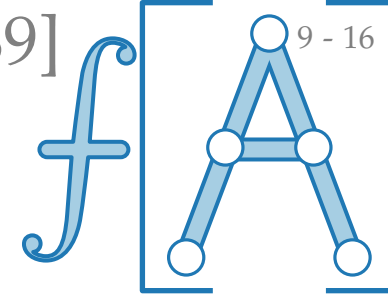
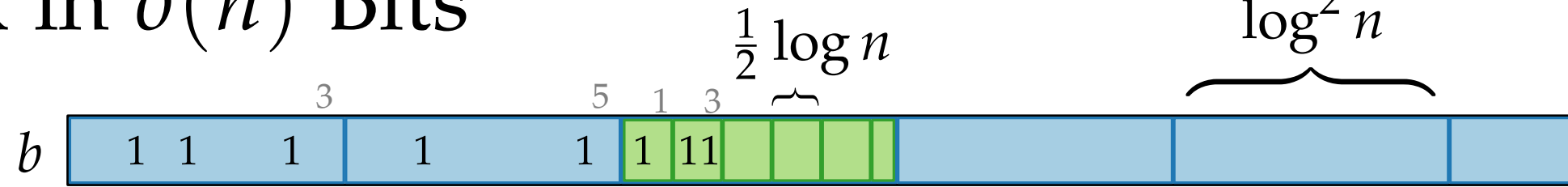
$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$

2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**

und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n)$ Bits

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



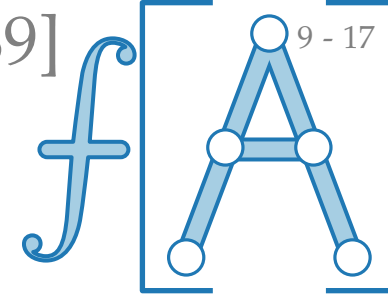
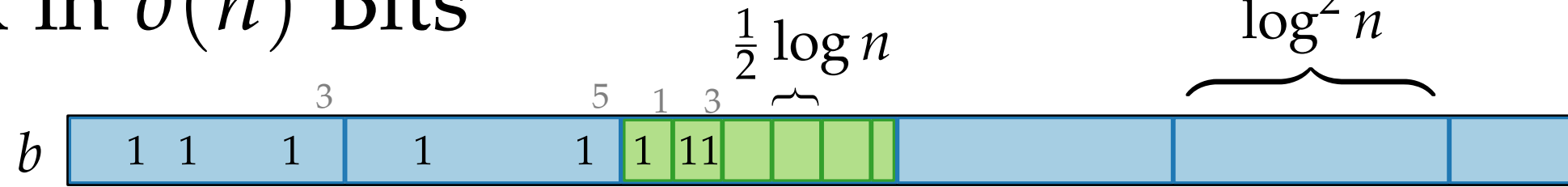
nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]

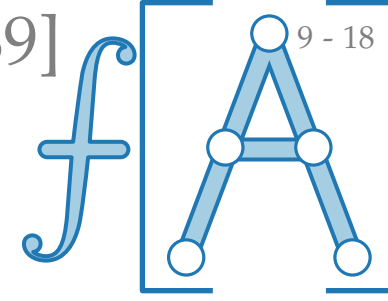
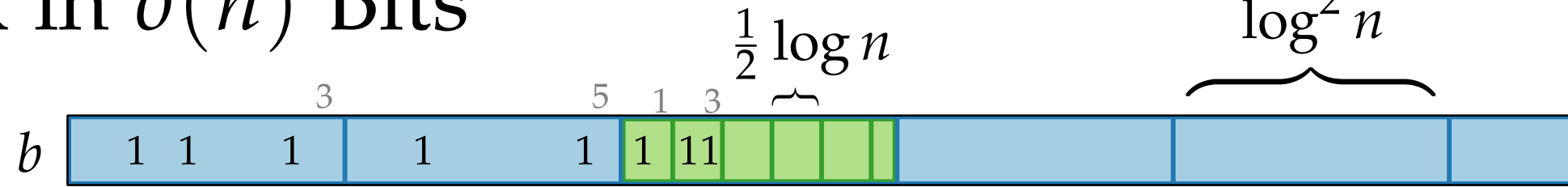


nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits
 $\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log^2 n}}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}}\right) = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits
 $\Rightarrow \mathcal{O}\left(\underbrace{\frac{1}{2}}_{\# \text{ Teilst.}} \underbrace{\log^2 n}_{\text{rel. RANK}}\right) = \log \log n \text{ Bits}$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

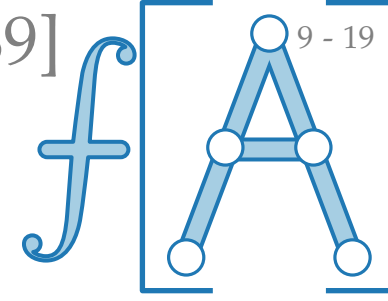
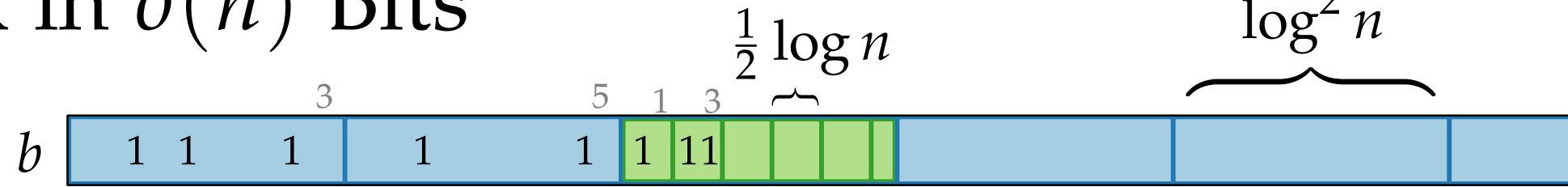
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log^2 n}}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}}\right) = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log n}}_{\# \text{ Teilst.}} \underbrace{1}_{\text{rel. RANK}}\right) \text{ Bits}$$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

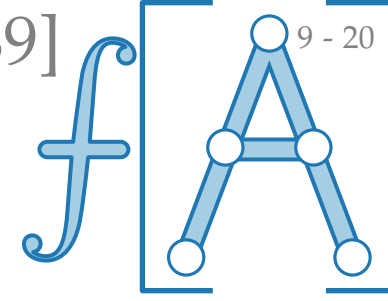
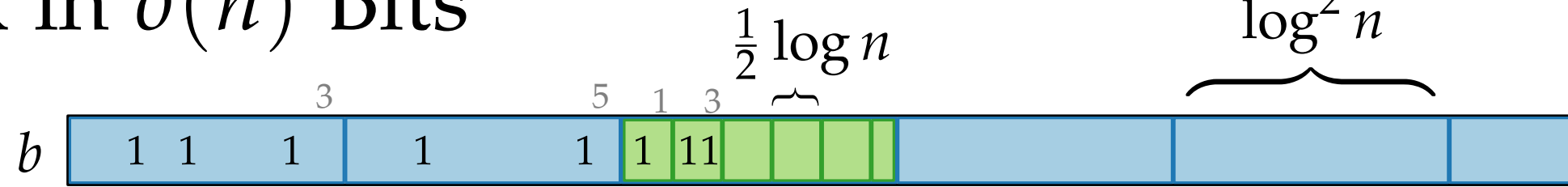
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \text{ Bits}$$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

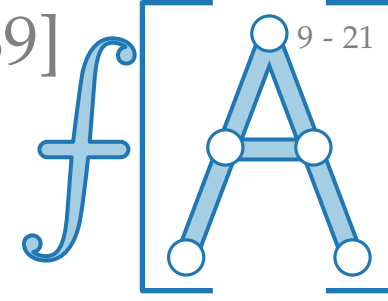
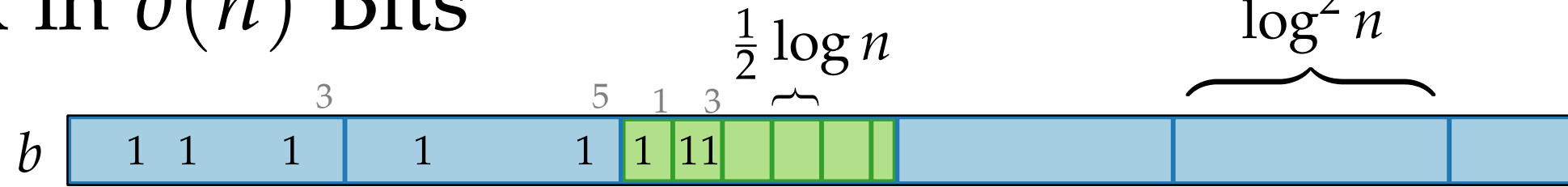
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]

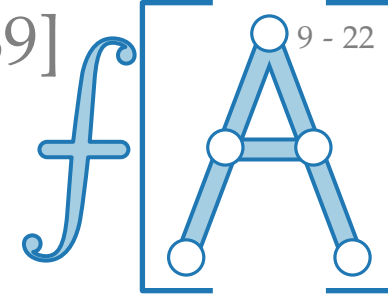
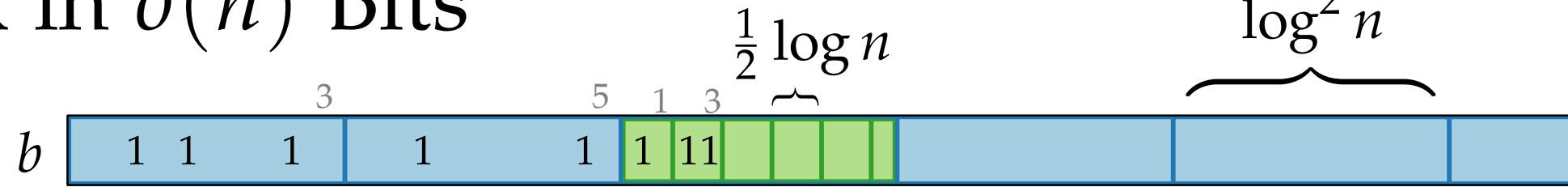


nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant

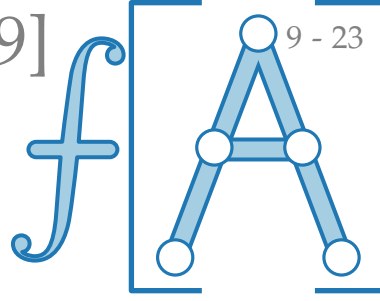
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

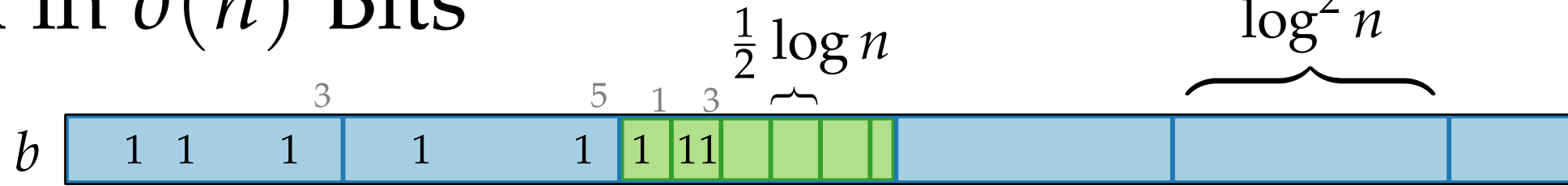
$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant



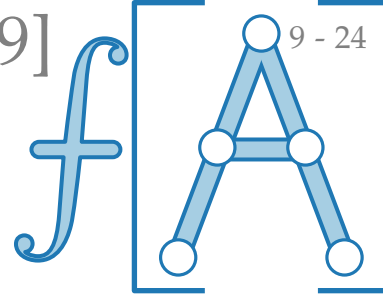
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

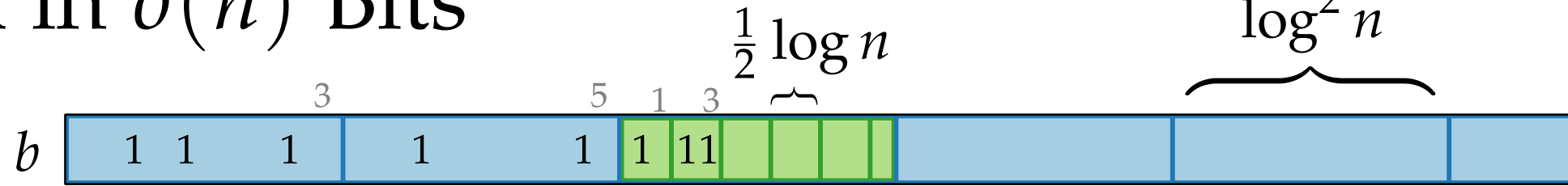
$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
davon gibt es *viele*

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant



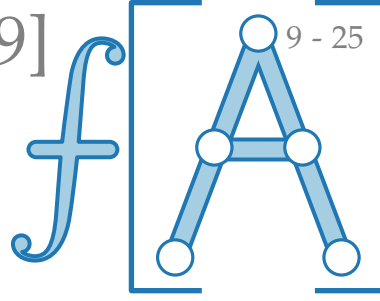
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

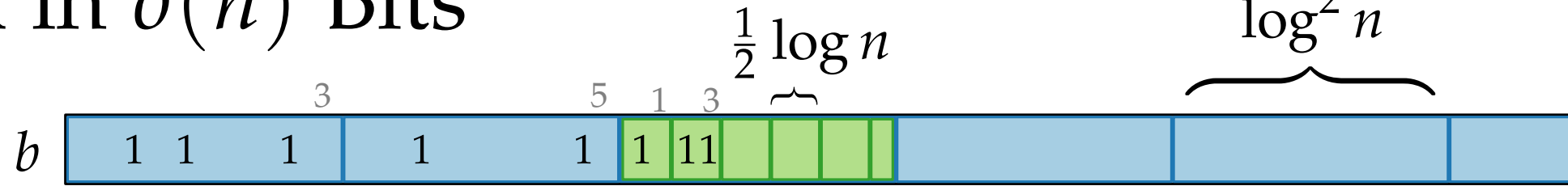
$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
 davon gibt es $2^{\frac{1}{2} \log n}$ ↓
viele

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



nicht prüfungsrelevant



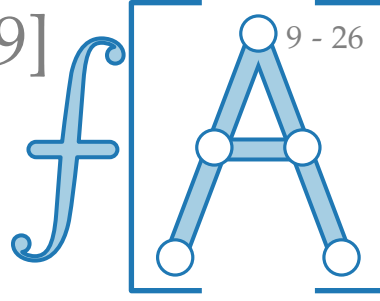
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

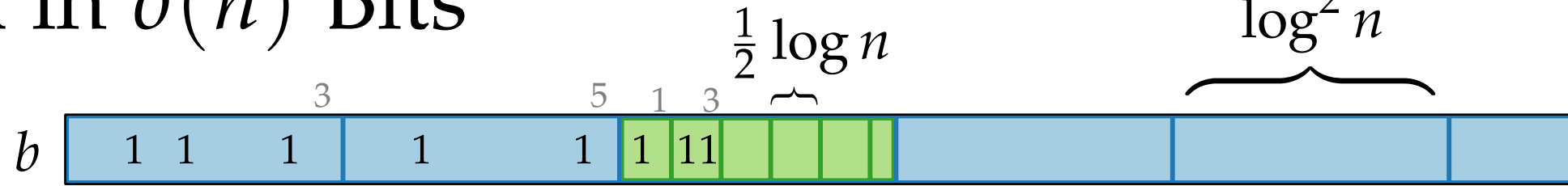
$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
 davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}}$ ↓
viele

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



9 - 26



nicht prüfungsrelevant

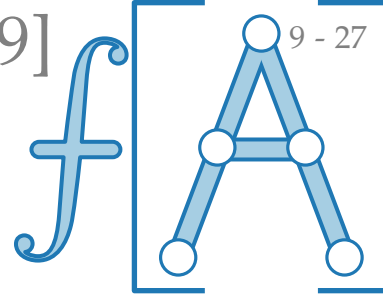
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

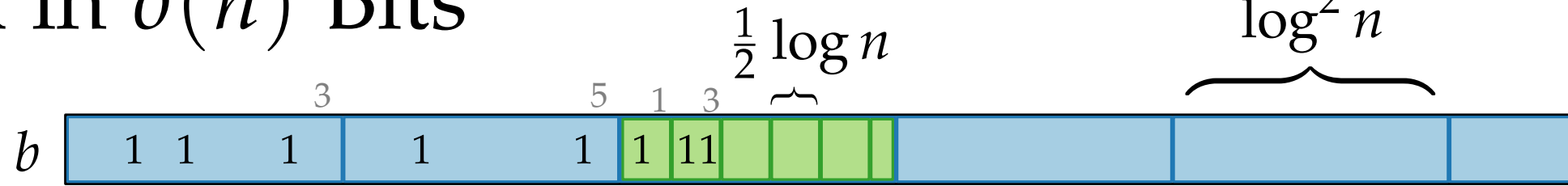
$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



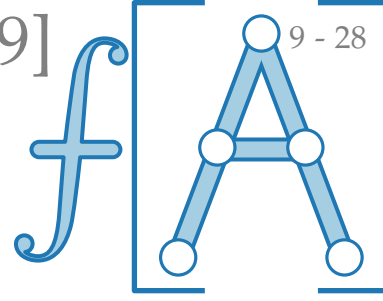
nicht prüfungsrelevant



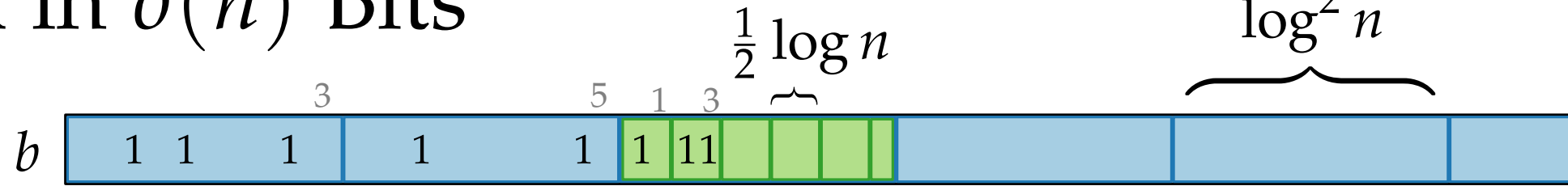
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele $\Rightarrow \mathcal{O}(\quad) \text{ Bits}$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



9 - 28



nicht prüfungsrelevant

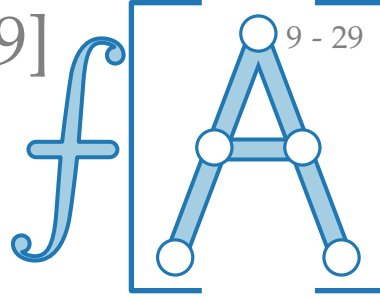
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

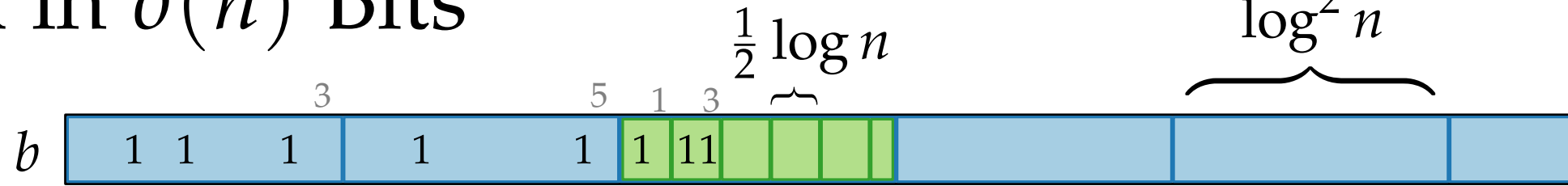
$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele $\Rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \text{ Bits}$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



9 - 29



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

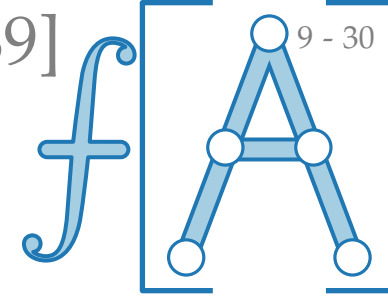
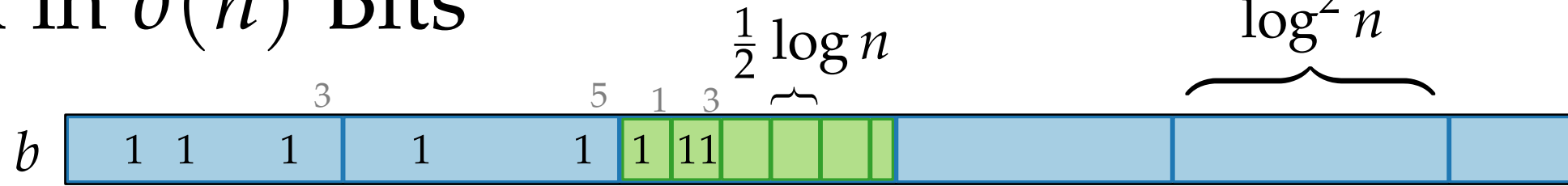
$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele

$$\Rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{Anfrage}} \subseteq o(n) \text{ Bits}$$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]

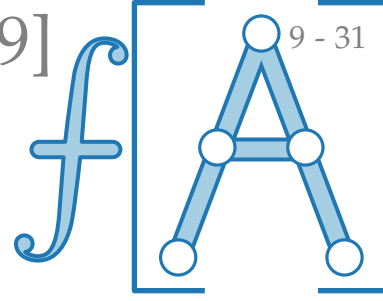


nicht prüfungsrelevant

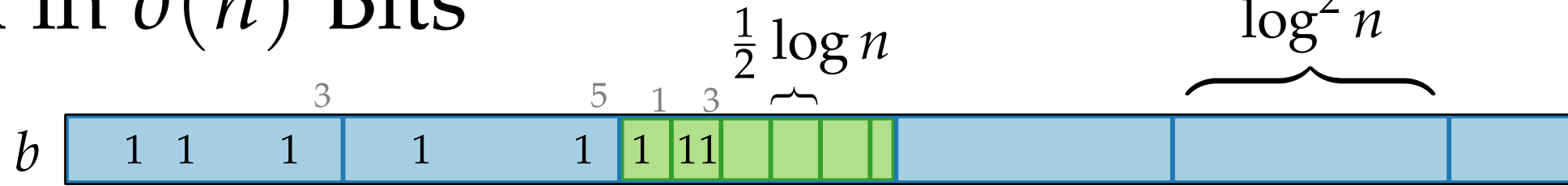
1. Zerlege in $(\log^2 n)$ -Bit **Stücke** und speichere kumulativen RANK: je $\log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\text{\# Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke** und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\text{\# Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
 davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele $\Rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{Anfrage}} \underbrace{\log \log n}_{\text{Antwort}} \text{ Bits}$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



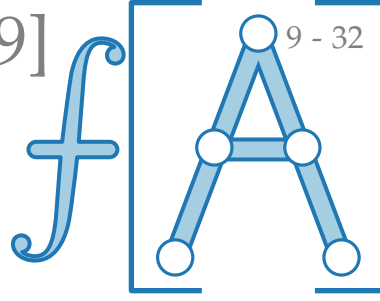
nicht prüfungsrelevant



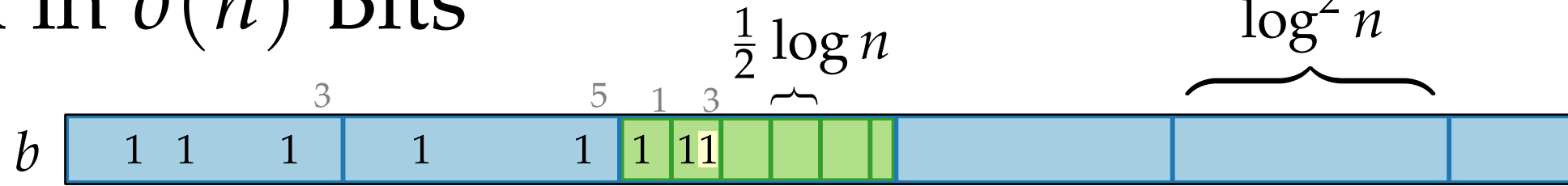
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele
 $\Rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{Anfrage}} \underbrace{\log \log n}_{\text{Antwort}} \subseteq o(n) \text{ Bits}$

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



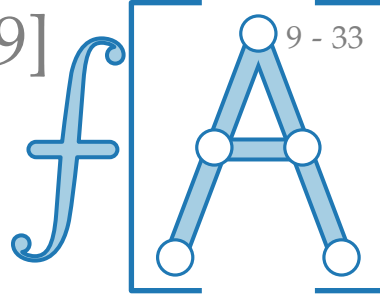
nicht prüfungsrelevant



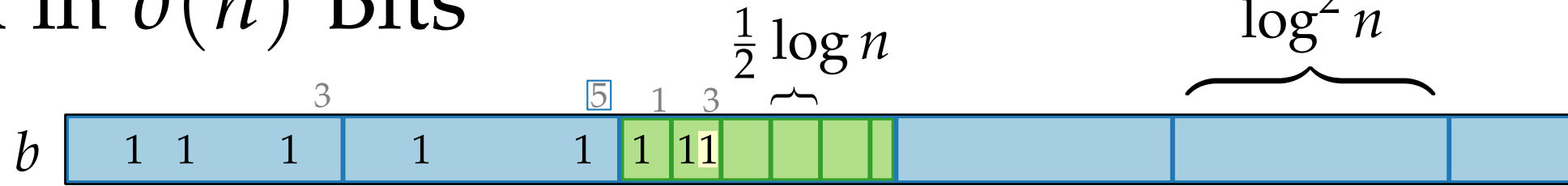
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
 davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele $\Rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{Anfrage}} \underbrace{\log \log n}_{\text{Antwort}} \subseteq o(n) \text{ Bits}$
4. RANK =

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



9 - 33



nicht prüfungsrelevant

1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

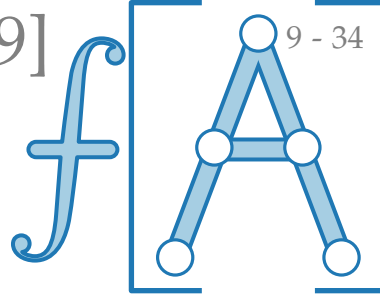
$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\# \text{ Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\# \text{ Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
 davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele

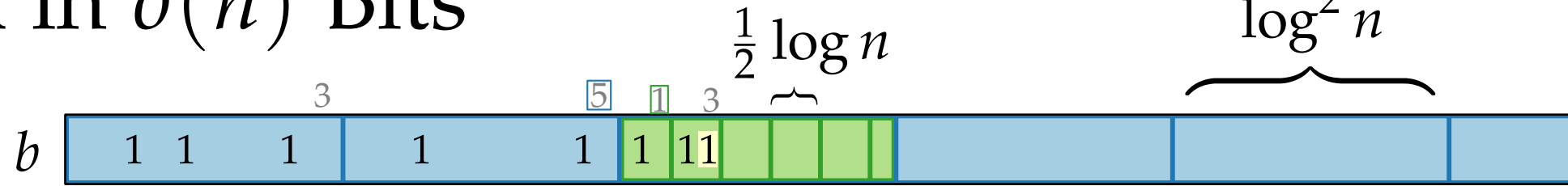
$$\Rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{Anfrage}} \underbrace{\log \log n}_{\text{Antwort}} \subseteq o(n) \text{ Bits}$$
4. RANK = RANK des **Stücks**

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



9 - 34

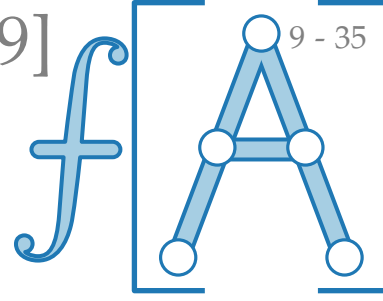


nicht prüfungsrelevant

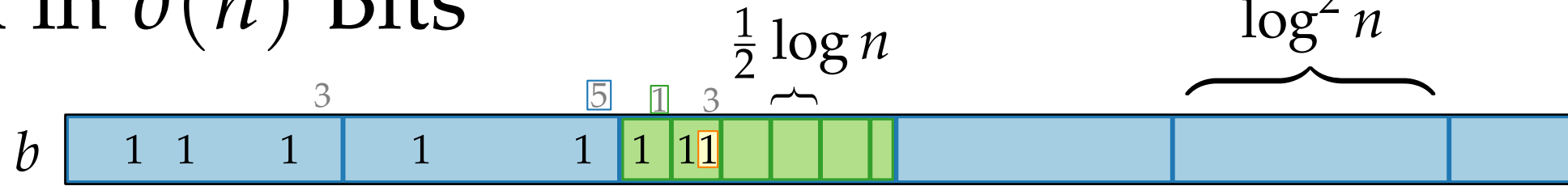
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n}\right)}_{\text{\# Stücke}} \underbrace{\log n}_{\text{RANK}} = \mathcal{O}\left(\frac{n}{\log n}\right) \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\text{\# Teilst.}} \underbrace{\log \log n}_{\text{rel. RANK}} \subseteq o(n) \text{ Bits}$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
 davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele $\Rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{Anfrage}} \underbrace{\log \log n}_{\text{Antwort}} \subseteq o(n) \text{ Bits}$
4. RANK = RANK des **Stücks**
+ relativer RANK des **Teilstücks** innerhalb des **Stücks**

RANK in $o(n)$ Bits

[Guy Jacobson, FOCS 1989]



9 - 35



nicht prüfungsrelevant

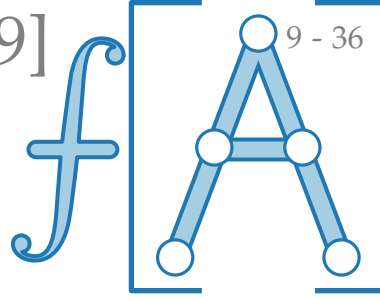
1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits

$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n} \log n\right)}_{\text{\# Stücke}} = \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\text{RANK}} \subseteq o(n) \text{ Bits}$$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits

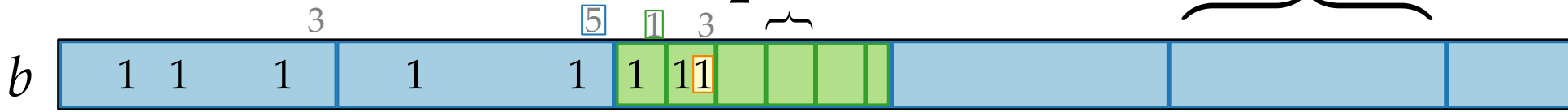
$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n} \log \log n\right)}_{\text{\# Teilst.}} \subseteq o(n) \text{ Bits}$$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
 davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele $\rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{Anfrage}} \underbrace{\log \log n}_{\text{Antwort}} \subseteq o(n) \text{ Bits}$
4. RANK = RANK des **Stücks**
 + relativer RANK des **Teilstücks** innerhalb des **Stücks**
 + relativer RANK des Elements innerhalb **Teilstück**

RANK in $o(n)$ Bits + $\mathcal{O}(1)$ Zeit

[Guy Jacobson, FOCS 1989]



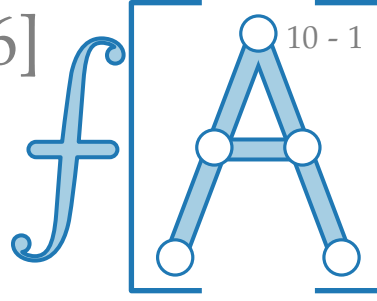
nicht prüfungsrelevant



1. Zerlege in $(\log^2 n)$ -Bit **Stücke**
und speichere kumulativen RANK: je $\log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log^2 n} \log n\right)}_{\text{\# Stücke}} = \underbrace{\mathcal{O}\left(\frac{n}{\log n}\right)}_{\text{RANK}} \subseteq o(n) \text{ Bits}$
2. Zerlege **Stücke** in $(\frac{1}{2} \log n)$ -Bit **Teilstücke**
und speichere kumulativen RANK innerhalb **Stück**: je $\log(\log^2 n) = 2 \log \log n$ Bits
 $\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{\log n} \log \log n\right)}_{\text{\# Teilst.}} \subseteq o(n) \text{ Bits}$
3. Benutze **Suchtabelle** für *alle möglichen* Bitstrings und Anfragen für **Teilstücke**
 davon gibt es $2^{\frac{1}{2} \log n} = n^{\frac{1}{2}} = \sqrt{n}$ viele $\Rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{Anfrage}} \underbrace{\log \log n}_{\text{Antwort}} \subseteq o(n) \text{ Bits}$
4. RANK = RANK des **Stücks**
 + relativer RANK des **Teilstücks** innerhalb des **Stücks** $\Rightarrow \mathcal{O}(1) \text{ Zeit}$
 + relativer RANK des Elements innerhalb **Teilstück**

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



b



nicht prüfungsrelevant

J. Ian Munro
* 1947, Canada



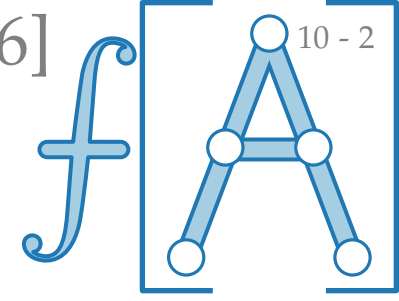
SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]

$\log n \log \log n$ 1's



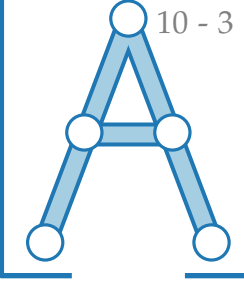
1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array



nicht prüfungsrelevant

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



10 - 3

nicht prüfungsrelevant



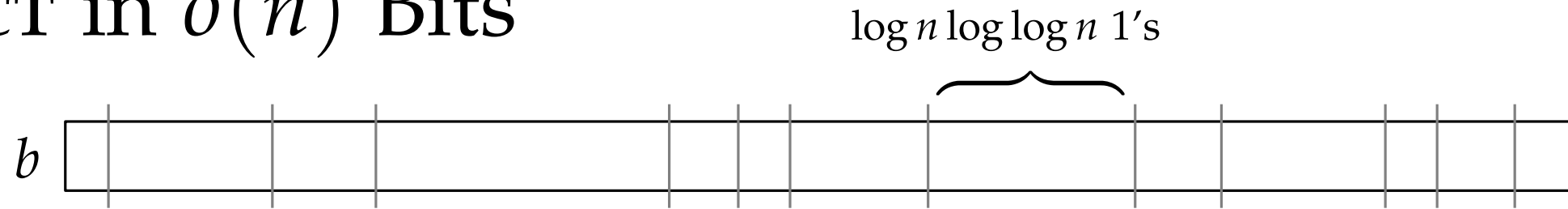
1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\underbrace{\hspace{2cm}}_{\text{\# Gruppen}} \underbrace{\hspace{1cm}}_{\text{Index}}\right)$$

Bits

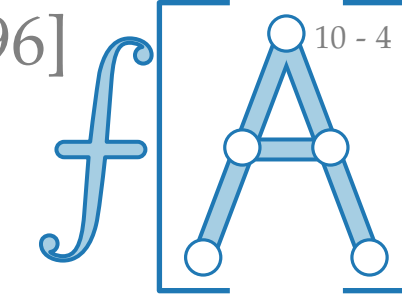
SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log n \log \log n}}_{\text{\# Gruppen}} \underbrace{\quad}_{\text{Index}}\right)$$

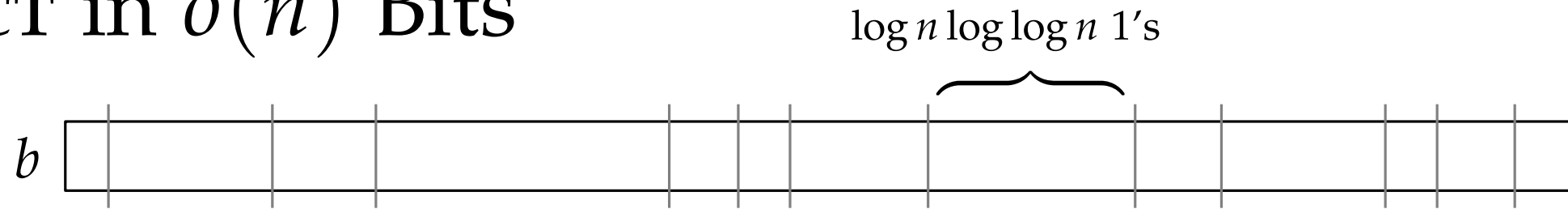


nicht prüfungsrelevant

Bits

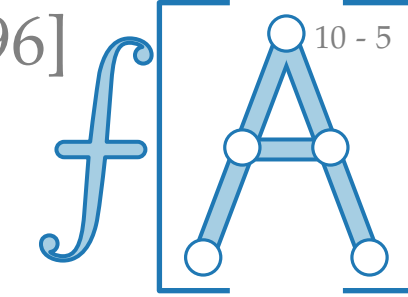
SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log n \log \log n}}_{\text{\# Gruppen}} \underbrace{\log n}_{\text{Index}}\right)$$



nicht prüfungsrelevant

Bits

SELECT in $o(n)$ Bits

$\log n \log \log n$ 1's



nicht prüfungsrelevant

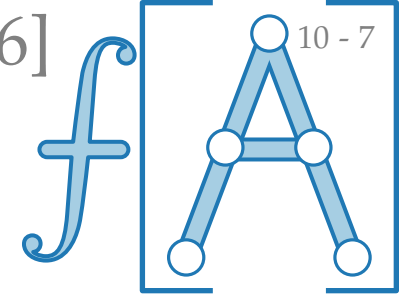
1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log n \log \log n}}_{\text{\# Gruppen}} \underbrace{\log n}_{\text{Index}}\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right)$$

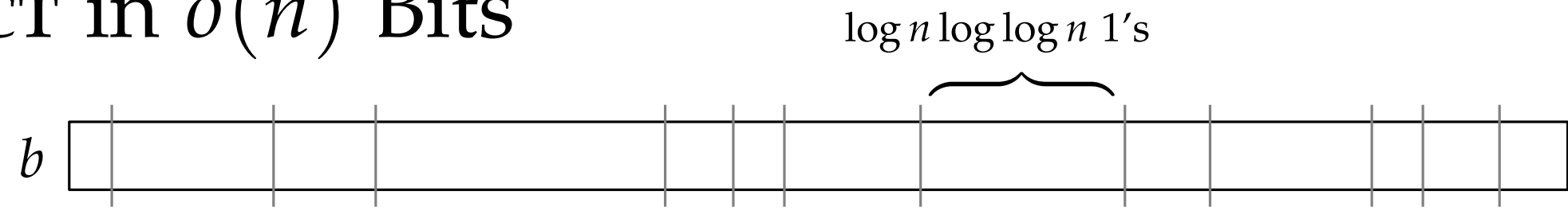
Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



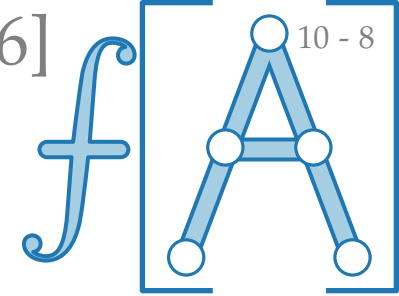
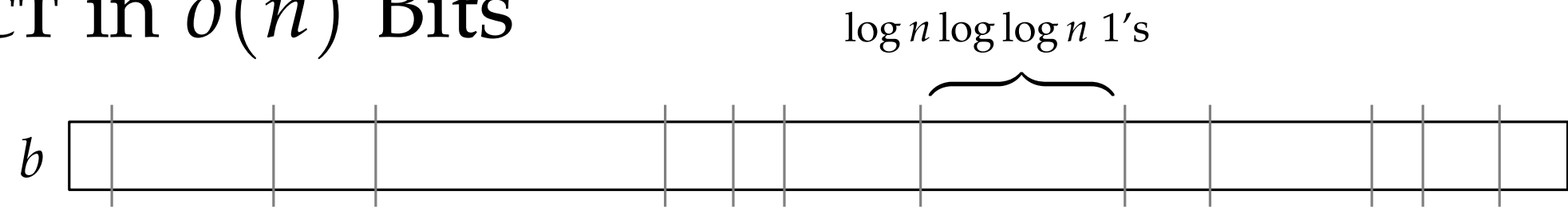
nicht prüfungsrelevant



1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{\log n \log \log n}}_{\text{\# Gruppen}} \underbrace{\log n}_{\text{Index}}\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

SELECT in $o(n)$ Bits



nicht prüfungsrelevant

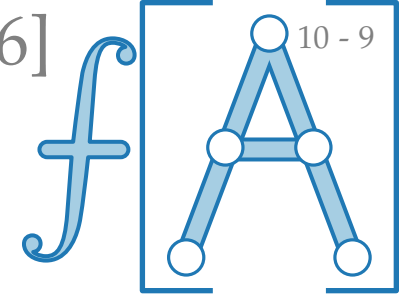
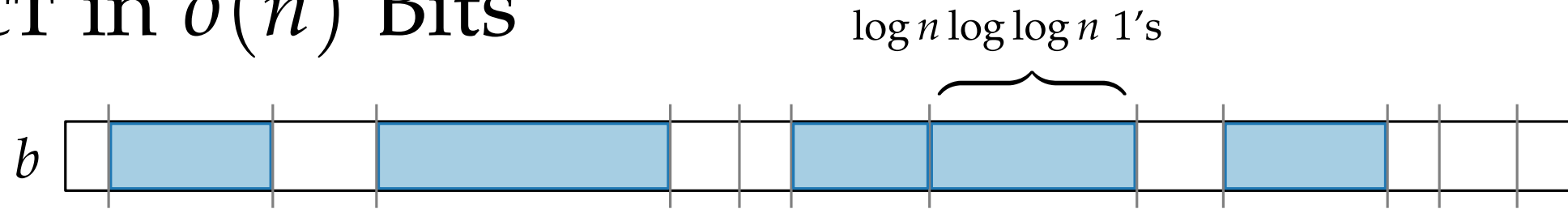
1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

2. Innerhalb **Stück** von $(\log n \log \log n)$ 1-Bits mit insgesamt r Bits:

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

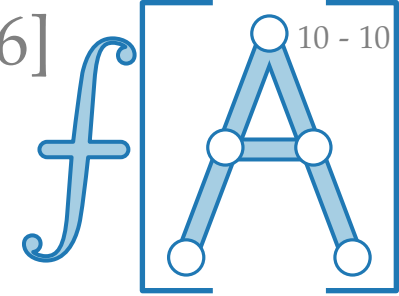
$$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

2. Innerhalb **Stück** von $(\log n \log \log n)$ 1-Bits mit insgesamt r Bits:

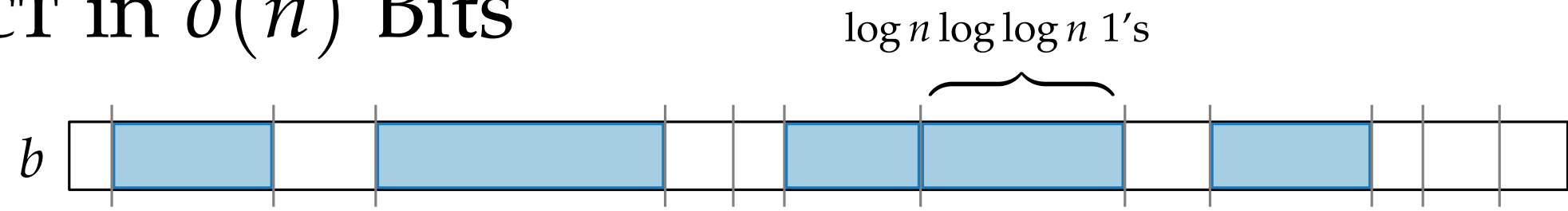
wenn $r \geq (\log n \log \log n)^2$

dann speichere Indizes der 1-Bits in **Stück** in Array

SELECT in $o(n)$ Bits



nicht prüfungsrelevant



1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

2. Innerhalb **Stück** von $(\log n \log \log n)$ 1-Bits mit insgesamt r Bits:

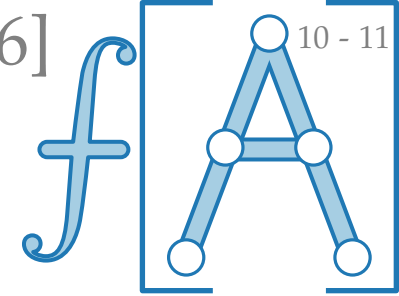
wenn $r \geq (\log n \log \log n)^2$

dann speichere Indizes der 1-Bits in **Stück** in Array

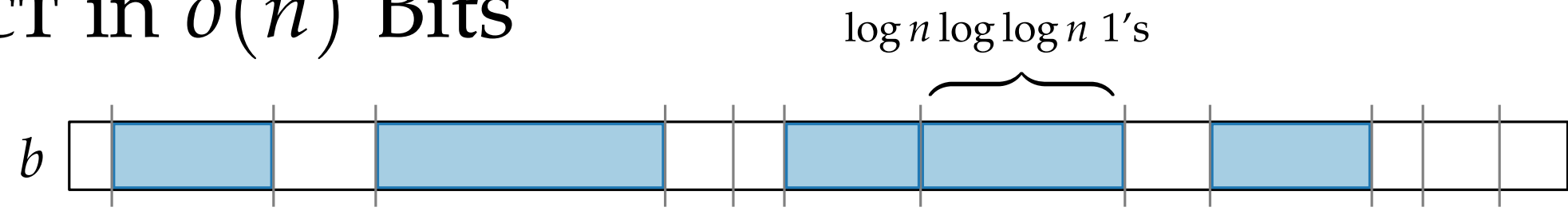
$$\Rightarrow \mathcal{O}\left(\underbrace{\hspace{1.5cm}}_{\# \text{ Stücke}} \underbrace{\hspace{1.5cm}}_{\# \text{ 1-Bits}} \underbrace{\hspace{1.5cm}}_{\text{index}}\right)$$

Bits

SELECT in $o(n)$ Bits



nicht prüfungsrelevant



1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

2. Innerhalb **Stück** von $(\log n \log \log n)$ 1-Bits mit insgesamt r Bits:

wenn $r \geq (\log n \log \log n)^2$

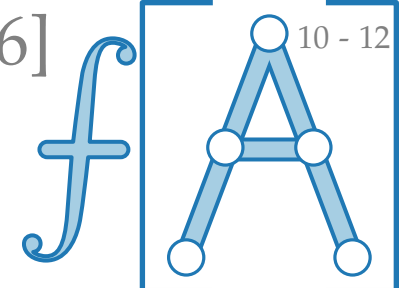
dann speichere Indizes der 1-Bits in **Stück** in Array

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{(\log n \log \log n)^2}}_{\# \text{ Stücke}} \underbrace{\hspace{1cm}}_{\# \text{ 1-Bits}} \underbrace{\hspace{1cm}}_{\text{index}}\right)$$

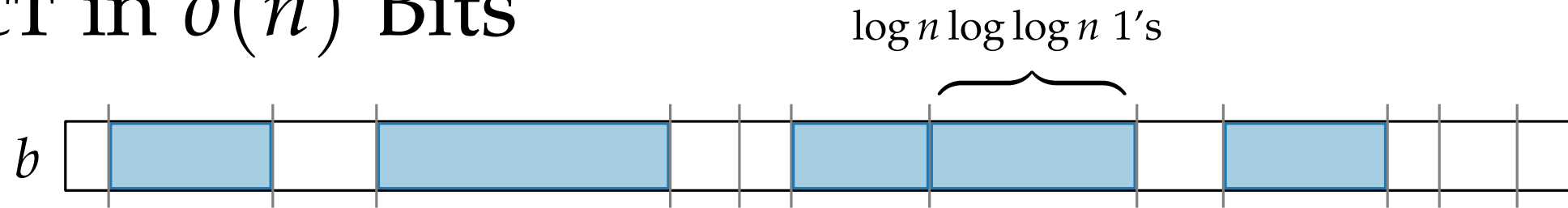
Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant



1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

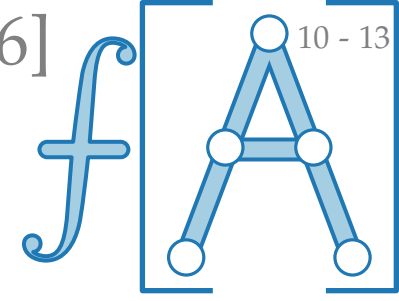
2. Innerhalb **Stück** von $(\log n \log \log n)$ 1-Bits mit insgesamt r Bits:

wenn $r \geq (\log n \log \log n)^2$

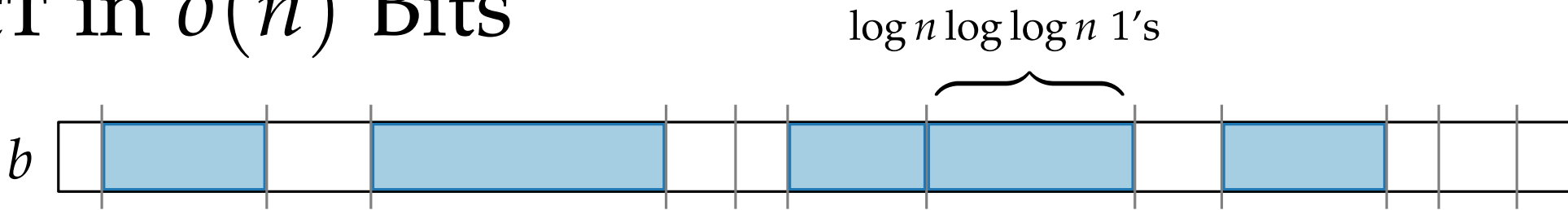
dann speichere Indizes der 1-Bits in **Stück** in Array

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{(\log n \log \log n)^2}}_{\# \text{ Stücke}} \underbrace{(\log n \log \log n)}_{\# \text{ 1-Bits}} \underbrace{(\log n \log \log n)}_{\text{index}}\right)$$

Bits



SELECT in $o(n)$ Bits



nicht prüfungsrelevant

1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

2. Innerhalb **Stück** von $(\log n \log \log n)$ 1-Bits mit insgesamt r Bits:

wenn $r \geq (\log n \log \log n)^2$

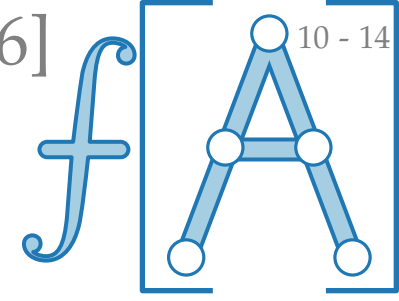
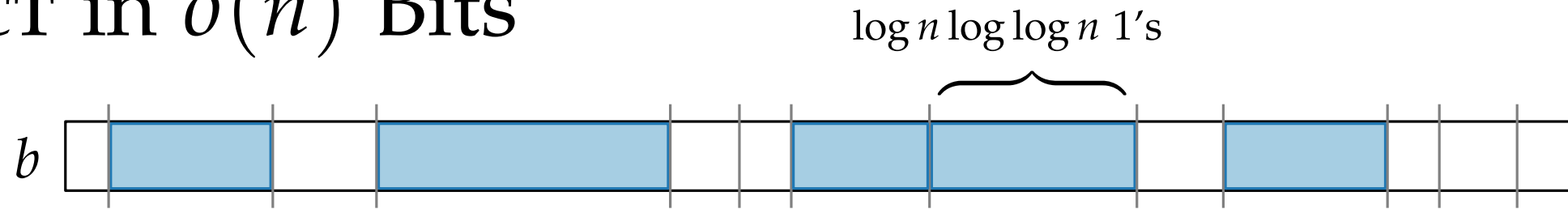
dann speichere Indizes der 1-Bits in **Stück** in Array

$$\Rightarrow \mathcal{O}\left(\underbrace{\frac{n}{(\log n \log \log n)^2}}_{\# \text{ Stücke}} \underbrace{(\log n \log \log n)}_{\# \text{ 1-Bits}} \underbrace{\log n}_{\text{index}}\right)$$

Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

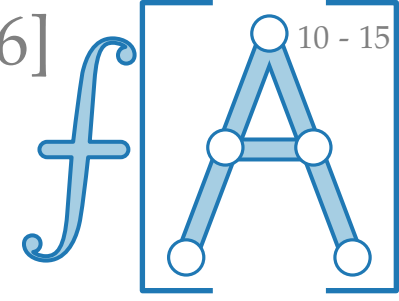
2. Innerhalb **Stück** von $(\log n \log \log n)$ 1-Bits mit insgesamt r Bits:

wenn $r \geq (\log n \log \log n)^2$

dann speichere Indizes der 1-Bits in **Stück** in Array

$$\Rightarrow \underbrace{\mathcal{O}\left(\frac{n}{(\log n \log \log n)^2}\right)}_{\# \text{ Stücke}} \underbrace{(\log n \log \log n)}_{\# \text{ 1-Bits}} \underbrace{\log n}_{\text{index}} = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

- $$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

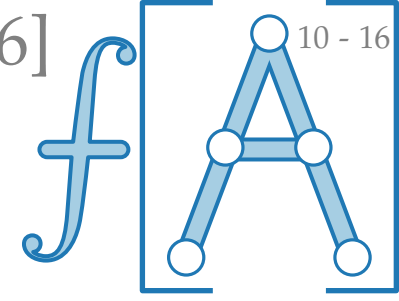
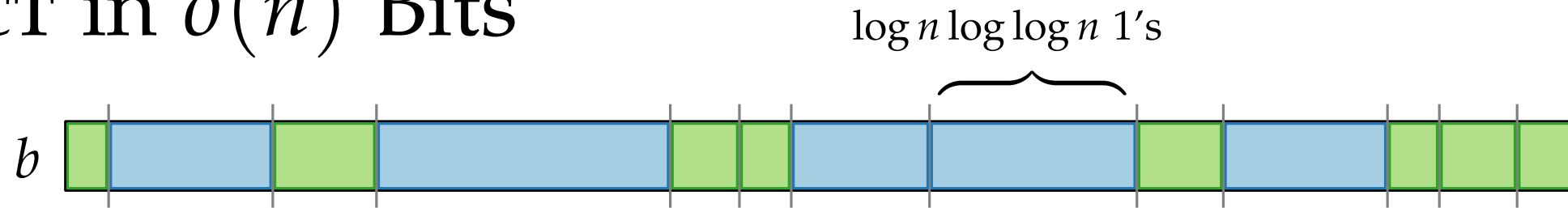
- wenn $r \geq (\log n \log \log n)^2$

$$\Rightarrow \mathcal{O}\left(\frac{n}{(\log n \log \log n)^2} (\log n \log \log n) \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

sonst Problem ist reduziert zu Bitstrings der Länge $r < (\log n \log \log n)^2$

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

1. Speichere Indizes von jedem $(\log n \log \log n)$ -ten 1-Bit in Array

$$\Rightarrow \mathcal{O}\left(\frac{n}{\log n \log \log n} \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \subseteq o(n) \text{ Bits}$$

2. Innerhalb **Stück** von $(\log n \log \log n)$ 1-Bits mit insgesamt r Bits:

wenn $r \geq (\log n \log \log n)^2$

dann speichere Indizes der 1-Bits in **Stück** in Array

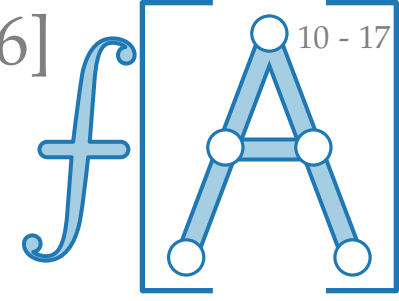
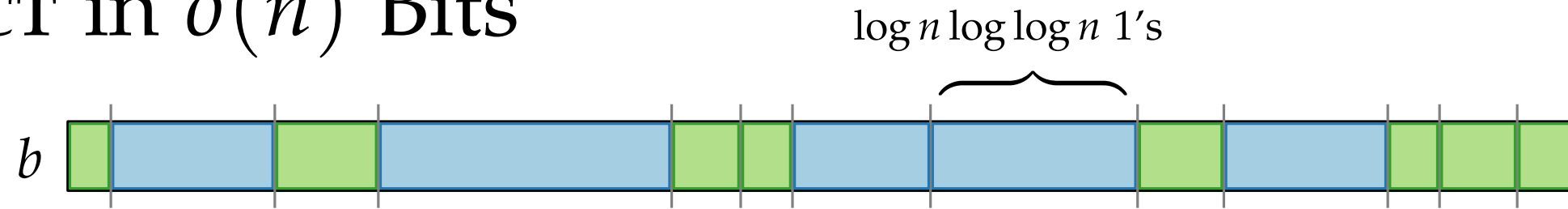
$$\Rightarrow \mathcal{O}\left(\frac{n}{(\log n \log \log n)^2} (\log n \log \log n) \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

sonst Problem ist reduziert zu Bitstrings der Länge $r < (\log n \log \log n)^2$

3. Wiederhole 1. and 2. auf reduzierten Bitstrings

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]

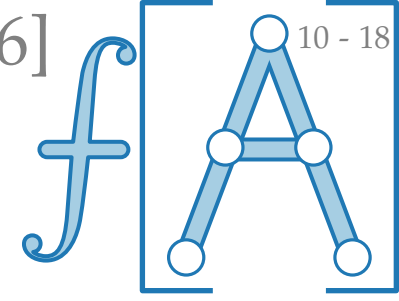
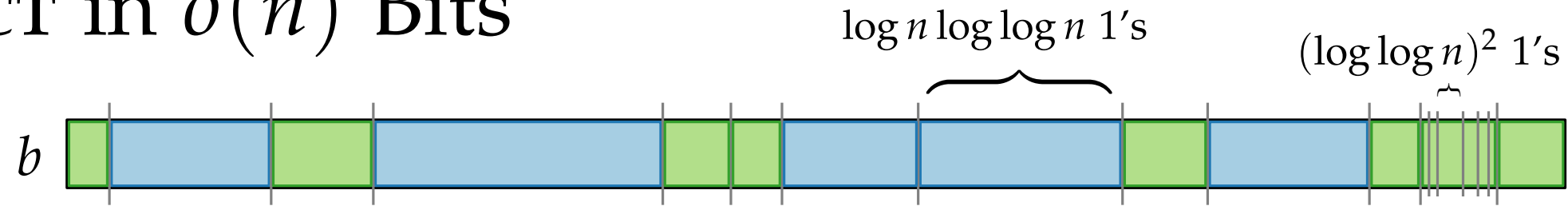


nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



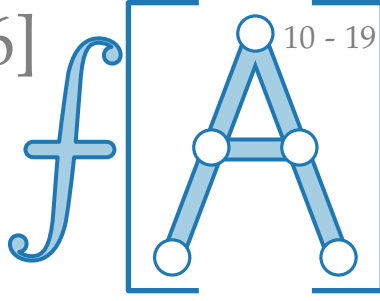
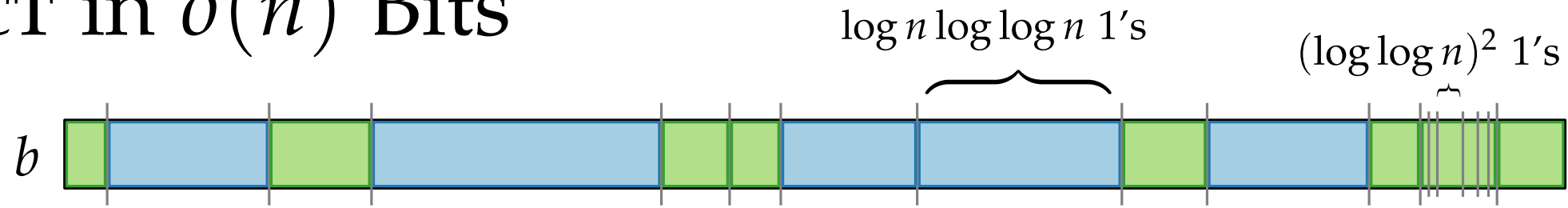
nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

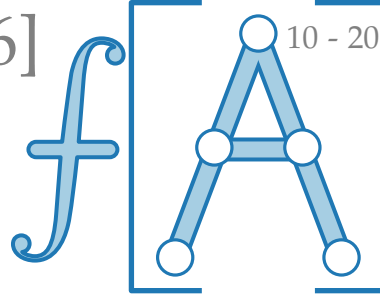
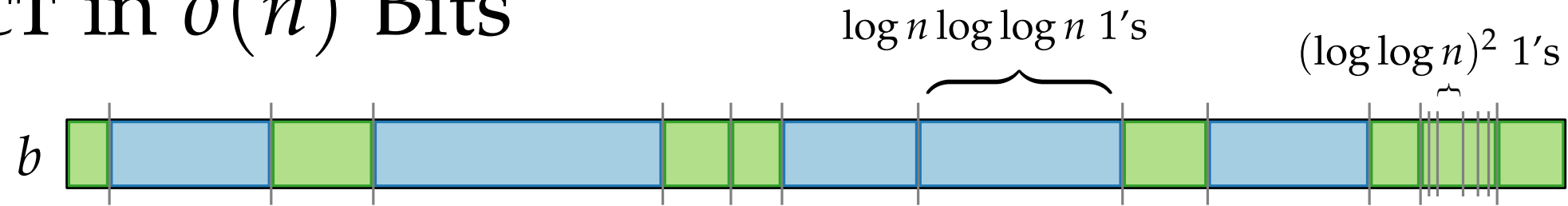
1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\underbrace{\quad}_{\# \text{ Teilstücke }} \underbrace{\quad}_{\text{rel. index }}\right)$$

Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

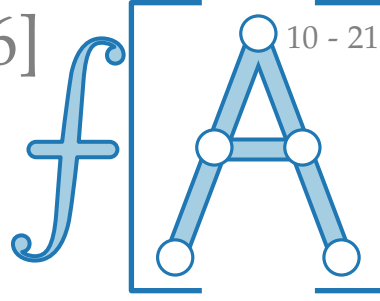
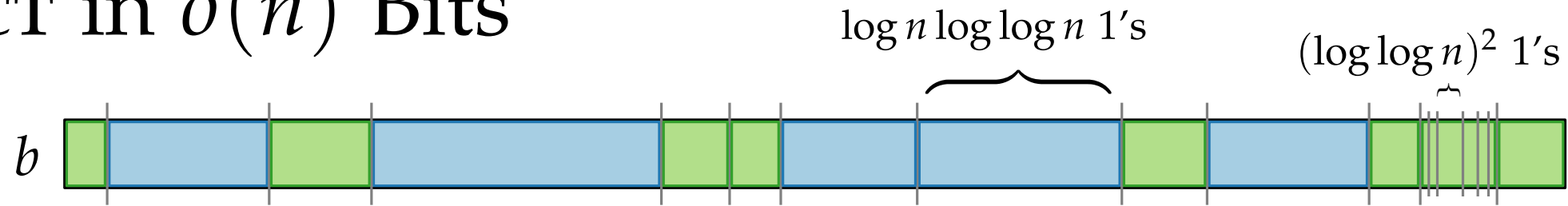
1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\underbrace{\frac{n}{(\log \log n)^2}}_{\text{\# Teilstücke}} \underbrace{\quad}_{\text{rel. index}}\right)$$

Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

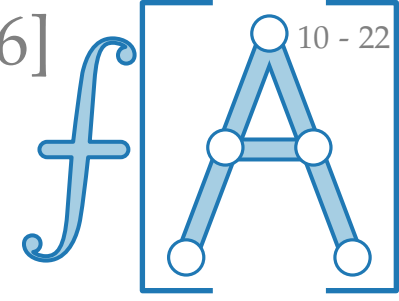
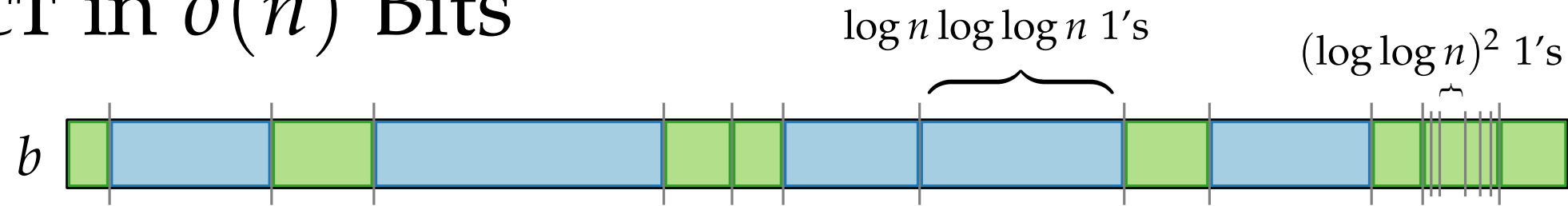
1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\underbrace{\frac{n}{(\log \log n)^2}}_{\text{\# Teilstücke}} \underbrace{\log \log n}_{\text{rel. index}}\right)$$

Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

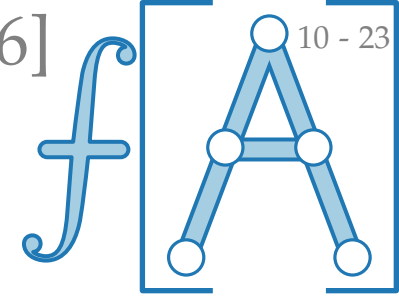
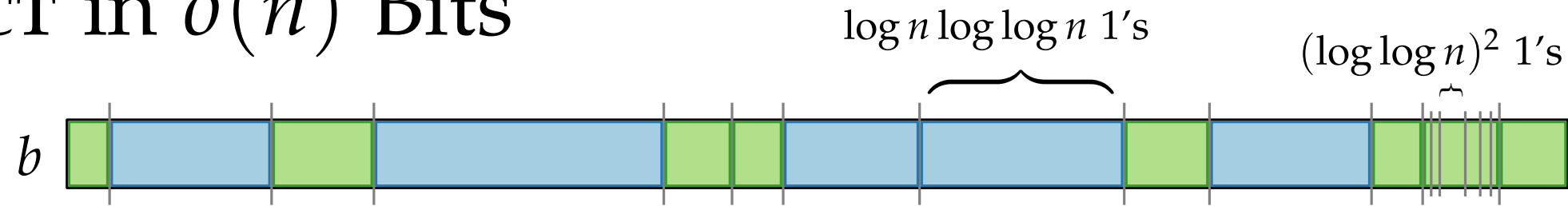
3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \underbrace{\mathcal{O}\left(\frac{n}{(\log \log n)^2}\right)}_{\# \text{ Teilstücke}} \underbrace{\log \log n}_{\text{rel. index}} = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

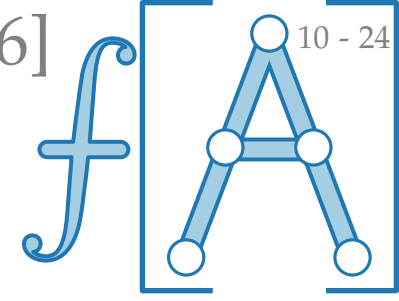
3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^2} \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

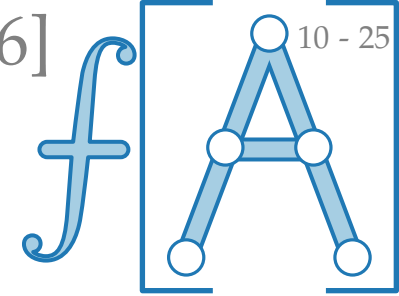
[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):
- 1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array
 $\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^2} \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right)$ Bits
 - 2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:
wenn $r' \geq (\log \log n)^4$
dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

- 1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

wenn $r' \geq (\log \log n)^4$

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

Bits

nicht prüfungsrelevant



nicht prüfungsrelevant

nicht prüfungsrelevant

nicht prüfungsrelevant

nicht prüfungsrelevant

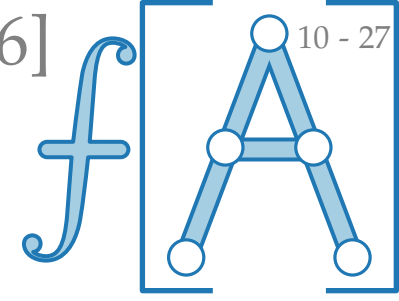
nicht prüfungsrelevant

nicht prüfungsrelevant

nicht prüfungsrelevant

nicht prüfungsrelevant

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

- 1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

wenn $r' \geq (\log \log n)^4$

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

$$\rightarrow \mathcal{O}\left(\underbrace{\frac{n}{(\log \log n)^4}}_{\# \text{ Teilgruppen}} \underbrace{(\log \log n)^2}_{\# \text{ 1-Bits}} \underbrace{1}_{\text{rel. index}}\right) : \text{Bits}$$

nicht prüfungsrelevant



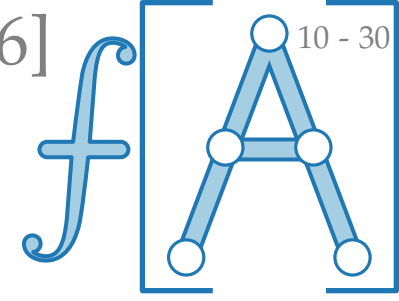
- # Bits

nicht prüfungsrelevant



- $$\rightarrow \mathcal{O}\left(\underbrace{\frac{n}{(\log \log n)^4}}_{\# \text{ Teilgruppen}} \underbrace{(\log \log n)^2}_{\# \text{ 1-Bits}} \underbrace{\log \log n}_{\text{rel. index}}\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

- 1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

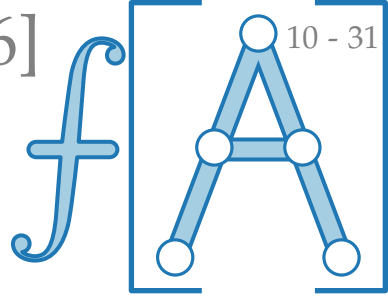
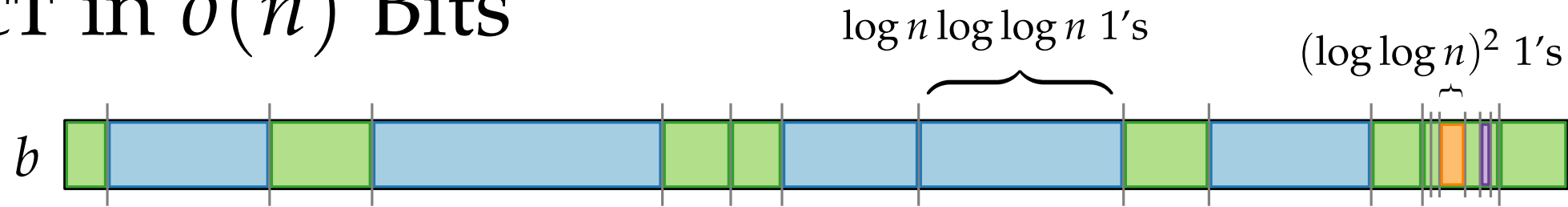
2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

sonst Problem ist reduziert zu Bitstrings der Länge $r' < (\log \log n)^4$

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



10 - 31

nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^2} \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

wenn $r' \geq (\log \log n)^4$

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^4} (\log \log n)^2 \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

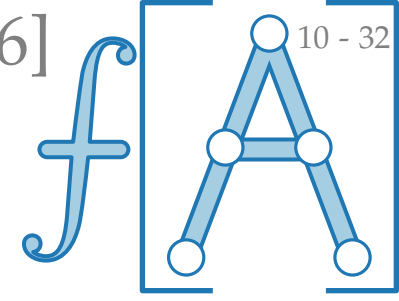
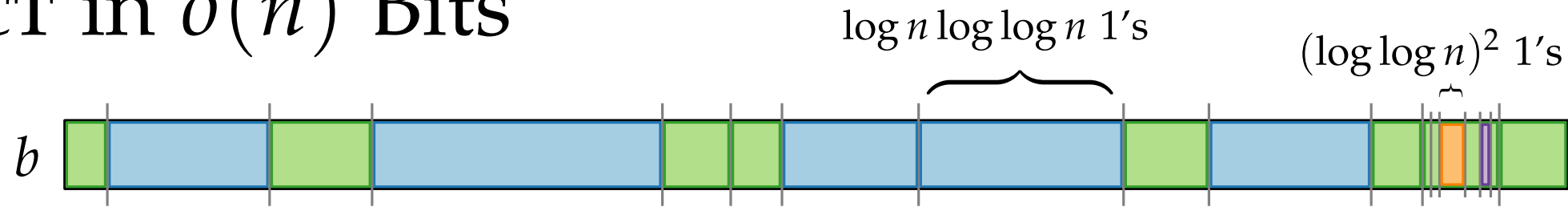
sonst Problem ist reduziert zu Bitstrings der Länge $r' < (\log \log n)^4$

4. Benutze Suchtabelle für Bitstrings der Länge

$$r' \leq (\log \log n)^4 \leq \frac{1}{2} \log n$$

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^2} \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

wenn $r' \geq (\log \log n)^4$

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^4} (\log \log n)^2 \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

sonst Problem ist reduziert zu Bitstrings der Länge $r' < (\log \log n)^4$

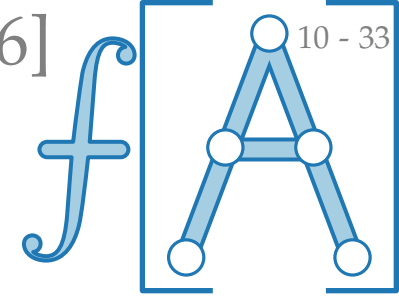
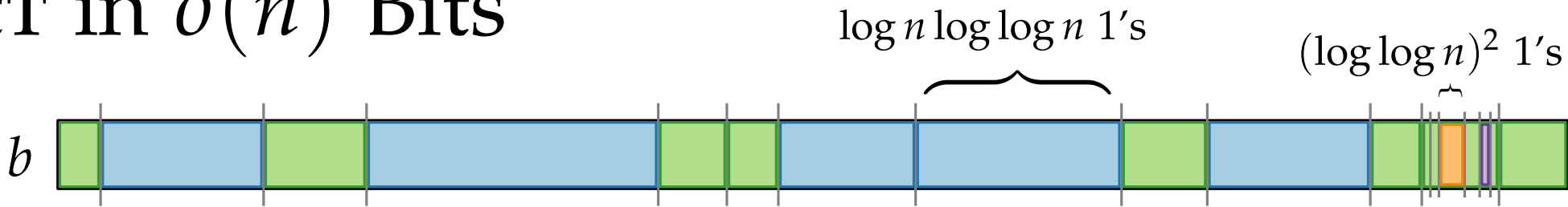
4. Benutze Suchtabelle für Bitstrings der Länge

$$r' \leq (\log \log n)^4 \leq \frac{1}{2} \log n$$

$$\rightarrow \mathcal{O}\left(\underbrace{\quad}_{\text{Bitstring}} \underbrace{\quad}_{\text{query } j} \underbrace{\quad}_{\text{Antwort}}\right) \text{ Bits}$$

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^2} \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

wenn $r' \geq (\log \log n)^4$

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^4} (\log \log n)^2 \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

sonst Problem ist reduziert zu Bitstrings der Länge $r' < (\log \log n)^4$

4. Benutze Suchtabelle für Bitstrings der Länge

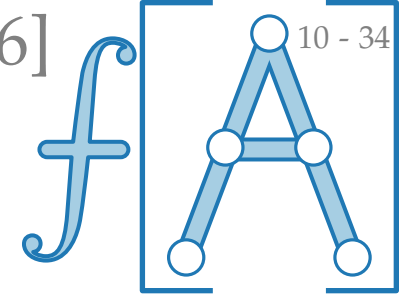
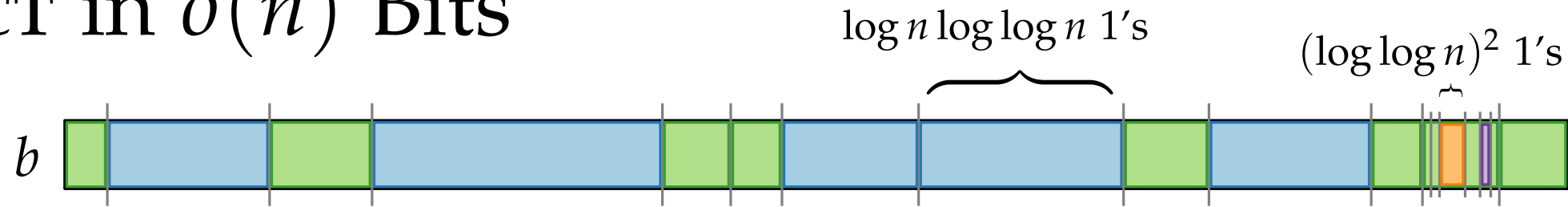
$$r' \leq (\log \log n)^4 \leq \frac{1}{2} \log n$$

$$\rightarrow \mathcal{O}\left(\underbrace{\sqrt{n}}_{\text{Bitstring}} \cdot \underbrace{\quad}_{\text{query } j} \cdot \underbrace{\quad}_{\text{Antwort}}\right)$$

Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^2} \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

wenn $r' \geq (\log \log n)^4$

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^4} (\log \log n)^2 \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

sonst Problem ist reduziert zu Bitstrings der Länge $r' < (\log \log n)^4$

4. Benutze Suchtabelle für Bitstrings der Länge

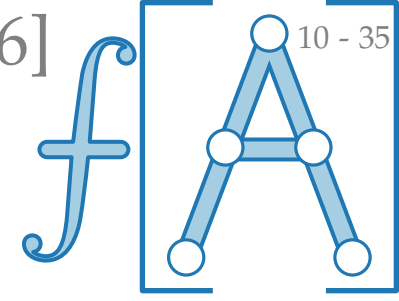
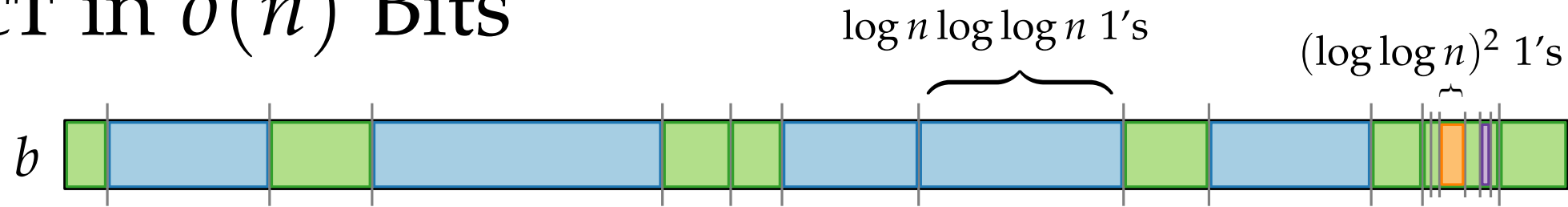
$$r' \leq (\log \log n)^4 \leq \frac{1}{2} \log n$$

$$\rightarrow \mathcal{O}\left(\underbrace{\sqrt{n}}_{\text{Bitstring}} \underbrace{\log n}_{\text{query } j} \underbrace{1}_{\text{Antwort}}\right)$$

Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^2} \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

wenn $r' \geq (\log \log n)^4$

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^4} (\log \log n)^2 \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

sonst Problem ist reduziert zu Bitstrings der Länge $r' < (\log \log n)^4$

4. Benutze Suchtabelle für Bitstrings der Länge

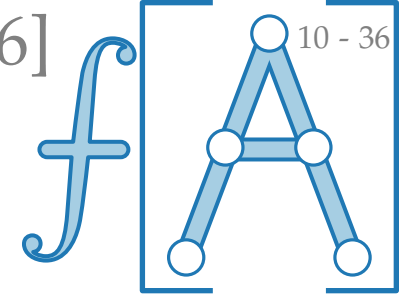
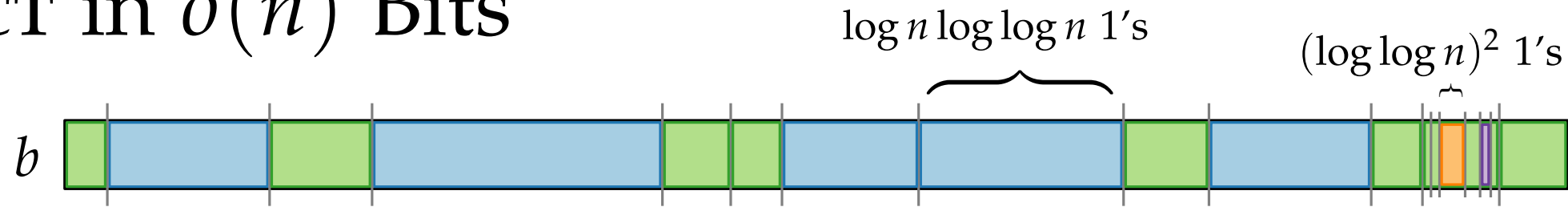
$$r' \leq (\log \log n)^4 \leq \frac{1}{2} \log n$$

$$\rightarrow \mathcal{O}\left(\underbrace{\sqrt{n}}_{\text{Bitstring}} \underbrace{\log n}_{\text{query } j} \underbrace{\log \log n}_{\text{Antwort}}\right)$$

Bits

SELECT in $o(n)$ Bits

[Clark & Munro, SODA 1996]



nicht prüfungsrelevant

3. Wiederhole 1. and 2. auf reduziertem Bitstring ($r < (\log n \log \log n)^2$):

1' Speichere relative Indizes für jedes $(\log \log n)^2$ -te 1-Bit in Array

$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^2} \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

2' Innerhalb **Teilstück** von $(\log \log n)^2$ 1-Bits mit insgesamt r' Bits:

wenn $r' \geq (\log \log n)^4$

dann speichere relative Indizes der 1-Bits in **Teilstück** in Array

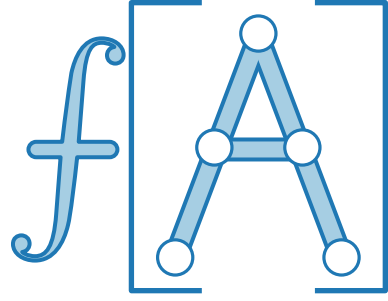
$$\rightarrow \mathcal{O}\left(\frac{n}{(\log \log n)^4} (\log \log n)^2 \log \log n\right) = \mathcal{O}\left(\frac{n}{\log \log n}\right) \text{ Bits}$$

sonst Problem ist reduziert zu Bitstrings der Länge $r' < (\log \log n)^4$

4. Benutze Suchtabelle für Bitstrings der Länge

$$r' \leq (\log \log n)^4 \leq \frac{1}{2} \log n$$

$$\rightarrow \underbrace{\mathcal{O}(\sqrt{n})}_{\text{Bitstring}} \underbrace{\log n}_{\text{query } j} \underbrace{\log \log n}_{\text{Antwort}} = o(n) \text{ Bits}$$

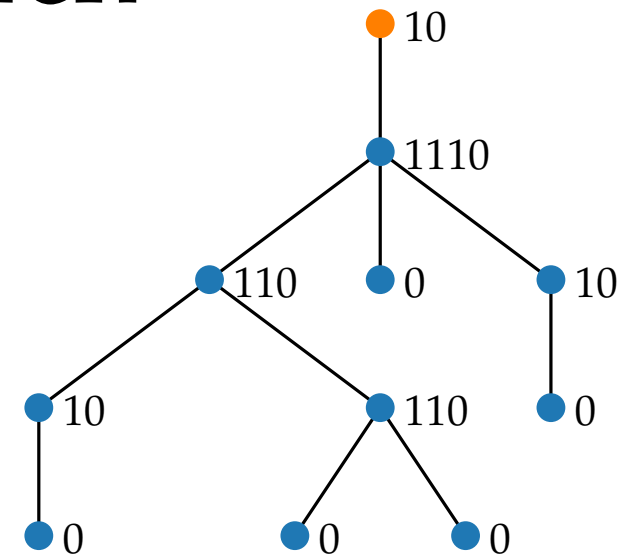


Fortgeschrittene Algorithmen

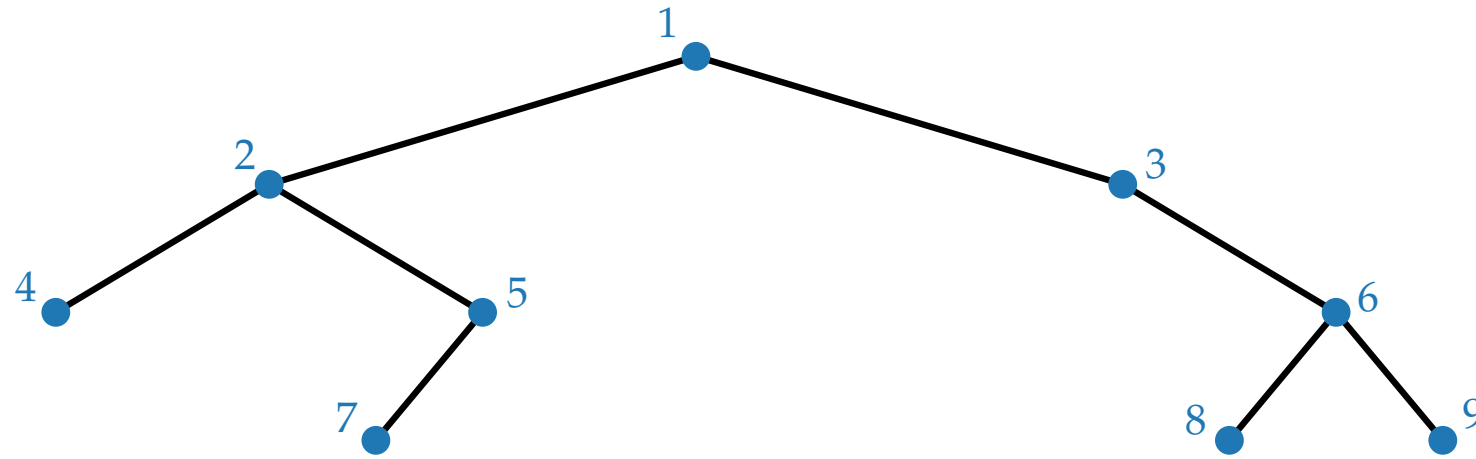
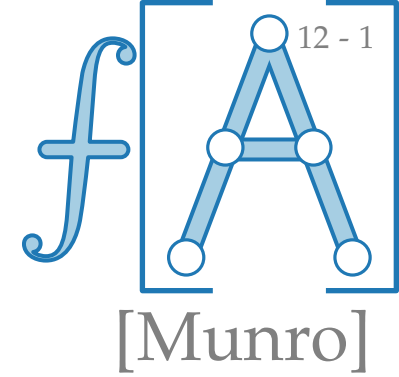
16. Vorlesung Datenstrukturen II:

Platzeffiziente Datenstrukturen (Succinct Data Structures)

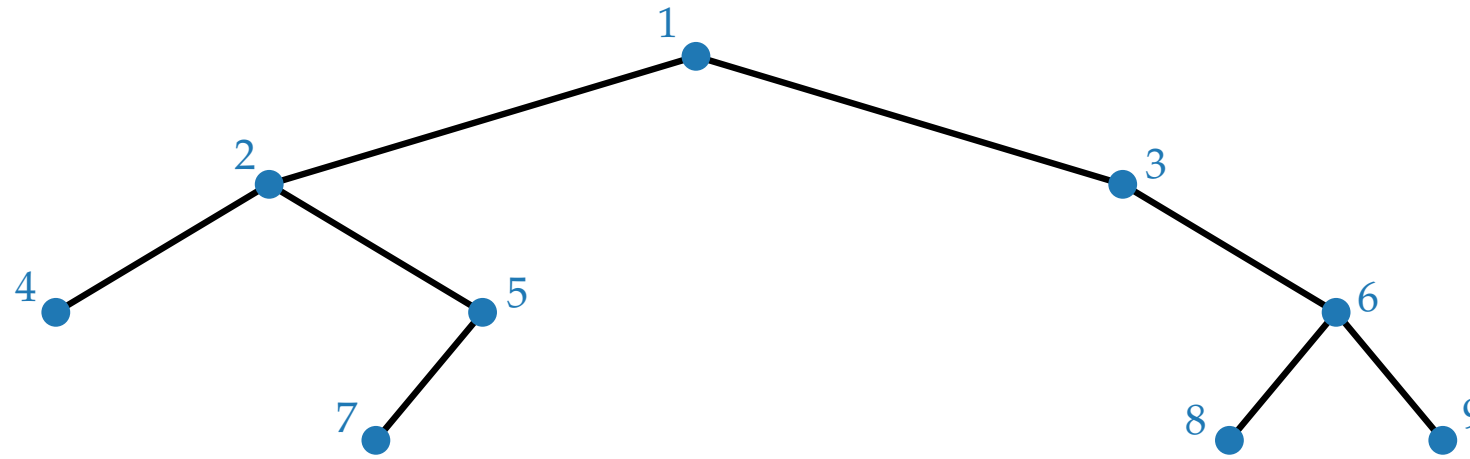
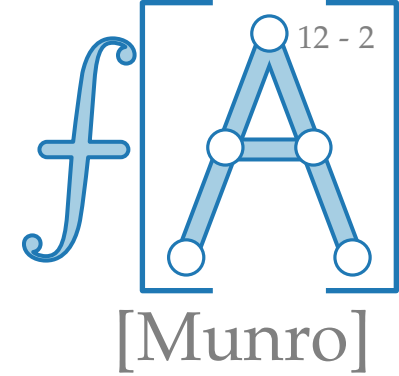
Teil IV: Binärbäume



Platzsparende Repräsentation von Binärbäumen

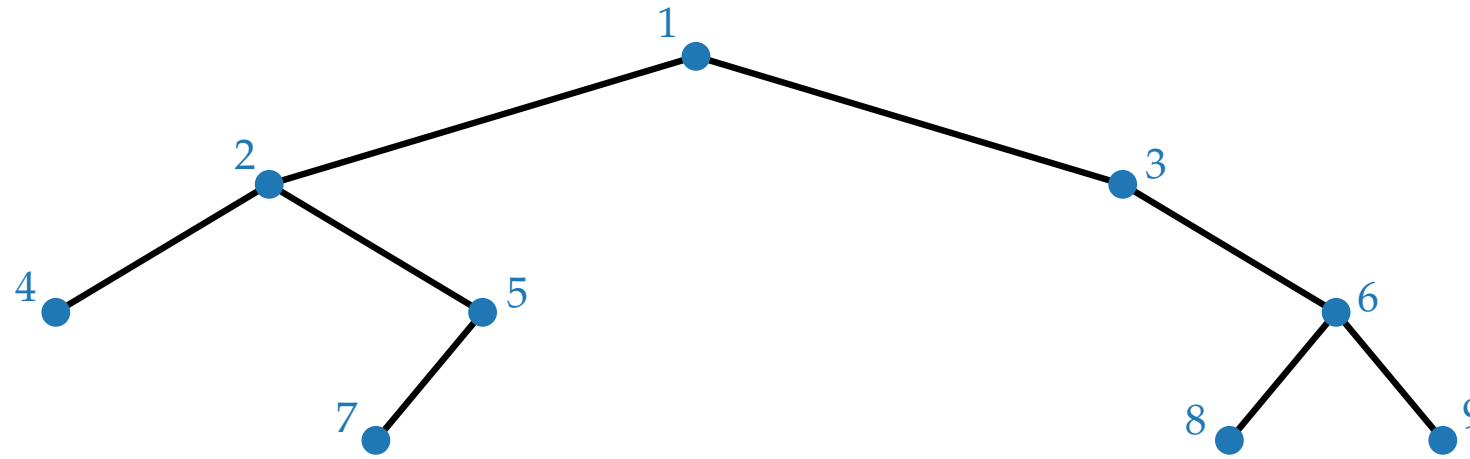
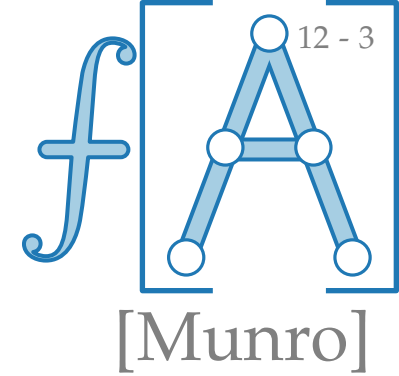


Platzsparende Repräsentation von Binärbäumen



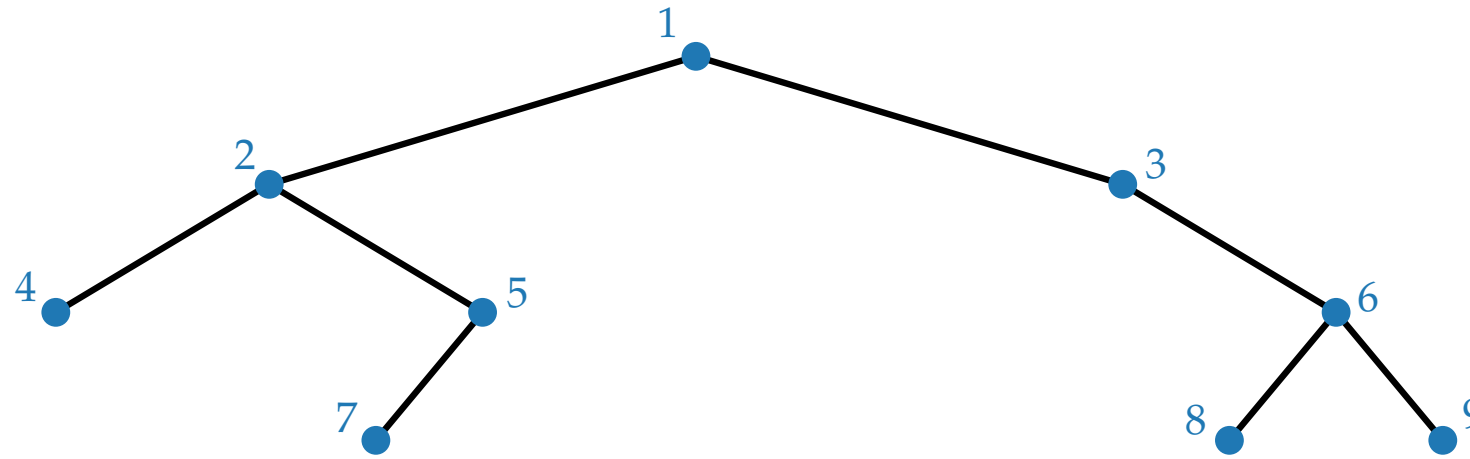
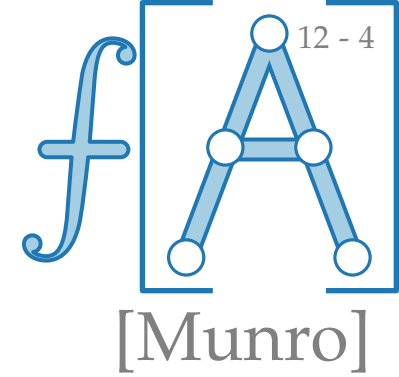
Anzahl der Binärbäume auf n Knoten:

Platzsparende Repräsentation von Binärbäumen



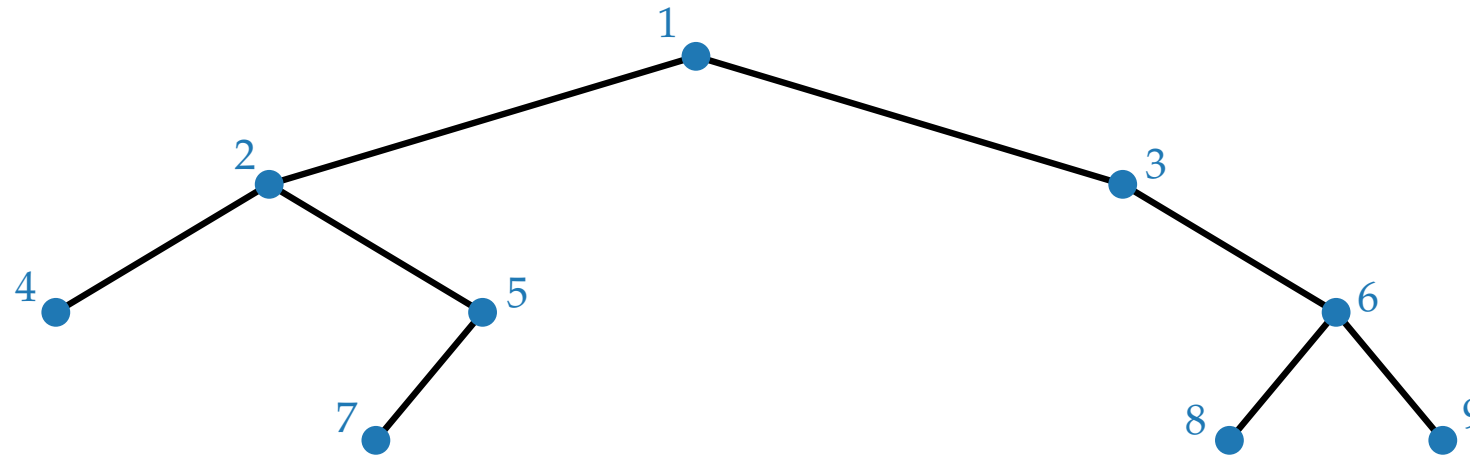
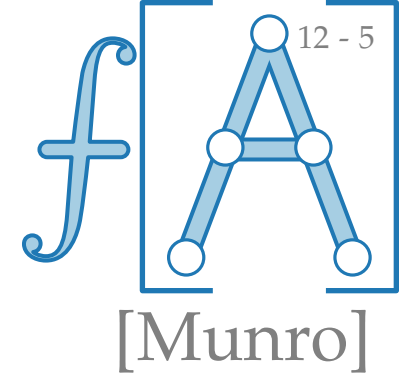
Anzahl der Binärbäume auf n Knoten: C_n

Platzsparende Repräsentation von Binärbäumen



Anzahl der Binärbäume auf n Knoten: $C_n = \frac{1}{n+1} \binom{2n}{n}$

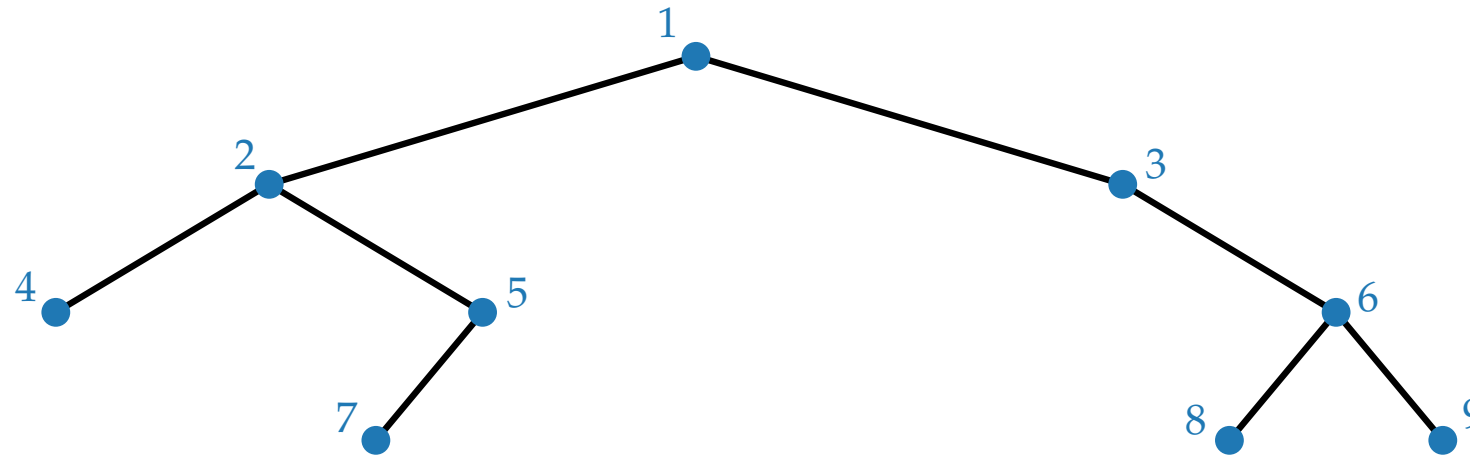
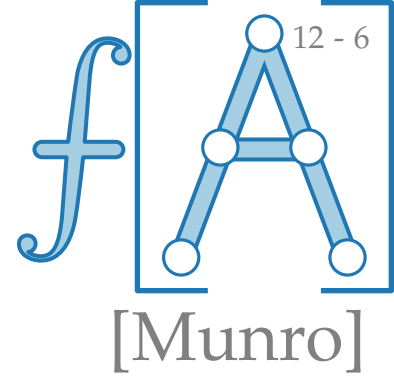
Platzsparende Repräsentation von Binärbäumen



Anzahl der Binärbäume auf n Knoten: $C_n = \frac{1}{n+1} \binom{2n}{n}$

$\log C_n = 2n + o(n)$ (nach Stirlings Approximation)

Platzsparende Repräsentation von Binärbäumen

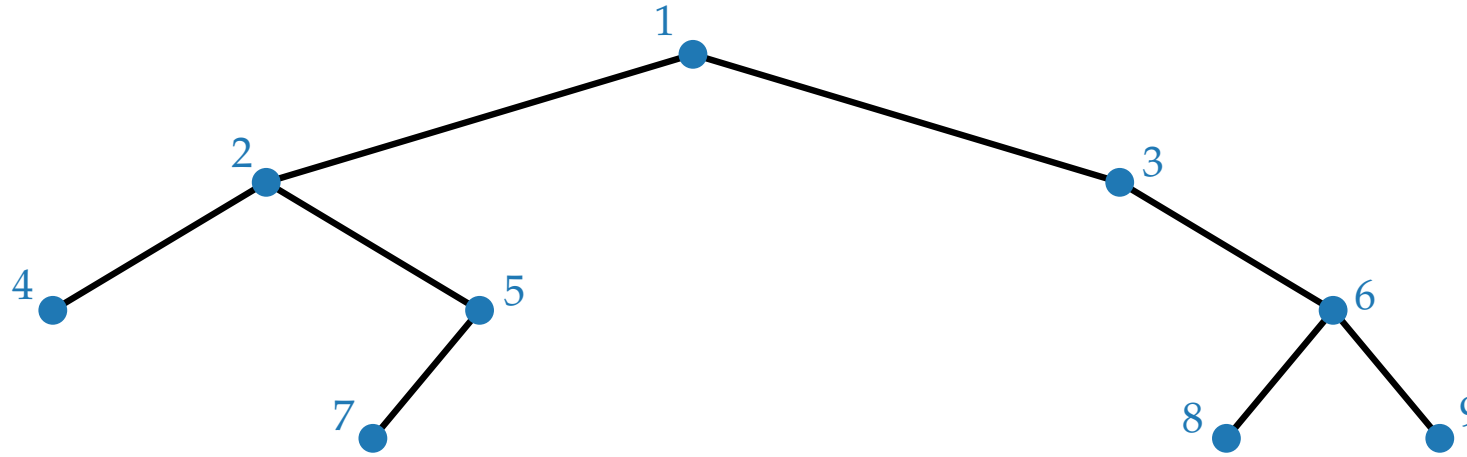
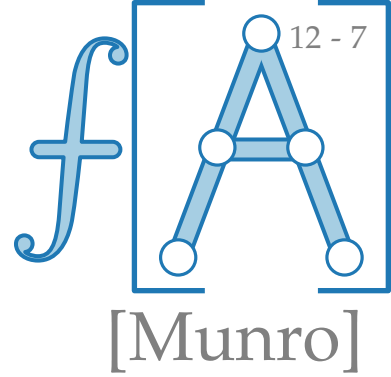


Anzahl der Binärbäume auf n Knoten: $C_n = \frac{1}{n+1} \binom{2n}{n}$

$\log C_n = 2n + o(n)$ (nach Stirlings Approximation)

$\Rightarrow$ Wir können $2n + o(n)$ Bits verwenden, um Binärbäume zu repräsentieren.

Platzsparende Repräsentation von Binärbäumen



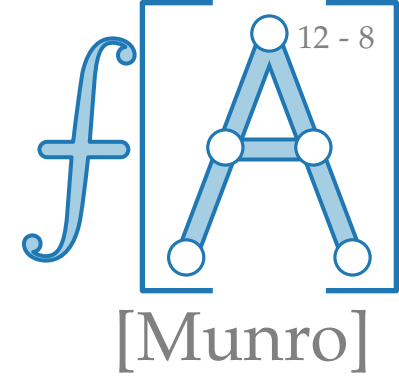
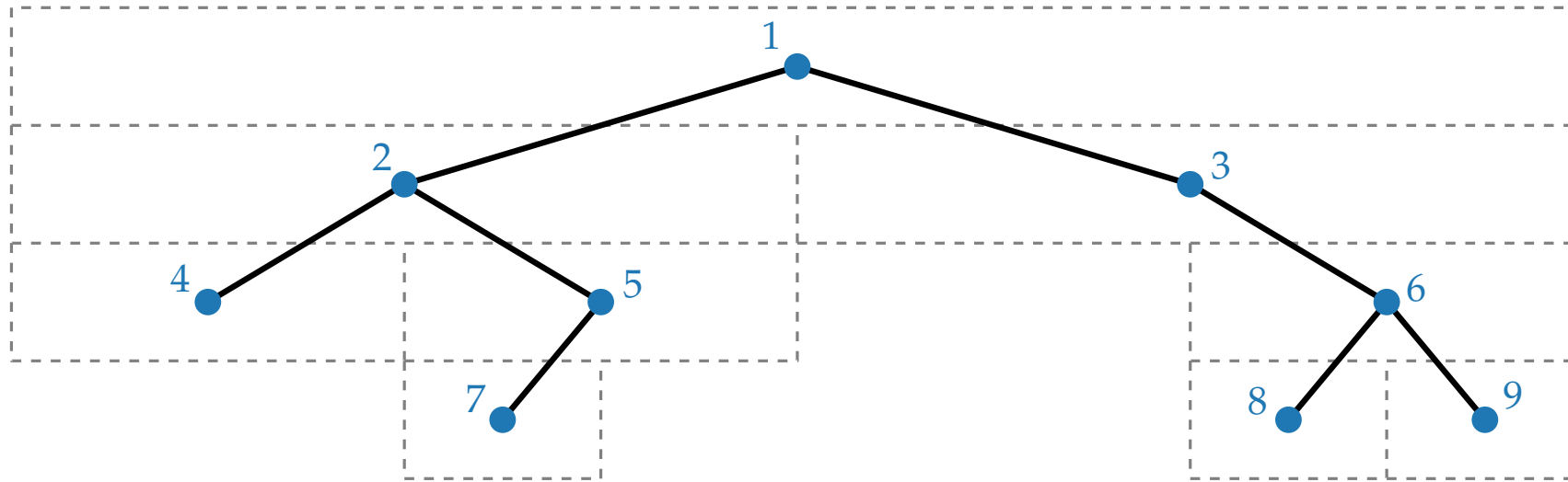
Anzahl der Binärbäume auf n Knoten: $C_n = \frac{1}{n+1} \binom{2n}{n}$

$$\log C_n = 2n + o(n) \text{ (nach Stirlings Approximation)}$$

$\Rightarrow$ Wir können $2n + o(n)$ Bits verwenden, um Binärbäume zu repräsentieren.

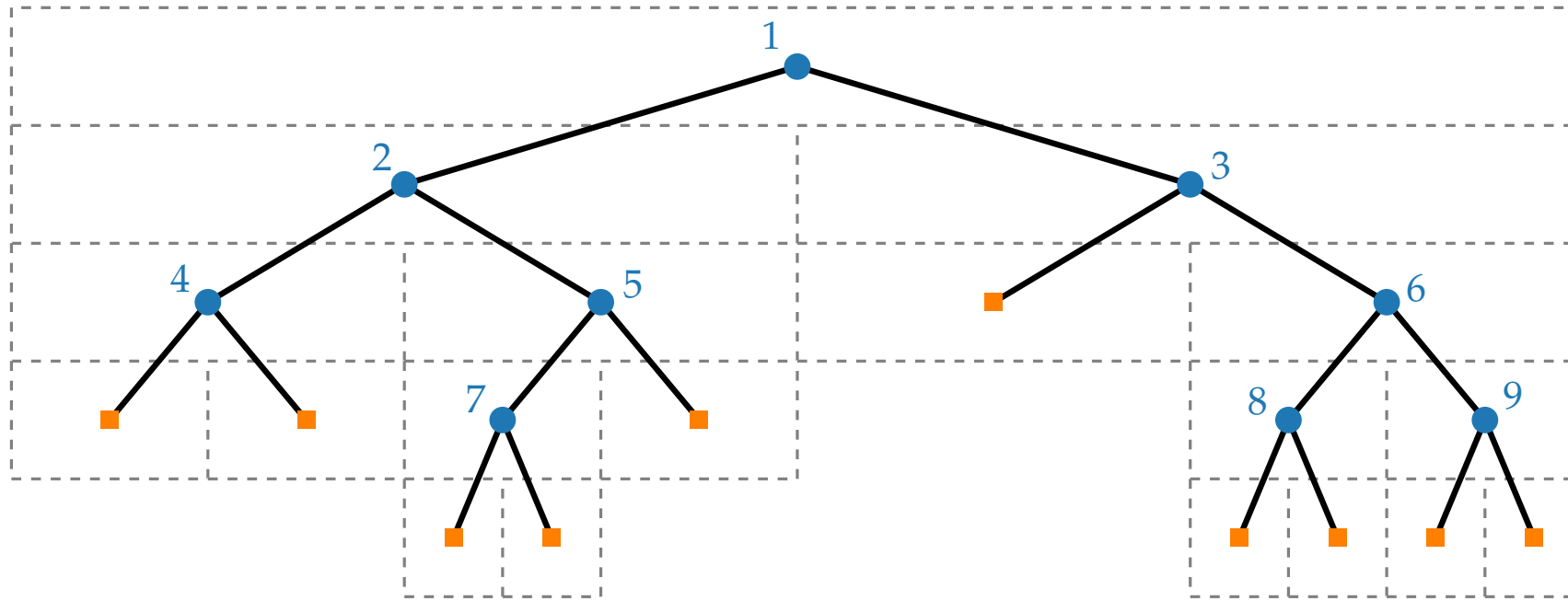
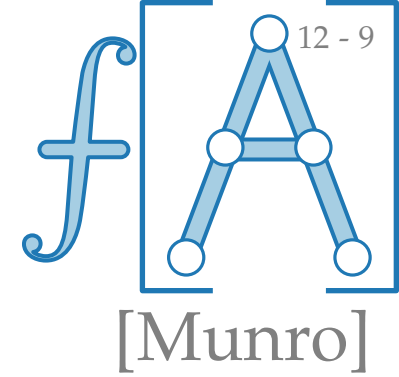
Schwierigkeit ist, wenn Binärbaum nicht voll ist.

Platzsparende Repräsentation von Binärbäumen



Idee.

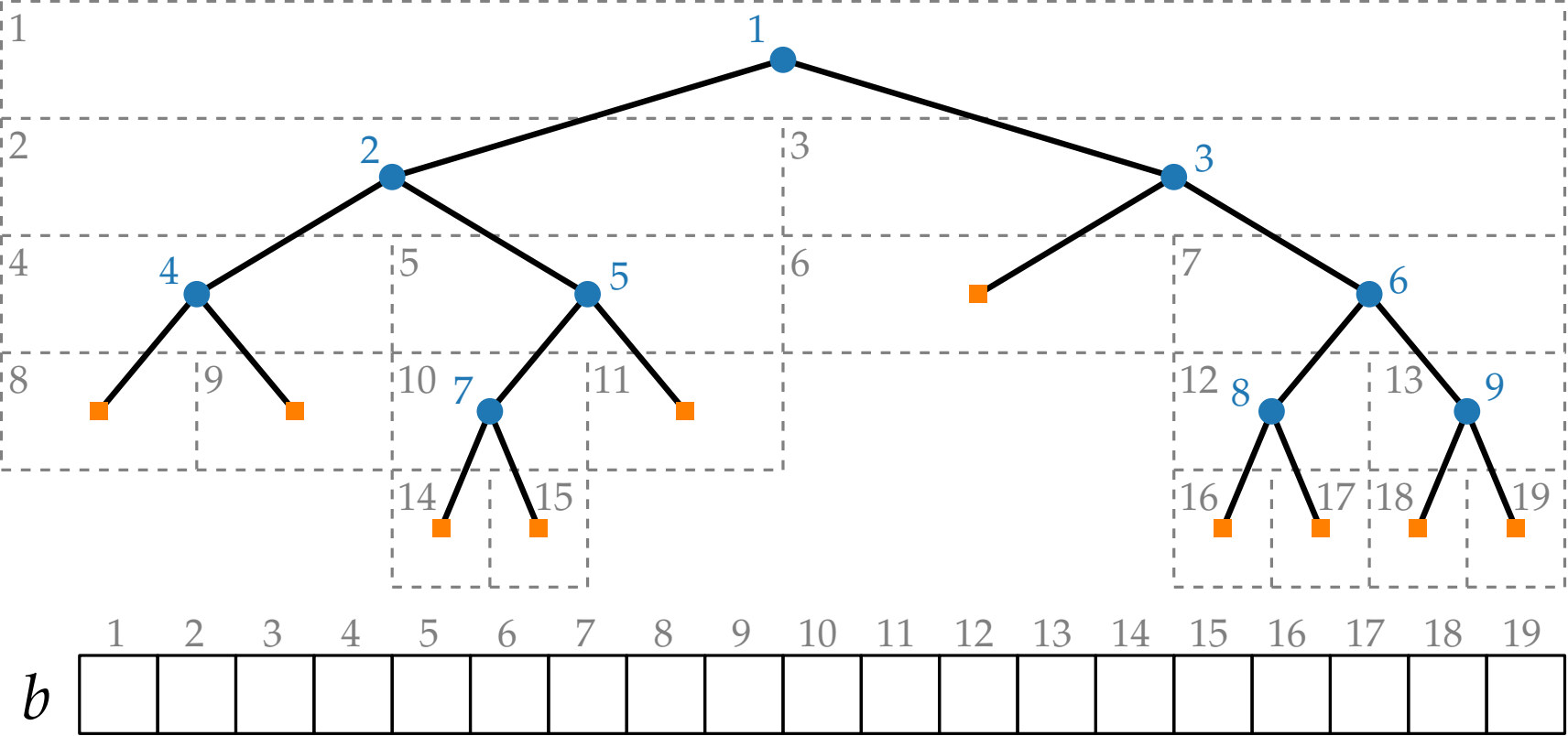
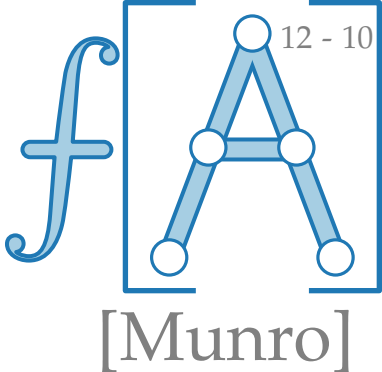
Platzsparende Repräsentation von Binärbäumen



Idee.

- Füge **Dummy Blätter** hinzu

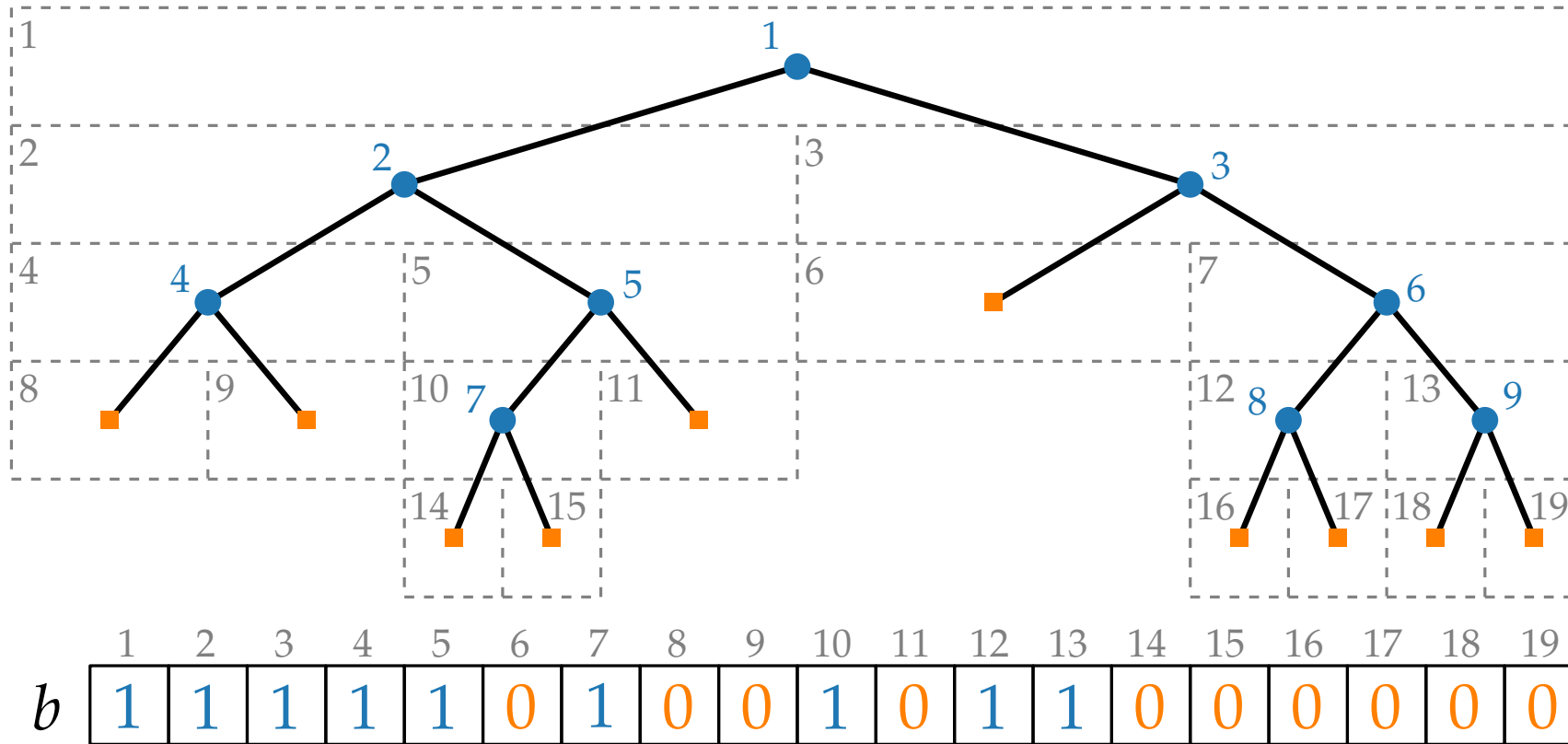
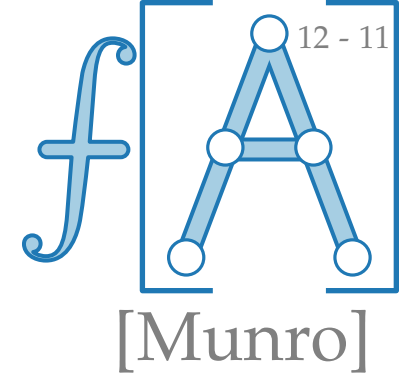
Platzsparende Repräsentation von Binärbäumen



Idee.

■ Füge **Dummy Blätter** hinzu

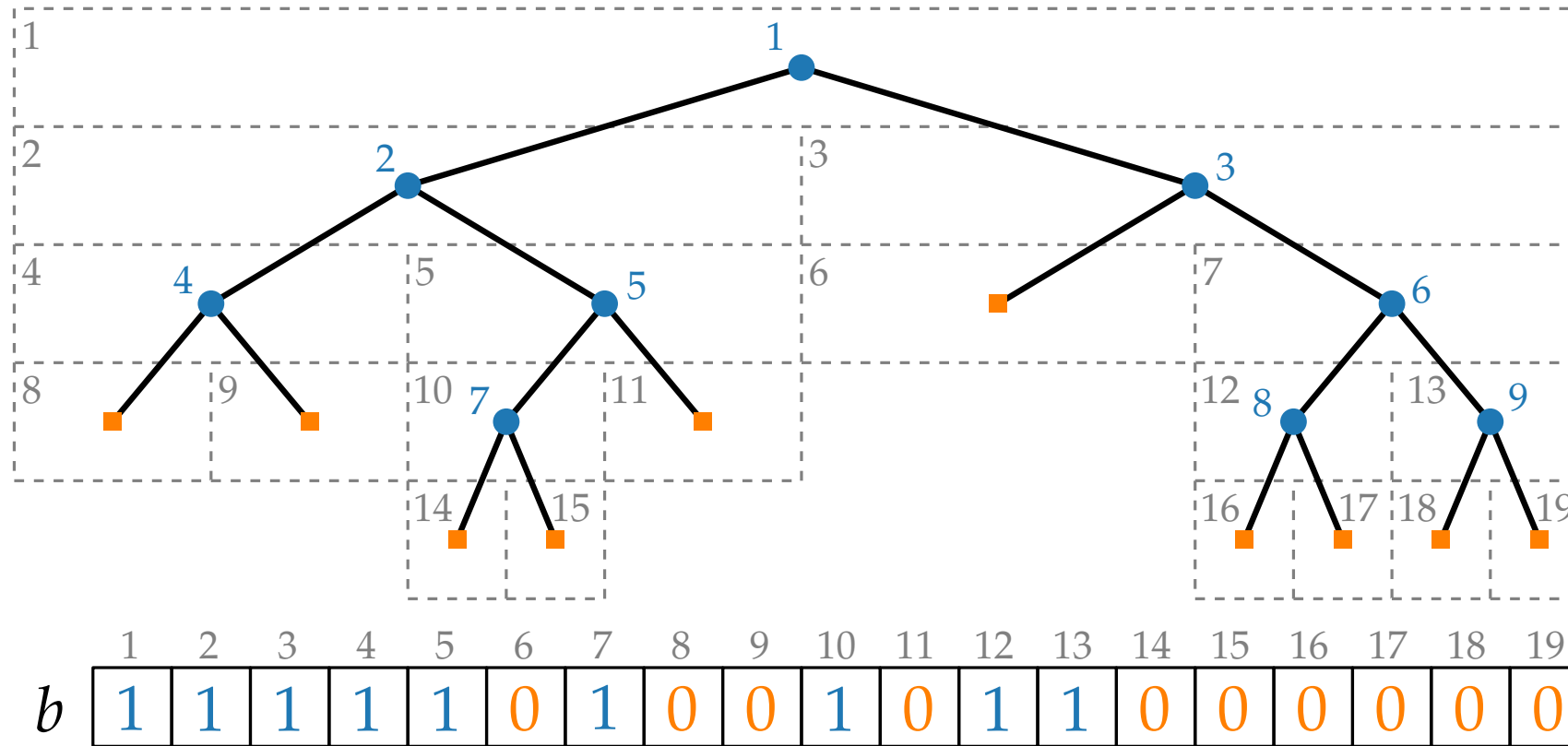
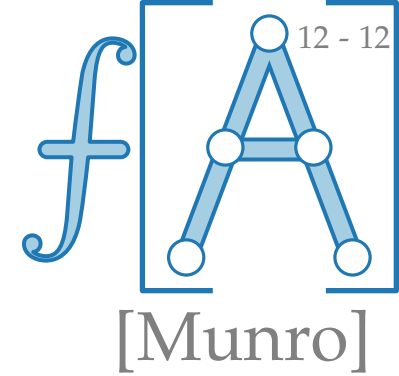
Platzsparende Repräsentation von Binärbäumen



Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0

Platzsparende Repräsentation von Binärbäumen



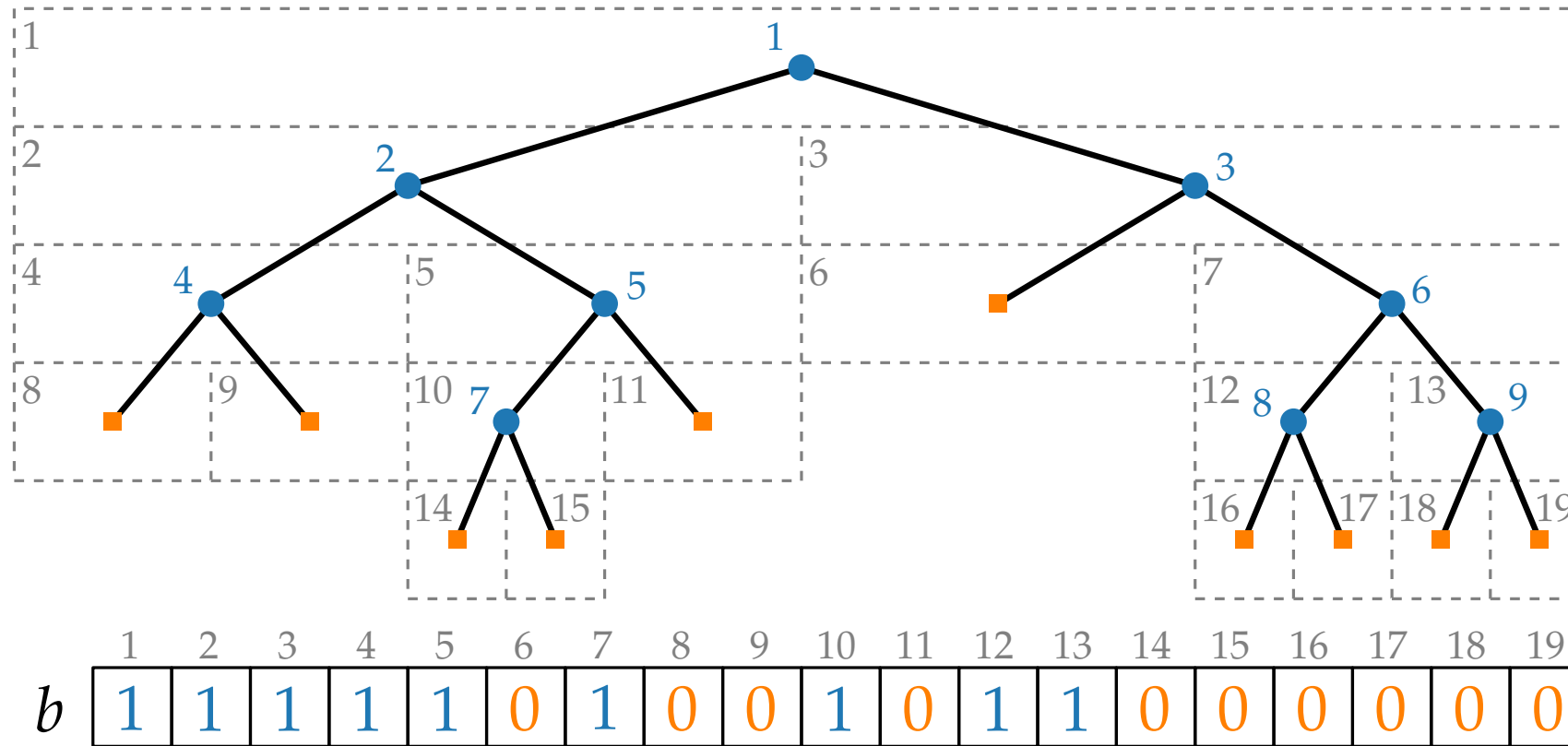
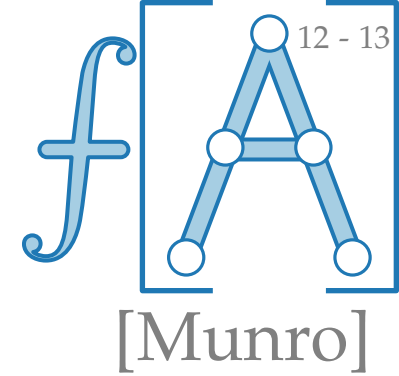
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0

Operationen.

- $\text{PARENT}(i) = ?$
- $\text{LEFTCHILD}(i) = ?$
- $\text{RIGHTCHILD}(i) = ?$

Platzsparende Repräsentation von Binärbäumen



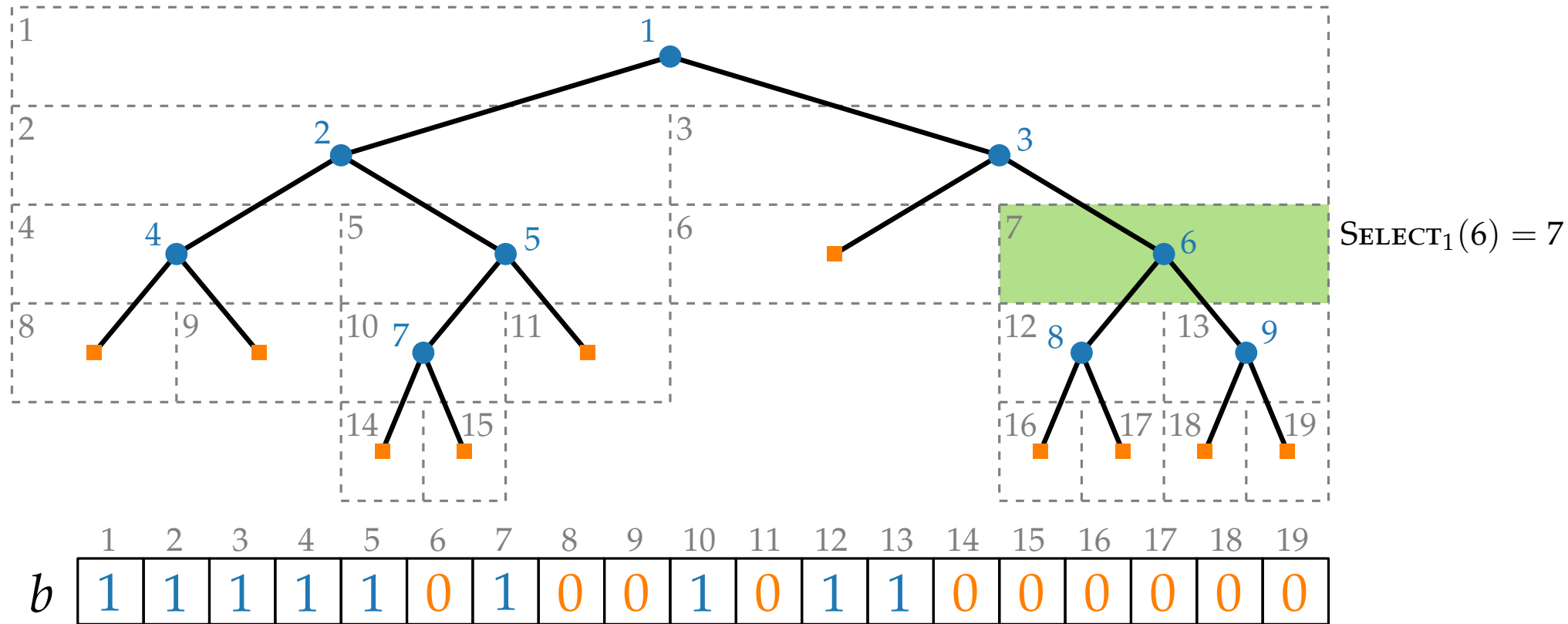
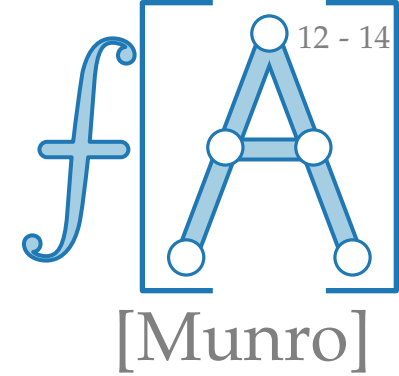
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

Operationen.

- $\text{PARENT}(i) = ?$
- $\text{LEFTCHILD}(i) = ?$
- $\text{RIGHTCHILD}(i) = ?$

Platzsparende Repräsentation von Binärbäumen



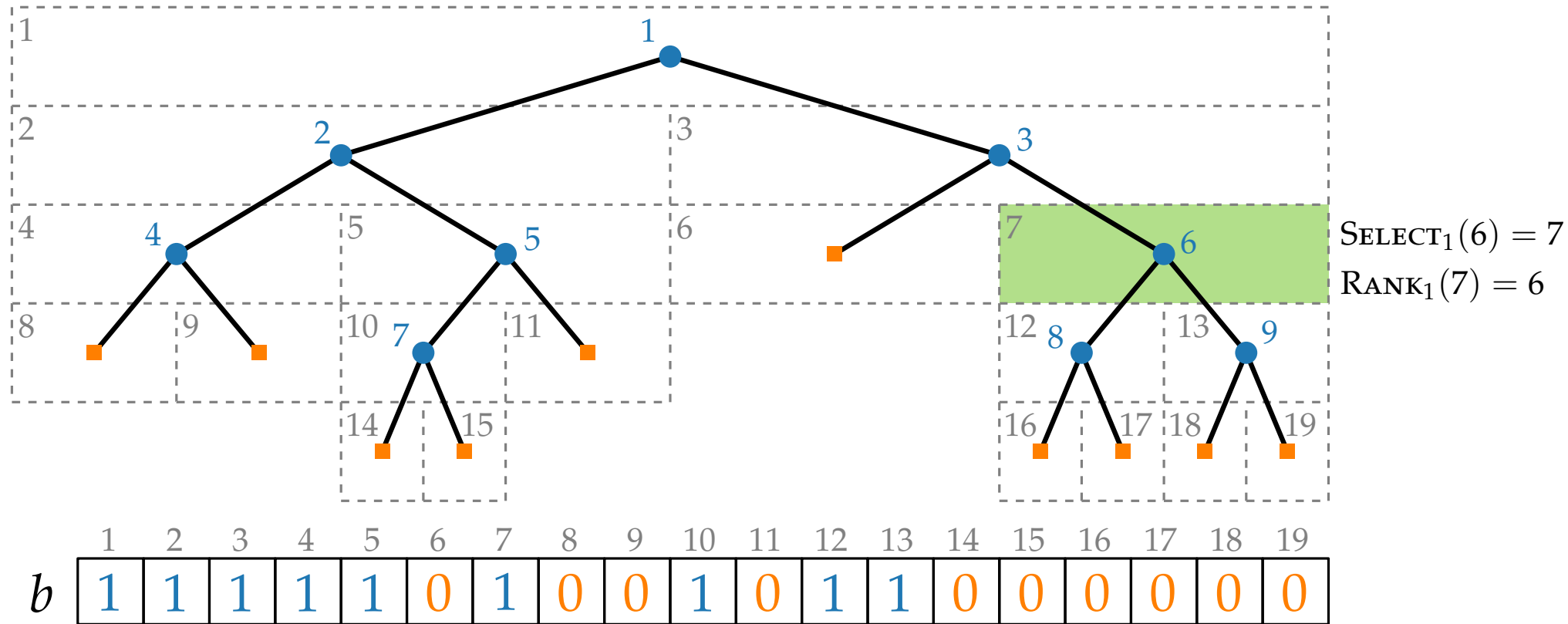
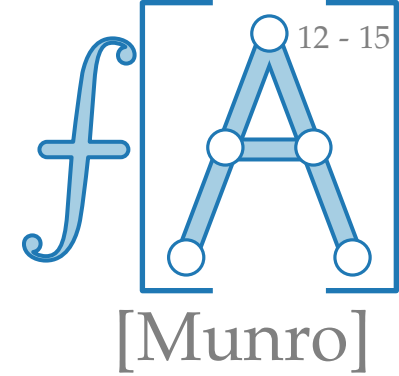
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) = ?
- RIGHTCHILD(i) = ?

Platzsparende Repräsentation von Binärbäumen



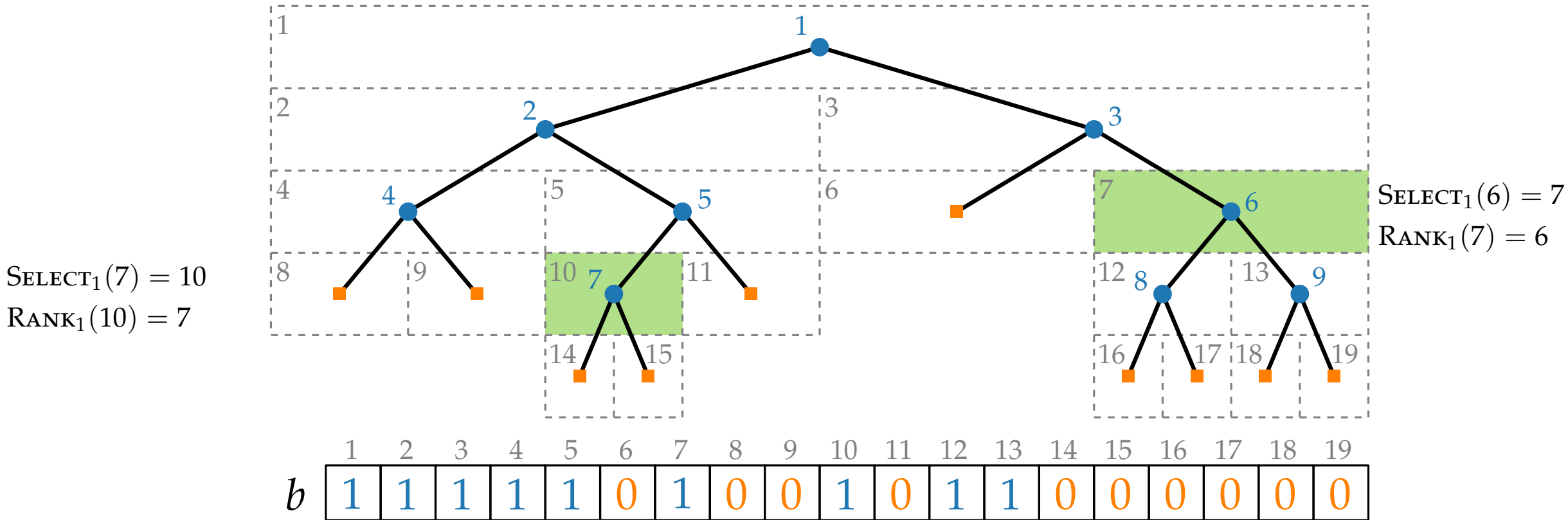
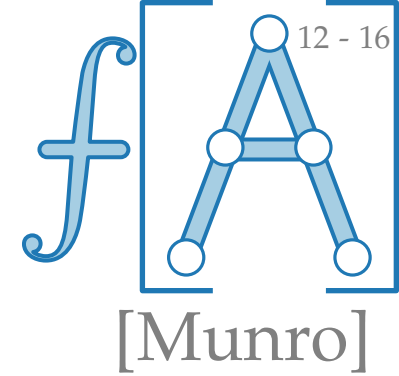
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

Operationen.

- $\text{PARENT}(i) = ?$
- $\text{LEFTCHILD}(i) = ?$
- $\text{RIGHTCHILD}(i) = ?$

Platzsparende Repräsentation von Binärbäumen

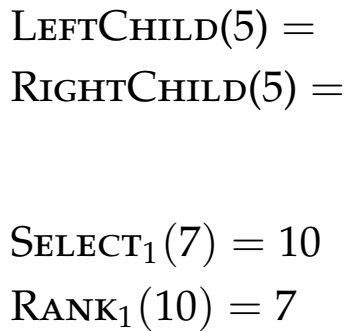


Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

Operationen.

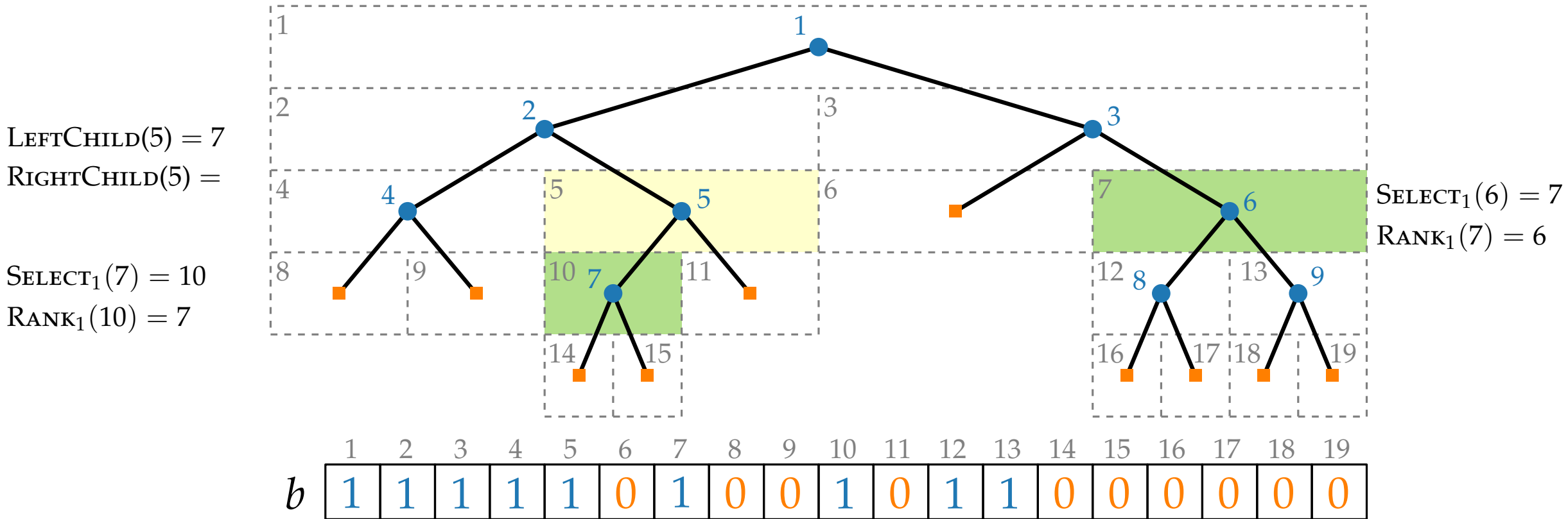
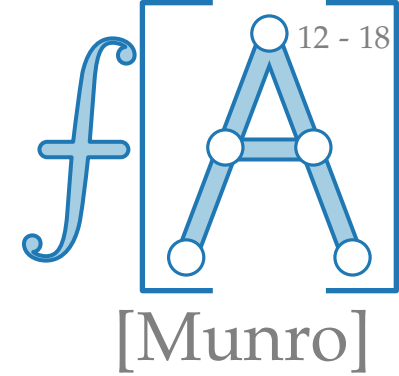
- $\text{PARENT}(i) = ?$
- $\text{LEFTCHILD}(i) = ?$
- $\text{RIGHTCHILD}(i) = ?$



- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze $RANK_1$ und $SELECT_1$

- $\text{PARENT}(i) = ?$
- $\text{LEFTCHILD}(i) = ?$
- $\text{RIGHTCHILD}(i) = ?$

Platzsparende Repräsentation von Binärbäumen



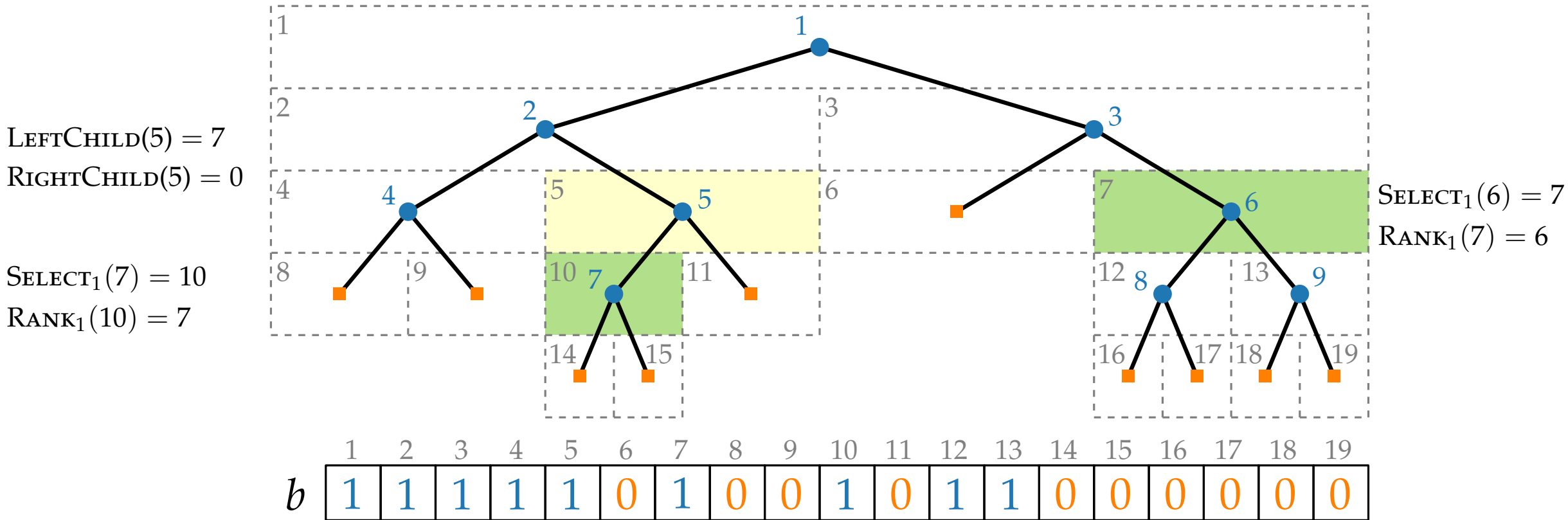
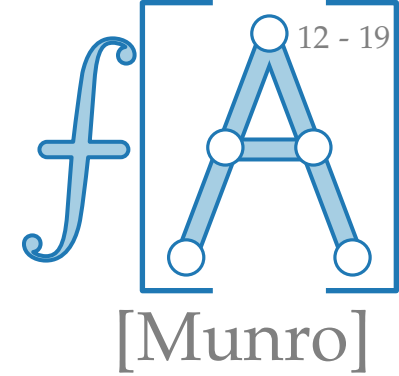
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) = ?
- RIGHTCHILD(i) = ?

Platzsparende Repräsentation von Binärbäumen



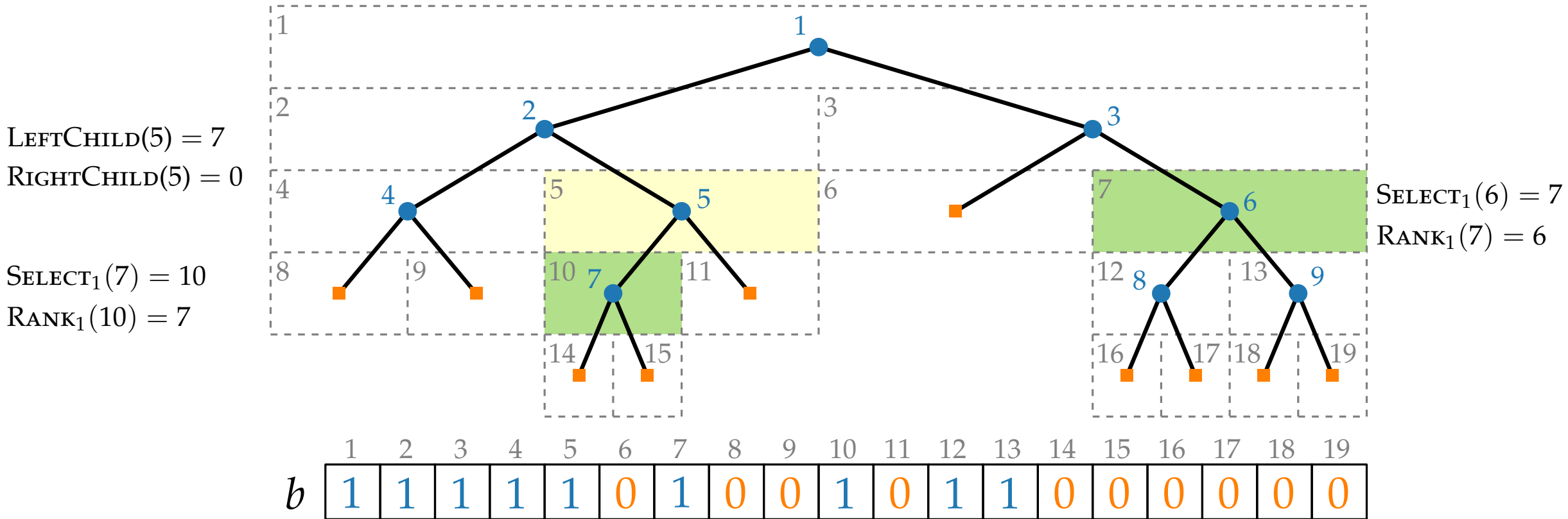
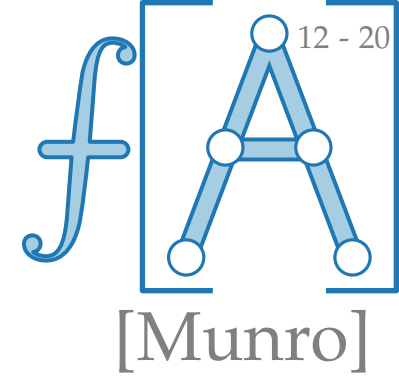
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

Operationen.

- $\text{PARENT}(i) = ?$
- $\text{LEFTCHILD}(i) = ?$
- $\text{RIGHTCHILD}(i) = ?$

Platzsparende Repräsentation von Binärbäumen



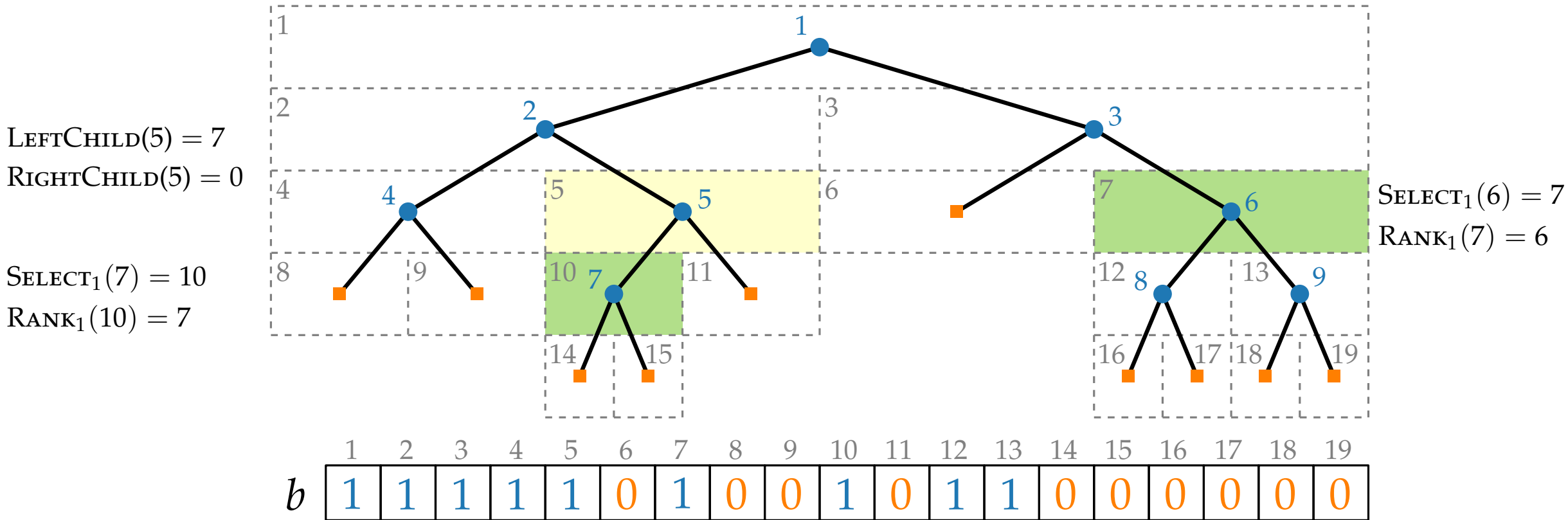
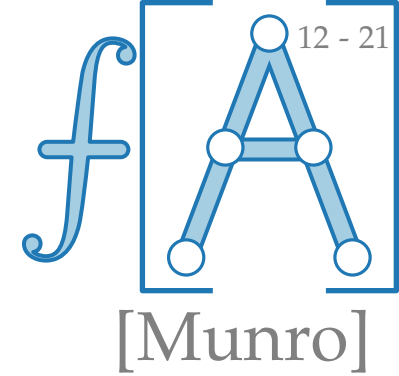
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) = $\left\{ \begin{array}{l} 2i \\ 2i+1 \end{array} \right.$
- RIGHTCHILD(i) = ?

Platzsparende Repräsentation von Binärbäumen



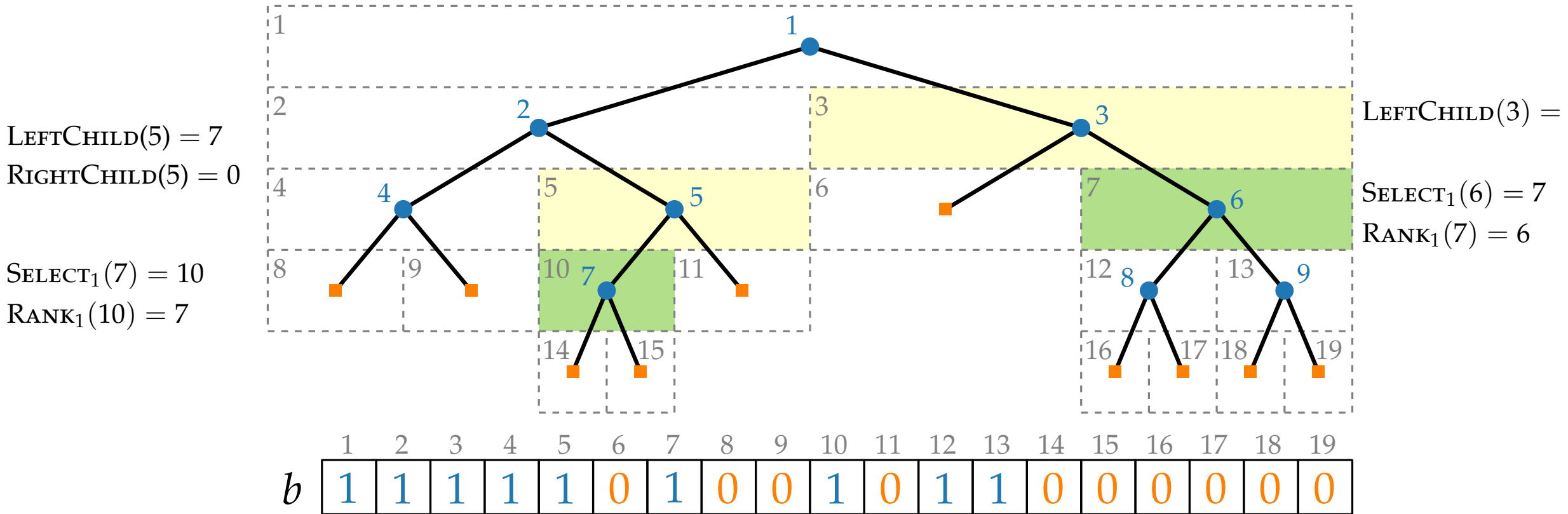
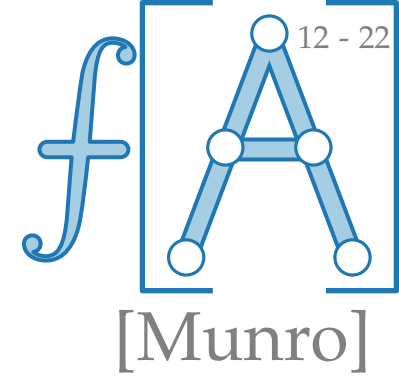
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) = $\begin{cases} \text{RANK}_1(2i) \end{cases}$
- RIGHTCHILD(i) = ?

Platzsparende Repräsentation von Binärbäumen



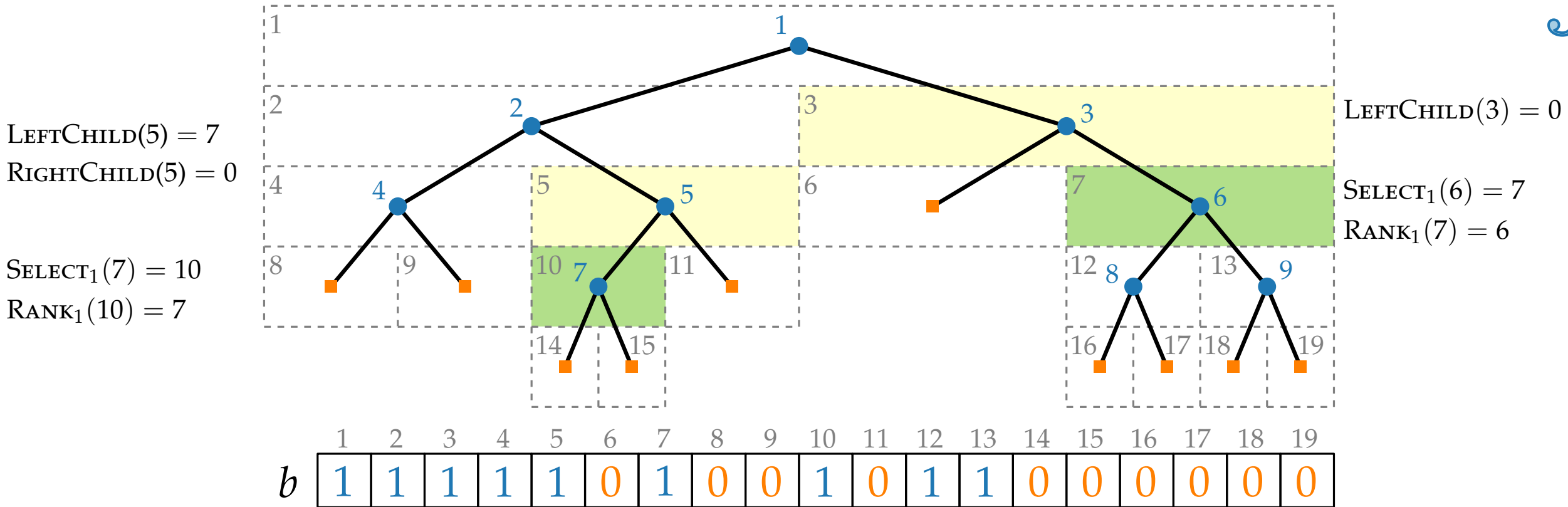
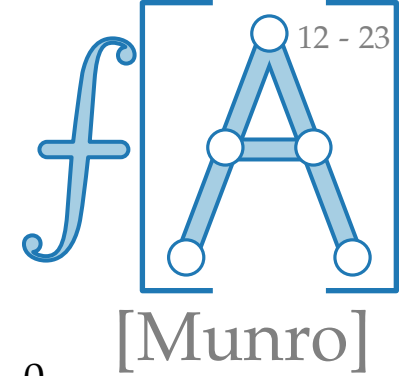
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) = $\begin{cases} \text{RANK}_1(2i) \end{cases}$
- RIGHTCHILD(i) = ?

Platzsparende Repräsentation von Binärbäumen



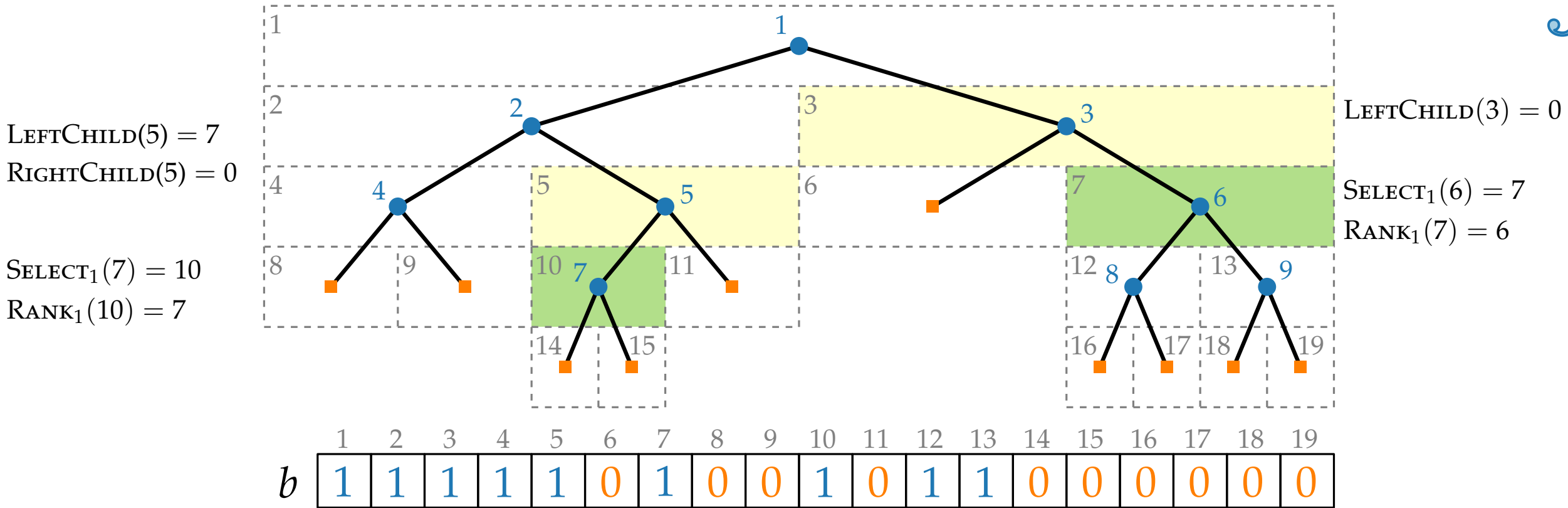
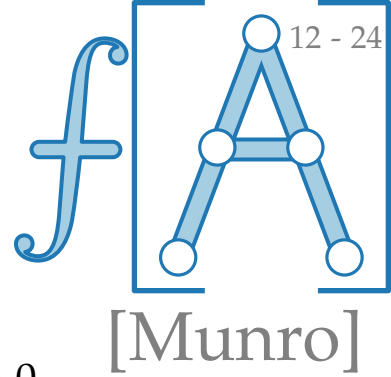
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) = $\begin{cases} \text{RANK}_1(2i) \end{cases}$
- RIGHTCHILD(i) = ?

Platzsparende Repräsentation von Binärbäumen



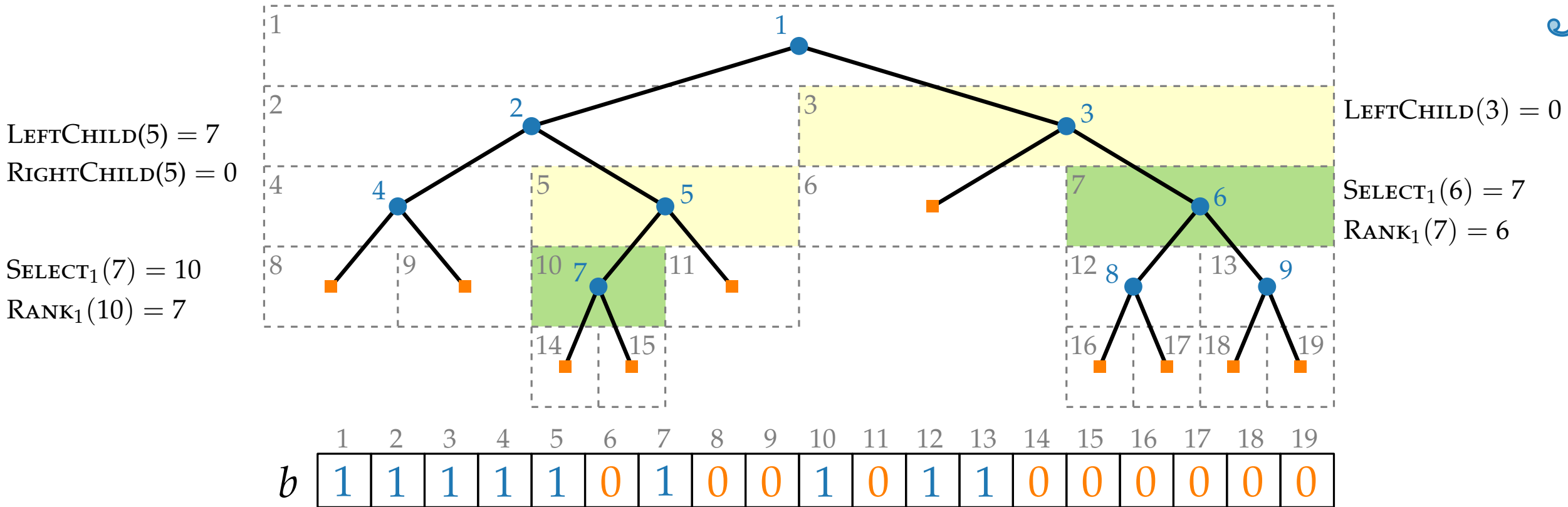
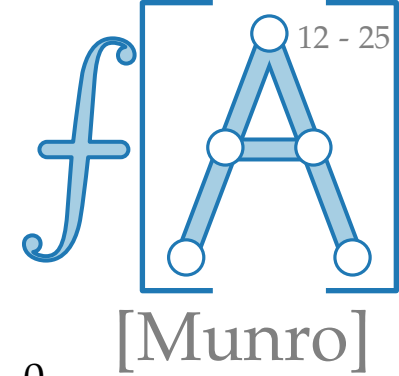
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) = $\begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$
- RIGHTCHILD(i) = ?

Platzsparende Repräsentation von Binärbäumen



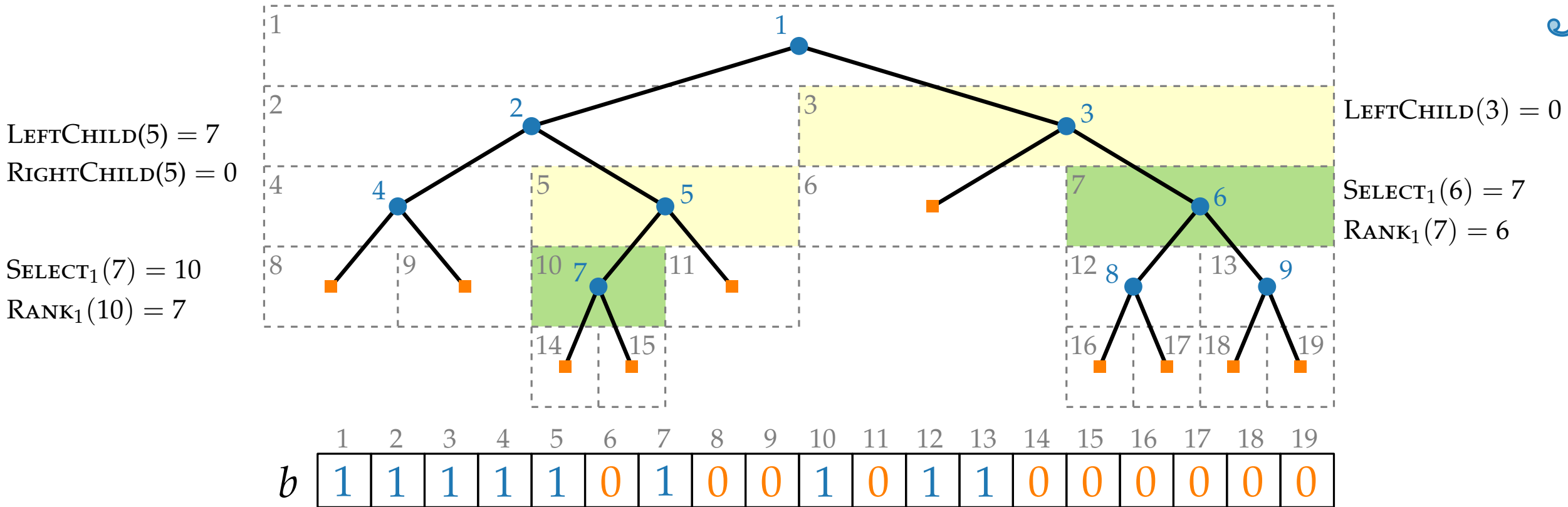
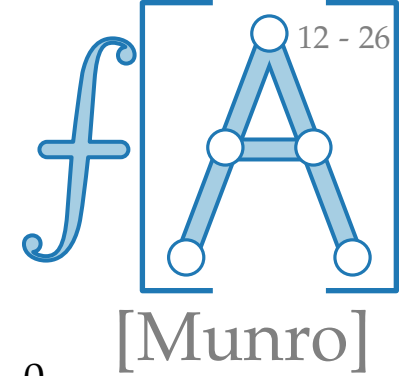
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) =
$$\begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$$
- RIGHTCHILD(i) = ?

Platzsparende Repräsentation von Binärbäumen



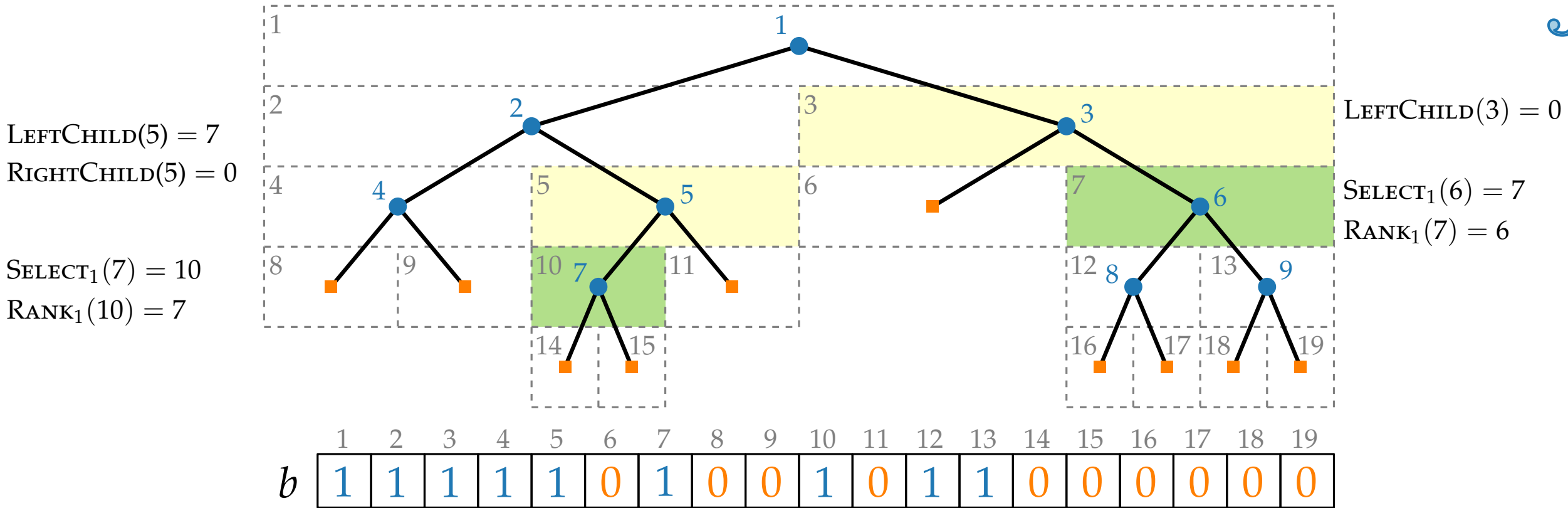
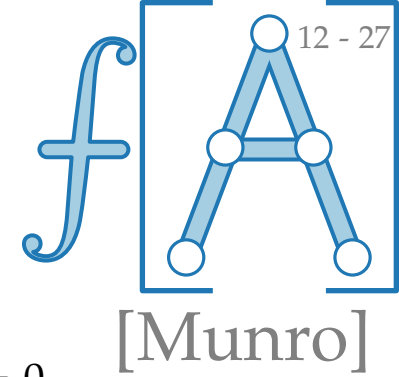
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

Operationen.

- $\text{PARENT}(i) = ?$
- $\text{LEFTCHILD}(i) = \begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$
 $= b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = ?$

Platzsparende Repräsentation von Binärbäumen



Idee.

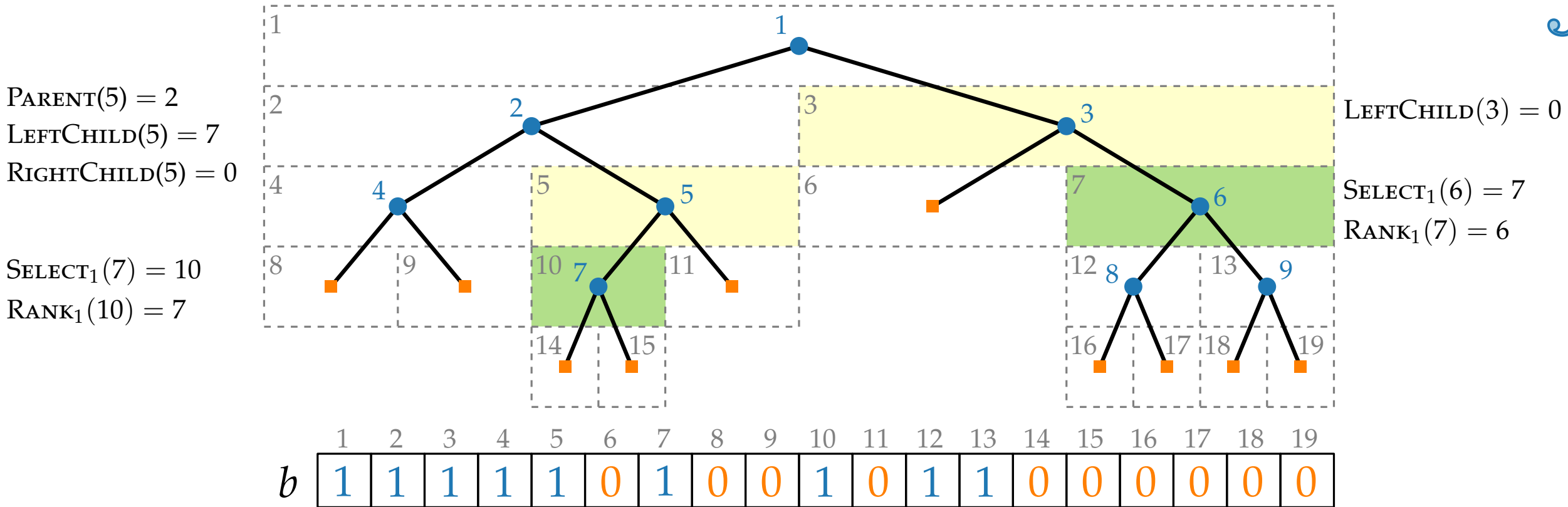
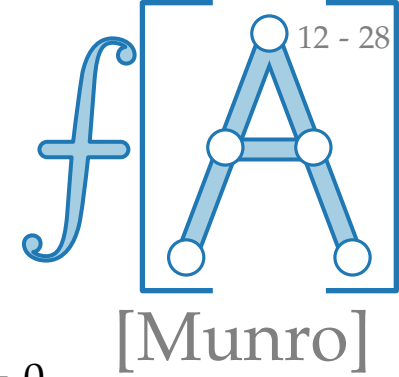
- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- PARENT(i) = ?
- LEFTCHILD(i) =
$$\begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$$

$= b[2i] \cdot \text{RANK}_1(2i)$
- RIGHTCHILD(i) = $b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Platzsparende Repräsentation von Binärbäumen



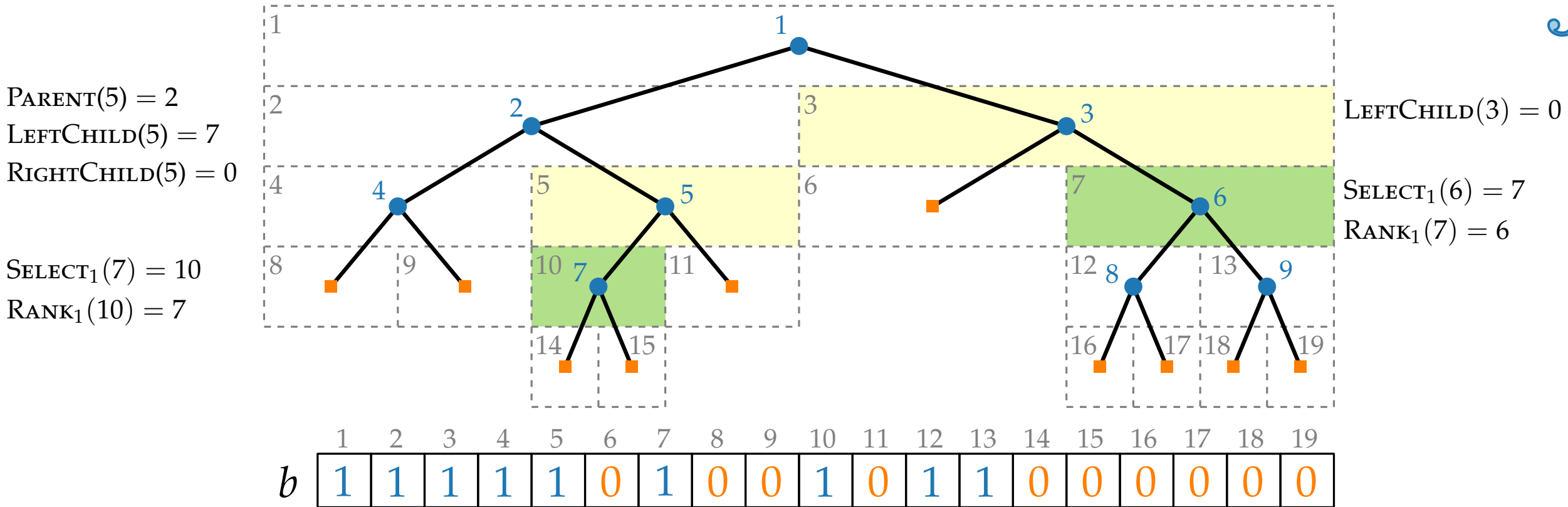
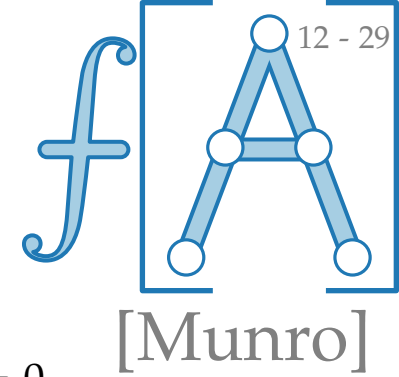
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

Operationen.

- $\text{PARENT}(i) = ?$
- $\text{LEFTCHILD}(i) = \begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$
 $= b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Platzsparende Repräsentation von Binärbäumen



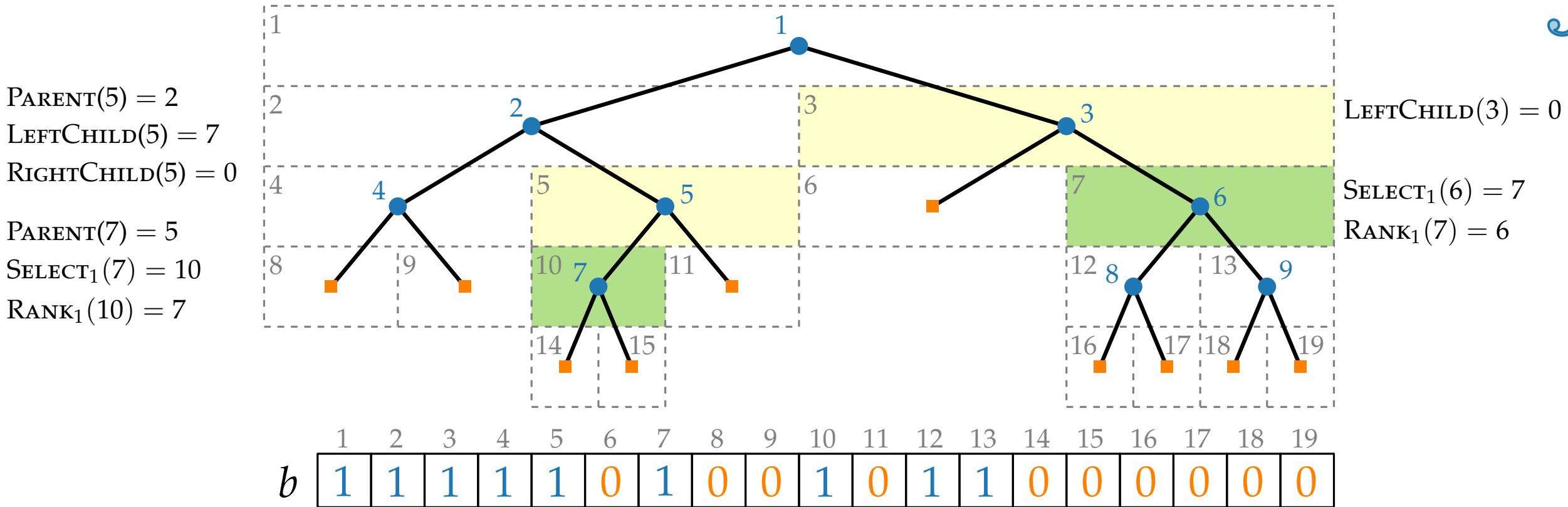
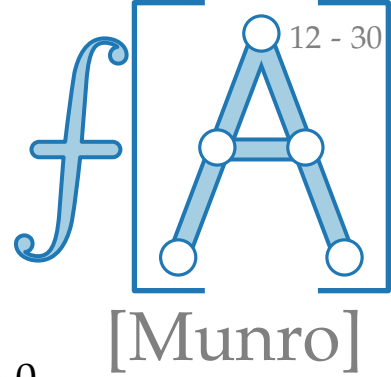
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

Operationen.

- $\text{PARENT}(i) = i / 2$ $= b[2i] \cdot \text{RANK}_1(2i)$
- $\text{LEFTCHILD}(i) = \begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Platzsparende Repräsentation von Binärbäumen



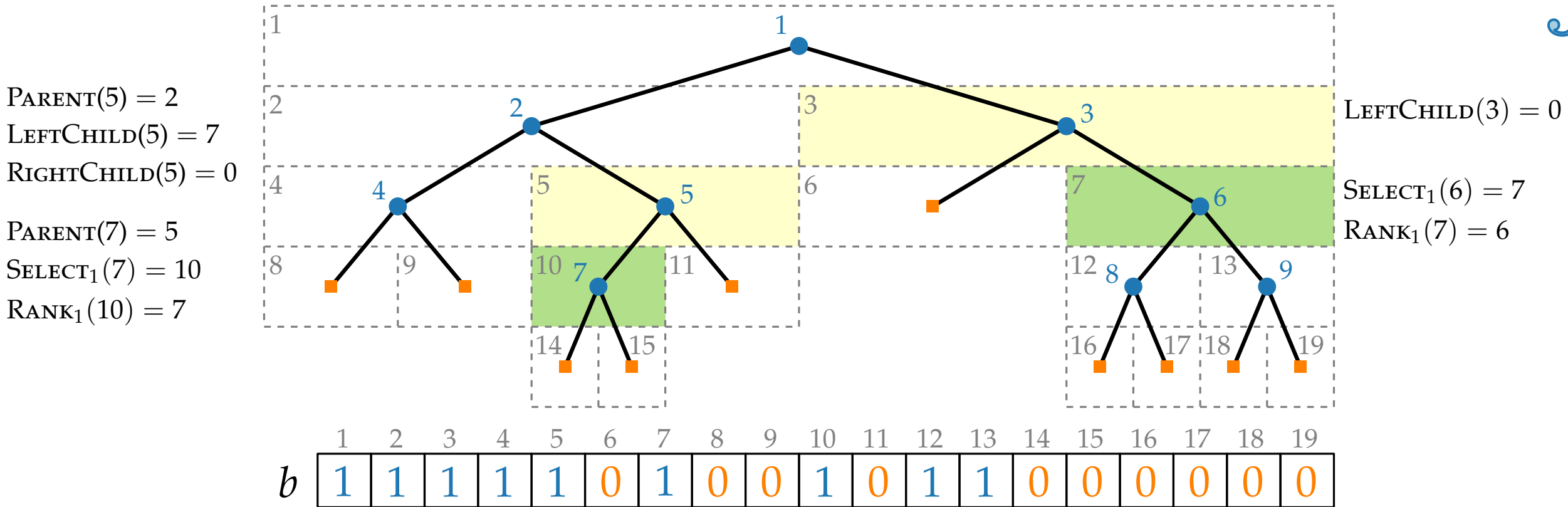
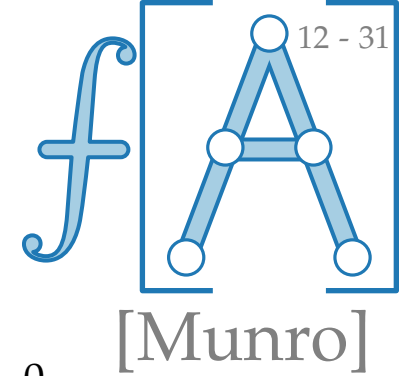
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

Operationen.

- $\text{PARENT}(i) = i / 2$ $= b[2i] \cdot \text{RANK}_1(2i)$
- $\text{LEFTCHILD}(i) = \begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Platzsparende Repräsentation von Binärbäumen



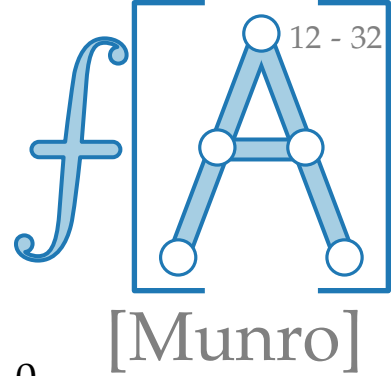
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK_1 und SELECT_1

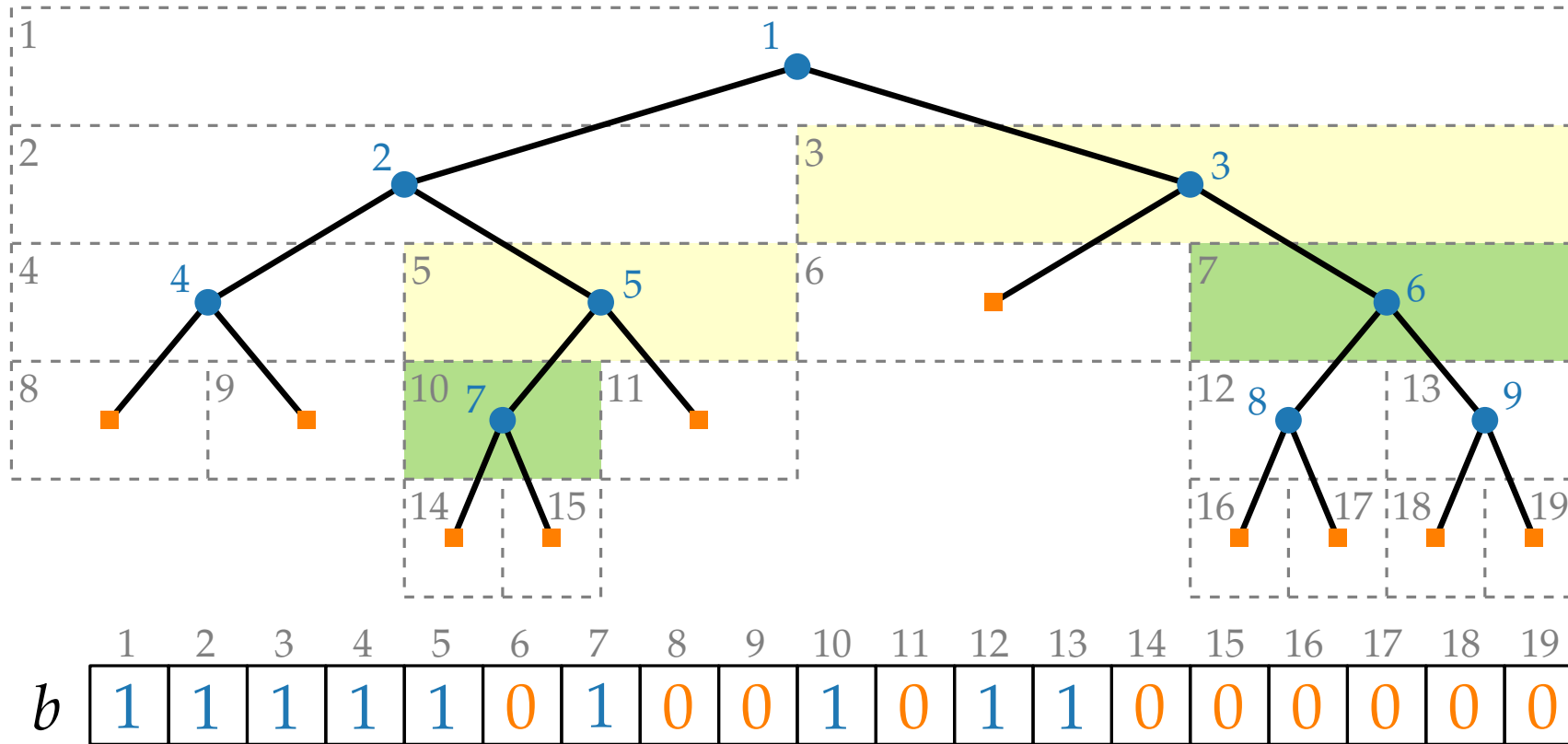
Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$ $= b[2i] \cdot \text{RANK}_1(2i)$
- $\text{LEFTCHILD}(i) = \begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Platzsparende Repräsentation von Binärbäumen



PARENT(5) = 2
 LEFTCHILD(5) = 7
 RIGHTCHILD(5) = 0
 PARENT(7) = 5
 SELECT₁(7) = 10
 RANK₁(10) = 7



LEFTCHILD(3) = 0

SELECT₁(6) = 7

RANK₁(7) = 6

Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

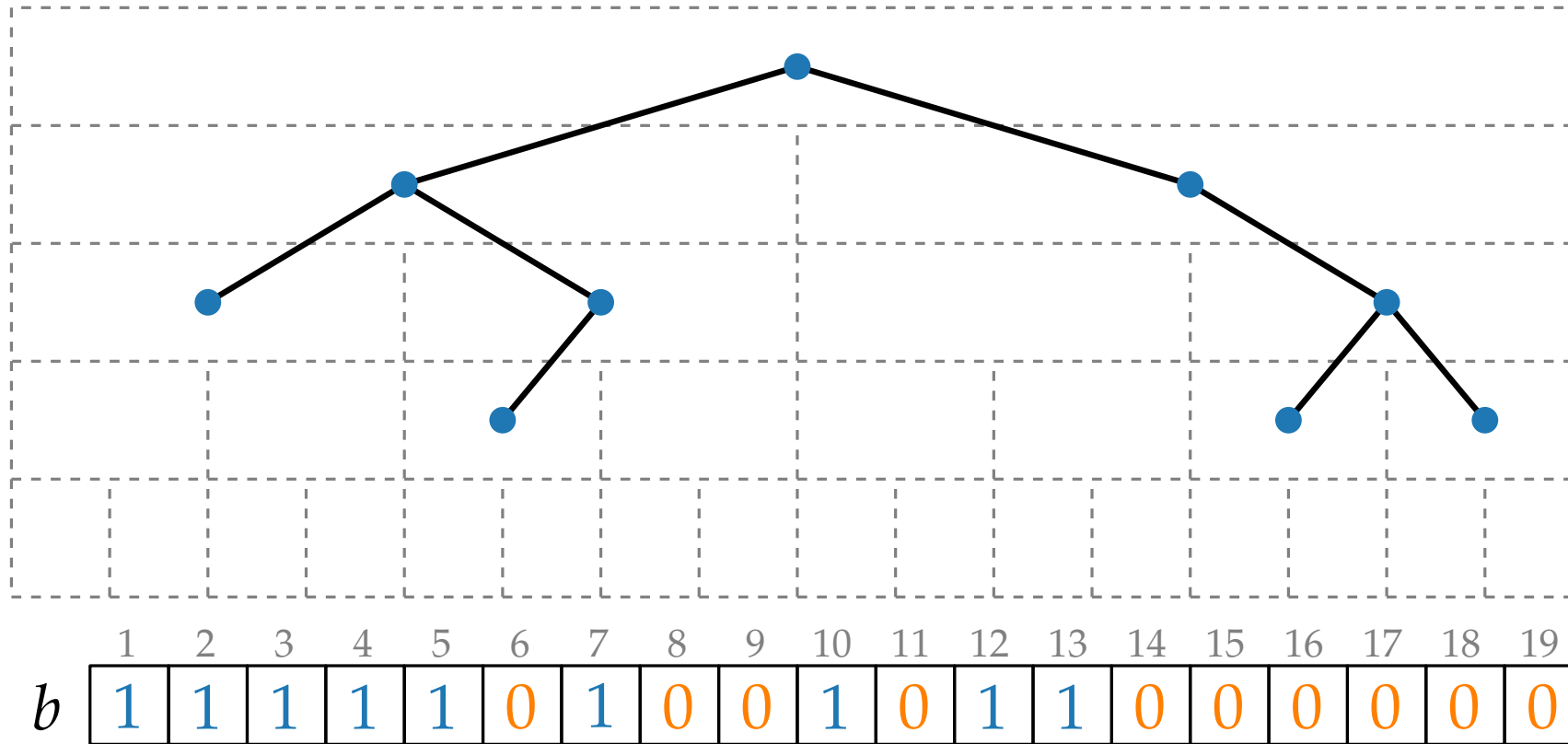
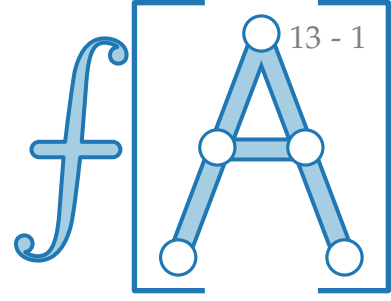
Idee.

- Füge **Dummy Blätter** hinzu
- Lese **echte Knoten** als 1
- Lese **Dummy Blätter** als 0
- Benutze RANK₁ und SELECT₁

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$ $= b[2i] \cdot \text{RANK}_1(2i)$
- $\text{LEFTCHILD}(i) = \begin{cases} 0 & \text{falls } b[2i] = 0 \\ \text{RANK}_1(2i) & \text{sonst} \end{cases}$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

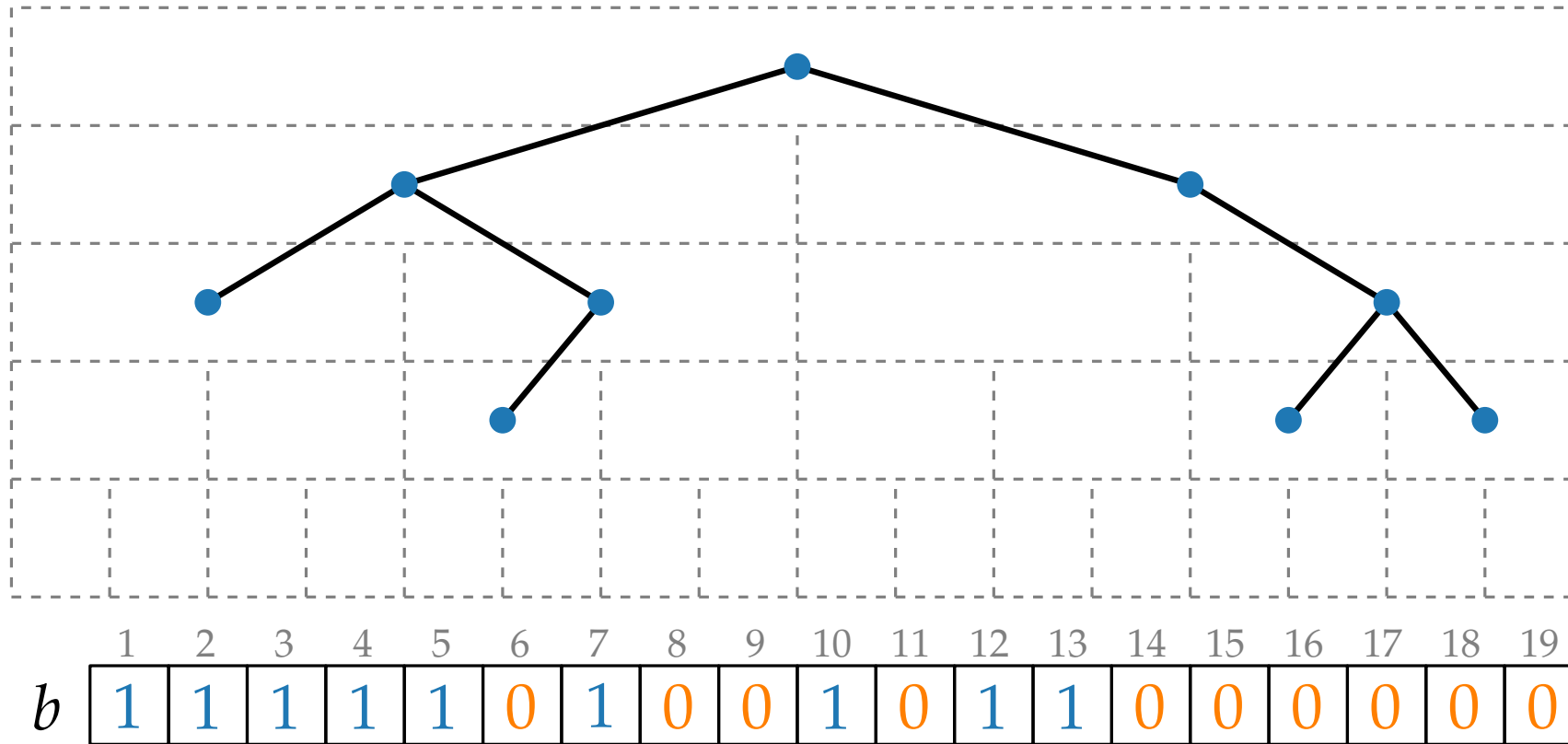
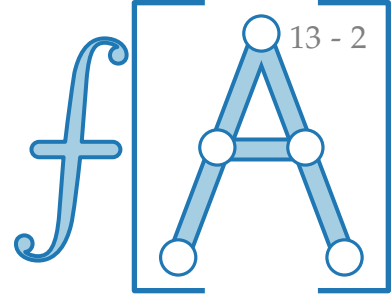
- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

Andere Idee.

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

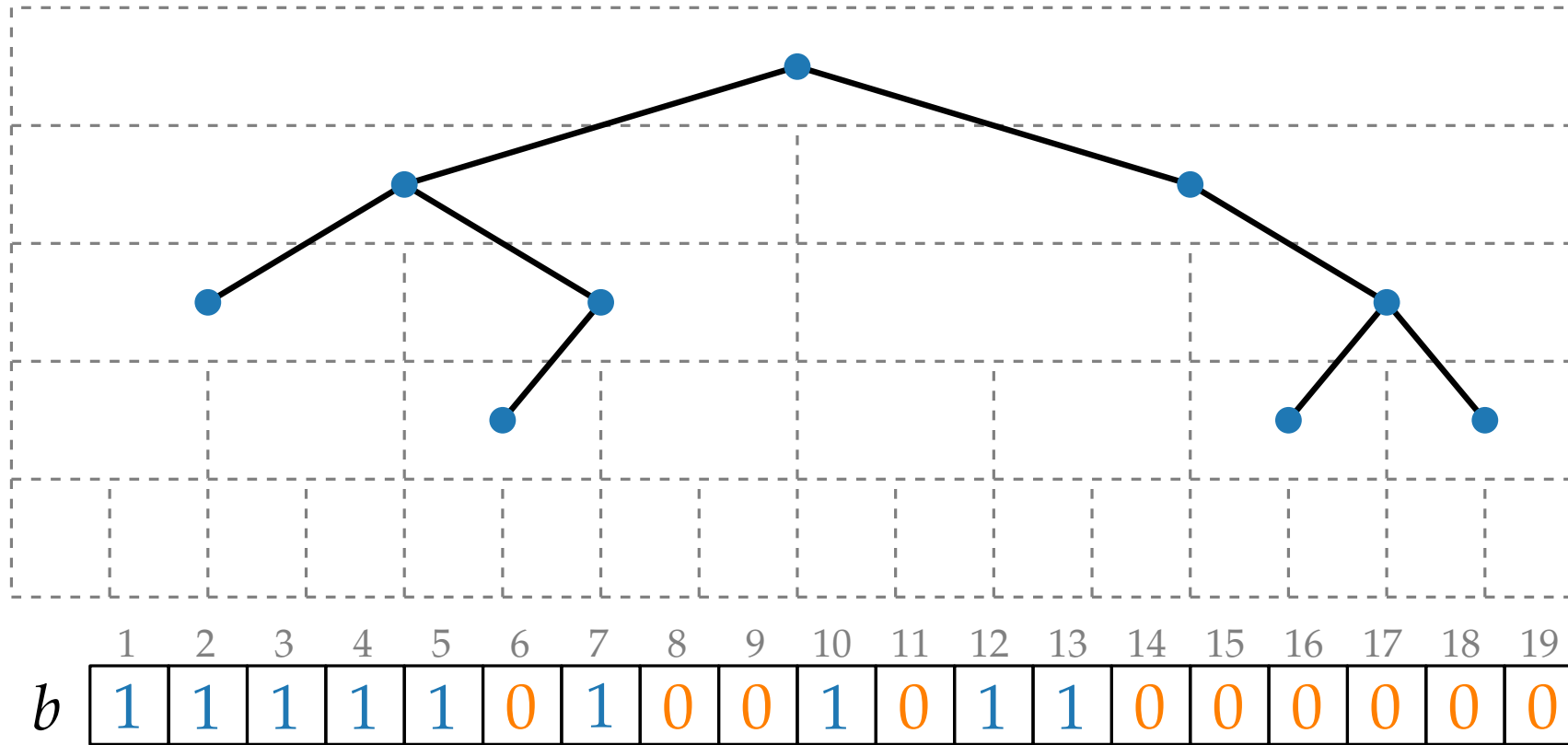
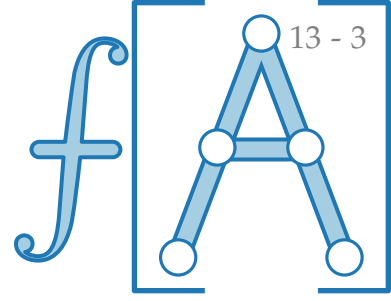
Andere Idee.

- Schreibe 1

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

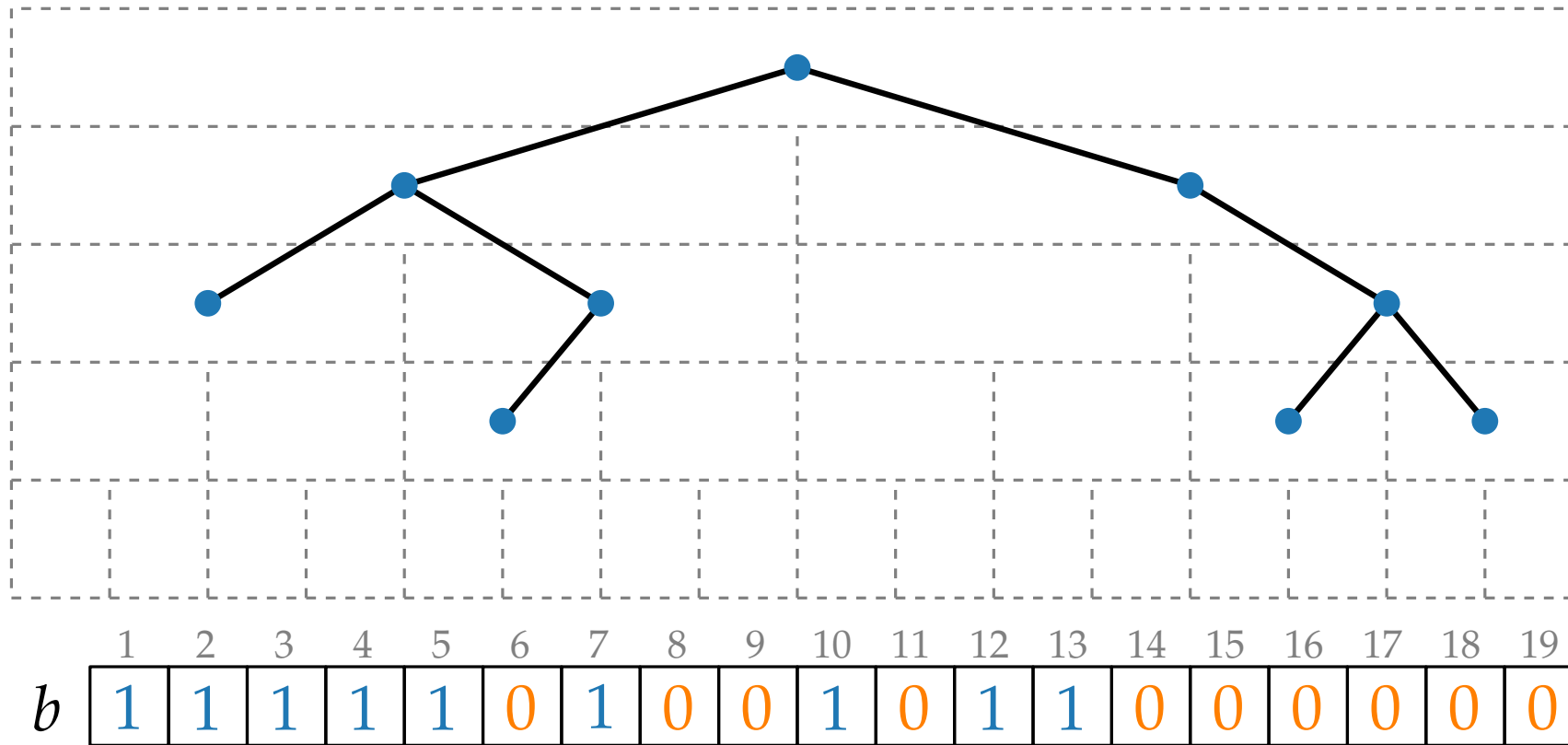
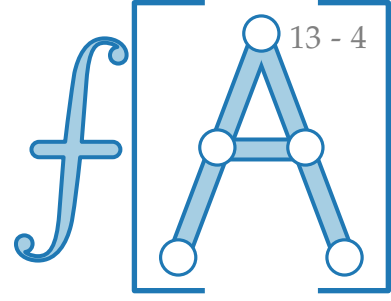
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

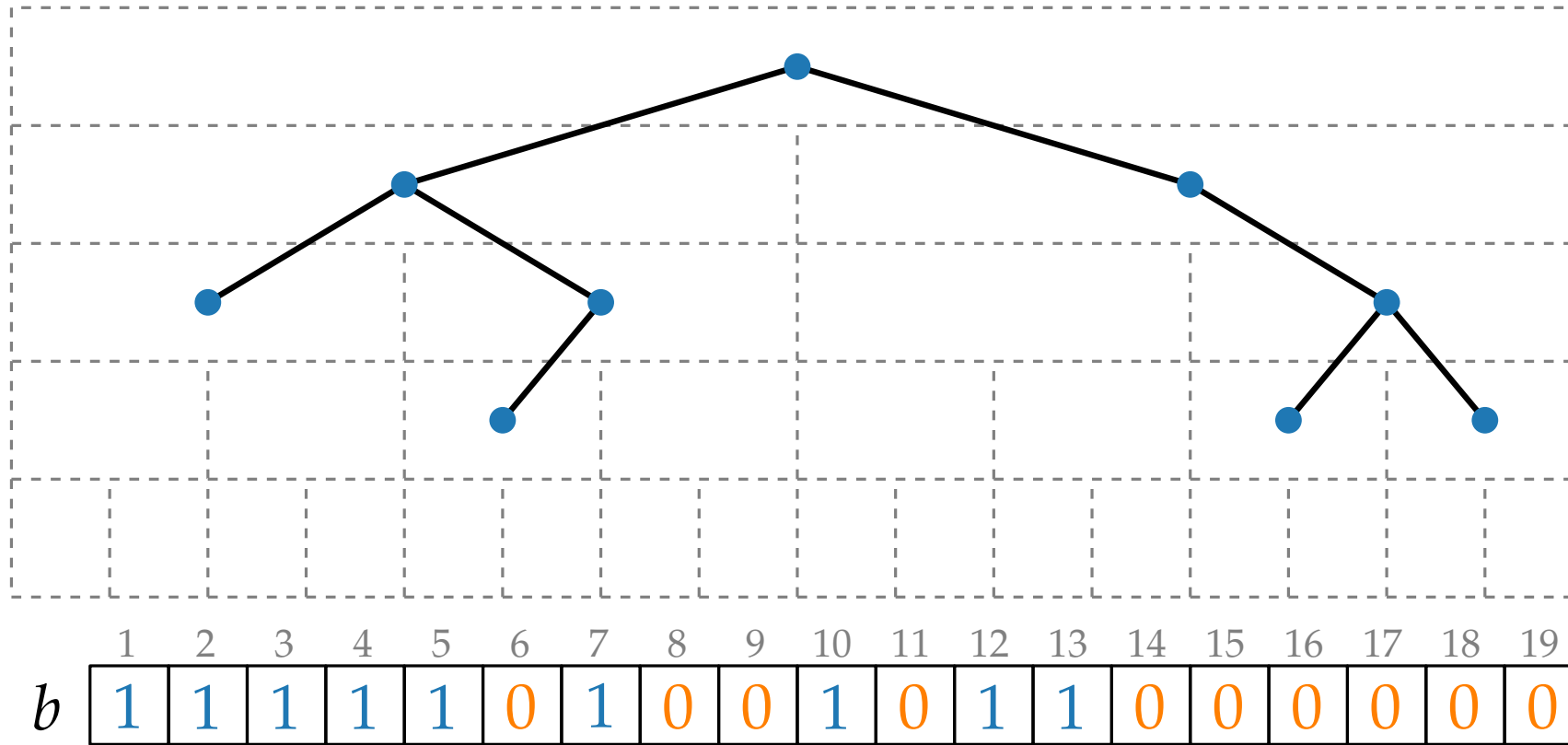
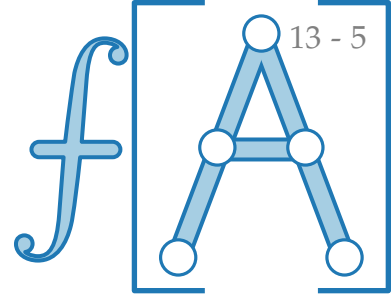
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

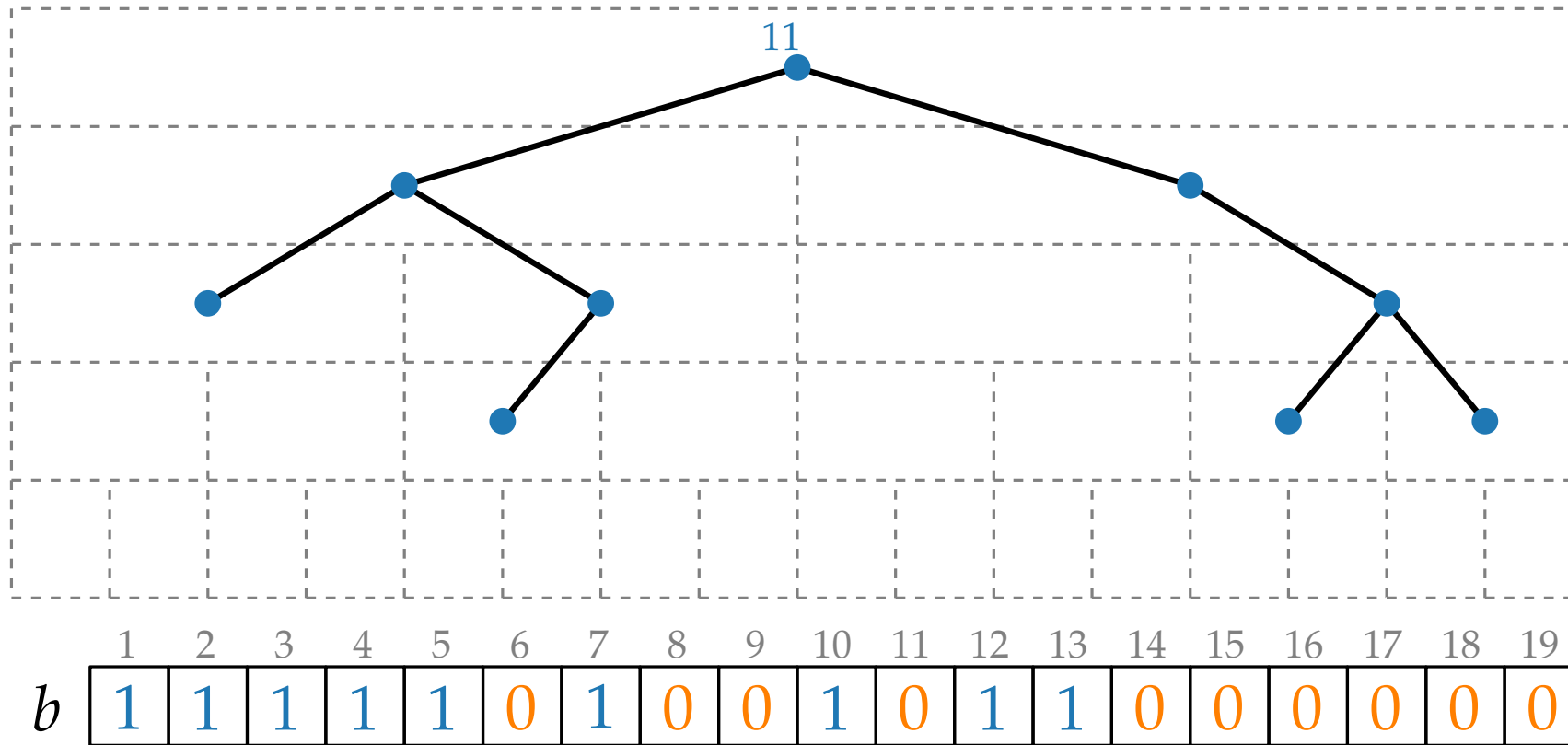
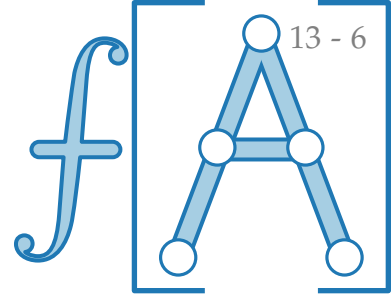
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

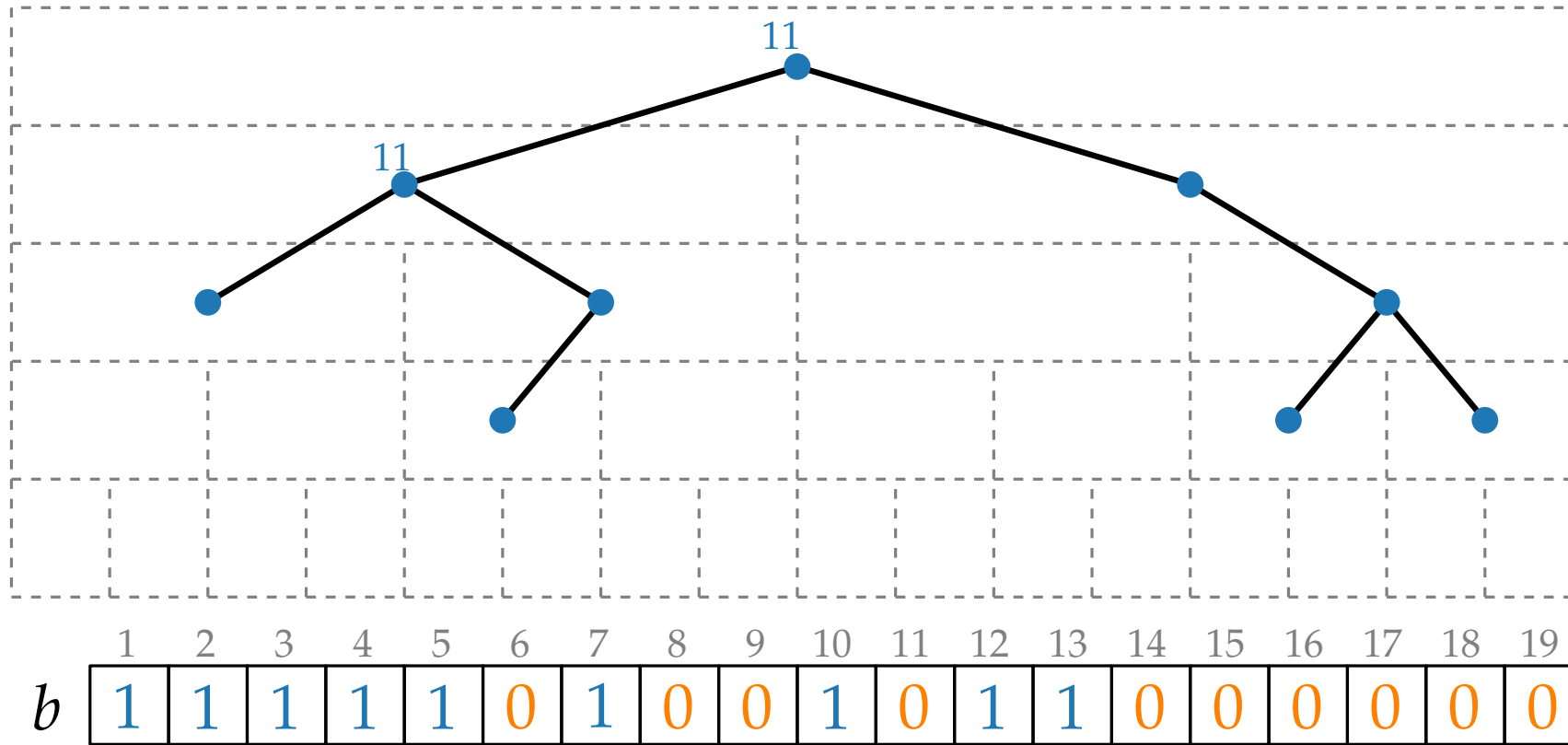
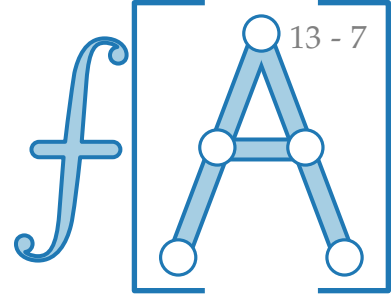
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

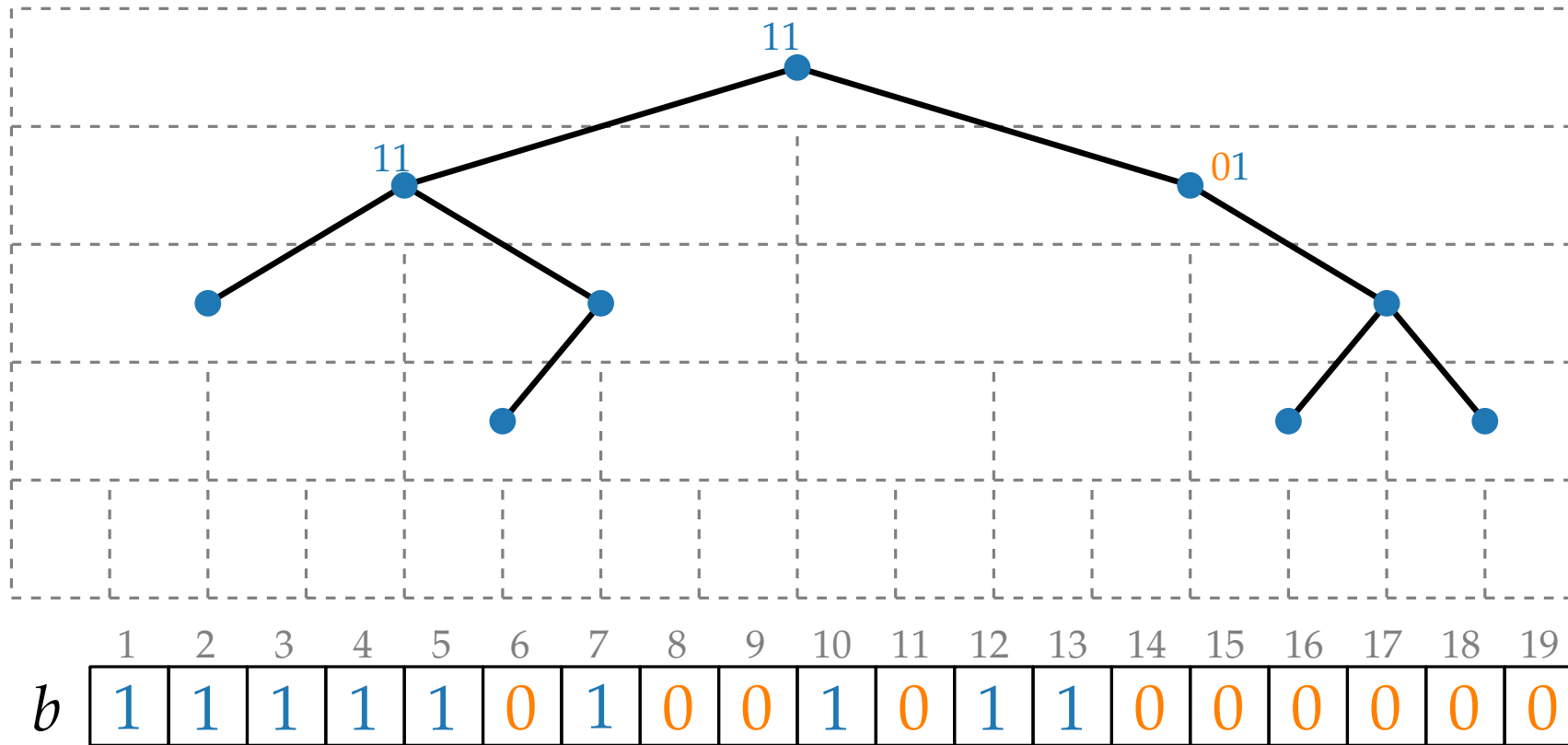
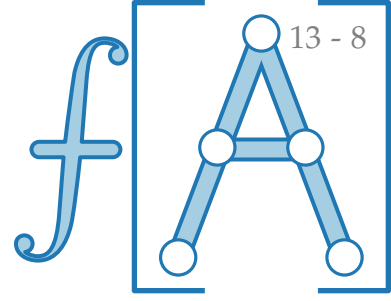
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

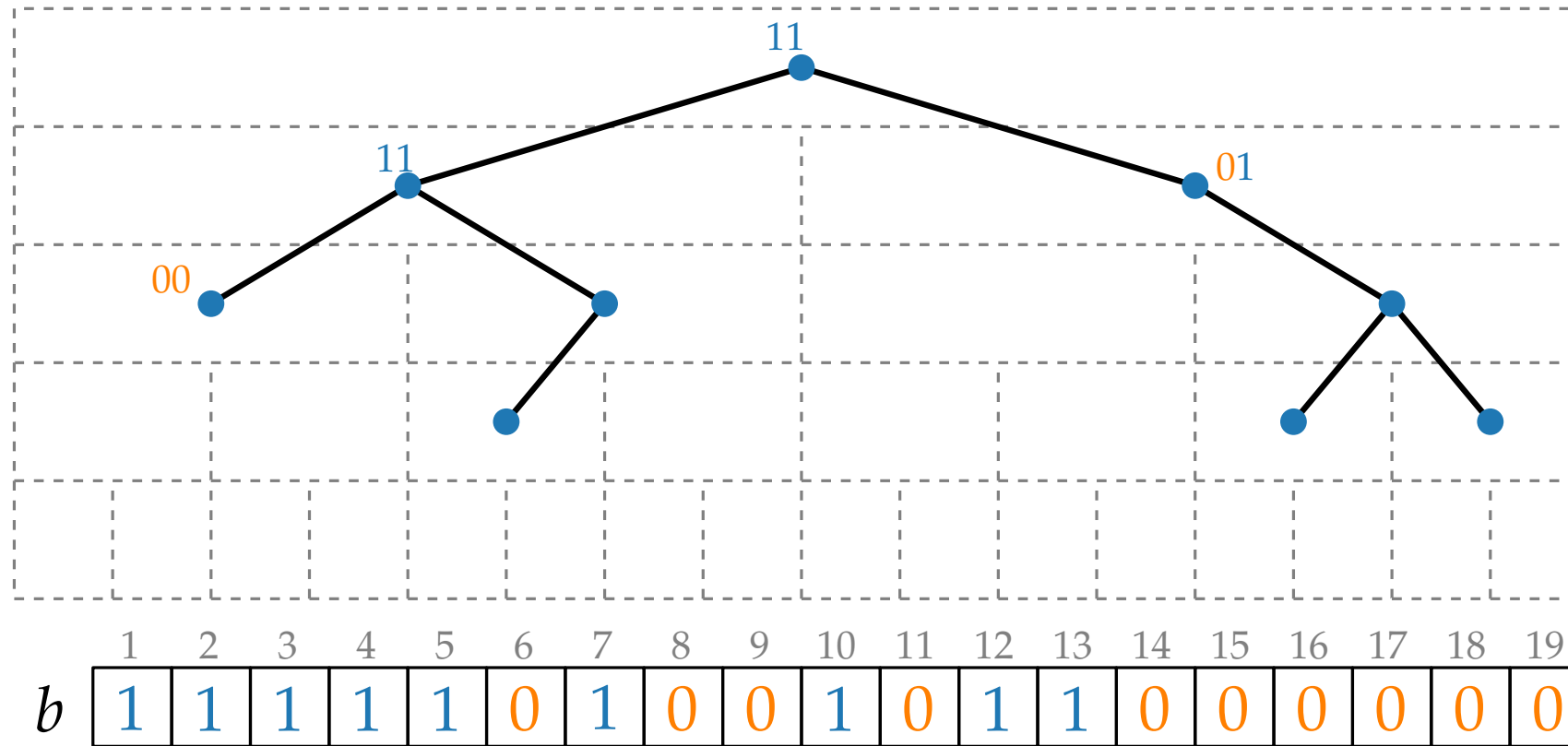
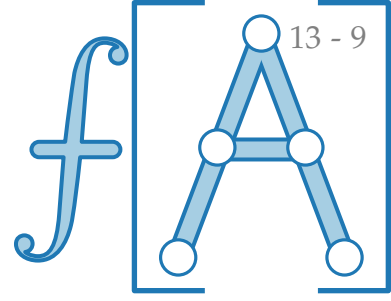
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

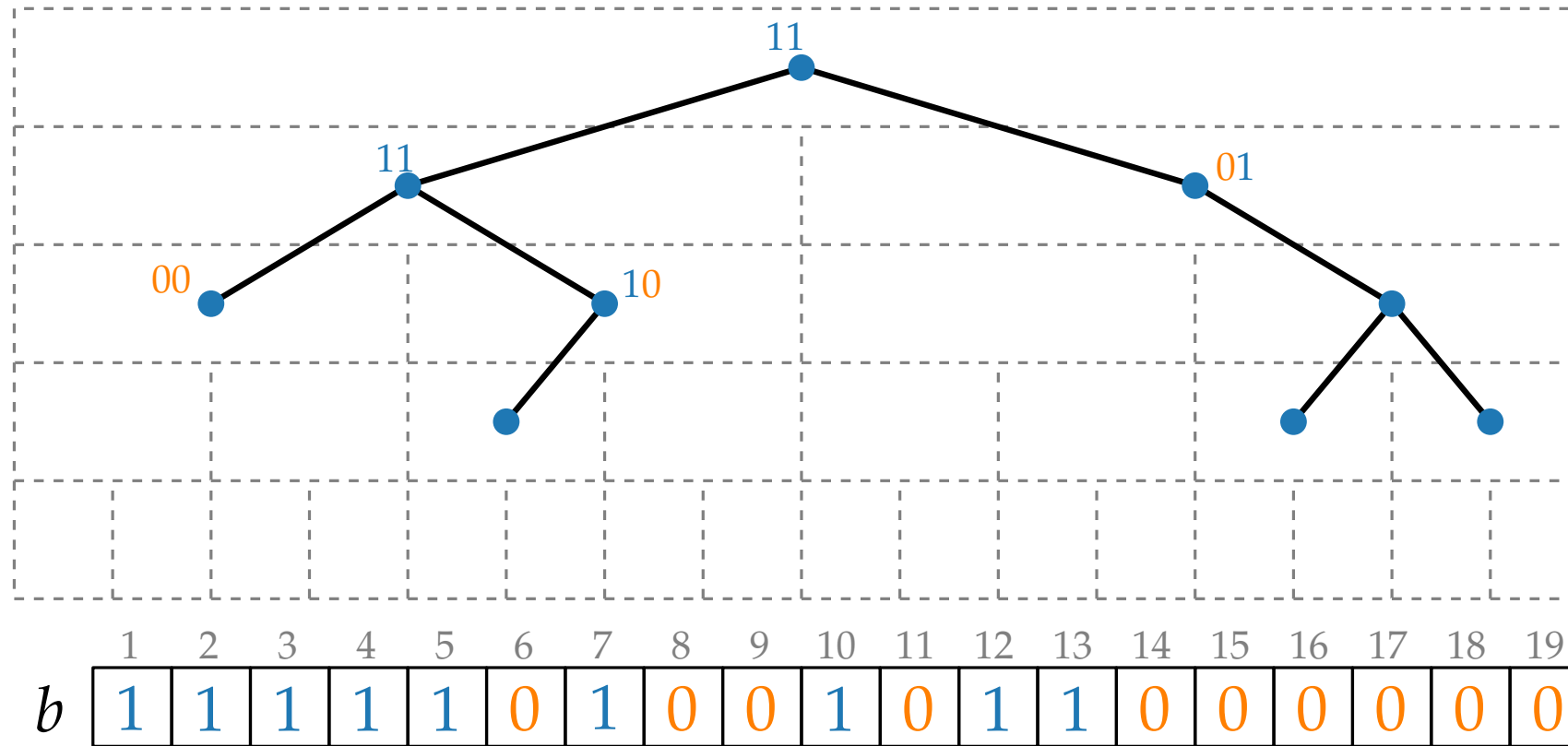
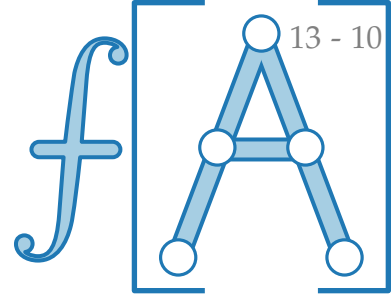
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

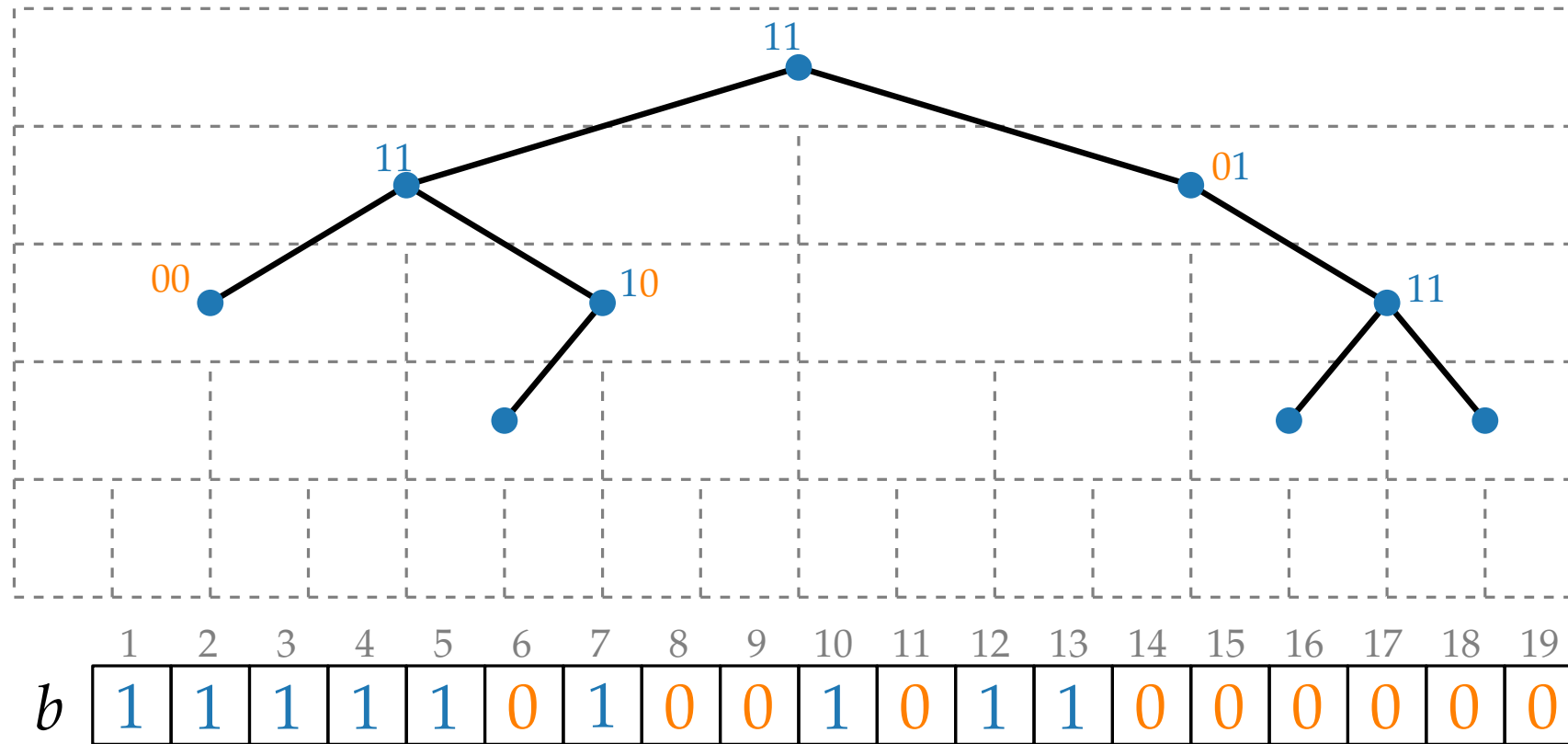
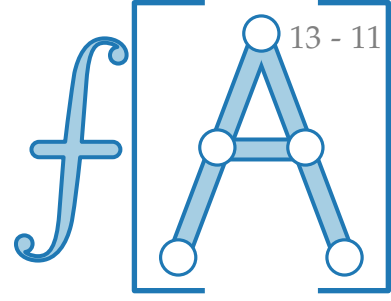
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

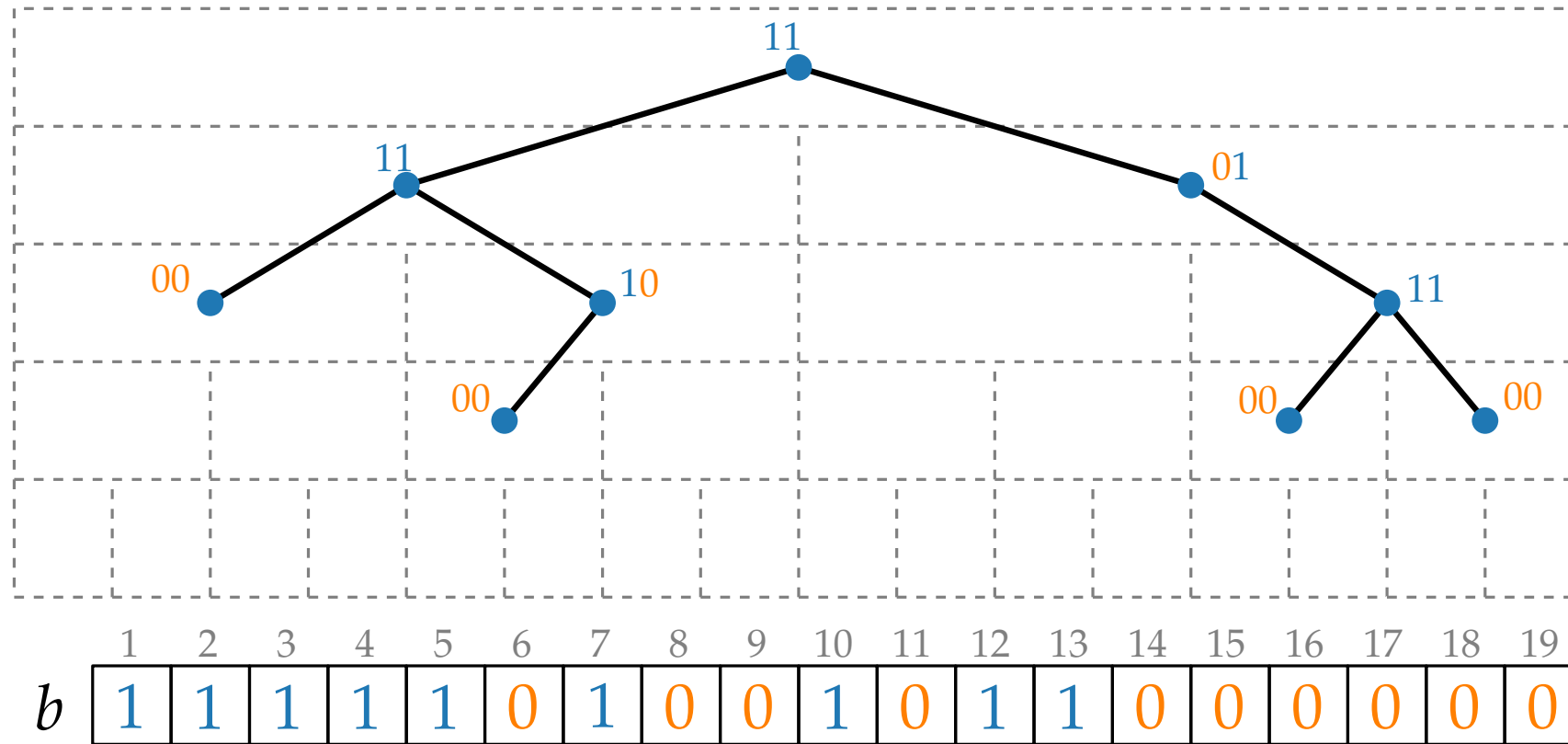
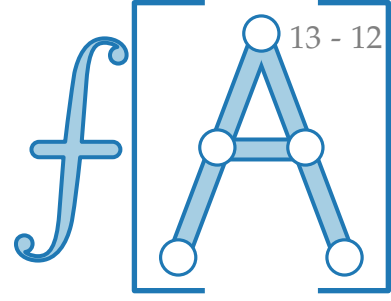
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

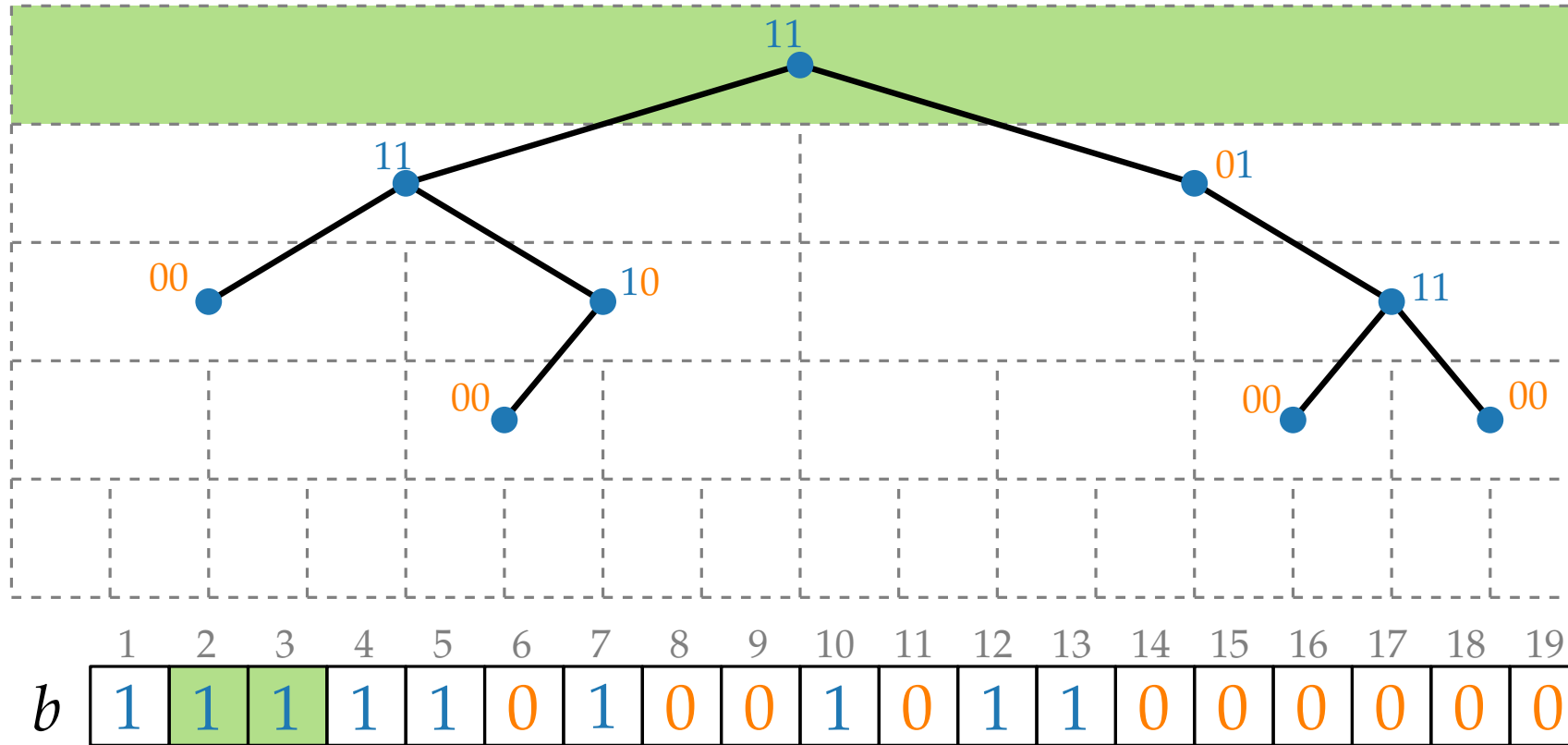
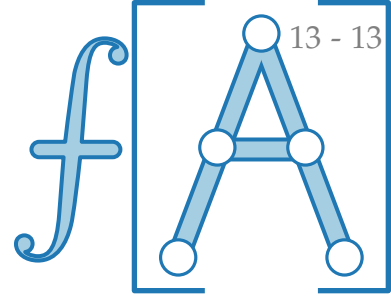
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

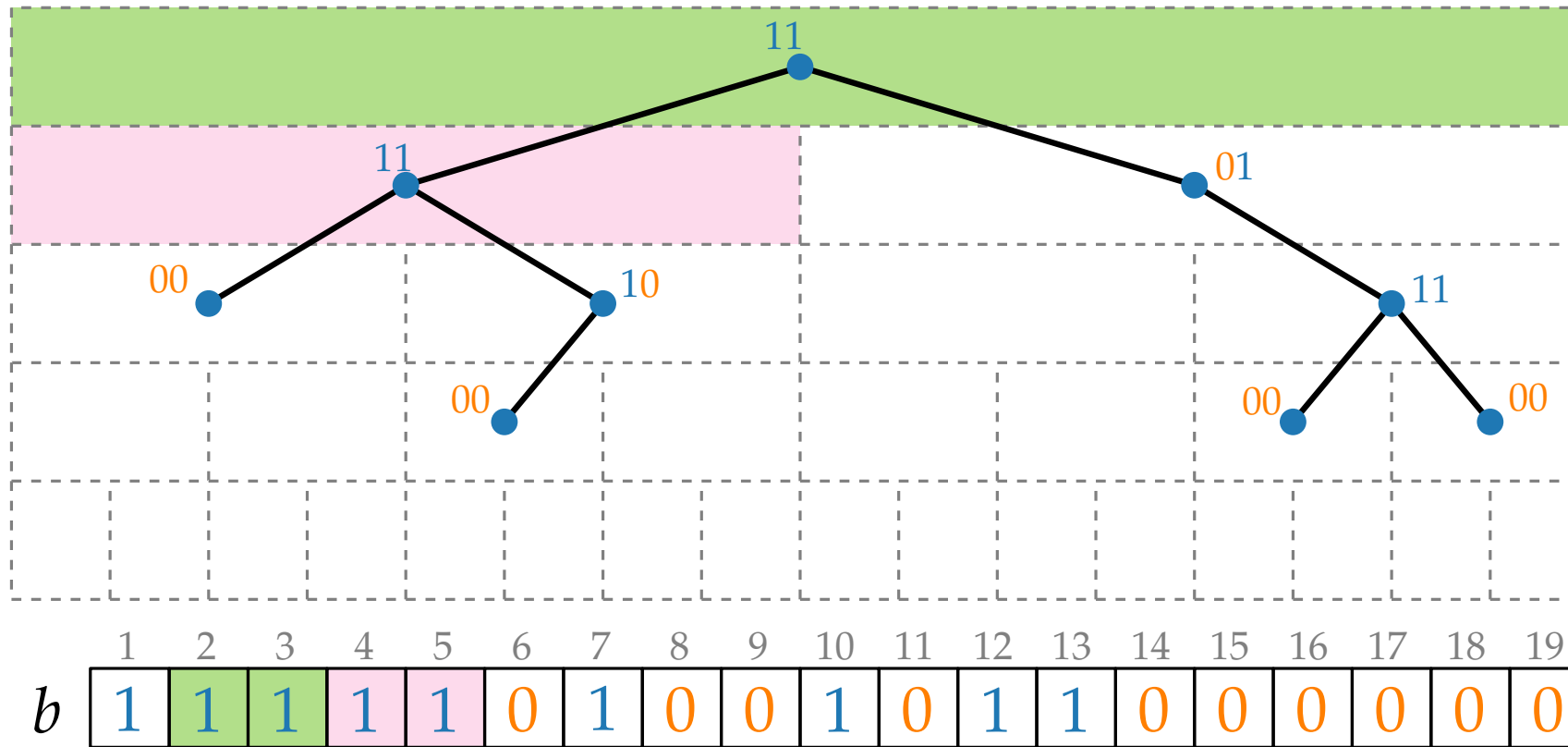
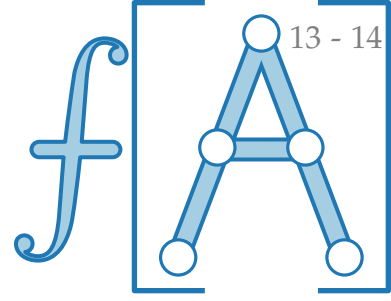
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

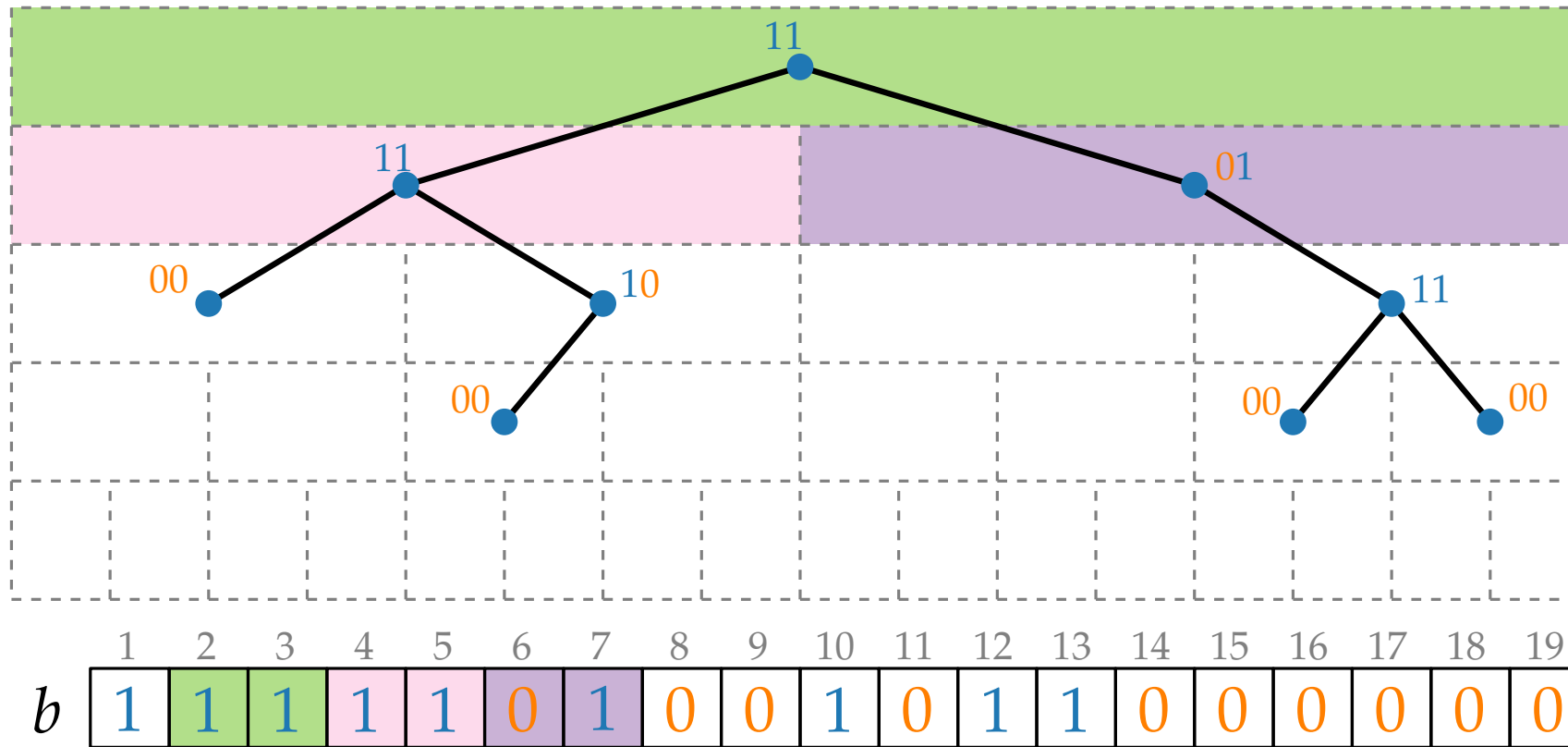
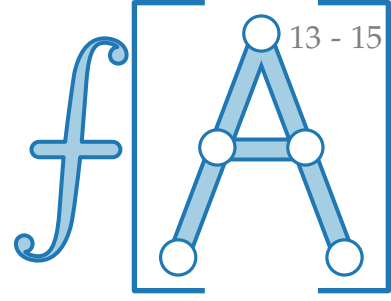
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

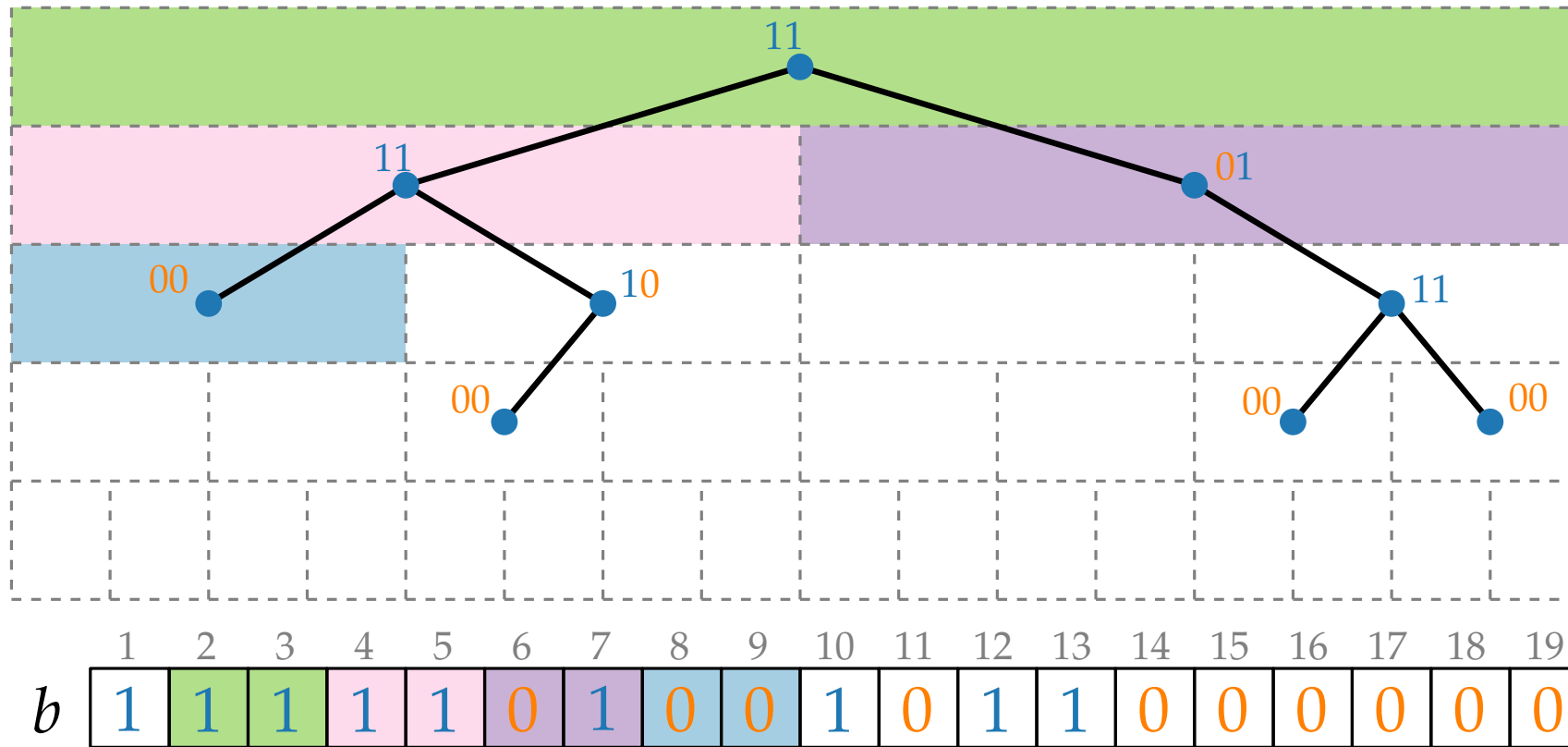
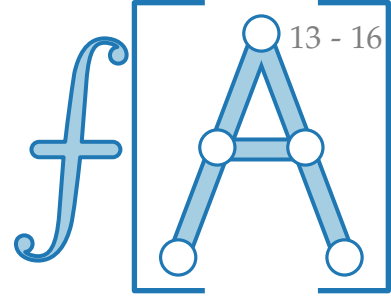
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

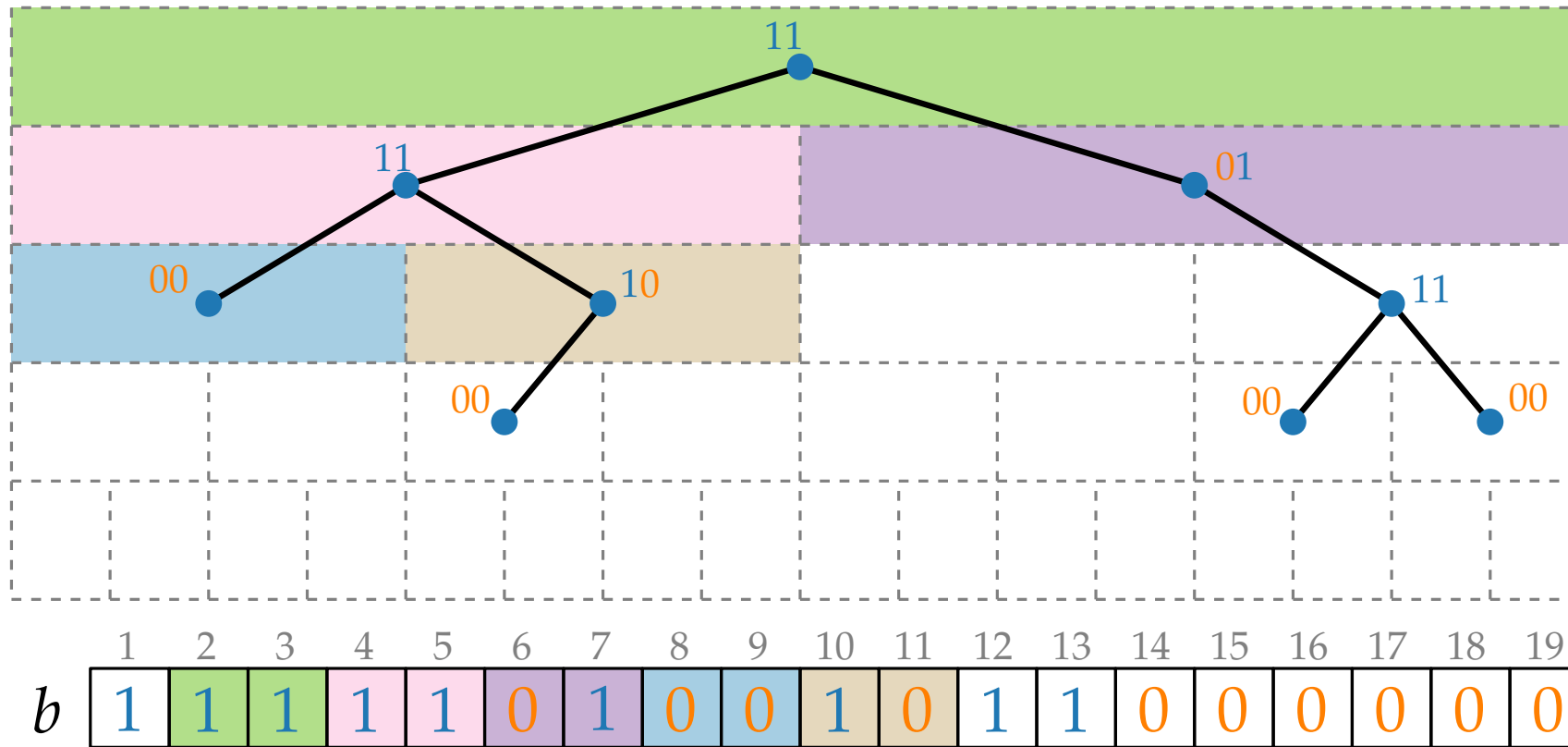
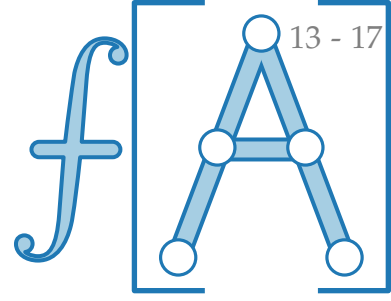
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

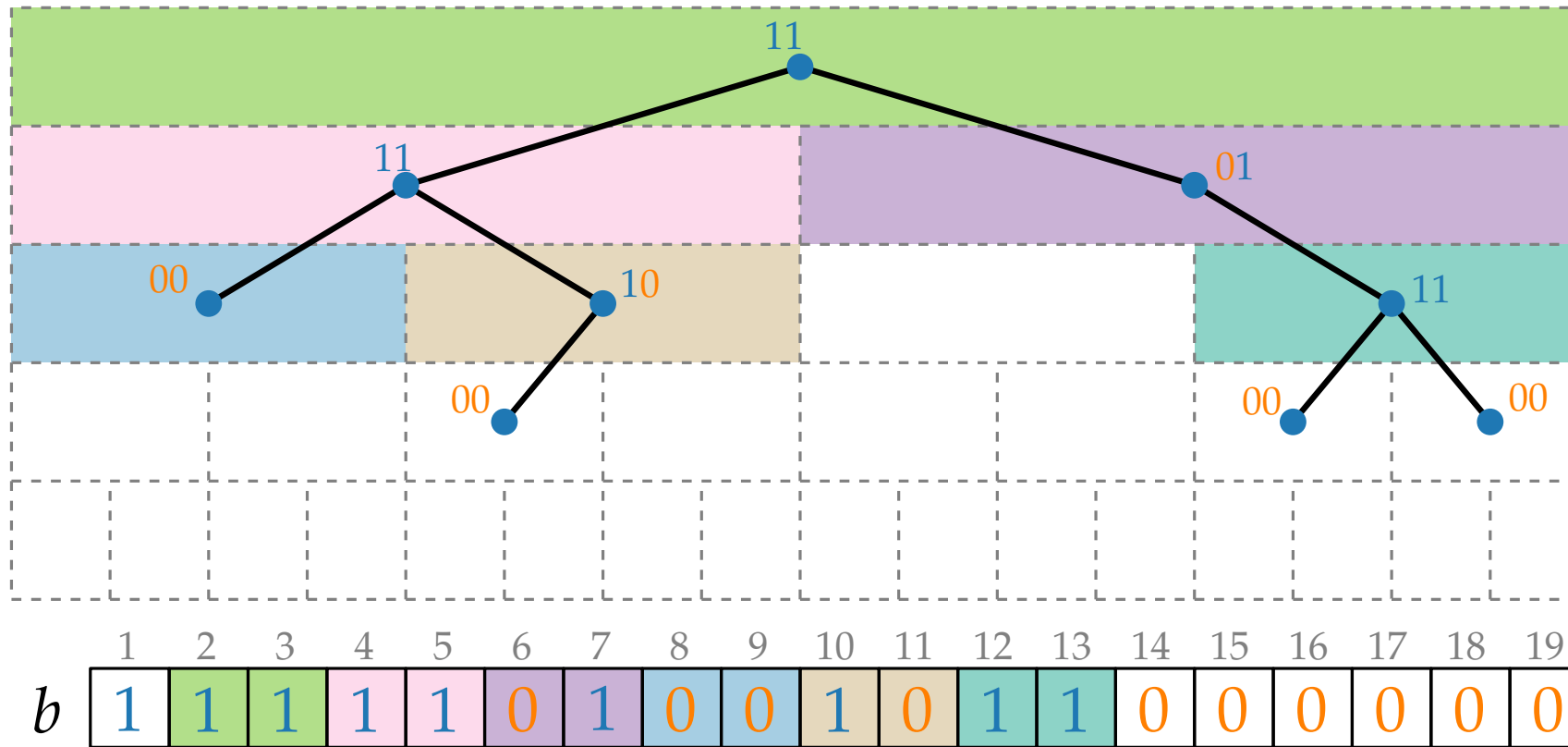
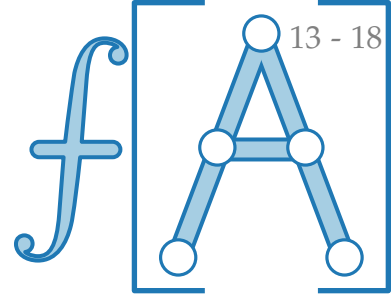
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

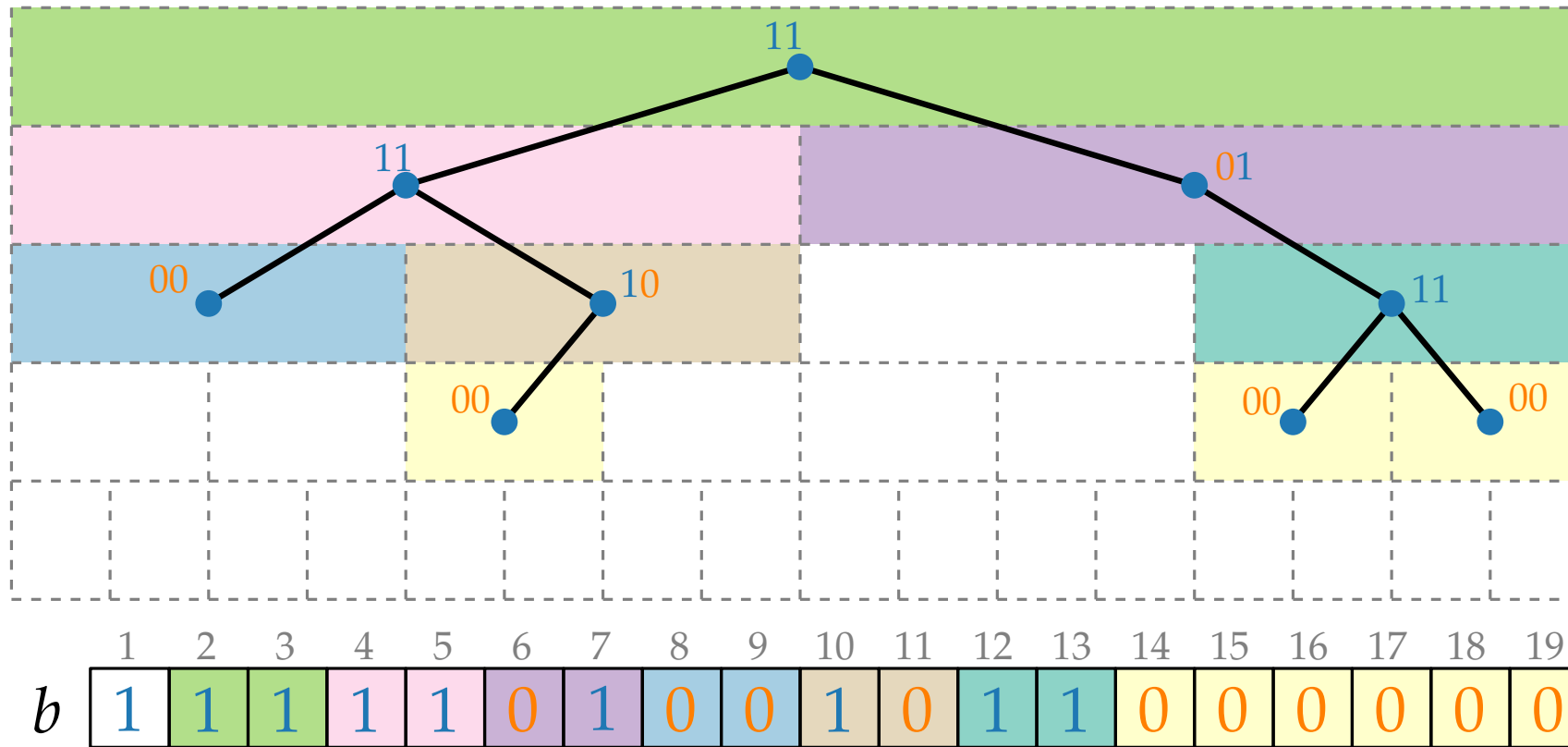
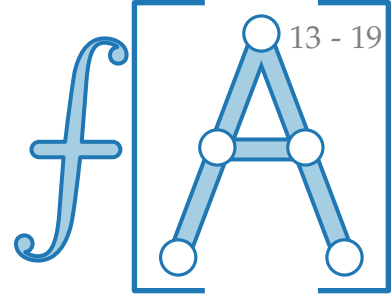
Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

Alt. platzsparende Repr. von Binärbäumen



Platz.

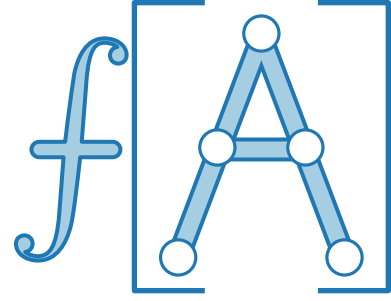
- $2n + 1$ Bits für b
- $o(n)$ für RANK und SELECT

Andere Idee.

- Schreibe 1
- Schreibe für jeden Knoten 2 Bits:
 - 1 falls linkes Kind ex., 0 sonst
 - 1 falls rechtes Kind ex., 0 sonst

Operationen.

- $\text{PARENT}(i) = \lfloor \text{SELECT}_1(i) / 2 \rfloor$
- $\text{LEFTCHILD}(i) = b[2i] \cdot \text{RANK}_1(2i)$
- $\text{RIGHTCHILD}(i) = b[2i + 1] \cdot \text{RANK}_1(2i + 1)$

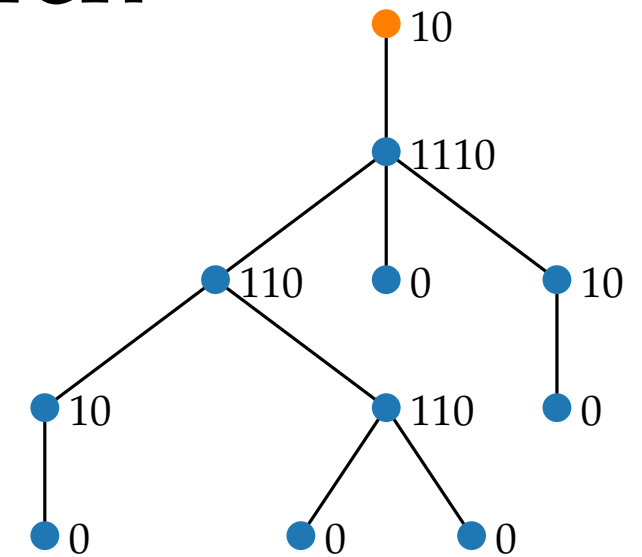


Fortgeschrittene Algorithmen

16. Vorlesung Datenstrukturen II:

Platzeffiziente Datenstrukturen (Succinct Data Structures)

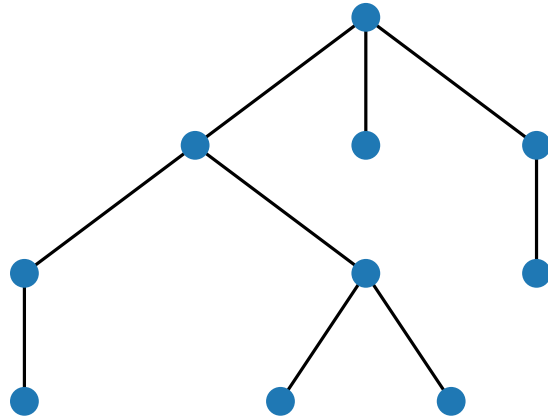
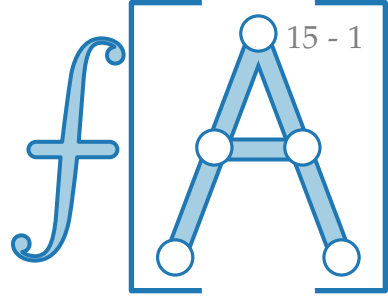
Teil V: Allgemeine Bäume



Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]

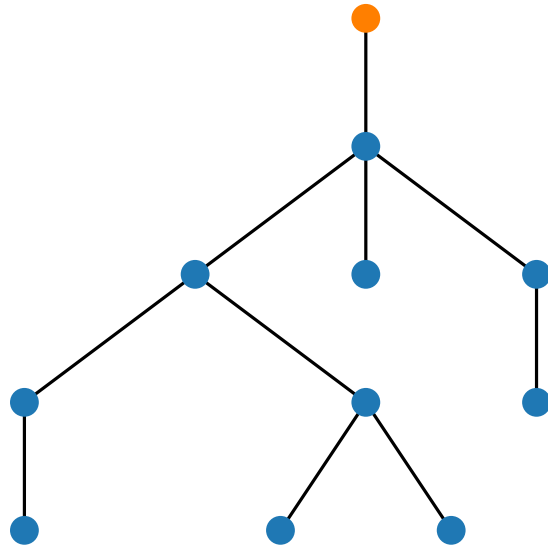
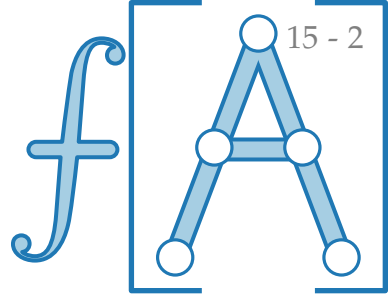
LOUDS = Level Order Unary Degree Sequence



Platzsparende Repr. von Bäumen – LOUDS

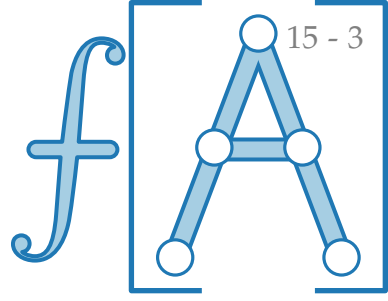
[Guy Jacobson, FOCS 1989]

LOUDS = Level Order Unary Degree Sequence



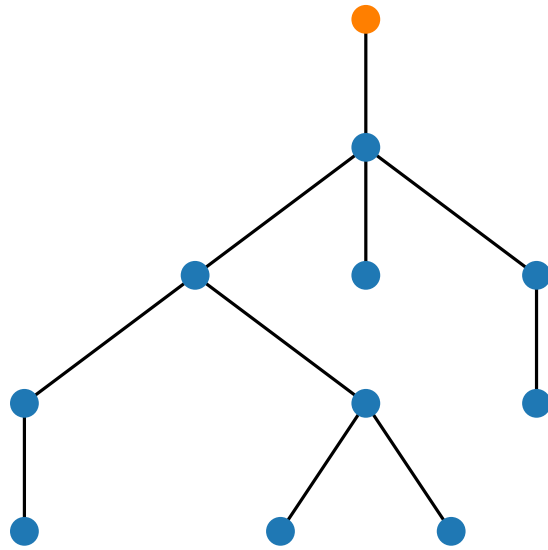
Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



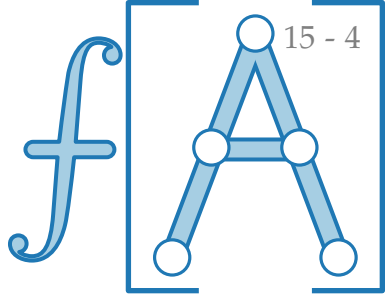
LOUDS = Level Order Unary Degree Sequence

■ Unäre Dekodierung des ausgehenden Grads



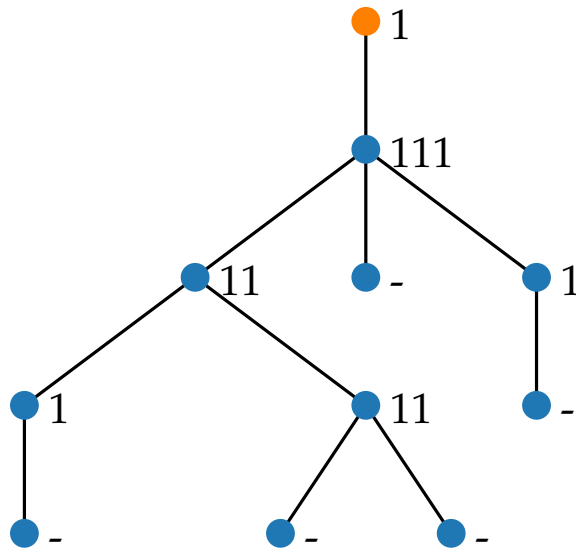
Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



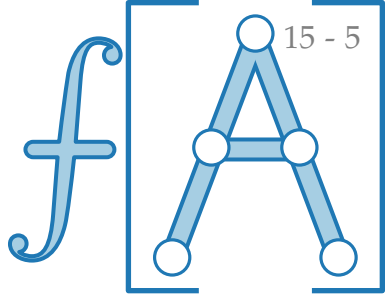
LOUDS = Level Order Unary Degree Sequence

■ Unäre Dekodierung des ausgehenden Grads

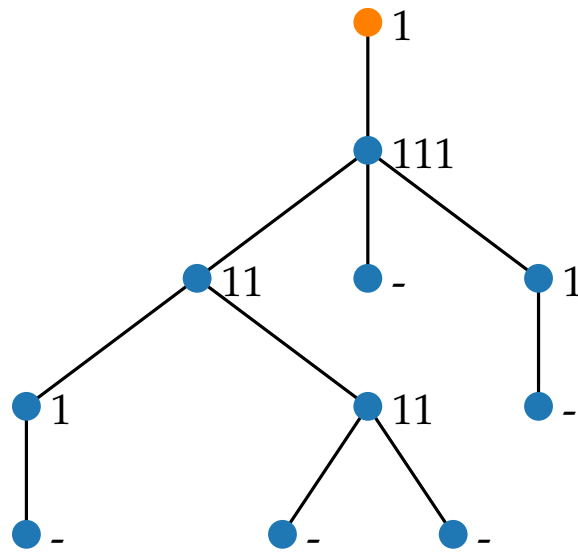


Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



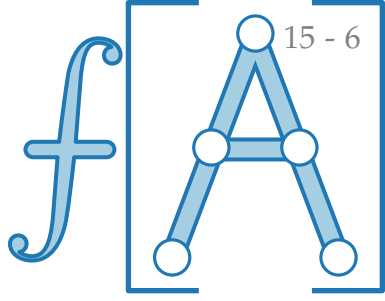
LOUDS = Level Order Unary Degree Sequence



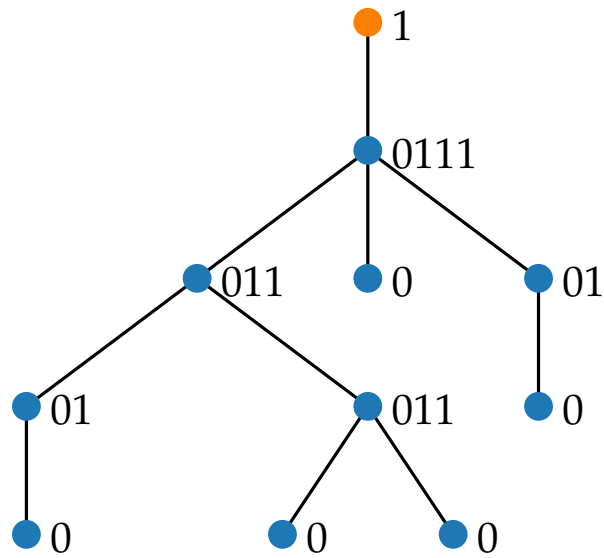
- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



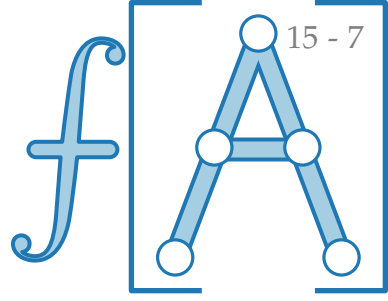
LOUDS = Level Order Unary Degree Sequence



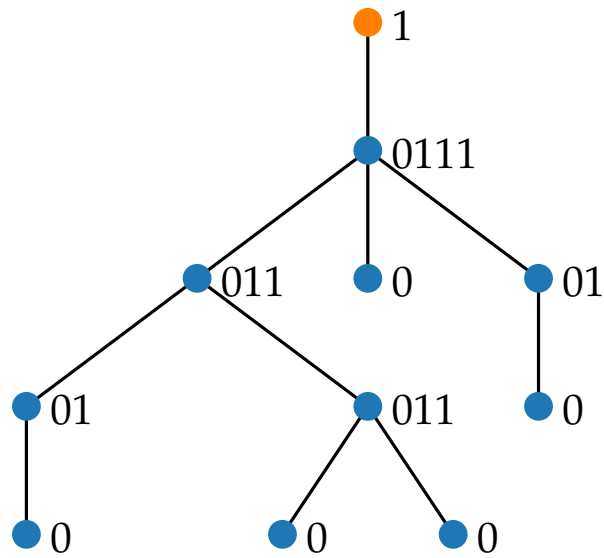
- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



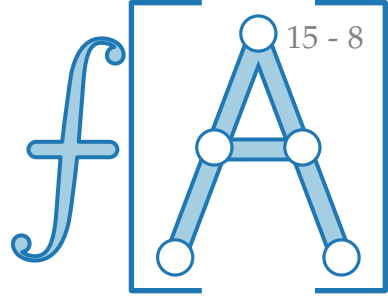
■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

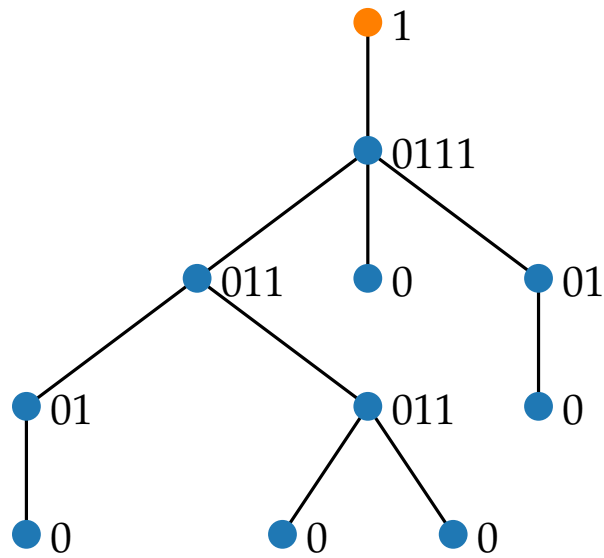
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads getrennt durch je eine 0
- gibt LOUDS Sequenz

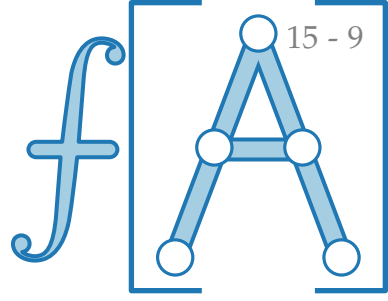
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

Platz.

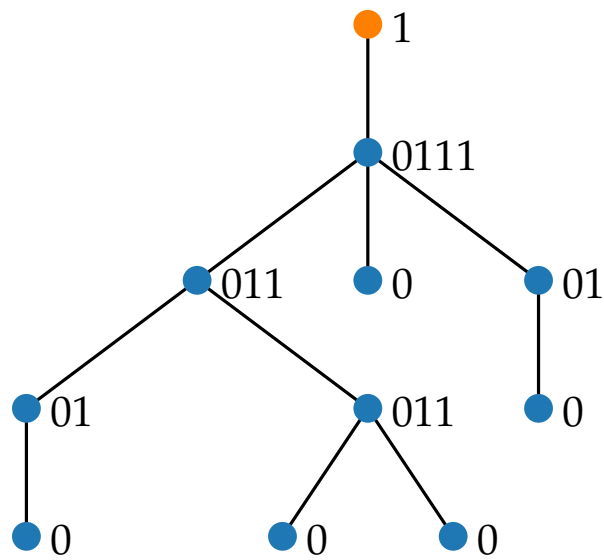
- Jeder Knoten (außer Wurzel) ist zweimal repräsentiert, nämlich mit einer 1 und einer 0

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0
- gibt LOUDS Sequenz

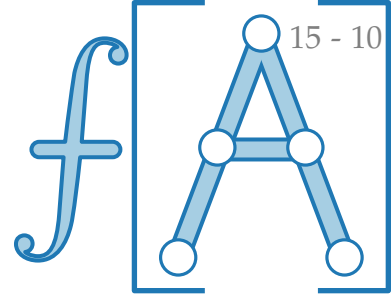
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

Platz.

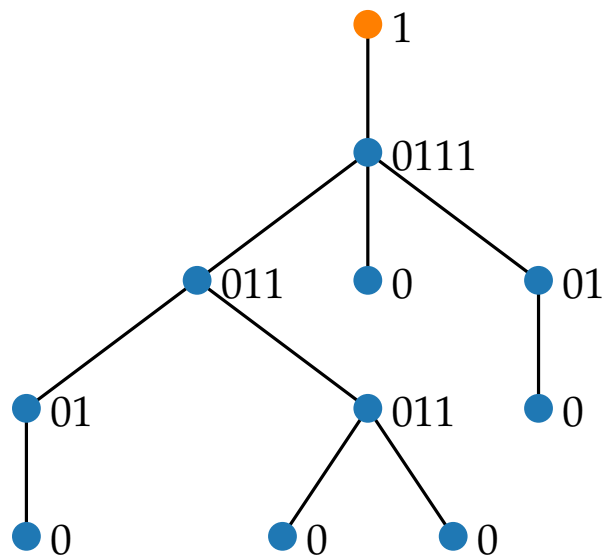
- Jeder Knoten (außer Wurzel) ist zweimal repräsentiert,
nämlich mit einer 1 und einer 0
- $o(n)$ Bits für RANK und SELECT

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

Platz.

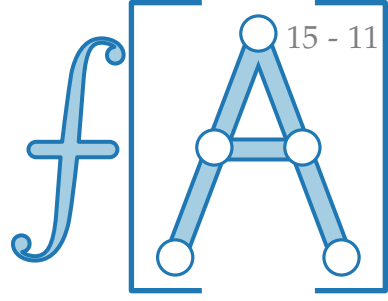
■ Jeder Knoten (außer Wurzel) ist zweimal repräsentiert,
nämlich mit einer 1 und einer 0

■ $o(n)$ Bits für RANK und SELECT

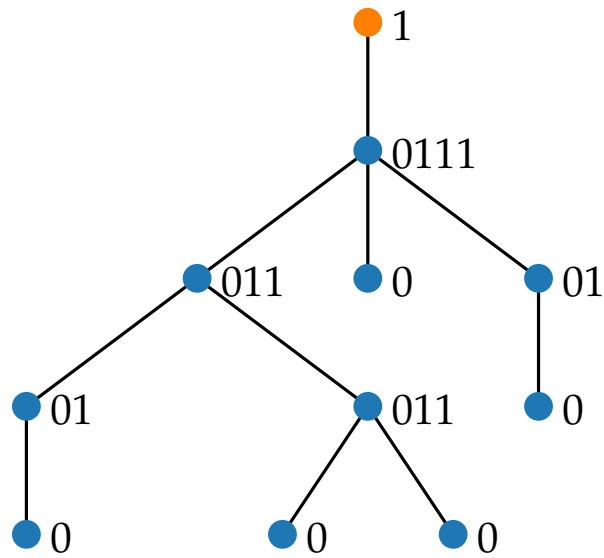
$$\Rightarrow 2n + o(n) \text{ Bits}$$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0
- gibt LOUDS Sequenz

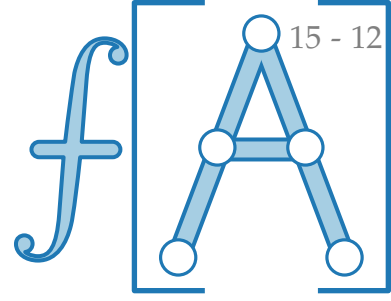
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

Operationen.

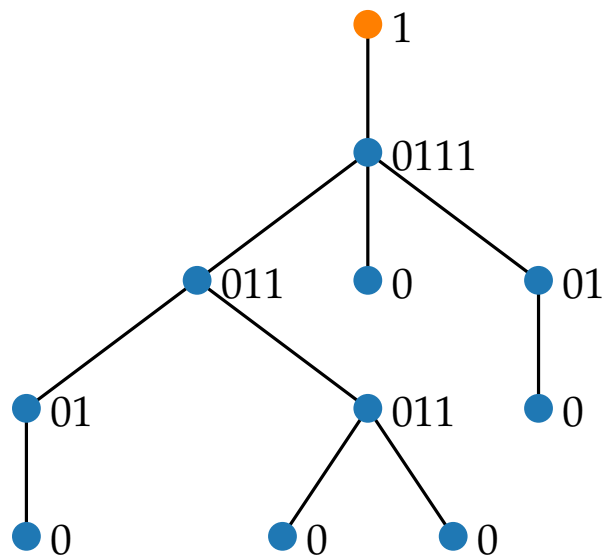
- Knotenobjekte/-werte in weiterem Array

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

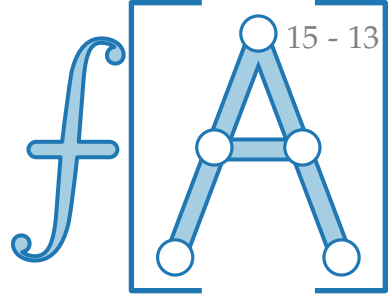
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

Operationen.

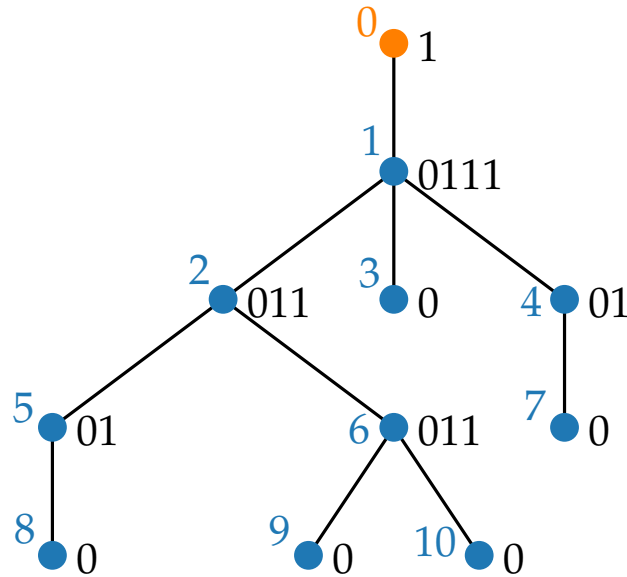
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads getrennt durch je eine 0
- gibt LOUDS Sequenz

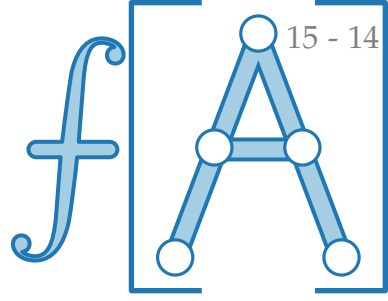
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

Operationen.

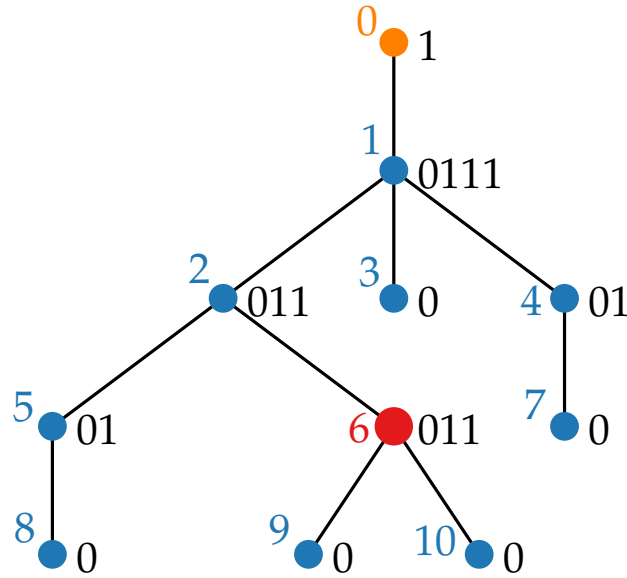
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0
- gibt LOUDS Sequenz

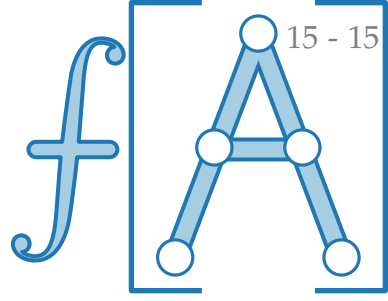
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

Operationen.

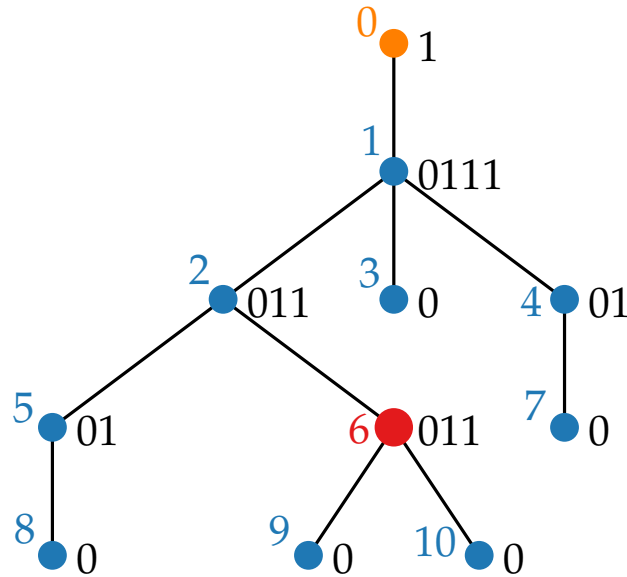
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0
- gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

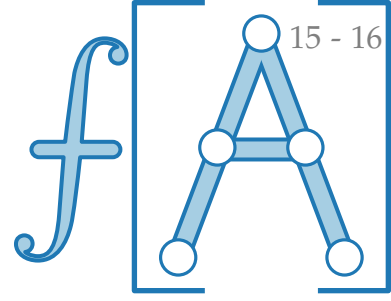
$\text{SELECT}_1(6) =$

Operationen.

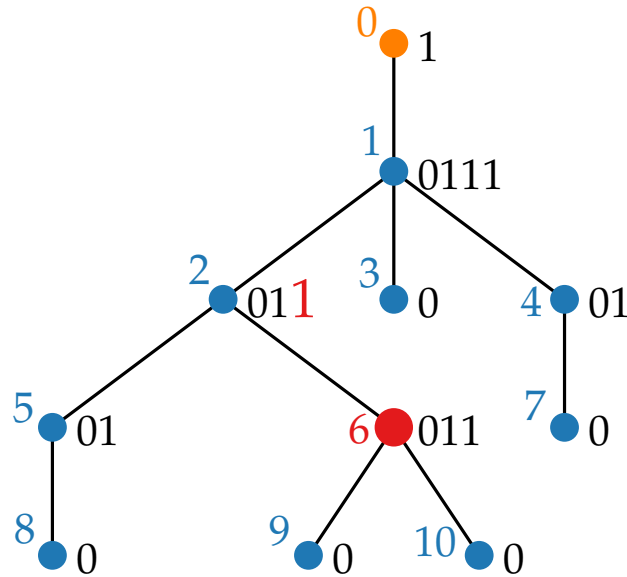
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0
- gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

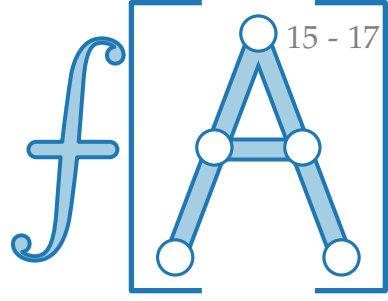
$\text{SELECT}_1(6) =$

Operationen.

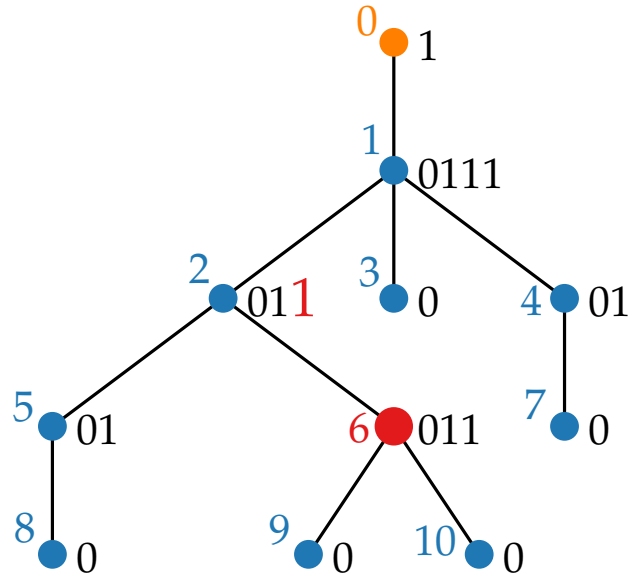
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0
- gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

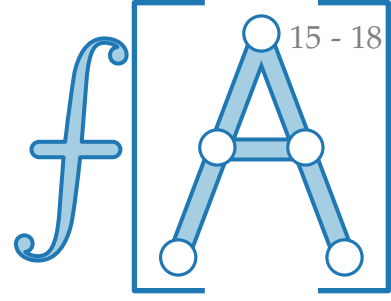
$\text{SELECT}_1(6) =$

Operationen.

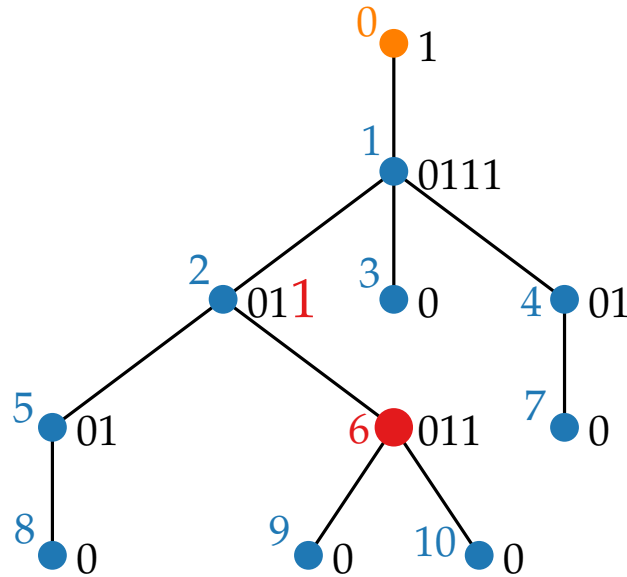
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0
- gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

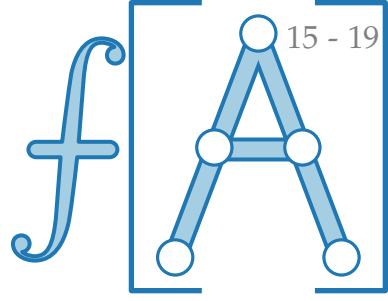
$$\text{SELECT}_1(6) = 8$$

Operationen.

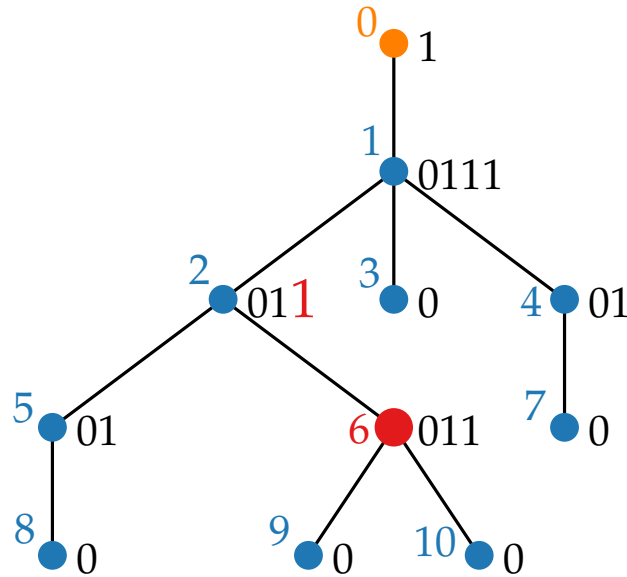
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



- Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0
- gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

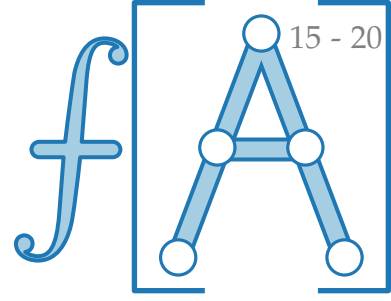
$$\text{SELECT}_1(6) = 8$$

Operationen.

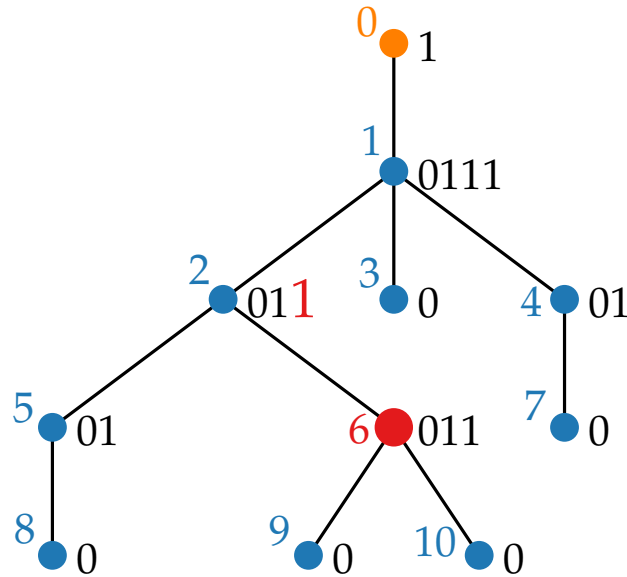
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an
- $\text{SELECT}_0(i)$ gibt Position der 0 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

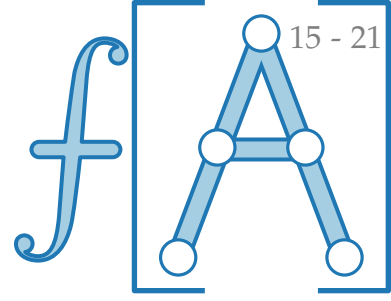
$$\text{SELECT}_0(6) =$$

Operationen.

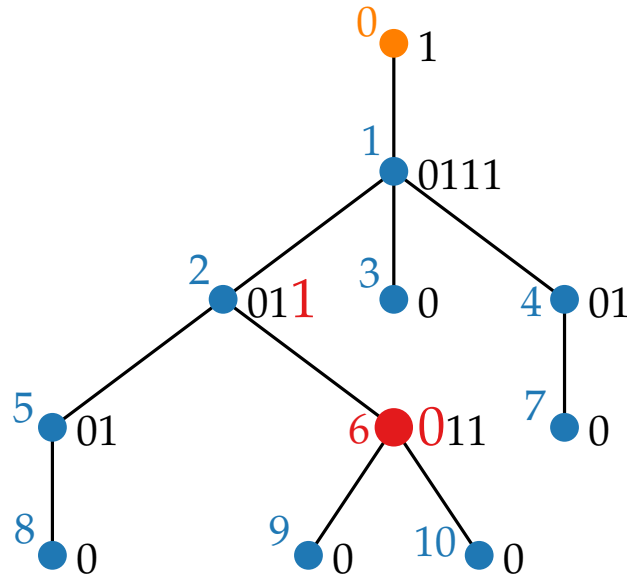
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an
- $\text{SELECT}_0(i)$ gibt Position der 0 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

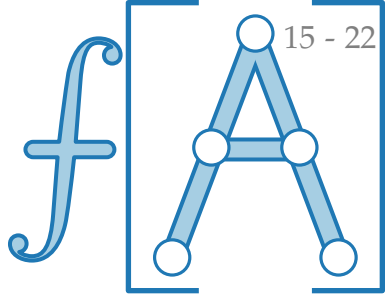
$$\text{SELECT}_0(6) =$$

Operationen.

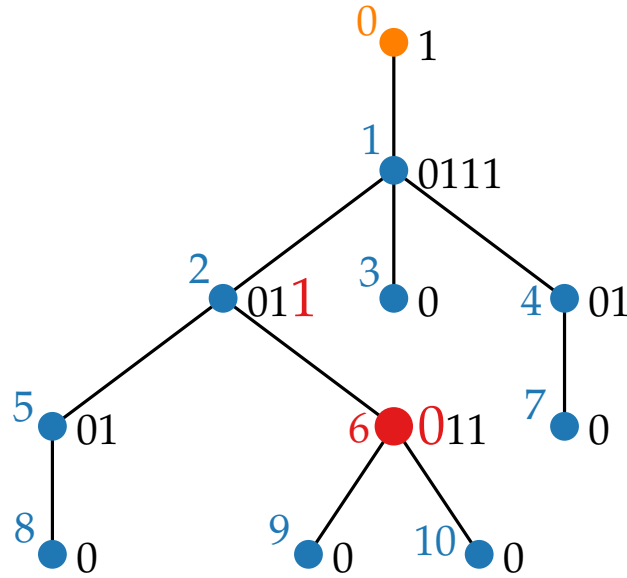
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an
- $\text{SELECT}_0(i)$ gibt Position der 0 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

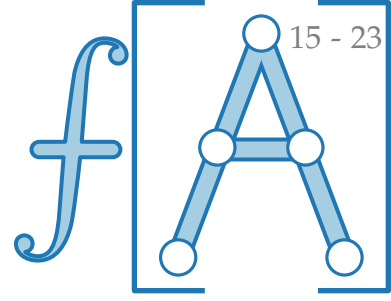
$$\text{SELECT}_0(6) =$$

Operationen.

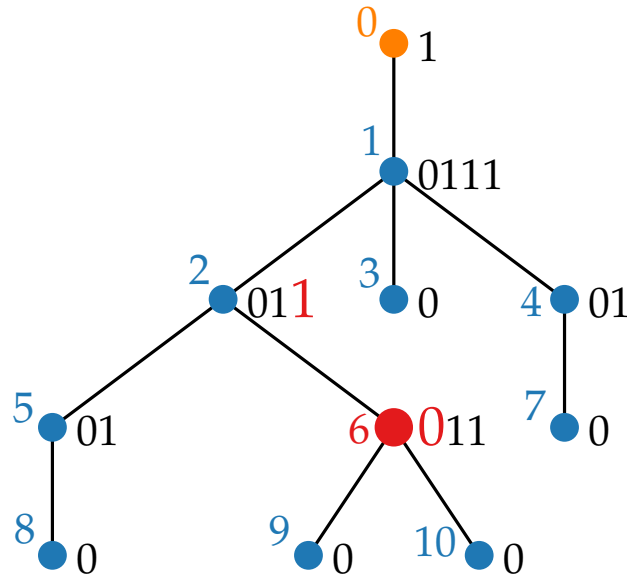
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an
- $\text{SELECT}_0(i)$ gibt Position der 0 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

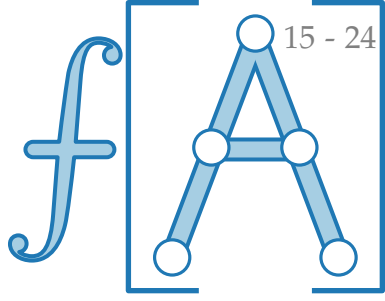
$$\text{SELECT}_0(6) = 17$$

Operationen.

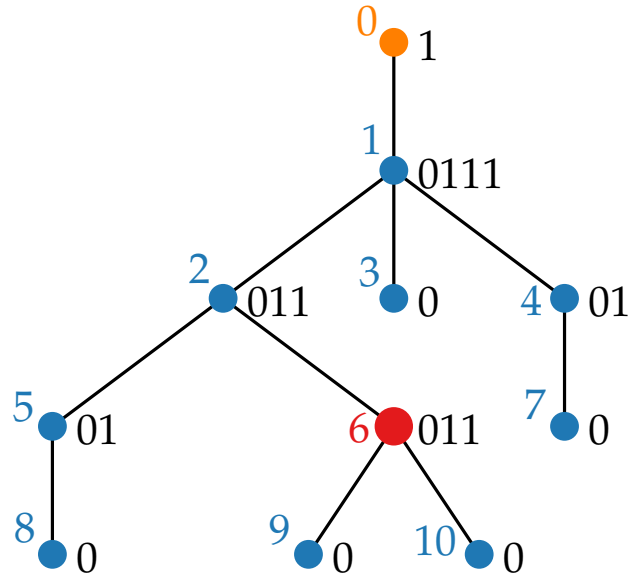
- Knotenobjekte/-werte in weiterem Array
- $\text{SELECT}_1(i)$ gibt Position der 1 von Knoten i in LOUDS Sequenz an
- $\text{SELECT}_0(i)$ gibt Position der 0 von Knoten i in LOUDS Sequenz an

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

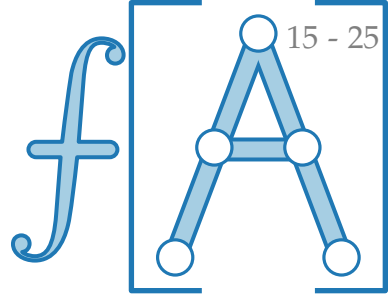
$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

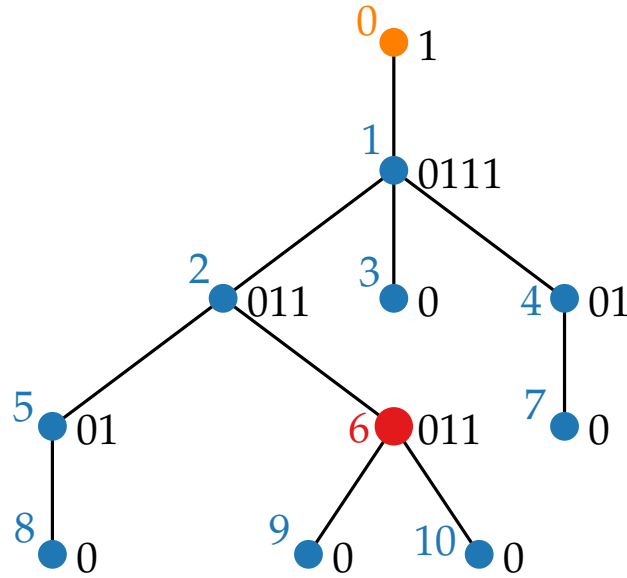
■ $\text{FIRSTCHILD}(i) =$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

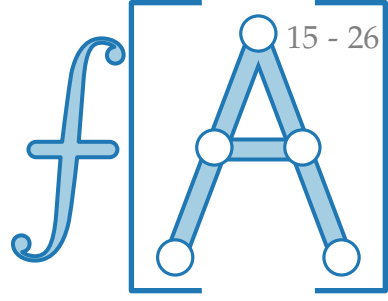
$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) =$

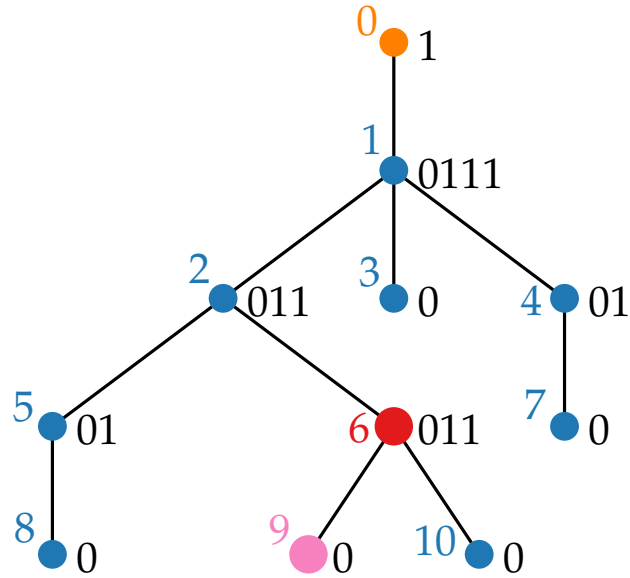
$\text{FIRSTCHILD}(6) =$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

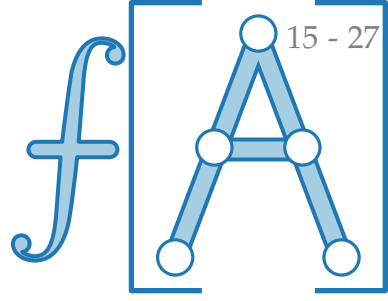
■ $\text{FIRSTCHILD}(i) =$

$\text{FIRSTCHILD}(6) =$

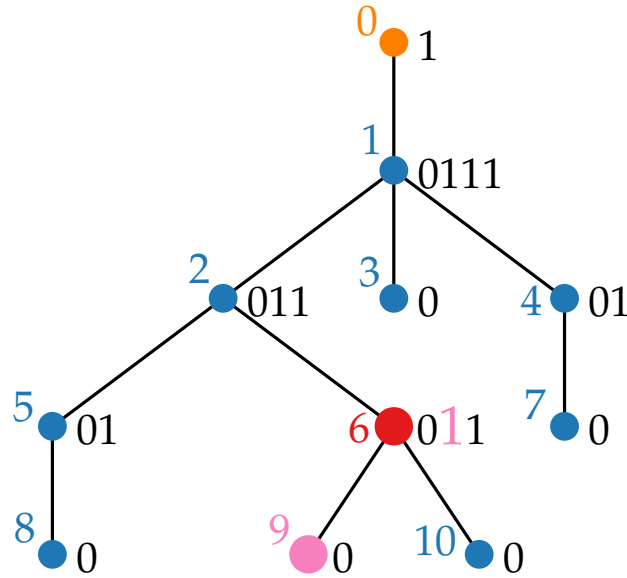
$= 9$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

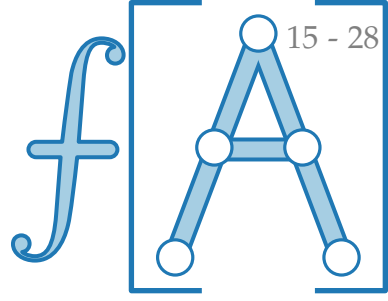
■ $\text{FIRSTCHILD}(i) =$

$\text{FIRSTCHILD}(6) =$

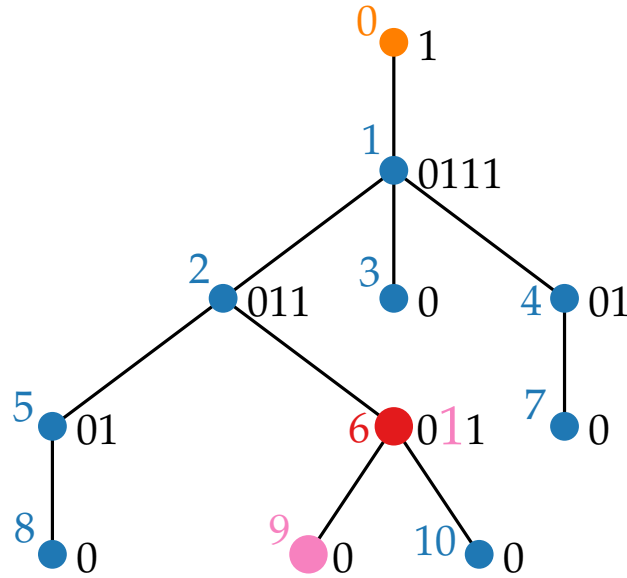
$= 9$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

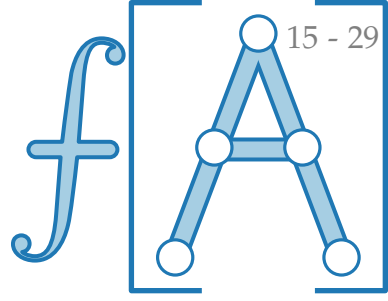
■ $\text{FIRSTCHILD}(i) =$

$\text{FIRSTCHILD}(6) =$

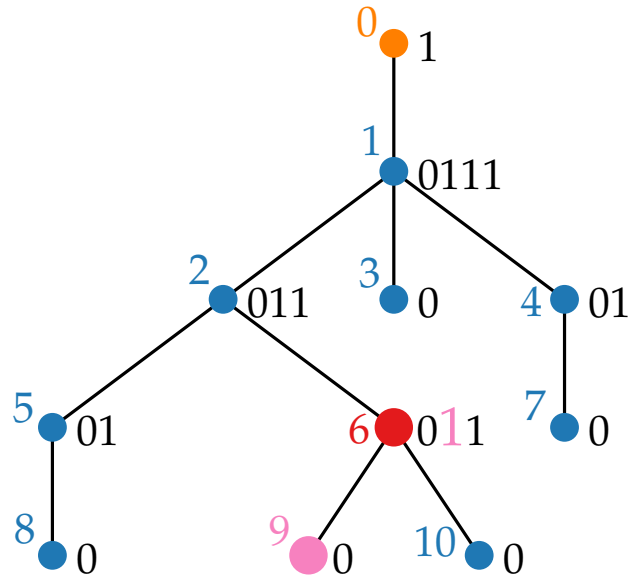
$= 9$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

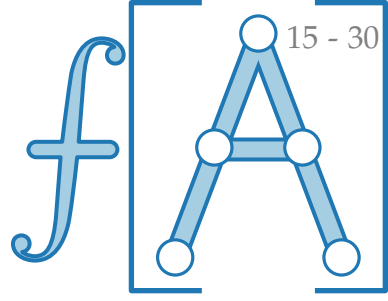
■ $\text{FIRSTCHILD}(i) =$

$$\text{FIRSTCHILD}(6) =$$

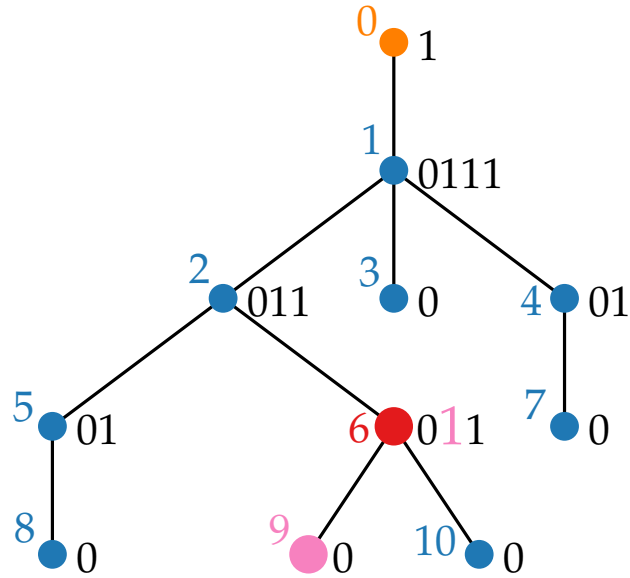
$$= \text{RANK}_1(15) = 9$$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

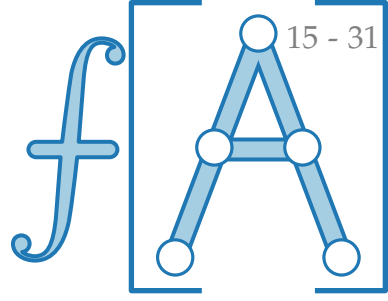
■ $\text{FIRSTCHILD}(i) =$

$$\text{FIRSTCHILD}(6) =$$

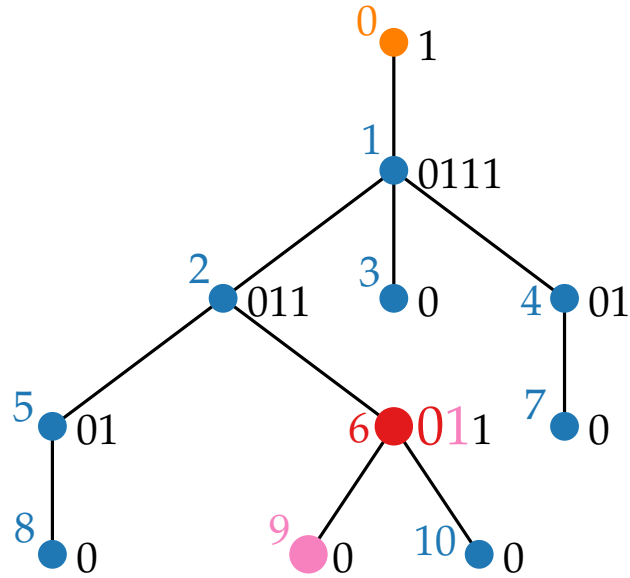
$$= \text{RANK}_1(15) = 9$$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

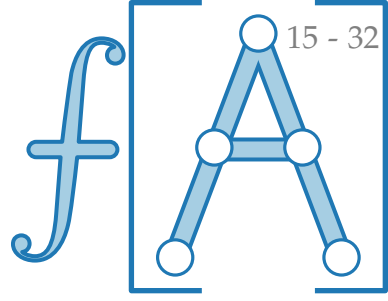
■ $\text{FIRSTCHILD}(i) =$

$$\text{FIRSTCHILD}(6) =$$

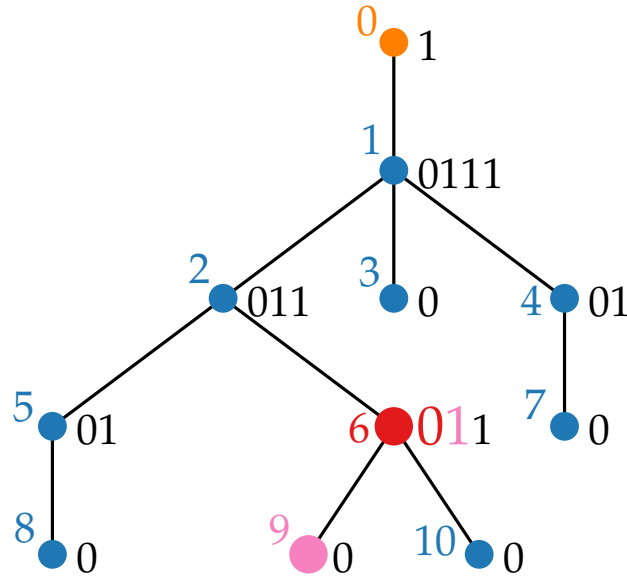
$$= \text{RANK}_1(15) = 9$$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

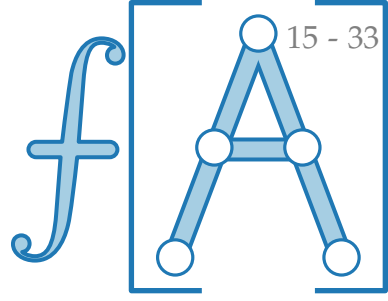
■ $\text{FIRSTCHILD}(i) =$

$$\text{FIRSTCHILD}(6) =$$

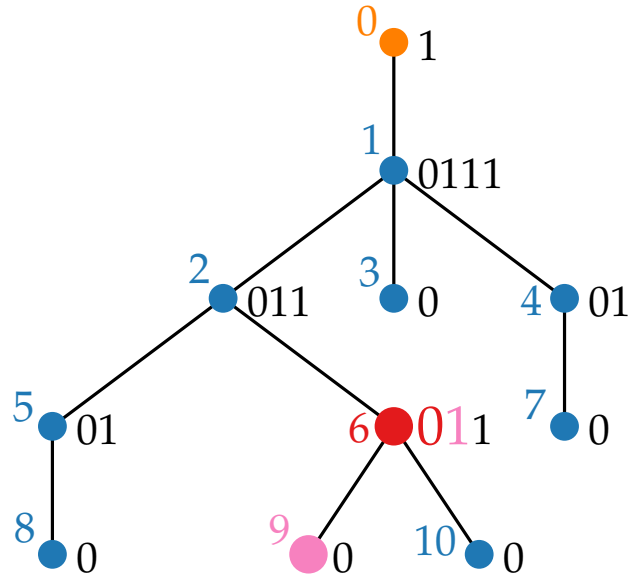
$$= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9$$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

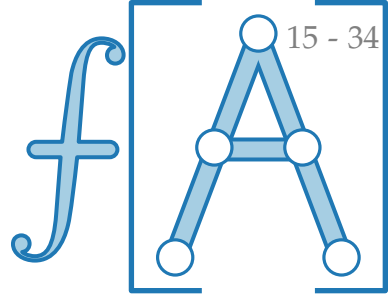
■ $\text{FIRSTCHILD}(i) =$

$$\text{FIRSTCHILD}(6) =$$

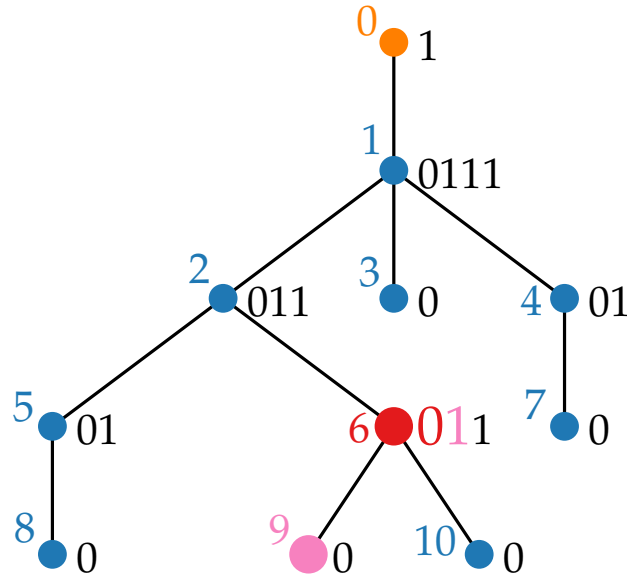
$$= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9$$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

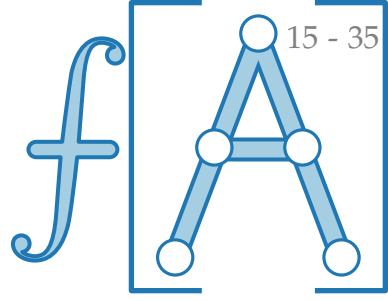
■ $\text{FIRSTCHILD}(i) =$

$$\text{FIRSTCHILD}(6) = \text{RANK}_1(\text{SELECT}_0(6) + 1)$$

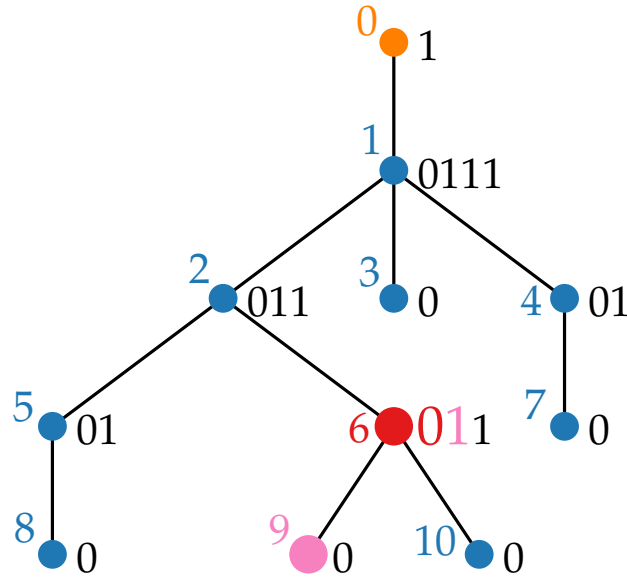
$$= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9$$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

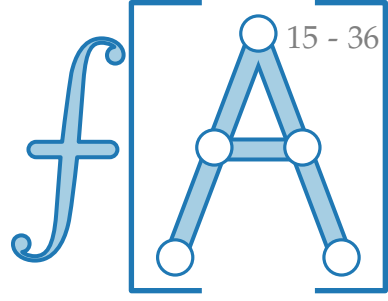
$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

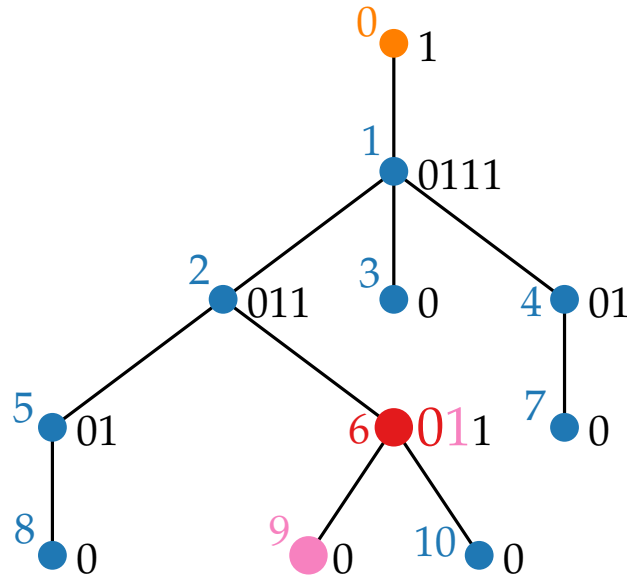
$$\begin{aligned}\text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9\end{aligned}$$

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

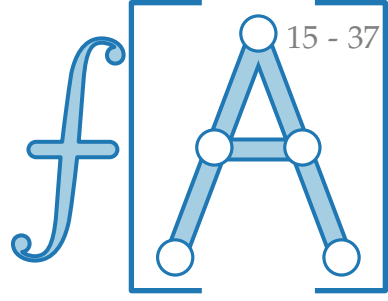
■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

$$\begin{aligned}\text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9\end{aligned}$$

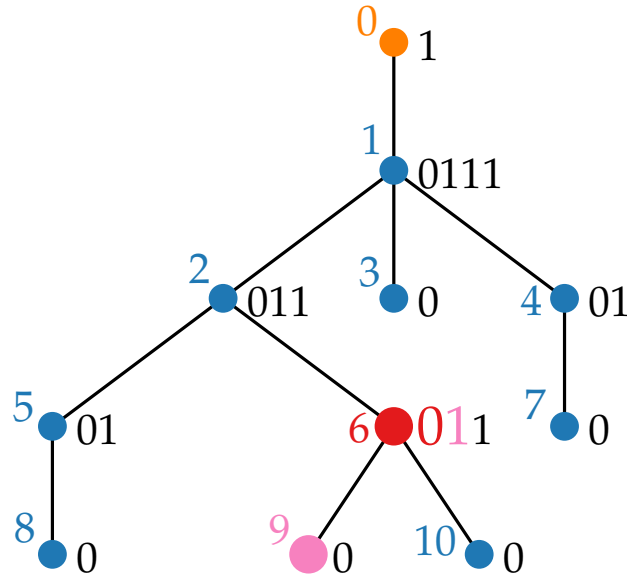
■ $\text{NEXTSIBLING}(i) = i + 1$ oder 0

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

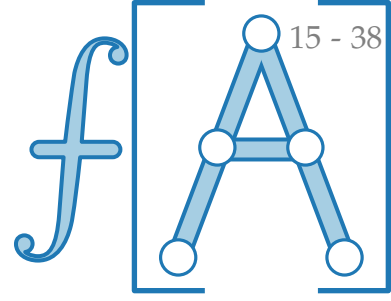
$$\begin{aligned}\text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9\end{aligned}$$

■ $\text{NEXTSIBLING}(i) = i + 1$ oder 0

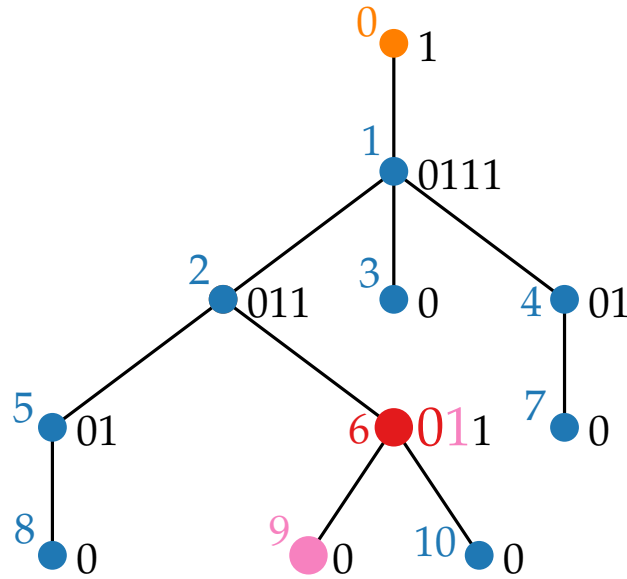
Übung: $\text{CHILD}(i, j)$: j -tes Kind von i

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

■ $\text{PARENT}(i) =$

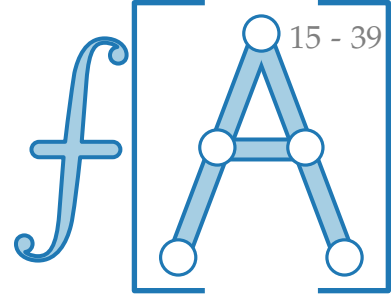
$$\begin{aligned}\text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9\end{aligned}$$

■ $\text{NEXTSIBLING}(i) = i + 1$ oder 0

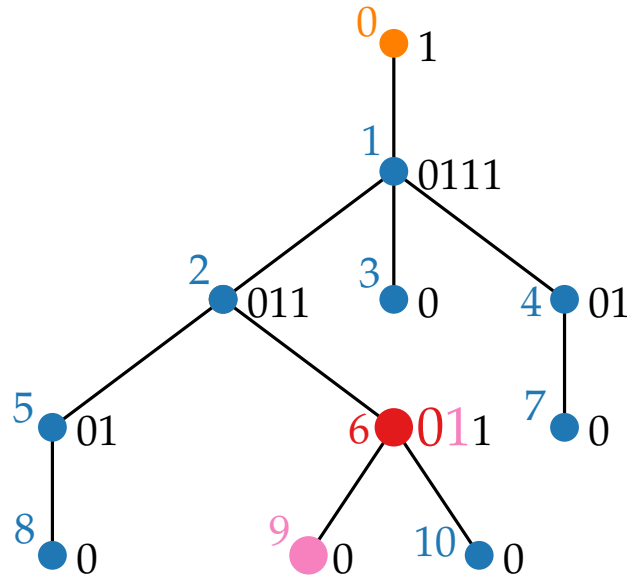
Übung: $\text{CHILD}(i, j)$: j -tes Kind von i

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

■ $\text{PARENT}(i) =$

$$\begin{aligned} \text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9 \end{aligned}$$

$$\text{PARENT}(9) =$$

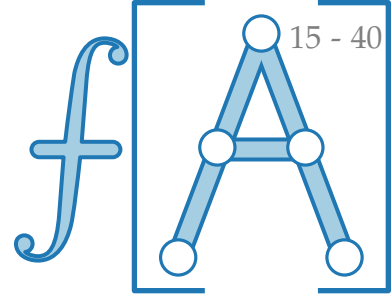
$$= 6$$

■ $\text{NEXTSIBLING}(i) = i + 1$ oder 0

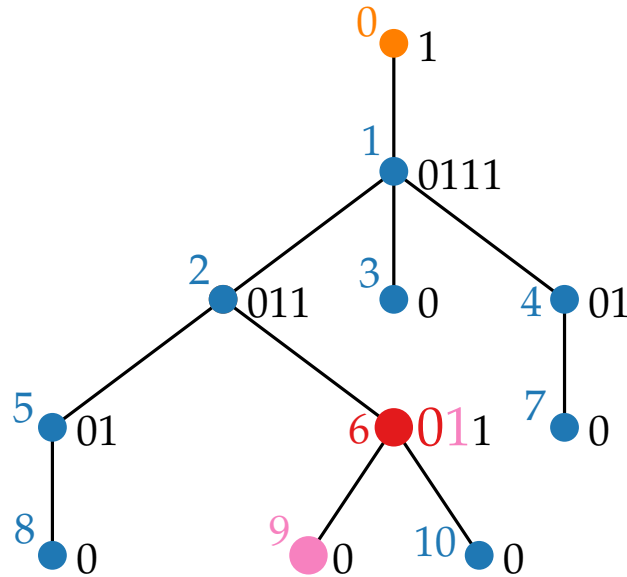
Übung: $\text{CHILD}(i, j)$: j -tes Kind von i

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

■ $\text{PARENT}(i) =$

$$\begin{aligned} \text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9 \end{aligned}$$

$$\text{PARENT}(9) =$$

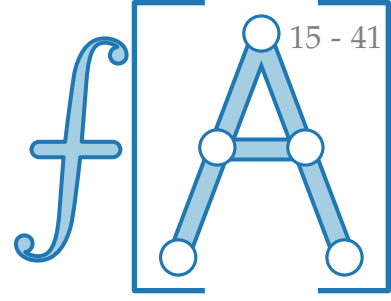
$$\text{RANK}_0(14) = 6$$

■ $\text{NEXTSIBLING}(i) = i + 1$ oder 0

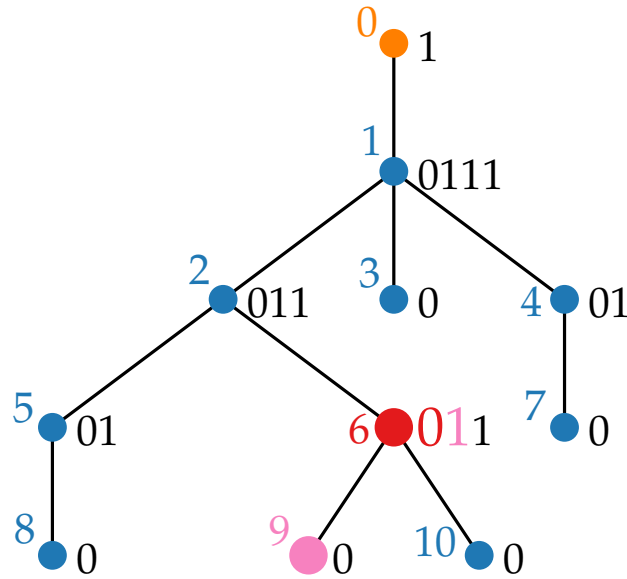
Übung: $\text{CHILD}(i, j)$: j -tes Kind von i

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

$$\begin{aligned}\text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9\end{aligned}$$

■ $\text{PARENT}(i) =$

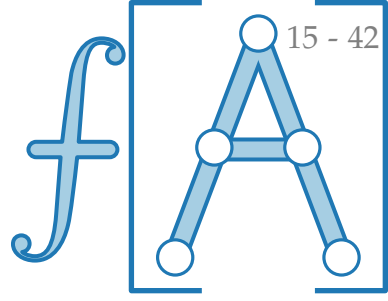
$$\begin{aligned}\text{PARENT}(9) &= \\ &= \text{RANK}_0(15) = \text{RANK}_0(14) = 6\end{aligned}$$

■ $\text{NEXTSIBLING}(i) = i + 1$ oder 0

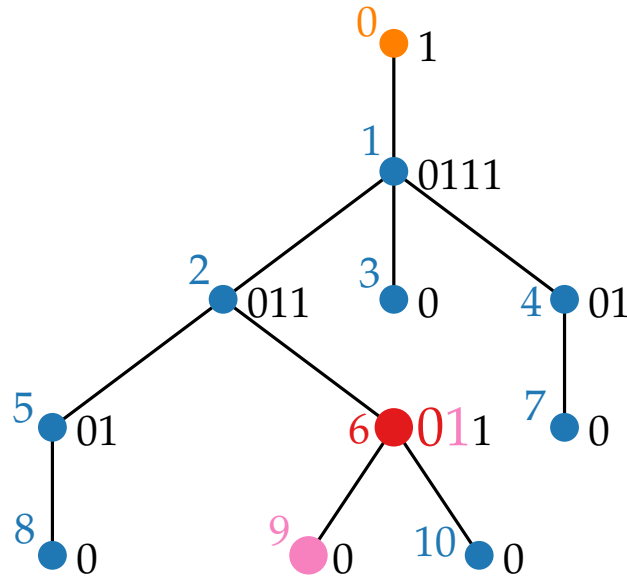
Übung: $\text{CHILD}(i, j)$: j -tes Kind von i

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

$$\begin{aligned}\text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9\end{aligned}$$

■ $\text{PARENT}(i) =$

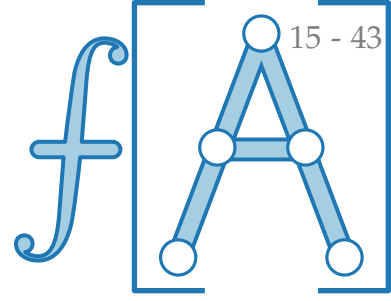
$$\begin{aligned}\text{PARENT}(9) &= \text{RANK}_0(\text{SELECT}_1(9)) \\ &= \text{RANK}_0(15) = \text{RANK}_0(14) = 6\end{aligned}$$

■ $\text{NEXTSIBLING}(i) = i + 1$ oder 0

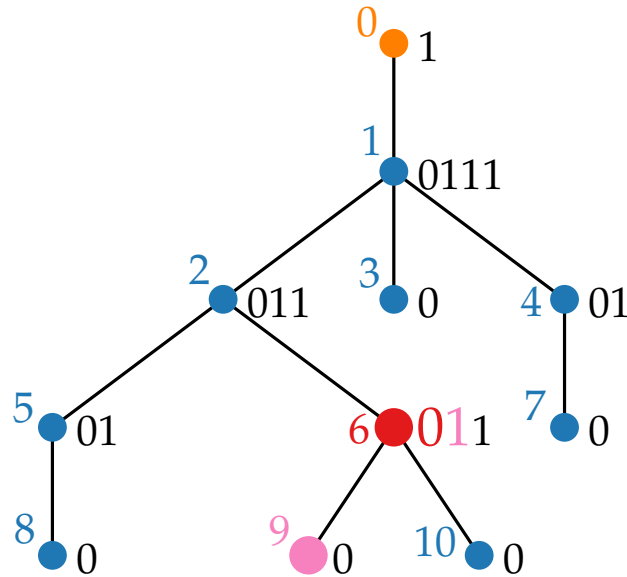
Übung: $\text{CHILD}(i, j)$: j -tes Kind von i

Platzsparende Repr. von Bäumen – LOUDS

[Guy Jacobson, FOCS 1989]



LOUDS = Level Order Unary Degree Sequence



■ Unäre Dekodierung des ausgehenden Grads
getrennt durch je eine 0

■ gibt LOUDS Sequenz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	0	1	1	1	0	1	1	0	0	1	0	1	0	1	1	0	0	0	0	0

$$\text{SELECT}_1(6) = 8$$

$$\text{SELECT}_0(6) = 17$$

■ $\text{FIRSTCHILD}(i) = \text{RANK}_1(\text{SELECT}_0(i) + 1)$

$$\begin{aligned}\text{FIRSTCHILD}(6) &= \text{RANK}_1(\text{SELECT}_0(6) + 1) \\ &= \text{RANK}_1(14 + 1) = \text{RANK}_1(15) = 9\end{aligned}$$

■ $\text{PARENT}(i) = \text{RANK}_0(\text{SELECT}_1(i))$

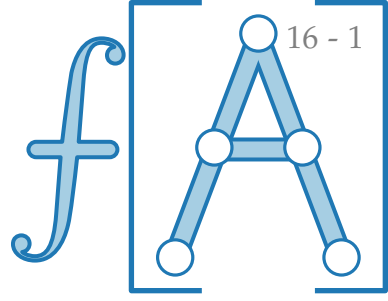
$$\begin{aligned}\text{PARENT}(9) &= \text{RANK}_0(\text{SELECT}_1(9)) \\ &= \text{RANK}_0(15) = \text{RANK}_0(14) = 6\end{aligned}$$

■ $\text{NEXTSIBLING}(i) = i + 1$ oder 0

Übung: $\text{CHILD}(i, j)$: j -tes Kind von i

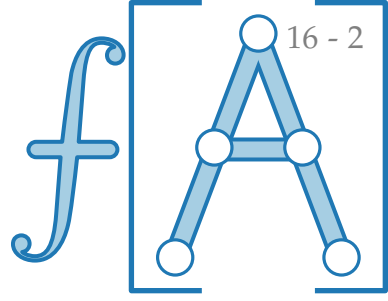
Diskussion

- Platzsparende Datenstrukturen
 - sind sehr platzeffizient
 - lassen schnelle Operationen zu



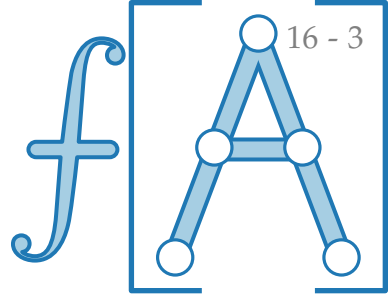
Diskussion

- Platzsparende Datenstrukturen
 - sind sehr platzeffizient
 - lassen schnelle Operationen zuaber
 - sind meistens statisch (dynamisch mit extra Kosten)



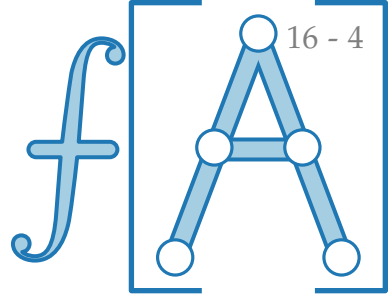
Diskussion

- Platzsparende Datenstrukturen
 - sind sehr platzeffizient
 - lassen schnelle Operationen zuaber
 - sind meistens statisch (dynamisch mit extra Kosten)
 - Anzahl der Operationen ist beschränkt

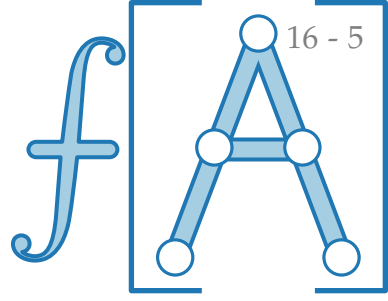


Diskussion

- Platzsparende Datenstrukturen
 - sind sehr platzeffizient
 - lassen schnelle Operationen zuaber
 - sind meistens statisch (dynamisch mit extra Kosten)
 - Anzahl der Operationen ist beschränkt
 - komplex → schwerer zu implementieren



Diskussion



- Platzsparende Datenstrukturen
 - sind sehr platzeffizient
 - lassen schnelle Operationen zuaber
 - sind meistens statisch (dynamisch mit extra Kosten)
 - Anzahl der Operationen ist beschränkt
 - komplex ➡ schwerer zu implementieren
- RANK und SELECT bilden die Basis für viele platzsparende Repräsentationen